



לוגו בית הספר :

שם בית הספר : עירוני א' למדעים ולאומנויות, מודיעין

שם הפרויקט : Securio

שם התלמיד : יואב פאר

ת"ז : 333057891

שם המורה : ורד בת-חן רודריק

התמחות : תכנון ותכנות מערכות בטלפונים ניידים תחת מערכת הפעלה Android



לוגו האפליקציה :

9.4.2026

תאריך הגשה :

תוכן עניינים

2	מבוא
2	הרקע לפרויקט
2	תהליך המחקר
4	אתגרים מרכזיים
10	מבנה/ארכיטקטורה
10	תכנון ותיעוד מסכי הפרויקט
22	מימוש הפרויקט
22	פירוט מסכים
33	פירוט עצמים
37	פירוט מחלקות Azure Functions
41	לוגיקת השרת (Business Logic)
56	מחלקות נוספות
59	מחלקות Helper
72	עוזרי לקוח נוספים
81	בסיס נתונים טבלאי
87	מדריך למשתמש
88	הרחבות
89	רפלקציה
90	ביבליוגרפיה
91	נספחים

מבוא

הרקע לפרויקט

שם הפרויקט:

Securio - Secure Password Manager

תיאור קצר של הפרויקט:

Securio היא אפליקציית Native לאנדרואיד המהווה מנהל סיסמאות מאובטח מבוסס ענן. המערכת בנויה על ארכיטקטורת Client-Side Encryption, המבטיחה שכל פעולות ההצפנה והפענוח מתבצעות על מכשיר המשתמש בלבד באמצעות פרוטוקול AES-256-GCM ללא חשיפת מפתחות ההצפנה לשרת. הפרויקט משלב צד-לקוח המפותח ב-Xamarin.Android עם תשתית Backend מבוססת Azure Functions ומסד נתונים Azure SQL. ייחודה של המערכת הוא בשירות רקע (Foreground Service) ייעודי המבצע סריקות יומיומיות ומנטר באופן אקטיבי את חוסן הסיסמאות וחשיפתן בדליפות מידע.

מטרת הפרויקט:

המטרה המרכזית של הפרויקט היא לספק פתרון אבטחתי מקיף שאינו מסתפק רק באחסון פסיבי, אלא מציע הגנה אקטיבית ודינמית על זהויות דיגיטליות. הפרויקט שואף ליישם מודל Zero Knowledge, שבו פרטיות המשתמש מובטחת ברמה המתמטית והטכנולוגית, כך שגם במקרה של חדירה לשרתים, המידע יישאר חסוי. יעד מרכזי נוסף הוא הפיכת האבטחה לאוטומטית; המערכת נועדה לחסוך מהמשתמש את הצורך בבדיקה ידנית של "בראות" הכספת שלו על ידי זיהוי אוטונומי של סיסמאות חלשות, ישנות או דלופות (מול מאגרי HIBP) ושליחת התראות בזמן אמת. בכך, הפרויקט שואף להנגיש רמת אבטחה של ארגונים גדולים למשתמש הפרטי בצורה פשוטה ויעילה.

קהל היעד:

קהל היעד של Securio כולל משתמשי אנדרואיד המחפשים פתרון ריכוזי ומאובטח לניהול עשרות חשבונות ושירותים דיגיטליים. המערכת פונה במיוחד למשתמשים בעלי מודעות גבוהה לפרטיות, המעוניינים בשירות שאינו חשוף למידע הרגיש שלהם. בנוסף, האפליקציה מיועדת לאנשים המעוניינים בהגנה אקטיבית וקבלת התרעות בזמן אמת במידה ואחת מסיסמאותיהם דורשת החלפה.

הסיבות לבחירת פרויקט:

הבחירה בפרויקט זה נבעה מעניין אישי עמוק בתחום אבטחת המידע ומהרצון לעבור מזיהוי פרצות אבטחה למתן מענה מעשי ומניעתי כבר משלב התכנון והפיתוח. פיתוח האפליקציה מאפשר לי לשלב ידע תיאורטי עם יישום מעשי של פרוטוקולי הצפנה מתקדמים, כגון AES-256-GCM ו-PBKDF2, ובכך להעמיק את הבנתי בבניית מערכות חסינות. בנוסף, הצבתי לעצמי אתגר טכנולוגי לפתח לראשונה מערכת Backend מלאה ורצינית, תוך שימוש בשירותי ענן של Azure Functions וניהול בסיסי נתונים ב-Azure SQL, כדי להרחיב את סל הכלים שלי כמפתח תוכנה.

תהליך המחקר

מחקר על תחום הידע:

בעידן הדיגיטלי הנוכחי, שבו המשתמש הממוצע מנהל עשרות חשבונות בשירותים שונים, סיסמאות הפכו לנקודת התורפה המרכזית באבטחת המידע האישי. נתונים סטטיסטיים מדוח ה-DBIR של Verizon מדגישים את חומרת הבעיה, ומראים כי למעלה מ-68% מפרצות האבטחה קשורות לשימוש בסיסמאות חלשות, גנובות או ממוחזרות. תופעה זו מולידה את איום ה-Credential Stuffing, שבו תוקפים

משתמשים במאגרי מידע שדלפו כדי לנסות ולחדור לחשבונות נוספים של אותו משתמש. הצורך בפתרון אבטחתי יעיל הוליד את תחום ניהול הסיסמאות (Password Management), שמטרתו לספק מענה לכשל האנושי מבלי להתפשר על נוחות המשתמש. עם זאת, האתגר המשמעותי ביותר במערכות אלו הוא שאלת האמון: כיצד ניתן לאחסן את כל המידע הרגיש ביותר של המשתמש בשרת חיצוני מבלי לחשוף אותו למנהלי המערכת או לתוקפים פוטנציאליים?

כדי לענות על אתגר זה, הפרויקט מתבסס על מודל ה-ZKA (Zero Knowledge Architecture). תפיסה תכנונית שבה לספק השירות אין כל גישה למידע הגולמי של המשתמש, והיא נשענת על עקרון ה-Client Side Encryption. במודל זה, כל פעולות ההצפנה והפענוח מתבצעות אך ורק על מכשיר הקצה, כך שמפתחות ההצפנה הפרטיים לעולם אינם עוזבים את המכשיר ואינם נשלחים לענן. המשמעות היא שגם במקרה של פריצה למסדי הנתונים של השרת, המידע השמור בהם יהיה מוצפן בצורה חזקה וחסרת ערך עבור התוקף. גישה זו, המכונה לעיתים "אמון אפס" (Zero Trust), מעבירה את השליטה המלאה בפרטיות לידי המשתמש. במקום להסתמך על הבטחות של חברות תוכנה לשמירה על הפרטיות, ה-ZKA מייצר מגבלה טכנולוגית ומתמטית שמונעת טכנית גישה למידע, ובכך מיישמת את עקרון ה-Security by Design כחלק בלתי נפרד מתשתית המערכת.

טכנולוגיות קיימות בשוק:

במסגרת תכנון הפרויקט, ביצעתי סקירה של מנהלי הסיסמאות המובילים בשוק כדי לעמוד על הפערים הקיימים בין נוחות המשתמש לבין רמת האבטחה והפרטיות המוצעת.

השירות הנפוץ ביותר, **Google Password Manager**, מציע נוחות מרבית וסנכרון אוטומטי, אך לוקה במודל הפרטיות שלו. ניהול מפתחות ההצפנה תלוי לרוב בחשבון המשתמש בגוגל, מה שמותיר יכולת גישה תיאורטית לנתונים. לעומת זאת, המערכת שלי מיישמת הצפנה בצד הלקוח בלבד, שבה מפתח ה-AES נגזר במכשיר ואינו עוזב אותו לעולם, כך שלשרת אין יכולת טכנית לפענח את המידע.

בבחינת השירות **LastPass**, עלה הצורך בהצפנה מלאה של כל שדות המידע ללא יוצא מן הכלל. פרצות אבטחה בעבר (כגון זו של 2022) הוכיחו כי אי-הצפנה של מטא-נתונים, כמו כתובות URL, מאפשרת לתוקפים לזהות אתרים רגישים של המשתמש. הפרויקט שלי נותן מענה לכך על ידי הצפנת כל שדה ב-AES-256-GCM ושימוש ב-PBKDF2 עם 600,000 איטרציות, רף המקשה משמעותית על מתקפות Brute-force.

שירותים מתקדמים כמו **1Password** מציעים אבטחה גבוהה מאוד, אך הם דורשים לרוב מנוי חודשי ומהווים מערכת סגורה. הבידול המרכזי של הפרויקט שלי מול פתרונות אלו הוא שירות ניטור הרקע (Foreground Service) שפועל 24/7 ומתריע על דליפות מידע בזמן אמת גם כשהאפליקציה סגורה – יכולת שאינה קיימת בחלק נכבד מהגרסאות החינמיות של המתחרים.

גם בהשוואה ל-**Bitwarden**, המבוסס על קוד פתוח, זיהיתי פער בתחום הניטור האקטיבי. בעוד שב-Bitwarden בדיקת הדליפות היא לרוב פעולה יזומה שהמשתמש נדרש להפעיל ידנית, המערכת שלי מבצעת ניטור אוטומטי מול מאגר Have I Been Pwned. השימוש בפרוטוקול k-Anonymity מאפשר לבצע את הבדיקות הללו בצורה אנונימית לחלוטין, ובכך משלב הגנה אקטיבית עם פרטיות מקסימלית.

לסיכום, בעוד שהשוק מציע פתרונות בשלים, ניכר כי קיים צורך ממשי בפתרון המנגיש הגנה אקטיבית ופרטיות מוחלטת כברירת מחדל ובאופן אוטומטי, וזהו הערך המוסף של המערכת שפיתחתי.

אתגרים מרכזיים

במהלך תכנון ופיתוח המערכת, ניצבתי בפני מספר אתגרים משמעותיים שנבעו מהדרישות המחמירות לאבטחת מידע מחד, ומהצורך בפונקציונליות אוטונומית מאידך.

האתגר הראשון והמורכב ביותר היה יצירת מנגנון לניטור דליפות מידע מול שירות HIBP; הקושי נבע מהמתח המובנה שבין ארכיטקטורת הצפנה בצד-הלקוח, שבה המידע חסוי לחלוטין בפני השרת, לבין הצורך של השרת לבצע בדיקות תקופתיות ברקע עבור המשתמש גם כאשר האפליקציה אינה פעילה והמפתחות אינם זמינים לפענוח.

הפתרון שנבחר להתמודדות עם אתגר הניטור האוטומטי מבוסס על שמירת ערך ה-SHA-1 Hash של כל סיסמה בשרת, בנפרד מהמידע המוצפן. בחירה זו מהווה פשרה הנדסית מודעת: מצד אחד, היא מאפשרת ל-Azure Function לבצע סריקות רקע עצמאיות מול שירות HIBP באמצעות פרוטוקול k-Anonymity מבלי שהמשתמש יצטרך לפענח את הנתונים במכשירו. מנגד, שמירת Hash בשרת חושפת סיכון תיאורטי שבו תוקף שיחדור למסד הנתונים ינסה לשחזר את הסיסמה המקורית באמצעות מתקפת Rainbow Table. עם זאת, סיכון זה מתקבל כסביר במסגרת מטרות הפרויקט: אם סיסמה מסוימת פגיעה מספיק כדי להופיע בטבלאות ריינובו מוכנות מראש, הרי שהיא ממילא תופיע במאגרי הסיסמאות הפרוצות של HIBP. במקרה כזה, המערכת תתריע למשתמש על הפרצה ותנחה אותו להחליף את הסיסמה לסיסמה חזקה וייחודית שאינה מופיעה במאגרים אלו, ובכך תנטרל את הפגיעות.

כחלק מתהליך קבלת ההחלטות, הועלתה חלופה היפותטית - ניהול "רשימה שחורה" של מיליוני סיסמאות נפוצות ישירות על השרת ובדיקת סיסמאות המשתמשים מולה. כדי להפוך זאת למאובטח, היה צורך לשמור את הרשימה השחורה לאחר שעברה אלגוריתם הצפנה/גיבוב עם מספר איטרציות רב (בדומה ל-PBKDF2) עם "מלח" (Salt) קבוע לכל הסיסמאות כדי לאפשר השוואה ישירה. חלופה זו נפסלה משום שהיא דורשת משאבי חישוב ואחסון אדירים בשרת (כדי להשוות כל סיסמה מול מיליארדי רשומות מוצפנות) ואינה מתאימה לפרויקט תיכון.

```
public class VaultItem  
{  
    //...  
    public string Sha1Hash { get; set; }    // Unsalted SHA-1 hash for HIBP breach lookup  
    //...  
}
```

האתגר השני עסק בסוגיית האחסון המקומי במכשיר הקצה; עלה צורך להגן על נתונים קריטיים להמשכיות העבודה, כגון מלחים (Salts) להזדהות וטוקני גישה (JWT), תוך התמודדות עם העובדה שרכיבי אחסון סטנדרטיים באנדרואיד אינם מספקים הגנה.

הפתרון הוא שימוש ב-SecureStorage של Xamarin.Essentials — ספרייה המגובה על ידי Android Keystore System — להגנה על המלחים (Salts), מפתח הכספת (VaultKey) וטוקן ה-JWT המאוחסנים במכשיר. SecureStorage מצפין אוטומטית נתונים עם AES-256-GCM בגיבוי חומרה (Hardware-backed Keystore), ומבטיח שגם אם גורם חיצוני ישיג גישה למערכת הקבצים, המידע הרגיש יישאר בלתי קריא. גישה זו מספקת הגנה מפני גישה לא מורשית של אפליקציות אחרות ותוקפים בעלי גישה פיזית למכשיר.

```

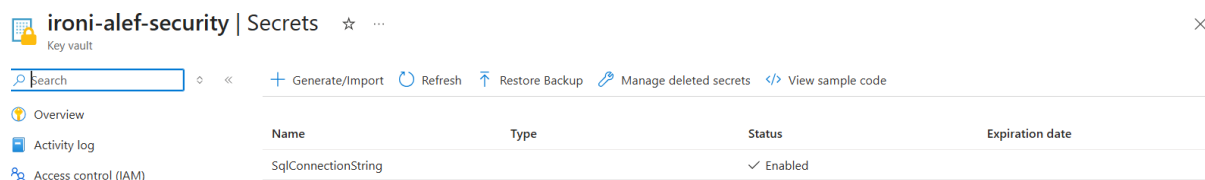
public static class StorageHelper
{
    // These keys identify the data in the encrypted file
    private const string KeyUserId = "user_id";
    //...
    /// <summary>Saves the user's unique ID to secure storage.</summary>
    public static async Task SaveUserId(int id)
    {
        await SecureStorage.SetAsync(KeyUserId, id.ToString());
    }

    /// <summary>Retrieves the user's unique ID from secure storage, returning 0 if not found
    or invalid.</summary>
    public static async Task<int> GetUserId()
    {
        var id = await SecureStorage.GetAsync(KeyUserId);
        return int.TryParse(id, out int result) ? result : 0;
    }
    //...
}

```


האתגר השלישי עסק בניהול סודות תשתיתיים בתוך סביבת ענן בצד השרת; היה עליי להבטיח כי פרטי הגישה הרגישים למסד הנתונים ב-Azure לא יהיו חשופים כחלק מקוד המקור של ה-Functions, דבר המהווה נקודת תורפה קריטית בכל מערכת מבוזרת.

הפתרון לאבטחת מחרוזת החיבור לדאטאבייס התבסס על שילוב של Azure Key Vault יחד עם Managed Identity. במקום לשמור את פרטי הגישה בהגדרות האפליקציה (App Settings) כטקסט פשוט, המחרוזת אוחסנה כ-"Secret" בתוך כספת הכתובות המאובטחת (Key Vault). ה-Azure Function הוגדרה עם זהות מנוהלת המאפשרת לה לגשת לכספת ולמשוך את המחרוזת בזמן הריצה בלבד, ללא צורך בחשיפת פרטי זיהוי בתוך הקוד או בממשקי הניהול הגלויים.



// Get the connection string from the Azure Key Vault

```
string sqlConn = Environment.GetEnvironmentVariable("SqlConnectionString");
```

מבנה/ארכיטקטורה

תכנון ותיעוד מסכי הפרויקט

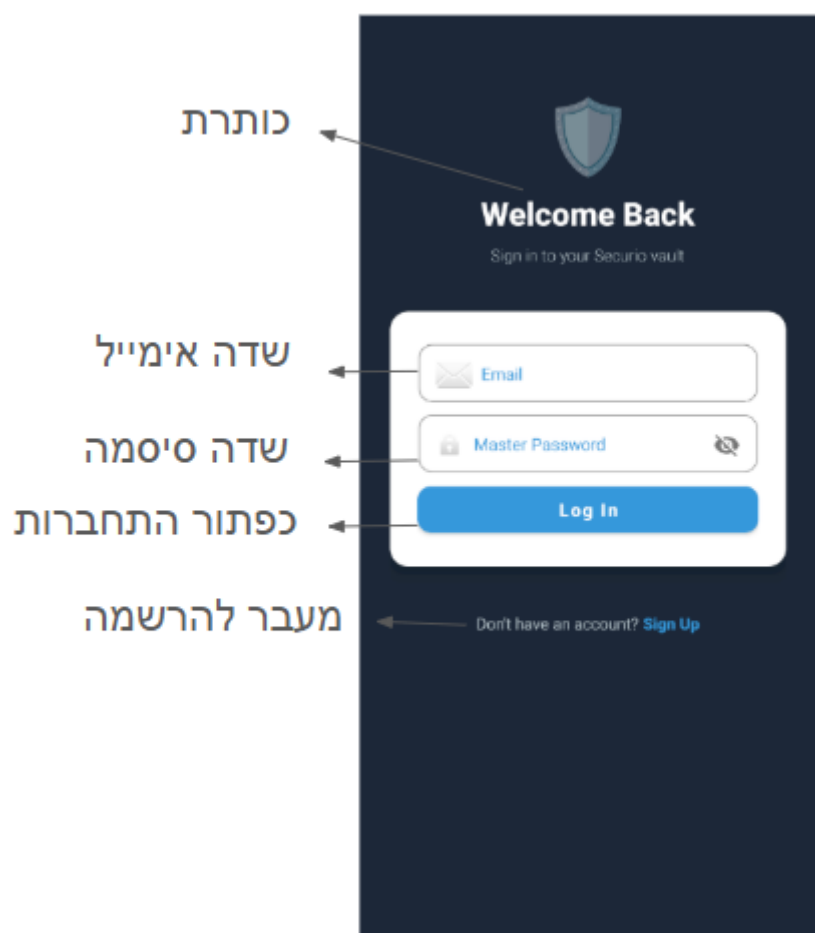
מסך כניסה

מסך זה הוא נקודת הכניסה לאפליקציה. בעת פתיחת האפליקציה, מסך זה בודק האם קיים JWT תקף באחסון המחובר לחשבון. אם כן, המשתמש מועבר ישירות לכספת הסיסמאות שלו. אם לא, הוא מועבר לדף ההתחברות. בנוסף, המסך מבקש הרשאת התראות ומפעיל את שירות ניטור הסיסמאות ברקע.



מסך התחברות

במסך זה המשתמש מזדהה באמצעות כתובת מייל וסיסמת העל שלו. בלחיצה על "התחברות" נשלחת בקשת GetSalt לשרת לאחזור AuthSalt ו-EncryptionSalt, ומהם נגזר מפתח PBKDF2 בצד הלקוח. המפתח מושווה מול השרת לאימות; בהצלחה, נשמרים ה-JWT ומפתח הכספת ב-SecureStorage והמשתמש מועבר לכספת הסיסמאות. משתמשים חדשים יכולים לעבור מכאן למסך ההרשמה.



מסך הרשמה

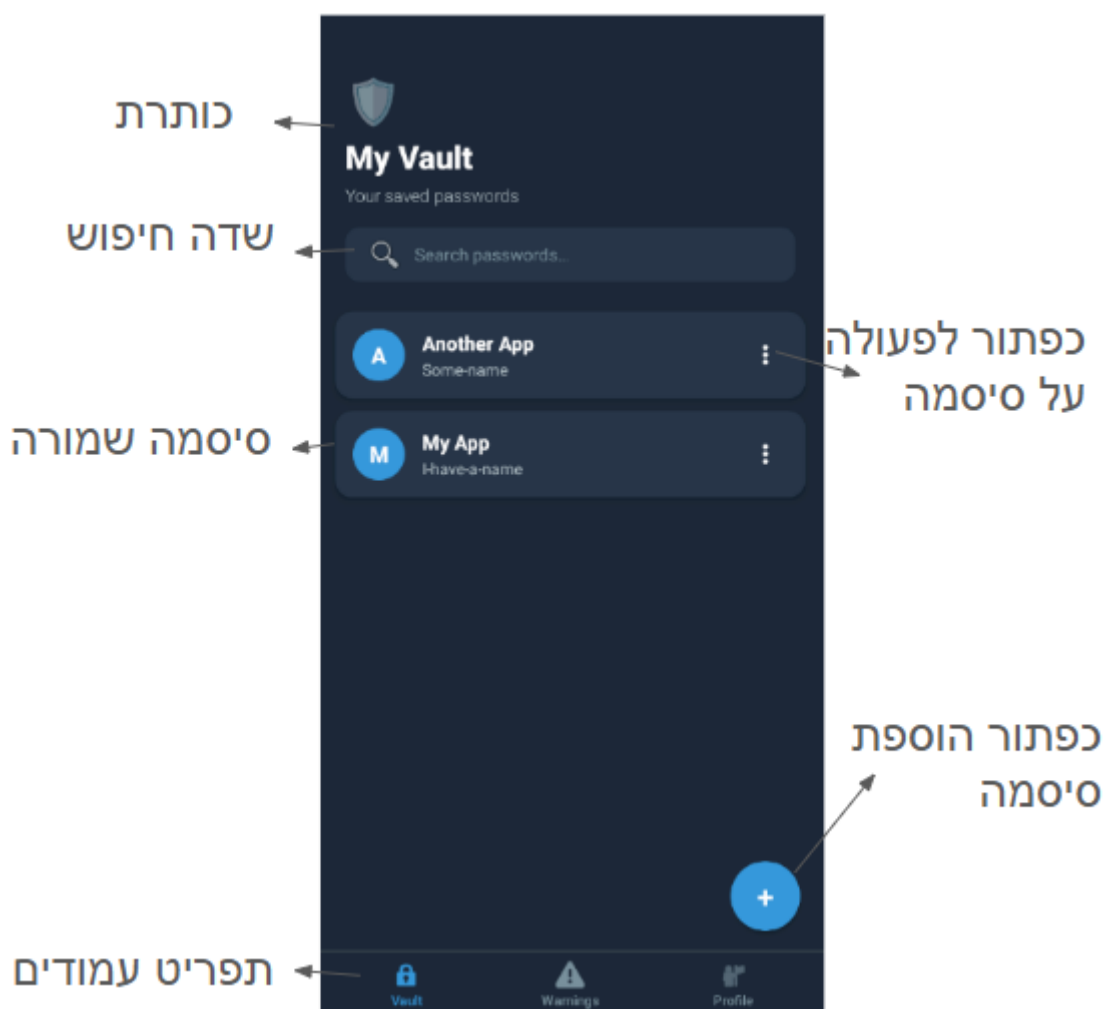
במסך זה המשתמש נרשם לאפליקציה על ידי הזנת שם משתמש, מייל וסיסמת על. בתהליך ההרשמה נוצרים שני מלחים ייחודיים (AuthSalt לאימות, EncryptionSalt לכספת), ונגזר מפתח PBKDF2. הסיסמה נבדקת מול HIBP לפני שמירה. המסך כולל מד חוזק סיסמה בזמן אמת וכפתור ליצירת סיסמה חזקה אוטומטית. עם סיום ההרשמה המוצלחת המשתמש מועבר ישירות לכספת הסיסמאות.

The image shows a 'Create Account' form with the following fields and annotations:

- כותרת** (Title): Points to the 'Create Account' header.
- שדה שם משתמש** (Username field): Points to the 'Username' input field.
- שדה אימייל** (Email field): Points to the 'Email' input field.
- שדה סיסמה** (Master Password field): Points to the 'Master Password' input field.
- כפתור הגרלת סיסמה חזקה** (Generate strong password button): Points to the 'Generate strong password' button.
- שדה אישור סיסמה** (Confirm Master Password field): Points to the 'Confirm Master Password' input field.
- כפתור הרשמה** (Sign Up button): Points to the 'Sign Up' button.
- מעבר להתחברות** (Already have an account? Log in): Points to the 'Log in' link.

מסך כספת הסיסמאות

זהו המסך הראשי של האפליקציה, המציג את רשימת כל פריטי הכספת השמורים. הנתונים נטענים מהמטמון בזיכרון (SessionHelper.CachedVault). ניתן לחפש פריטים לפי שם אתר או שם משתמש, ולהוסיף פריט חדש. לחיצה על פריט קיים פותחת את PasswordOptionsBottomSheet לביצוע פעולות. ניווט בין הכספת, מסך האזהרות ומסך הפרופיל מתבצע דרך ה-BottomNavFragment התחתון.



מסך הוספת סיסמה

במסך זה המשתמש מוסיף חשבון חדש לכספת: שם אתר/אפליקציה, שם משתמש, סיסמה (עם אימות) והערות. הסיסמה מוצפנת ב-AES-256-GCM בצד הלקוח לפני שליחה; נגזר ממנה SHA-1 Hash לבדיקת HIBP. המסך כולל מד חוזק סיסמה בזמן אמת, כפתור ליצירת סיסמה חזקה, ובדיקת כפילויות מול הכספת הקיימת לפני שמירה.

כפתור חזרה

כותרת עמוד

שדה אפליקציה/אתר

שדה שם באפליקציה

שדה סיסמה

כפתור הגרלת סיסמה חזקה

שדה אישור סיסמה

שדה הערות

כפתור שמירה

Add Password
Save a new credential to your vault

App or site name

Username or email

Password

Generate password

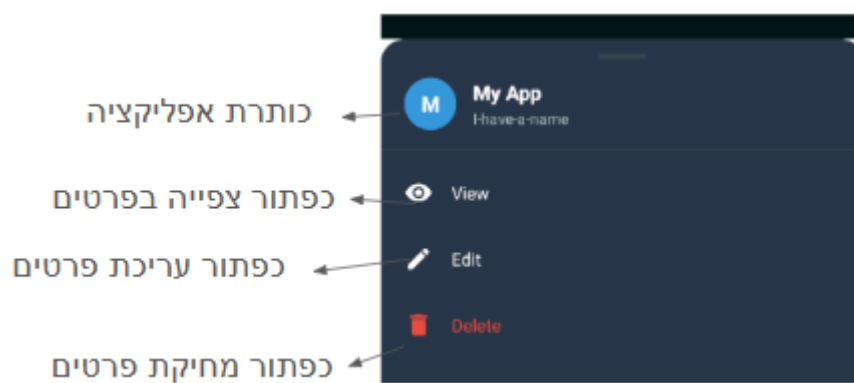
Confirm Password

Notes (optional)

Save

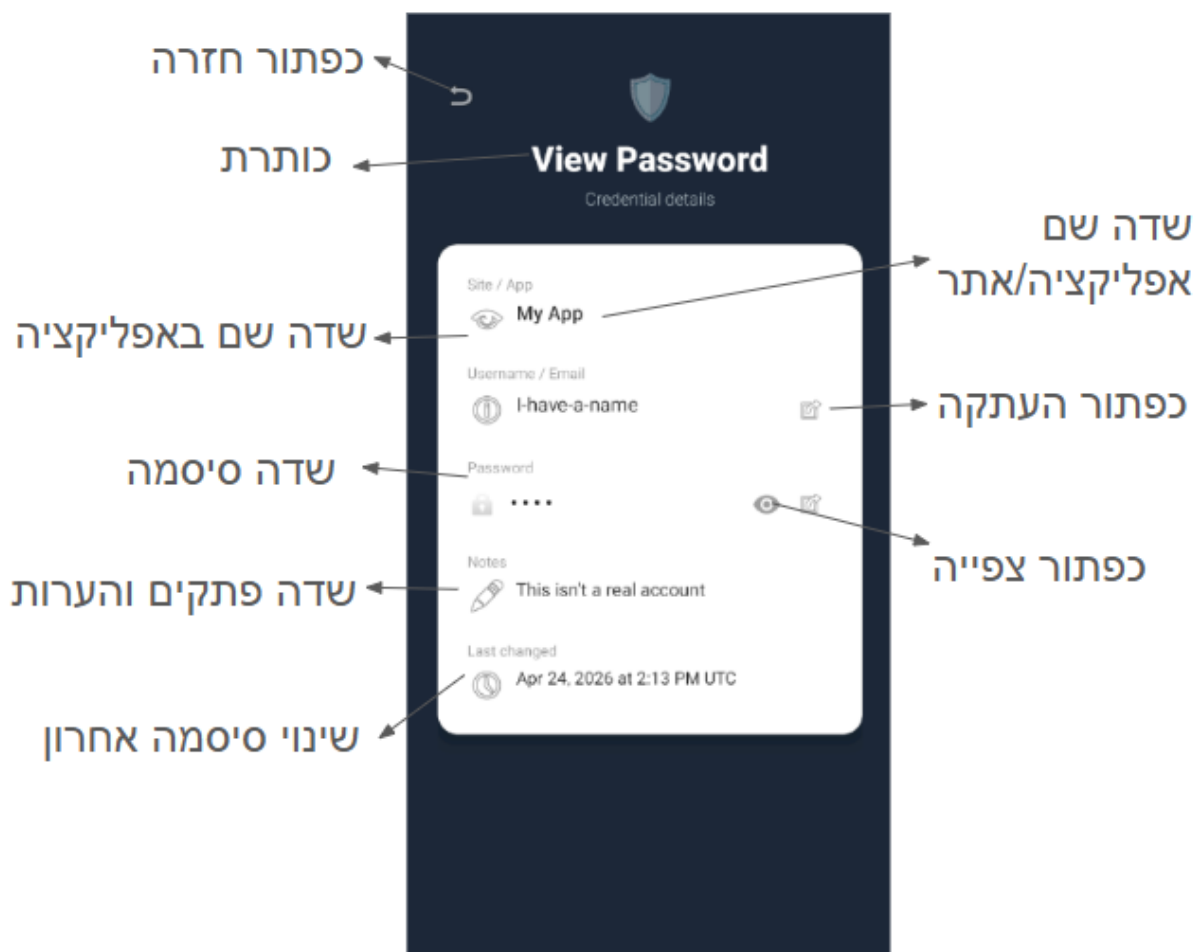
תפריט פעולה על סיסמה

בעת לחיצה על פריט מהרשימה, נפתח BottomSheetDialogFragment המציג כותרת עם שם האתר, שם המשתמש ושלוש **פעולות**: צפייה בפרטי החשבון, עריכת המידע הקיים, ומחיקת החשבון מהכספת (עם דיאלוג אישור).



מסך צפייה בסיסמה

מסך לקריאה בלבד של פרטי הכספת. מציג שם האתר/אפליקציה, שם המשתמש, הסיסמה (מוסתרת כברירת מחדל עם אפשרות חשיפה), הערות, ותאריך עדכון אחרון. הסיסמה מופענחת AES-GCM-256 בצד הלקוח לצורך הצגה בלבד. כפתורי העתקה זמינים לשם המשתמש ולסיסמה.



מסך עריכת סיסמה

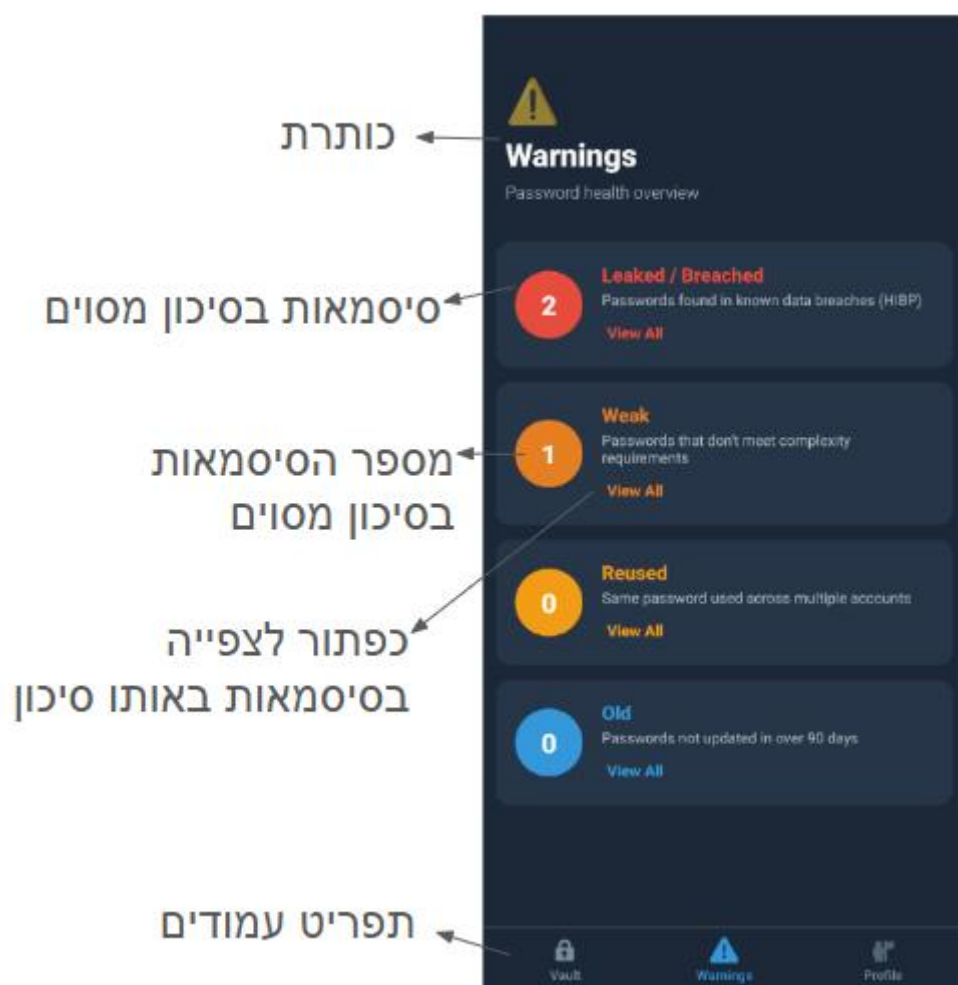
מסך לעריכת פריט קיים בכספת. מציג את הערכים הנוכחיים לעריכה (שם אתר, שם משתמש, הערות). אם הסיסמה שונתה — מוצפנת מחדש ב-AES-256-GCM ונגזר SHA-1 Hash חדש לבדיקת HIBP ; אם לא שונתה — ה-CipherText המקורי נשמר כפי שהוא. כולל מד חוזק סיסמה ובדיקת כפילויות כמו במסך ההוספה.

The screenshot shows the 'Edit Password' interface with the following fields and annotations:

- כפתור חזרה** (Back button): Points to the back arrow icon at the top left.
- כותרת** (Title): Points to the 'Edit Password' header.
- שדה אפליקציה/אתר** (App/site name field): Points to the 'App or site name' input field.
- שדה שם באפליקציה** (Username/email field): Points to the 'Username or email' input field.
- שדה סיסמה** (New Password field): Points to the 'New Password' input field.
- כפתור הגרלת סיסמה חזקה** (Generate strong password button): Points to the 'Generate password' button.
- שדה אישור סיסמה** (Confirm New Password field): Points to the 'Confirm New Password' input field.
- שדה פתקים והערות** (Notes field): Points to the 'Notes (optional)' text area.
- כפתור שמירת שינויים** (Update button): Points to the 'Update' button at the bottom.

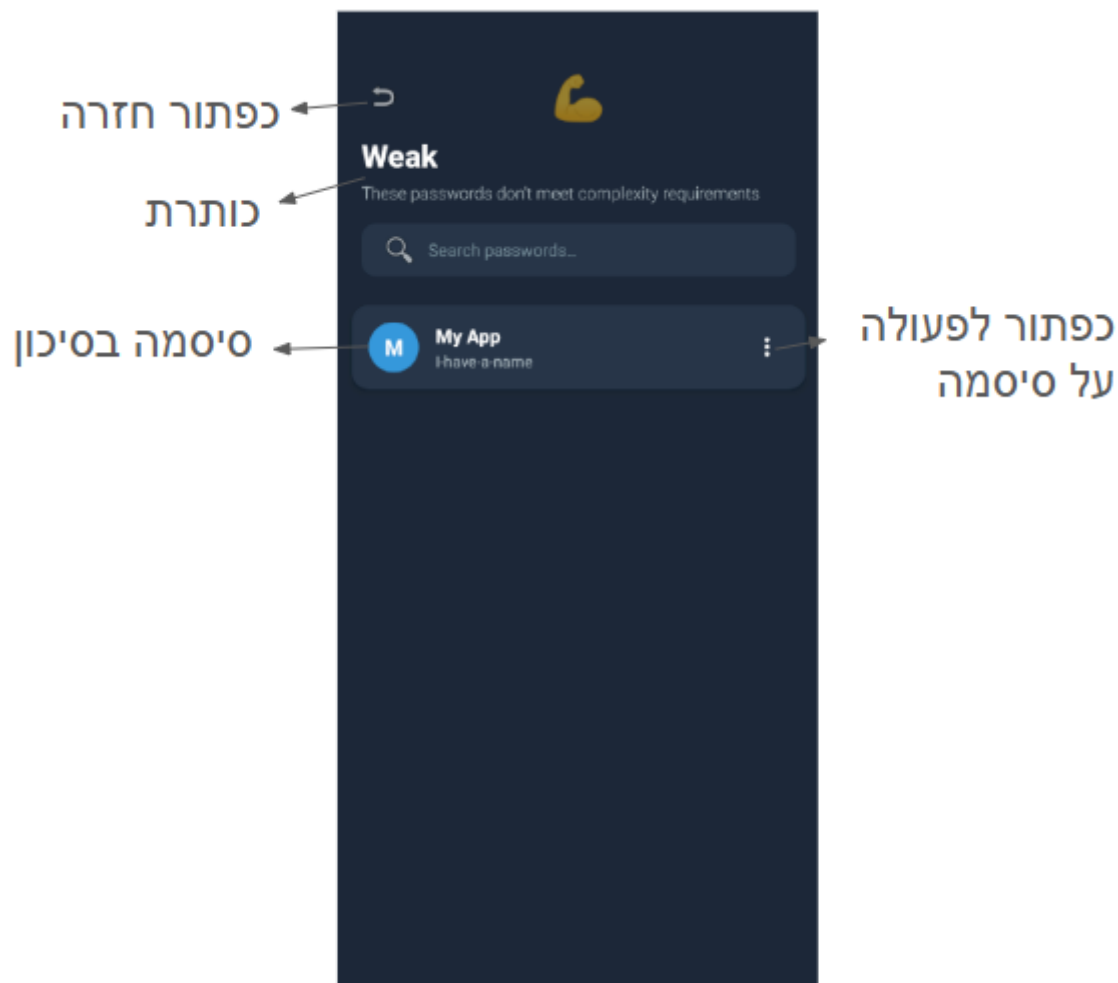
מסך אזהרות אבטחה

מסך זה מציג ארבעה כרטיסי סיכום: סיסמאות דלופות, חלשות, ממוחזרות (אותו SHA-1 Hash בכמה פריטים), וישנות (לא עודכנו מעל 90 יום). הנתונים נקראים מ-SessionHelper.CachedWarnings ומתעדכנים בכל שינוי בכספת. לחיצה על "הצג הכל" בכל קטגוריה מעברת ל-RiskDetailActivity עם הקטגוריה המתאימה.



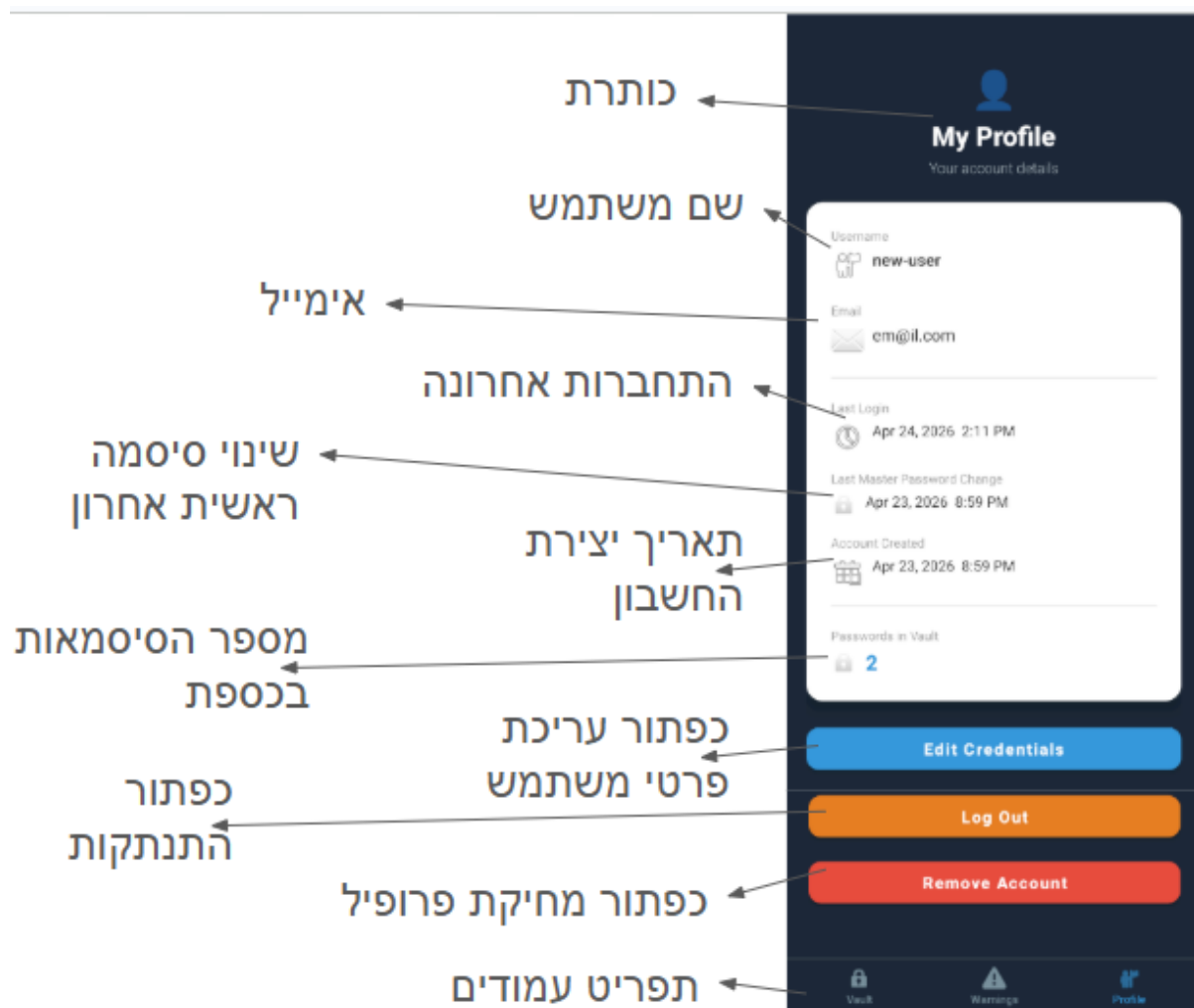
מסך סיסמאות לתיקון

מסך המציג את רשימת פריטי הכספת המשויכים לקטגוריית סיכון ספציפית (דלוף/חלש/ממוחזר/ישן). מקבל את שם הקטגוריה כפרמטר ומסנן את SessionHelper.CachedVault בהתאם. תומך בחיפוש טקסט. לחיצה על פריט פותחת את PasswordOptionsBottomSheet לצפייה, עריכה או מחיקה.



מסך פרופיל

מסך המציג את פרטי הפרופיל המשתמש: שם משתמש, כתובת מייל, מספר הסיסמאות בכספת, תאריך התחברות אחרון, תאריך שינוי סיסמת על אחרון ותאריך יצירת חשבון. הנתונים נטענים מהשרת בכל פתיחת המסך. כפתורים זמינים: עריכת פרטים, התנתקות (מנקה Session ו-SecureStorage), ומחיקת חשבון לצמיתות.



מסך עדכון פרטי משתמש

מסך לעריכת פרטי החשבון: שם משתמש, כתובת מייל, ואופציונלית סיסמת על חדשה. בשינוי סיסמת על — נגזרים מלחים חדשים ומפתח PBKDF2 חדש, כל פריטי הכספת מוצפנים מחדש במפתח החדש ונשלחים לשרת, ונבדק שהסיסמה החדשה אינה זהה לנוכחית או לאחת מ-4 הסיסמאות הקודמות לה (MasterPasswordHistory). הגדרות חדשות נשמרות ב-SecureStorage; הפרופיל מתרענן אוטומטית.

The screenshot shows the 'Edit Account' screen with the following fields and buttons, annotated with Hebrew text:

- כפתור חזרה** (Back button) - points to the top left arrow.
- כותרת** (Title) - points to the 'Edit Account' header.
- שדה שם משתמש** (Username field) - points to the 'Username' input field.
- שדה אימייל** (Email field) - points to the 'Email' input field.
- שדה סיסמה נוכחית** (Current Master Password field) - points to the 'Current Master Password' input field.
- שדה סיסמה חדשה** (New Master Password field) - points to the 'New Master Password' input field.
- כפתור הגרלת סיסמה חזקה** (Generate strong password button) - points to the 'Generate strong password' button.
- שדה אישור סיסמה** (Confirm New Master Password field) - points to the 'Confirm New Master Password' input field.
- כפתור שמור שינויים** (Save Changes button) - points to the 'Save Changes' button.

מימוש הפרויקט

פירוט מסכים

מסך כניסה (MainActivity)

פעילות הכניסה לאפליקציה. בעת ההפעלה, מאמתת את ה-JWT השמור מול השרת ומנתבת את המשתמש לכספת הסיסמאות (VaultActivity) או לדף ההתחברות (LoginActivity) בהתאמה. לאחר אימות מוצלח, מפעילה את PasswordMonitorService כשירות קדמי (Foreground Service) לניטור יומי של הסיסמאות. מבקשת הרשאת POST_NOTIFICATIONS עבור שליחת התראות במערכות Android 13 ומעלה.

Layout: activity_main.xml

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
RequestCodeNotificationPermission (const int)	Request code (1001) for the POST_NOTIFICATIONS runtime permission dialog on Android 13+.
_pendingVaultNavigation (bool)	Set to true when JWT is valid and vault is loaded; navigation to VaultActivity is deferred until after the permission dialog is resolved.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
OnCreate(Bundle)	Validates the stored JWT, restores the vault key and session cache, computes warnings, requests notification permission, and routes to Vault or Login.
OnRequestPermissionsResult(int, string[], Permission[])	Handles the POST_NOTIFICATIONS permission dialog result and calls NavigateToDest().
NavigateToDest()	Starts VaultActivity (valid session) or LoginActivity (no session) and finishes the current activity.
StartPasswordMonitor(Context) [static]	Starts PasswordMonitorService as an Android foreground service.

התחברות (LoginActivity)

המסך הראשון אליו מגיע משתמש שאינו מחובר, בו מתבצע תהליך ההזדהות (Authentication) של המשתמש. בנוסף, משתמשים חדשים מגיעים דרכו לדף ההרשמה.

Layout: activity_login.xml

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
OnCreate(Bundle)	Inflates layout, initialises views, and sets up event handlers.
InitializeViews()	Finds and assigns all view references.
SetupEventHandlers()	Wires click and text-change listeners.
OnLoginClicked()	Validates input, fetches salts, derives PBKDF2 key, calls VerifyLogin, saves JWT + vault key to SecureStorage, navigates to vault.
ValidateInputs()	Ensures email and password fields are filled and correctly formatted.
SetLoadingState(bool)	Shows/hides the progress bar and enables/disables the login button.
ShowError(Textview, string)	Displays an inline validation error beneath a field.
HideError(Textview)	Hides an inline validation error.
ShowGeneralError(string)	Displays a full-form error message.
ClearErrors()	Hides all inline and general error views.

הרשמה (SignupActivity)

פעילות הרשמה. מאפשרת למשתמש חדש ליצור חשבון על ידי הזנת שם משתמש, מייל וסיסמה על. בעת הרשמה, נוצרים שני מלחים ייחודיים (AuthSalt, EncryptionSalt), נגזר מפתח PBKDF2 עם 600,000 איטרציות, ומתבצעת בדיקת HIBP לוודא שהסיסמה לא מוכרת בדליפות. כולל מד חוזק סיסמה בזמן אמת וכפתור ליצירת סיסמה חזקה אוטומטית.

Layout: activity_signup.xml

פעולות:

Method Name	Use and Explanation
OnCreate(Bundle)	Inflates layout, initialises views, and sets up event handlers.
InitializeViews()	Finds and assigns all view references.
SetupEventHandlers()	Wires click and text-change listeners; attaches password strength indicator.
OnSignUpClicked()	Validates input, generates AuthSalt and EncryptionSalt, derives PBKDF2 key, calls RegisterUser, saves session, navigates to vault.
ValidateInputs()	Validates username, email, password strength, and password match.
SetLoadingState(bool)	Shows/hides progress bar and enables/disables the signup button.
ShowGeneralError(string)	Displays a full-form error message.
ClearErrors()	Hides all inline and general error views.

כספת הסיסמאות (VaultActivity)

מסך הבית, בו מוצגת רשימת הסיסמאות. משתמש מחובר מגיע לדף זה ישירות, ללא התחברות או הרשמה. במסך ניתן ליצור סיסמה חדשה, לחפש את הסיסמאות ולעבור לעמודי ביצוע פעולות על סיסמאות (צפייה, עריכה, מחיקה).

Layout: activity_vault.xml

תכונות:

Property Name	Use and Explanation
requestCodeNotificationPermission (const int)	Request code (9001) for the POST_NOTIFICATIONS permission on Android 13+.
postNotificationsPermission (const string)	Android manifest permission string "android.permission.POST_NOTIFICATIONS".
allEntries (List<VaultItem>)	Full, unfiltered list of vault items loaded from the session cache.

פעולות:

Method Name	Use and Explanation
onCreate(Bundle)	Inflates layout, sets up RecyclerView, bottom nav, event handlers, and loads vault from session.
initializeViews()	Finds and assigns all view references.
setUpRecyclerView()	Configures RecyclerView with PasswordBannerAdapter and item-click handlers.
setUpBottomNavFragment(Bundle)	Attaches BottomNavFragment on first creation.
setUpEventHandlers()	Wires FAB and search text-change listeners.
onBannerActionAt(int)	Opens PasswordOptionsBottomSheet via PasswordEntryActionsHelper for the entry at the given position.
onActivityResult(int, Result, Intent)	Handles results from AddPasswordActivity and EditPasswordActivity to sync and refresh the list.
loadVaultFromSession()	Copies SessionHelper.CachedVault into allEntries.
filterPasswords(string)	Filters allEntries by query and refreshes the adapter.
getDisplayedEntries()	Returns the currently displayed (possibly filtered) list.
refreshList()	Re-applies the current search filter.
updateEmptyState()	Shows/hides the empty-state layout based on adapter item count.
syncEntryCache()	Pushes CachedVault into VaultEntryCache so EditPasswordActivity can perform duplicate checking.
requestNotificationPermissionIfNeeded()	Requests POST_NOTIFICATIONS on Android 13+ if not already granted.

הוספת סיסמה (AddPasswordActivity)

פעילות להוספת פריט חדש לכספת. המשתמש מזין שם אתר/אפליקציה, שם משתמש, סיסמה והערות. הסיסמה מוצפנת ב-AES-256-GCM בצד הלקוח לפני שליחה לשרת, ונשלח ה-SHA-1 Hash שלה לבדיקת HIBP. מזהה גם סיסמאות כפולות (Duplicate Detection) הזהות לפריטים קיימים בכספת.

Layout: activity_entry.xml

תכונות:

Property Name	Use and Explanation
ResultEntryId / ResultSiteName / ResultUsername / ResultNotes / ResultIV / ResultTag / ResultCipherText / ResultSha1Hash / ResultIsLeaked / ResultLastUpdate (const string)	Keys for the Intent extras returned to VaultActivity on successful save.
RequestCodeAdd (const int)	Request code (1001) used when VaultActivity starts this activity for result.
existingEntries (List<VaultItem>)	Copy of the current vault used for duplicate-password detection before saving.

פעולות:

Method Name	Use and Explanation
OnCreate(Bundle)	Inflates layout, initialises views, loads existing entries, sets up event handlers.
InitializeViews()	Finds and assigns all view references.
ConfigureForAddMode()	Sets title and subtitle labels for add-password mode.
PopulateExistingEntries()	Copies SessionHelper.CachedVault into existingEntries for duplicate checking.
SetupEventHandlers()	Wires Save, Generate, and password text-change listeners.
OnSaveClicked()	Validates input, encrypts the password AES-256-GCM, computes SHA-1 hash, sends to server, returns result to caller.
ValidateInputs()	Validates site name, password strength, password match, and duplicate check.
IsDuplicate(string)	Returns true if the SHA-1 hash of the new password matches any existing vault entry.
UpdatePasswordStrengthIndicator(string)	Updates the strength ProgressBar and hint label via FormUiHelper.
ShowError(Textview, string)	Displays an inline error below a specific field.
HideError(Textview)	Hides an inline error.
ClearErrors()	Hides all error views.
SetLoadingState(bool)	Shows/hides progress bar and enables/disables Save.

צפייה בסיסמה (ViewPasswordActivity)

פעילות לתצוגה לקריאה בלבד של פרטי פריט כספת. מפענחת את הסיסמה בצד הלקוח לצורך הצגתה. מציגה שם האתר/אפליקציה, שם המשתמש, הסיסמה (עם אפשרות הצגה/הסתרה), הערות, תאריך עדכון אחרון, ואינדיקטור אם הסיסמה זוהתה כדלופה. מאפשרת העתקת שם משתמש וסיסמה ללוח.

Layout: activity_view_password.xml

תכונות:

Property Name	Use and Explanation
ExtraSiteName / ExtraUsername / ExtraNotes / ExtraIV / ExtraTag / ExtraCipherText / ExtraLastUpdate (const string)	Intent-extra keys used to pass the vault item data into this activity.
decryptedPassword (string)	The plaintext password decrypted AES-256-GCM on entry; held in memory for the lifetime of the activity only.
passwordVisible (bool)	Tracks whether the password is currently shown in plain text or masked.

פעולות:

Method Name	Use and Explanation
OnCreate(Bundle)	Inflates layout, initialises views, populates fields with decrypted data.
InitializeViews()	Finds and assigns all view references.
PopulateFields()	Reads Intent extras, decrypts the password AES-256-GCM, and fills all text views.
SetupEventHandlers()	Wires the back button, copy-username, toggle-password, and copy-password handlers.
UpdatePasswordDisplay()	Switches between masked and plain-text display of the password.
CopyToClipboard(string, string)	Places the given text onto the Android clipboard with the specified label.

עריכת סיסמה (EditPasswordActivity)

פעילות לעריכת פריט קיים בכספת. מציגה את הערכים הנוכחיים לעריכה. אם הסיסמה שונתה, מצפינה אותה מחדש ב-AES-256-GCM, מעדכנת את ה-SHA-1 Hash ומבצעת בדיקת HIBP. כולל בדיקת כפילויות (לא כולל הפריט הנוכחי עצמו).

Layout: activity_entry.xml

תכונות:

Property Name	Use and Explanation
ExtraEntryId / ExtraSiteName / ExtraUsername / ExtraNotes / ExtraIV / ExtraTag / ExtraCipherText / ExtraSha1Hash / ExtraIsLeaked (const string)	Intent-extra keys carrying the existing entry data into the activity.
ResultEntryId / ResultSiteName / ResultUsername / ResultNotes / ResultIV / ResultTag / ResultCipherText / ResultSha1Hash / ResultIsLeaked / ResultLastUpdate (const string)	Keys for the Intent extras returned to the caller on successful save.
RequestCodeEdit (const int)	Request code (1002) used when the parent activity starts this activity for result.
existingEntries (List<VaultItem>)	Copy of the current vault used for duplicate-password detection.

פעולות:

Method Name	Use and Explanation
OnCreate(Bundle)	Inflates layout, populates fields with existing entry data, loads existing entries, sets up event handlers.
InitializeViews()	Finds and assigns all view references.
PopulateExistingValues()	Fills form fields with entry values from Intent extras.
PopulateExistingEntries()	Copies SessionHelper.CachedVault into existingEntries.
SetupEventHandlers()	Wires Save, Generate, and password text-change listeners.
OnSaveClicked()	If password changed: re-encrypts and recomputes SHA-1; sends update to server; returns result.
ValidateInputs()	Validates site name, password strength, password match, and duplicate check.
IsDuplicate(string)	Returns true if the new SHA-1 matches an existing entry (excluding the entry being edited).
UpdatePasswordStrengthIndicator(string)	Updates the strength bar and hint label.
ShowError(Textview, string)	Displays an inline error.
HideError(Textview)	Hides an inline error.
ClearErrors()	Hides all error views.
SetLoadingState(bool)	Shows/hides progress bar and enables/disables Save.

מסך האזהרות (WarningsActivity)

במסך זה המשתמש יכול לראות כמה סיסמאות שלו התגלו בהדלפה, חלשות, ממוחזרות ו/או ישנות ודורשות שינוי. מכל כפתור הוא יכול לעבור לרשימה של האפליקציות שבאותו סיכון.

Layout: activity_warnings.xml

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
OnCreate(Bundle)	Inflates layout, initializes views, attaches bottom nav, wires "View All" buttons, and displays warnings.
OnResume()	Re-displays warnings on every resume so counts stay current after vault changes.
InitializeViews()	Finds and assigns all view references.
SetupBottomNavFragment(Bundle)	Attaches the BottomNavFragment on first creation.
SetupViewAllButtons()	Wires the four "View All" buttons to launch RiskDetailActivity with the correct category.
DisplayWarnings()	Reads counts from SessionHelper.CachedWarnings (recomputing if null) and sets the four counter TextViews.

סיסמאות בסיכון מסוים (RiskDetailActivity)

פעילות המציגה את רשימת הפריטים המשויכים לקטגוריית סיכון ספציפית (דלוף/חלש/ממוחזר/ישן). מקבלת את הקטגוריה כפרמטר ומסננת את פריטי הכספת המאוחסנים בזיכרון. כוללת שורת חיפוש ומשתמשת ב- PasswordBannerAdapter ו- PasswordOptionsBottomSheet לביצוע פעולות על הפריטים.

Layout: *activity_risk_detail.xml*

תכונות:

Property Name	Use and Explanation
ExtraRiskCategory (const string)	Intent-extra key for the risk category string passed by WarningsActivity.
CategoryLeaked / CategoryWeak / CategoryReused / CategoryOld (const string)	String constants for the four risk categories: "leaked", "weak", "reused", "old".
adapter (PasswordBannerAdapter)	RecyclerView adapter that renders the risk-filtered entry list.
riskEntries (List<VaultItem>)	The subset of vault items that belong to the current risk category.
category (string)	The active risk category received from the Intent.

פעולות:

Method Name	Use and Explanation
OnCreate(Bundle)	Reads category from Intent, initialises views, configures header, sets up RecyclerView and handlers, loads risk entries.
InitializeViews()	Finds and assigns all view references.
ConfigureHeader()	Sets emoji, title, and subtitle based on the active risk category.
SetupRecyclerView()	Configures RecyclerView with PasswordBannerAdapter.
OnBannerActionAt(int)	Shows PasswordOptionsBottomSheet via PasswordEntryActionsHelper for the entry at the given position.
OnActivityResult(int, Result, Intent)	Handles edit results from EditPasswordActivity; updates riskEntries and reloads the list.
LoadRiskEntriesAsync()	Calls WarningsHelper.GetItemsAtRiskAsync() to filter the vault for the active category.
FilterPasswords(string)	Filters riskEntries by the search query and updates the adapter.
GetFilteredEntries(string)	Returns entries matching the search query.
GetDisplayedEntries()	Returns all riskEntries or the filtered subset.
RefreshList()	Re-applies the current search filter.
UpdateEmptyState()	Shows/hides the empty-state layout.
SyncEntryCache()	Pushes CachedVault into VaultEntryCache for duplicate checking in EditPasswordActivity.

פרופיל (ProfileActivity)

פעילות הצגת פרופיל המשתמש. מציגה שם משתמש, מייל, מספר הסיסמאות בכספת, תאריך ההתחברות האחרונה ותאריך שינוי סיסמת העל האחרון. נתונים נטענים מהשרת עם fallback לנתונים מאוחסנים מקומית. כוללת כפתורים לעריכת פרטי חשבון, יציאה מהחשבון ומחיקת חשבון לצמיתות.

Layout: activity_profile.xml

תכונות:

Property Name	Use and Explanation
requestCodeEditAccount (const int)	Request code (2001) used when launching EditAccountActivity for result.

פעולות:

Method Name	Use and Explanation
onCreate(Bundle)	Inflates layout, initialises views, sets up bottom nav, wires event handlers, loads profile.
initializeViews()	Finds and assigns all view references.
setupBottomNavFragment(Bundle)	Attaches the BottomNavFragment on first creation.
setupEventHandlers()	Wires Edit, Logout, and Delete-account button handlers.
loadProfileAsync()	Fetches user profile from the server via ProfileService and populates the TextViews.
onActivityResult(int, Result, Intent)	When EditAccountActivity returns ResultUpdated=true, reloads the profile.
logoutAsync()	Calls ClearSessionAsync, ends the in-memory session, and navigates to LoginActivity.
deleteAccountAsync()	Shows confirmation dialog; on confirm, deletes account server-side and calls StorageHelper.ClearAll().

עריכת פרטי חשבון (EditAccountActivity)

פעילות עריכת פרטי חשבון. מאפשרת שינוי שם משתמש, מייל, וסיסמת על. בעת שינוי סיסמת על, נגזרים מלחים חדשים ומפתח PBKDF2 חדש, נבדקת הסיסמה החדשה מול 4 הסיסמאות האחרונות להימנעות מחזרה, וכל פריטי הכספת מוצפנים מחדש במפתח החדש. מוחזרת לProfileActivity עם ResultUpdated=true לטעינת הנתונים המעודכנים.

Layout: activity_edit_account.xml

תכונות:

Property Name	Use and Explanation
RequestCodeEditAccount (const int)	Request code (2001) identifying this activity to its caller (ProfileActivity).
ResultUpdated (const string)	Intent-extra key returned on success, signalling ProfileActivity to reload the profile.
originalUsername (string)	The current username loaded at startup, used to detect changes.
originalEmail (string)	The current email loaded at startup, used to detect changes.

פעולות:

Method Name	Use and Explanation
OnCreate(Bundle)	Inflates layout, initializes views, wires event handlers, loads current profile.
InitializeViews()	Finds and assigns all view references.
SetupEventHandlers()	Wires Save, Generate, and password text-change listeners.
LoadCurrentProfileAsync()	Fetches profile from server and pre-fills the form fields.
OnSaveClicked()	Validates changes; if password changed: checks history, re-derives keys, re-encrypts vault, sends UpdateUser; returns result.
ValidateInputs()	Validates username, email, current password, and optionally new password fields.
UpdatePasswordStrengthIndicator(string)	Updates strength bar and hint for the new-password field.
ShowError(Textview, string)	Displays an inline error.
HideError(Textview)	Hides an inline error.
ClearErrors()	Hides all error views.
SetLoadingState(bool)	Shows/hides progress bar and enables/disables Save.

פירוט עצמים

משתמש (User)

המחלקה מייצגת את חשבון המשתמש ב-Securio בבסיס הנתונים.

תכונות:

<u>Property</u>	<u>Description and Use</u>
Id (int)	Primary key; auto-assigned by the database.
Username (string)	Display name shown in the profile and warnings screens.
Email (string)	Unique email address used as the login credential.
MasterPasswordKey (string)	PBKDF2-derived key (600,000 iterations, SHA-256) stored as Base64; never the raw password.
AuthSalt (string)	Random 32-byte salt (Base64) used to derive MasterPasswordKey; sent to the client on login.
EncryptionSalt (string)	Random 32-byte salt (Base64) used to derive the AES-256 vault encryption key on the client.
LastLogin (DateTime)	UTC timestamp of the most recent successful login.
LastPasswordUpdate (DateTime)	UTC timestamp of the most recent master-password change; used by PasswordCheckManager.
CreatedAt (DateTime)	UTC timestamp of account creation.
PasswordCount (int)	Computed count of vault items; populated by GetUserProfileAsync (not stored directly in the DB).
PasswordSha1Hash (string)	SHA-1 of the plaintext master password for HIBP check at registration; never persisted to the DB.

פריט כספת (VaultItem)

המחלקה מייצגת פריט מוצפן בכספת הסיסמאות של המשתמש. מכילה את נתוני החשבון, נתוני הצפנת הסיסמה, ה-SHA-1 Hash לבדיקת HIBP, דגל IsLeaked לאחסון תוצאת הבדיקה, ו-LastUpdate למעקב גיל הסיסמה.

Property	Description and Use
Id (int)	Primary key; auto-assigned by the database.
UserId (int)	Foreign key linking the entry to its owner in the Users table.
AccountName (string)	Name of the service or application (e.g. "Google", "Amazon").
AccountUsername (string)	Login name or email used for the specific service.
IV (string)	AES-GCM 96-bit initialisation vector, Base64-encoded.
Tag (string)	AES-GCM 128-bit authentication tag (Base64); verifies integrity on decryption.
CipherText (string)	AES-GCM encrypted password, Base64-encoded.
Notes (string)	Optional free-text notes associated with the entry.
Sha1Hash (string)	Unsalted SHA-1 hash of the plaintext password; used for HIBP breach lookup and reuse detection.
IsLeaked (bool)	Set by the backend via HIBP at add/update time; true if the password appears in a known breach.
LastUpdate (DateTime)	UTC timestamp of the most recent password change; used for the "old password" (>90 days) warning.
PasswordChanged (bool)	Transient client-side flag indicating the ciphertext was re-encrypted; never persisted to the DB.

סיסמת על ישנה (MasterPasswordHistory)

המחלקה מייצגת סיסמת על ישנה בבסיס הנתונים כדי למנוע שימוש חוזר באותה סיסמת על

תכונות:

<u>Property</u>	<u>Description and Use</u>
Id (int)	Primary key; auto-assigned by the database.
UserId (int)	Foreign key referencing the Users table (CASCADE DELETE).
PasswordKey (string)	The PBKDF2-derived MasterPasswordKey that was active at the time of archival.
AuthSalt (string)	The AuthSalt used to derive PasswordKey; returned to the client for no-reuse comparison.
CreatedAt (DateTime)	UTC timestamp of when this password was originally set.

שירות ניטור רקע (PasswordMonitorService)

שירות Android קדמי (Foreground Service) עם START_STICKY הפועל ברצף. בעת הפעלה, יוצר ערוץ התראות, מציג התראת שירות מתמשכת ומתזמן מחזור בדיקה אחת ל-24 שעות באמצעות `Handler.PostDelayed`. עם כל מחזור, שולח את מזהה המשתמש לנקודת הקצה `PasswordCheck` בשרת ומציג התראה אם נמצאו בעיות. מופעל גם על ידי `BootReceiver` לאחר הפעלה מחדש של המכשיר.

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
<code>IntervalMs (const long)</code>	Repeat interval in milliseconds: 24 h ($24 \times 60 \times 60 \times 1000 = 86,400,000$ ms).
<code>_handler (Android.OS.Handler)</code>	Android Handler bound to the main looper, used to schedule and re-schedule the periodic task.
<code>_runnable (Java.Lang.Runnable)</code>	The repeating task that calls <code>RunCycleAsync()</code> and re-schedules itself after <code>IntervalMs</code> .

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
<code>OnCreate()</code>	Creates the notification channel, starts the mandatory foreground notification, and schedules the first check.
<code>OnStartCommand(Intent, StartCommandFlags, int)</code>	Returns <code>START_STICKY</code> so Android automatically restarts the service if killed.
<code>OnBind(Intent)</code>	Returns null; this is not a bound service.
<code>OnDestroy()</code>	Removes pending Handler callbacks to stop rescheduling.
<code>RunCycleAsync()</code>	Executes one password-health check and re-schedules itself via <code>Handler.PostDelayed</code> .
<code>PerformCheckAsync(Context) [static]</code>	Reads the stored <code>UserId</code> from <code>SecureStorage</code> , calls the <code>PasswordCheck</code> Azure Function, and posts a notification if issues are found.

פירוט מחלקות Azure Functions

פונקציות אימות (AuthFunctions)

מחלקת Azure Functions לניהול endpoint-ים של אימות. כל פונקציה חושפת HTTP Trigger, מפרשת את גוף הבקשה, ומאצילה ל-AuthManager.

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
_authManager (AuthManager)	Holds authentication business logic; injected via DI.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
Register (POST "RegisterUser")	Parses a User object from the request body; calls RegisterAsync; returns 200 on success or 409 on duplicate email.
Login (POST "VerifyLogin")	Parses email + MasterPasswordKey; calls VerifyLoginAsync; returns 200 with JWT or 401 on invalid credentials.
GetSalts (POST "GetSalts")	Parses Email from the body; calls GetUserSaltsAsync; returns SaltData with 200 or 404 when the user is not found.
ValidateToken (GET "ValidateToken")	Reads the Authorization header, validates the JWT; returns 200 with UserId on success or 401 when invalid/expired.

פונקציות כספת (VaultItemFunctions)

מחלקת Azure Functions לניהול פריטי הכספת. כל פונקציה דורשת JWT ב-Authorization header. UserId נגזר מה-JWT למניעת זיוף בעלות.

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
_vaultItemManager (VaultItemManager)	Holds vault-item CRUD logic; injected via DI.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
AddVaultItem (POST "AddVaultItem")	Validates JWT, extracts UserId from token, calls AddVaultItemAsync; returns the saved item with its new Id.
UpdateVaultItem (POST "UpdateVaultItem")	Validates JWT, extracts UserId from token, calls UpdateVaultItemAsync; returns the updated item.
GetVaultItems (GET "GetVaultItems")	Validates JWT, extracts UserId from token; returns the list of encrypted vault items for that user.
DeleteVaultItem (POST "DeleteVaultItem")	Validates JWT, parses {Id} from the body, calls DeleteVaultItemAsync with ownership check.

פונקציות משתמש (UserFunctions)

מחלקת Azure Functions לניהול פרופיל וחשבון. כל פונקציה דורשת JWT.

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
_userManager (UserManager)	Holds user-profile management logic; injected via DI.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
GetProfile (GET "GetProfile")	Validates JWT, calls GetProfileAsync; returns full profile data including password count.
UpdateUser (POST "UpdateUser")	Validates JWT, parses UpdateAccountRequest, calls UpdateUserAsync with optional re-encrypted vault items.
DeleteUser (POST "DeleteUser")	Validates JWT, calls DeleteUserAsync; database CASCADE removes vault items and password history.
GetPasswordHistory (GET "GetPasswordHistory")	Validates JWT, calls GetPasswordHistoryAsync; returns the last 4 old master-password entries for reuse checking.

פונקציות בדיקת סיסמאות (PasswordCheckFunctions)

מחלקת Azure Functions לחשיפת endpoint בדיקת הסיסמאות. אינו דורש UserId — JWT מגיע בגוף הבקשה לשימוש ה-PasswordMonitorService ברקע.

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
_manager (PasswordCheckManager)	Holds password-health summary logic; injected via DI.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
PasswordCheck (POST "PasswordCheck")	Parses {UserId} from the body; calls GetPasswordCheckAsync; returns PasswordCheckResult with BreachedCount, OldCount, and MasterPasswordOld. No JWT required.

לוגיקת השרת (Business Logic)

מנהל האימות (AuthManager)

מחלקה המנהלת את לוגיקת ההרשמה והכניסה. RegisterAsync בודקת HIBP ואת ייחודיות המייל, יוצרת משתמש ב-DB ומחזירה JWT. VerifyLoginAsync מאמתת את מפתח PBKDF2 שנגזר בלקוח ומחזירה JWT. GetUserSaltsAsync מחזירה AuthSalt ו-EncryptionSalt כדי שהלקוח יוכל לגזור את המפתח לפני שליחת הסיסמה.

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
_repo (IUserRepository)	Data access for user operations; injected via DI.
_hibp (IHibpService)	HIBP service for checking whether the password appears in a known breach; injected via DI.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
AuthManager(IUserRepository, IHibpService)	Constructor.
RegisterAsync(User user)	Checks HIBP for the new password, verifies email uniqueness, inserts the user, and returns ServerResponse<AuthData> with a JWT.
VerifyLoginAsync(string email, string key)	Validates the PBKDF2 MasterPasswordKey against the stored value, updates LastLogin, and returns ServerResponse<AuthData> with a JWT.
GetUserSaltsAsync(string email)	Returns ServerResponse<SaltData> containing AuthSalt and EncryptionSalt for the given email address.

מנהל פריטי הכספת (VaultItemManager)

מחלקה המנהלת לוגיקת CRUD של פריטי הכספת המוצפנים. בעת הוספה ועדכון (כאשר הסיסמה שונתה), בודקת HIBP ומעדכנת IsLeaked לפני שמירה.

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
_repo (IVaultItemRepository)	Data access for vault-item operations; injected via DI.
_hibp (IHibpService)	HIBP service for updating IsLeaked on add/update; injected via DI.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
VaultItemManager(IVaultItemRepository, IHibpService)	Constructor.
AddVaultItemAsync(VaultItem item)	Validates required fields, checks HIBP, persists the item, and returns ServerResponse<VaultItem> with the generated Id.
UpdateVaultItemAsync(VaultItem item)	Validates fields, re-checks HIBP when PasswordChanged=true, updates the record, and returns ServerResponse<VaultItem>.
GetVaultItemsAsync(int userId)	Returns ServerResponse<List<VaultItem>> with all vault items belonging to the specified user.
DeleteVaultItemAsync(int itemId, int userId)	Verifies ownership and deletes the item; returns ServerResponse<object>.

מנהל המשתמשים (UserManager)

מחלקה המנהלת פרופיל, עדכון ומחיקת חשבון. UpdateUserAsync מטפל בייחודיות מייל, בדיקת HIBP לסיסמה חדשה, שמירת היסטוריית סיסמות (MasterPasswordHistory) ו-bulk re-encryption של הכספת.

תכונות:

Property Name	Use and Explanation
_repo (IUserRepository)	Data access for user operations; injected via DI.
_vaultRepo (IVaultItemRepository)	Data access for vault items, used for bulk re-encryption; injected via DI.
_hibp (IHibpService)	HIBP service for checking the new master password on change; injected via DI.

פעולות:

Method Name	Use and Explanation
UserManager(IUserRepository, IVaultItemRepository, IHibpService)	Constructor.
GetProfileAsync(int userId)	Returns ServerResponse<User> with full profile data including PasswordCount.
UpdateUserAsync(int userId, User updated, bool passwordChanged, List<VaultItem>)	Updates username and email; when passwordChanged=true, checks HIBP, saves a history entry, updates keys, and bulk-updates vault ciphertexts.
DeleteUserAsync(int userId)	Deletes the account; database CASCADE removes vault items and password history.
GetPasswordHistoryAsync(int userId)	Returns the last 4 old master-password entries to support client-side reuse checking.

מנהל בדיקת הסיסמאות (PasswordCheckManager)

מחלקה המחשבת סיכום בריאות סיסמאות ללא צורך ב-Session פעיל. משמש את endpoint PasswordCheck שנקרא מ-PasswordMonitorService ברקע.

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
OldPasswordDays (const int = 90)	Age threshold in days; passwords older than this value are flagged as stale.
_userRepo (IUserRepository)	Data access for user data; injected via DI.
_vaultRepo (IVaultItemRepository)	Data access for vault items; injected via DI.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
PasswordCheckManager(IUserRepository, IVaultItemRepository)	Constructor.
GetPasswordCheckAsync(int userId)	Returns PasswordCheckResult? with BreachedCount (items where IsLeaked=true), OldCount (items with LastUpdate > 90 days ago), and MasterPasswordOld flag.

מאגרי הנתונים (Repositories)

ממשק מאגר המשתמשים (IUserRepository)

ממשק (interface) המגדיר את חוזה הגישה ל-DB לפעולות משתמש. מאפשר הזרקת תלויות (DI).

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
EmailExistsAsync(string email)	Returns true when the email address is already registered.
EmailExistsForOtherUserAsync(string email, int excludeUserId)	Returns true when the email belongs to a different user (used during update).
CreateUserAsync(User user)	Inserts a new user row and returns the auto-generated Id.
GetUserByEmailAsync(string email)	Returns the User entity for the given email, including auth fields.
GetUserByIdAsync(int userId)	Returns the full User entity for the given Id, including auth fields.
GetUserProfileAsync(int userId)	Returns the User entity enriched with a PasswordCount sub-query.
UpdateLastLoginAsync(int userId)	Sets LastLogin to the current UTC timestamp.
UpdateUserAsync(User, bool passwordChanged)	Updates account fields; when passwordChanged=true, also updates keys and LastPasswordUpdate.
DeleteUserAsync(int userId)	Deletes the user row; CASCADE removes vault items and password history.
GetLastPasswordHistoryAsync(int userId, int count)	Returns the most recent count entries from MasterPasswordHistory, ordered by CreatedAt descending.
AddPasswordHistoryAsync(int userId, string key, string salt, DateTime createdAt)	Inserts a new row into MasterPasswordHistory to archive the retired password key.

מאגרי המשתמשים (UserRepository)

מימוש IUserRepository. מנהל כל פניות SQL לטבלאות Users ו-MasterPasswordHistory ב-Azure SQL תוך שימוש ב-ADO.NET (SqlConnection, SqlCommand). כל פעולה פותחת חיבור עצמאי להבטחת Thread Safety.

תכונות:

Property Name	Use and Explanation
_connectionString (string)	Azure SQL connection string; injected via the constructor.

פעולות:

Method Name	Use and Explanation
UserRepository(string connectionString)	Constructor.
EmailExistsAsync(string email)	Executes SELECT COUNT(*) FROM Users WHERE Email = @email.
EmailExistsForOtherUserAsync(string email, int excludeUserId)	Executes SELECT COUNT(*) WHERE Email = @email AND Id <> @excludeUserId.
CreateUserAsync(User user)	Executes INSERT INTO Users ... OUTPUT INSERTED.Id; initialises LastLogin and LastPasswordUpdate.
GetUserByEmailAsync(string email)	SELECT with auth fields (MasterPasswordKey, AuthSalt, EncryptionSalt, etc.) filtered by Email.
GetUserByIdAsync(int userId)	SELECT with auth fields filtered by Id.
GetUserProfileAsync(int userId)	SELECT with a PasswordCount sub-query against VaultItems filtered by Id.
UpdateLastLoginAsync(int userId)	Executes UPDATE Users SET LastLogin = GETDATE() WHERE Id = @id.
UpdateUserAsync(User, bool passwordChanged)	UPDATE Users; when passwordChanged=true, also updates MasterPasswordKey, AuthSalt, EncryptionSalt, and LastPasswordUpdate.
DeleteUserAsync(int userId)	Executes DELETE FROM Users WHERE Id = @id.
GetLastPasswordHistoryAsync(int userId, int count)	Executes SELECT TOP(@count) FROM MasterPasswordHistory WHERE UserId = @uid ORDER BY CreatedAt DESC.
AddPasswordHistoryAsync(...)	Executes INSERT INTO MasterPasswordHistory.

ממשק מאגר פריטי הכספת (IVaultItemRepository)

ממשק המגדיר את חוזה הגישה ל-DB לפעולות פריטי כספת.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
AddVaultItemAsync(VaultItem item)	Inserts a new vault item and returns the auto-generated Id.
UpdateVaultItemAsync(VaultItem item)	Updates the item; returns true when the row was found and modified.
GetVaultItemsByUserIdAsync(int userId)	Returns all vault items belonging to the given user.
DeleteVaultItemAsync(int itemId, int userId)	Deletes by Id + UserId to enforce ownership.
BulkUpdateVaultItemsAsync(List<VaultItem>, int userId)	Updates IV, Tag, and CipherText for a list of items atomically within a single SQL transaction.

מאגר פריטי הכספת (VaultItemRepository)

מימוש `IVaultItemRepository`. מנהל SQL ישיר לטבלת `VaultItems`. `BulkUpdateVaultItemsAsync`. מתבצעת בטרנזקציה SQL אטומית להבטחת אטומיות ה-re-encryption.

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
<code>_connectionString</code> (string)	Azure SQL connection string; injected via the constructor.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
<code>VaultItemRepository(string connectionString)</code>	Constructor.
<code>AddVaultItemAsync(VaultItem item)</code>	Executes INSERT INTO VaultItems ... OUTPUT INSERTED.Id.
<code>UpdateVaultItemAsync(VaultItem item)</code>	UPDATE VaultItems; LastUpdate is refreshed only when PasswordChanged = 1.
<code>GetVaultItemsByUserIdAsync(int userId)</code>	SELECT all columns WHERE UserId = @uid ORDER BY LastUpdate DESC.
<code>DeleteVaultItemAsync(int itemId, int userId)</code>	DELETE WHERE Id = @id AND UserId = @uid (ownership check).
<code>BulkUpdateVaultItemsAsync(List<VaultItem>, int userId)</code>	Updates IV, Tag, and CipherText for every item in the list within a single SqlTransaction.

עוזרי שרת (Backend Helpers)

עוזר JWT (JwtHelper)

מחלקה סטטית ליצירה ואימות של JSON Web Tokens חתומים ב-HMAC-SHA256. ה-Secret נטען מ-Environment Variable "JwtSecret". הטוקן תקף 2 שעות ומכיל Claims: user_id, username, jti.

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
Secret (string) [static private]	HMAC-SHA256 signing key loaded from the "JwtSecret" environment variable at startup.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
GenerateJwtToken(int userId, string username) [static]	Creates a signed JWT with user_id and username claims; token expires after 2 hours.
ValidateToken(string token) [static]	Validates signature and expiry; returns a ClaimsPrincipal on success or null when invalid.
GetUserIdFromPrincipal(ClaimsPrincipal principal) [static]	Extracts the user_id claim value from the principal; returns 0 on failure.

שירות HIBP (HibpService / IhibpService)

IHibpService הוא ממשק המאפשר Mock בבדיקות. HibpService הוא המימוש: שולח 5 תווים ראשונים של SHA-1 Hash ל-`/api.pwnedpasswords.com/range/`, מקבל רשימת `suffix: count` ומשווה מקומית. במקרה של כשל רשת — מחזיר `false` (Fail Open).

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
<code>_http (HttpClient)</code>	Injected HttpClient instance; allows mock replacement in unit tests.
<code>ApiBase (const string)</code>	Base URL of the HIBP Pwned Passwords API: " <code>https://api.pwnedpasswords.com/range/</code> ".

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
<code>IsPasswordPwnedAsync(string sha1Hash)</code>	Sends the 5-character SHA-1 prefix to HIBP, checks whether the full suffix appears in the response; returns false on network failure (Fail Open).

אובייקטי העברת נתונים (DTOs)

תגובת שרת גנרית (ServerResponse<T>)

מחלקה גנרית המשמשת תבנית תגובה אחידה לכל ה-endpoint-ים. מבטיחה שכל תגובה תכיל Success, Message ו-Data. הלקוח תמיד בודק Success לפני גישה ל-Data.

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
Success (bool)	True when the operation completed successfully; false otherwise.
Message (string)	Human-readable description of the outcome.
Data (T)	The generic result payload (AuthData, List<VaultItem>, etc.); null on failure.

נתוני אימות (AuthData)

DTO הנשלח ב-ServerResponse<AuthData> לאחר הרשמה/כניסה מוצלחת. הלקוח שומר את Token ב-SecureStorage ומשתמש ב-UserId לשירות הניטור ברקע.

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
UserId (int)	Server-assigned identifier of the authenticated user; persisted in SecureStorage for background checks.
Username (string)	Display name of the authenticated user; shown in the profile screen.
Token (string)	Signed JWT attached to every authenticated request via Authorization: Bearer <token>.

נתוני מלחים (SaltData)

DTO הנשלח ב- <SaltData> ServerResponse בתגובת GetSalts. הלקוח משתמש ב-AuthSalt לגזירת מפתח האימות וב-EncryptionSalt לגזירת מפתח הכספת — שניהם 600k PBKDF2 איטרציות.

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
AuthSalt (string)	Base64-encoded 32-byte salt used by the client to derive MasterPasswordKey via PBKDF2.
EncryptionSalt (string)	Base64-encoded 32-byte salt used by the client to derive the AES-256 vault key via PBKDF2.

בקשת בדיקת סיסמאות (PasswordCheckRequest)

DTO לגוף הבקשה של PasswordCheck endpoint. אינו דורש JWT — מכיל רק UserId כדי שה- PasswordMonitorService יוכל לקרוא ללא Session פעיל.

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
UserId (int)	Identifier of the user whose passwords are being checked; stored in SecureStorage by the background monitor service.

תוצאת בדיקת סיסמאות (PasswordCheckResult)

DTO הנשלח חזרה מ-PasswordCheck endpoint עם סיכום בריאות הסיסמאות. ה-PasswordCheckDecision בלקוח מחליט על סמך תוצאה זו האם לשלוח התראה.

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
BreachedCount (int)	Number of vault items whose password appears in a known data breach (IsLeaked = true).
OldCount (int)	Number of vault items whose password has not been changed in more than 90 days.
MasterPasswordOld (bool)	True when the master password has not been changed in more than 90 days.

בקשת עדכון חשבון (UpdateAccountRequest)

DTO לגוף הבקשה של UpdateUser endpoint. כאשר PasswordChanged=true, מכיל מפתחות חדשים ורשימת פריטי כספת מוצפנים מחדש.

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
Username (string)	New display name for the account.
Email (string)	New email address for the account.
PasswordChanged (bool)	True when the master password was changed; triggers history archiving and vault re-encryption.
MasterPasswordKey (string)	New PBKDF2-derived key; relevant only when PasswordChanged = true.
AuthSalt (string)	New AuthSalt; relevant only when PasswordChanged = true.
EncryptionSalt (string)	New EncryptionSalt; relevant only when PasswordChanged = true.
PasswordSha1Hash (string)	SHA-1 hash of the new master password for HIBP checking; never stored in the database.
VaultItems (List<VaultItem>)	Re-encrypted vault items with updated IV, Tag, and CipherText; sent as a bulk update.

מחלקות נוספות

רשימת הסיסמאות (PasswordBannerAdapter)

RecyclerView Adapter המציג את רשימת פריטי הכספת. כל שורה מציגה אייקון עם האות הראשונה של שם האתר, שם האתר/אפליקציה ושם המשתמש. מספק אירועי ItemClick (לפתיחת PasswordOptionsBottomSheet) ו-EditClick לניווט ישיר לעריכה.

Layout (item): item_password_banner.xml

תכונות:

Property Name	Use and Explanation
TextViewIcon (TextView)	Displays the first letter of the account name as an avatar character.
TextViewSiteName (TextView)	Displays the service/application name.
TextViewUsername (TextView)	Displays the account username.
ImageViewEdit (ImageView)	Kebab icon; tapping it raises EditClickAction.
ItemClickAction (Action<int>)	Callback invoked when the full banner row is tapped; argument is the adapter position.
EditClickAction (Action<int>)	Callback invoked when the edit icon is tapped; argument is the adapter position.

פעולות:

Method Name	Use and Explanation
PasswordBannerViewHolder(View)	Constructor that finds and assigns all child view references and attaches click listeners.
OnItemClick(object, EventArgs)	Invokes ItemClickAction with the current adapter position.
OnEditClick(object, EventArgs)	Invokes EditClickAction with the current adapter position.

גיליון תחתון לפעולות (PasswordOptionsBottomSheet)

BottomSheetDialogFragment המציג תפריט עם שלוש אפשרויות: צפייה, עריכה ומחיקה. מציג אייקון, שם האתר ושם המשתמש של הפרט שנבחר. מאפשר מחיקה עם דיאלוג אישור.

Layout: *bottom_sheet_password_options.xml*

תכונות:

Property Name	Use and Explanation
items (List<VaultItem>)	The collection of vault items currently displayed in the RecyclerView.
ItemClick (event EventHandler<int>)	Raised when the user taps a banner row; argument is the tapped position.
EditClick (event EventHandler<int>)	Raised when the user taps the edit/options icon; argument is the tapped position.
ItemCount (int override)	Returns items.Count; required by RecyclerView.Adapter.

פעולות:

Method Name	Use and Explanation
PasswordBannerAdapter(List<VaultItem>)	Constructor that initialises the items list.
OnCreateViewHolder(ViewGroup, int)	Inflates item_password_banner.xml and wraps it in PasswordBannerViewHolder.
OnBindViewHolder(RecyclerView.ViewHolder, int)	Populates icon letter, site name, and username from the item at the given position; wires click callbacks.
UpdateData(List<VaultItem>)	Replaces the data set with the new list and calls NotifyDataSetChanged to refresh the RecyclerView.

ניווט תחתון (BottomNavFragment)

Fragment המציג תפריט ניווט תחתון עם שלושה לשוניות: כספת, אזהרות ופרופיל. מסמן את הלשונית הפעילה, מעלה אירוע TabSelected בעת לחיצה ומונע ניווט מיותר אם כבר נמצאים בלשונית הנבחרת.

Layout: *fragment_bottom_nav.xml*

תכונות:

Property Name	Use and Explanation
ArgSelectedTab (const string)	Bundle argument key for the initially selected tab name.
currentTab (string)	Name of the tab currently shown as selected ("vault", "warnings", or "profile").
TabSelected (event EventHandler<string>)	Raised when the user taps a tab; argument is the tab name.

פעולות:

Method Name	Use and Explanation
NewInstance(string) [static]	Factory method that creates a BottomNavFragment with the given tab pre-selected.
OnCreateView(LayoutInflater, ViewGroup, Bundle)	Inflates fragment_bottom_nav.xml.
OnViewCreated(View, Bundle)	Reads the initial tab from Arguments, attaches click listeners to the three tab containers, and calls SelectTab.
SelectTab(View, string)	Updates icon and label colours to reflect the newly active tab and raises TabSelected.

מחלקות Helper

תשתית תקשורת HTTP (BaseService)

BaseService היא מחלקה מופשטת (abstract) המהווה מנוע HTTP מרכזי. מספקת `PostAsync<T>` ו `GetAsync<T>` עם סדרה/ביטול סדרה ב-JSON, הזרקת Bearer token אוטומטית מה-StorageHelper, וטיפול בשגיאות 401/500. מחלקות שירות ספציפיות יורשות ממנה: AuthService לפעולות הזדהות, VaultService לפריטי כספת, ProfileService לניהול פרופיל, PasswordCheckServerService לבדיקת סיסמאות ברקע.

תכונות:

Property Name	Use and Explanation
_httpClient (HttpClient) [static]	Shared HttpClient instance used for all HTTP calls; static to avoid socket exhaustion.
_baseUrl (string)	Root URL of the Azure Functions backend, read from app configuration.

פעולות:

Method Name	Use and Explanation
PostAsync<T>(string endpoint, object data)	Serialises data to JSON, POSTs to the endpoint, injects the JWT header, and deserialises the response as ServerResponse<T>.
GetAsync<T>(string endpoint)	Performs a GET to the endpoint with the JWT header and returns a deserialised ServerResponse<T>.

שירות אימות (AuthService)

שירות ספציפי המנהל הרשמה וכניסה של משתמשים. יורש מ-BaseService.

פעולות:

Method Name	Use and Explanation
RegisterAsync(User newUser, string plainTextPassword)	POSTs the new user to "RegisterUser"; on success calls SetupAuthenticatedSession to derive keys and warm the vault cache.
LoginAsync(string email, string password)	Step 1 — POSTs to "GetSalts" to retrieve AuthSalt and EncryptionSalt. Step 2 — derives PBKDF2 key locally. Step 3 — POSTs to "VerifyLogin" with the hashed key. On success calls SetupAuthenticatedSession.
SetupAuthenticatedSession(AuthData data, string password, string salt) [private]	Persists UserId, JWT, and Username to SecureStorage; derives the AES vault key with DeriveKey; stores it; calls FetchAndCacheVaultAsync; computes and caches WarningsData.
FetchAndCacheVaultAsync() [static]	GETs vault items via VaultService.GetVaultItemsAsync and stores the result in SessionHelper.CachedVault.
ValidateTokenAsync()	GETs "ValidateToken"; returns true when the server confirms the stored JWT is still valid.

שירות כספת (VaultService)

שירות ספציפי לניהול פעולות CRUD של פריטי הכספת המוצפנים. יורש מ-BaseService.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
AddVaultItemAsync(VaultItem item)	POSTs the encrypted item to "AddVaultItem"; returns (Success, Message, VaultItem) with the server-assigned Id.
UpdateVaultItemAsync(VaultItem item)	POSTs the updated encrypted item to "UpdateVaultItem"; returns (Success, Message, VaultItem).
GetVaultItemsAsync()	GETs "GetVaultItems"; returns (Success, Message, List<VaultItem>) with all encrypted items for the authenticated user.
DeleteVaultItemAsync(int itemId)	POSTs {Id} to "DeleteVaultItem"; returns (Success, Message).

שירות פרופיל (ProfileService)

שירות ספציפי לניהול פרטי חשבון ופרופיל המשתמש. יורש מ-BaseService.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
GetProfileAsync()	GETs "GetProfile"; caches the result via StorageHelper.SaveProfileAsync; returns ServerResponse<User>.
UpdateAccountAsync(UpdateAccountRequest request)	POSTs the UpdateAccountRequest (with optional re-encrypted vault items) to "UpdateUser"; returns ServerResponse<object>.
DeleteAccountAsync()	POSTs an empty body to "DeleteUser"; returns ServerResponse<object>.
GetPasswordHistoryAsync()	GETs "GetPasswordHistory"; returns ServerResponse<List<MasterPasswordHistory>> with the last 4 archived password keys.

שירות בדיקת סיסמאות (PasswordCheckService)

שירות ספציפי לשליחת בקשת בדיקת הסיסמאות לשרת. יורש מ-BaseService. משמש את מסכי UI הדורשים JWT (בניגוד ל-PasswordCheckServerService המשמש את השירות ברקע).

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
CheckAsync(int userId)	POSTs {UserId} to "PasswordCheck"; returns (Success, Message, PasswordCheckResult) with BreachedCount, OldCount, and MasterPasswordOld.

שירות בדיקת סיסמאות ברקע (PasswordCheckServerService)

מחלקה סטטית לשליחת בקשת PasswordCheck ישירות מה-PasswordMonitorService, ללא JWT וללא BaseService. משתמש ב-HttpClient עצמאי ומאמץ Fail Open: כל שגיאת רשת מחזירה null ואינה מייצרת התראת שווא.

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
_http (HttpClient) [static]	Shared HttpClient instance used exclusively by the background monitor service.
Endpoint (const string)	Absolute URL of the PasswordCheck Azure Function.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
FetchAsync(int userId) [static]	Serialises PasswordCheckRequest and POSTs it to the PasswordCheck endpoint; deserialises ServerResponse<PasswordCheckResult>; returns the PasswordCheckResult on success or null on any failure.

מנהל בדיקות תקינות (ValidationHelper)

מחלקה סטטית לאימות שדות קלט וחישוב חוזק סיסמאות. מאמתת שם משתמש (3-30 תווים, מתחיל באות), מייל (פורמט RFC 5321 מרוכך) וסיסמה (לפחות 8 תווים, אות גדולה, קטנה, ספרה ותו מיוחד). מחשבת ציון חוזק 0-5 ומחזירה רמז עם הדרישה החסרה. כוללת גם פונקציית יצירת סיסמה חזקה אוטומטית (GenerateStrongPassword).

תכונות:

Property Name	Use and Explanation
UsernameRegex (Regex)	Compiled regex: 3-30 chars, must start with a letter, letters/digits/underscores/hyphens only.
EmailRegex (Regex)	Compiled RFC 5321-compatible regex for email format validation.
HasUppercase / HasLowercase / HasDigit / HasSpecialChar (Regex)	Four compiled regexes used to check individual password-strength criteria.

פעולות:

Method Name	Use and Explanation
ValidateEmail(string)	Returns ValidationResult with IsValid=true if the email matches the regex, otherwise Fail with a message.
ValidateUsername(string)	Returns ValidationResult.Ok if the username meets length and character requirements.
ValidatePassword(string)	Returns ValidationResult.Ok if the password meets all criteria (min length, upper, lower, digit, special char).
ValidatePasswordsMatch(string, string)	Returns ValidationResult.Ok if both strings are identical.
GetPasswordScore(string)	Returns a score 0-5 based on how many of the five strength criteria are met.
GetMissingCriteriaHint(string)	Returns a constructive hint string listing the first unmet strength criterion.
GenerateStrongPassword()	Creates a random password that meets all five strength criteria.

עוזר ממשק טפסים (FormUiHelper)

מחלקה סטטית לניהול ממשק המשתמש של טפסים. מציגה ומסתירה שגיאות שדה, מעדכנת את מד חוזק הסיסמה (5 מקטעים עם צבעי RAG) ומחשבת צבע לפי ציון.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
ShowError(Textview, string)	Sets the error Textview text and makes it visible.
HideError(Textview)	Clears the error Textview and hides it.
UpdatePasswordStrengthIndicator(string, ProgressBar, TextView, TextView, Resources)	Calculates score via ValidationHelper, tints the ProgressBar with a RAG colour, and shows a constructive hint label.
ScoreToColorRes(int)	Maps a strength score (1-5) to the corresponding colour resource ID for the progress-bar tint.

מנהל החיבור (SessionHelper)

מחלקה סטטית המנהלת את מפתח ההצפנה הפעיל ואת מטמון (Cache) הנתונים בזיכרון בלבד. StartSession שומרת את מפתח AES-256, EndSession מנקה אותו. מחזיקה גם את CachedVault (רשימת פריטי הכספת) ו-CachedWarnings (תוצאות בדיקת הבריאות) לגישה מהירה בין מסכים. נתונים אלה לעולם אינם נשמרים לדיסק - רק ב-RAM.

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
SessionVaultKey (string)	The AES-256 vault key derived from the master password; held in RAM only, never persisted to disk.
CachedVault (List<VaultItem>)	In-memory list of all vault items for the current session; single source of truth for the UI.
CachedWarnings (WarningsData)	Cached password-risk counters (Leaked/Weak/Reused/Old); set to null when the vault changes.
IsAuthenticated (bool)	True when SessionVaultKey is non-null; used as a session guard throughout the app.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
StartSession(string aesKey)	Sets SessionVaultKey in memory to mark the session as authenticated.
EndSession()	Clears SessionVaultKey, CachedVault, and CachedWarnings from memory; calls GC.Collect.
InvalidateWarnings()	Sets CachedWarnings to null so it is recomputed on the next access.
AddVaultItem(VaultItem)	Appends a new item to CachedVault.
UpdateVaultItem(VaultItem)	Finds the matching item by Id in CachedVault and replaces its fields in place.
RemoveVaultItem(int itemId)	Removes all items with the given Id from CachedVault.

מנהל האחסון (StorageHelper)

מחלקה סטטית לאחסון מאובטח בדיסק. עוטפת את SecureStorage של Xamarin.Essentials (הצפנת AES-256-GCM בגיבוי Android Keystore). שומרת JWT, מזהה משתמש, שם משתמש, מפתח כספת ונתוני פרופיל. ClearSessionAsync מנקה נתוני Session (JWT ומפתח כספת) ושומרת את מזהה המשתמש ושם המשתמש לשירות הרקע. ClearAll מנקה הכל (לאחר מחיקת חשבון).

תכונות:

Property Name	Use and Explanation
KeyUserId / KeyUsername / KeyJwt / KeyVaultKey / KeyEmail / KeyCreatedAt / KeyPasswordCount / KeyLastLogin / KeyLastPasswordChange (const string)	Private string constants that serve as storage key identifiers in SecureStorage.

פעולות:

Method Name	Use and Explanation
SaveUserId(int)	Writes the user's integer ID to SecureStorage as a string.
GetUserId()	Reads and parses the stored user ID; returns 0 if absent or invalid.
SaveUsername(string)	Persists the display name to SecureStorage.
GetUsername()	Returns the stored display name, or null if absent.
SaveJwt(string)	Writes the JWT to SecureStorage.
GetJwt()	Returns the stored JWT, or null if absent.
SaveVaultKey(string)	Writes the AES vault key to SecureStorage.
GetVaultKey()	Returns the stored AES vault key, or null if absent.
SaveProfileAsync(User)	Writes email, password count, CreatedAt, LastLogin, and LastPasswordChange to SecureStorage.
GetCachedProfileAsync()	Reads all profile fields from SecureStorage and returns a populated User object (null if absent).
ClearSessionAsync()	Removes JWT, VaultKey, Email, and profile timestamps; preserves UserId and Username for the background service.
ClearAll()	Calls SecureStorage.RemoveAll(); used only on account deletion.

מנהל ההצפנה (EncryptionHelper)

מחלקה סטטית לכל פעולות ההצפנה. GenerateSalt יוצר מלח 32-בייט אקראי. DeriveKey נגזר מפתח IV עם AES-256-GCM מבצעים SHA-256. Encrypt/Decrypt ו-600,000 איטרציות PBKDF2 ייחודי 96-ביט ותג אימות 128-ביט. ComputeSha1Hash מחשב גיבוב SHA-1 של הסיסמה לבדיקת HIBP.

תכונות:

Property Name	Use and Explanation
Iterations (const int)	PBKDF2 iteration count: 600,000; calibrated to make brute-force attacks computationally expensive per OWASP guidelines.

פעולות:

Method Name	Use and Explanation
GenerateSalt()	Creates a cryptographically strong 32-byte random salt and returns it as a Base64 string.
DeriveKey(string password, string saltBase64)	Derives a 256-bit key using PBKDF2-HMAC-SHA256 with 600,000 iterations; returns Base64.
EncryptAesGcm(string plaintext, string base64Key)	Encrypts plaintext with AES-256-GCM using a random 96-bit IV; returns (IV, Tag, CipherText) all Base64-encoded.
DecryptAesGcm(string iv, string tag, string cipherText, string base64Key)	Decrypts AES-256-GCM ciphertext using the provided IV, tag, and key; returns the plaintext string.
ComputeSha1Hash(string plaintext)	Computes the unsalted SHA-1 hash and returns it as a 40-character uppercase hex string.

עוזר ניווט תחתון (BottomNavHelper)

מחלקה סטטית לשיתוף לוגיקת מעבר בין לשוניות הניווט התחתון. ממפה שם לשונית לפעילות המתאימה ומסיים את הפעילות הנוכחית.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
Navigate(AppCompatActivity activity, string tab, string currentTab) [static]	If tab equals currentTab the call is a no-op. Otherwise creates an Intent for the matching Activity (VaultActivity, WarningsActivity, or ProfileActivity), starts it, and calls Finish() on the current activity.

מאזין שינוי טקסט (SimpleTextWatcher)

מתאם (Adapter) מינימלי של ITextWatcher המאפשר שימוש ב-lambda כ-Listener לשינוי טקסט ב- EditText של Android. מממש את שלוש המתודות הנדרשות אך רק AfterTextChanged מפעילה את ה-callback.

תכונות:

Property Name	Use and Explanation
_onChanged (Action<string>) [private]	Lambda callback invoked after each text change; supplied via the constructor.

פעולות:

Method Name	Use and Explanation
SimpleTextWatcher(Action<string> onChanged)	Constructor; stores the callback.
AfterTextChanged(Editable s)	Invokes _onChanged with the current string value after every text modification.
BeforeTextChanged(CharSequence s, int start, int count, int after)	Required by ITextWatcher; not used.
OnTextChanged(CharSequence s, int start, int before, int count)	Required by ITextWatcher; not used.

עוזרי לקוח נוספים

עוזר האזהרות (WarningsHelper + WarningsData)

WarningsData הוא DTO המחזיק ארבעה מונים: LeakedCount, WeakCount, ReusedCount, OldCount. WarningsHelper הוא מחלקה סטטית לחישוב סיכום אזהרות מהכספת בזיכרון. ComputeWarningsSync ללא רשת — משתמש ב-IsLeaked השמור. ComputeWarningsAsync בודק HIBP בזמן אמת (נקרא בלוגין בלבד).

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
OldPasswordDays (const int = 90) [WarningsHelper]	Age threshold in days; passwords last updated more than 90 days ago are flagged as stale.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
ComputeWarningsSync(IList<VaultItem> vault, string vaultKey)	Computes all four counters without network calls: LeakedCount from IsLeaked flags, WeakCount by decrypting and running ValidationHelper, ReusedCount by grouping SHA-1 hashes, OldCount by comparing LastUpdate.
ComputeWarningsAsync(IList<VaultItem> > vault, string vaultKey)	Queries HIBP in real time for each item (k-Anonymity), updates IsLeaked in the cache, then computes all four counters. Called once at login.
GetItemsAtRiskAsync(IList<VaultItem> vault, string vaultKey, string category)	Returns the subset of vault items that match the given risk category: "leaked", "weak", "reused", or "old".

קבלת החלטת התראה (PasswordCheckDecision)

מחלקה סטטית המחליטה אם ועל מה להתריע.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
ShouldNotify(PasswordCheckResult result) [static]	Returns true when BreachedCount > 0, OldCount > 0, or MasterPasswordOld = true.
BuildMessage(PasswordCheckResult result) [static]	Builds a human-readable alert string: "Securio alert: X leaked passwords, Y old passwords, master password not changed in 90+ days."

עוזר התראות (NotificationHelper)

מחלקה סטטית המטפלת בכל פעולות Android Notification עבור PasswordMonitorService. מפרידה לוגיקת Android API מ-PasswordCheckDecision.

תכונות:

<u>Property Name</u>	<u>Use and Explanation</u>
ChannelId (const string)	Notification channel identifier: "securio_monitor_channel".
ChannelName (const string)	Notification channel display name: "Password Monitor".
AlertNotificationId (const int)	Android notification ID (1001) for password-health alert notifications.
ForegroundNotificationId (const int)	Android notification ID (1002) for the persistent foreground-service notification.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
CreateChannel(Context context) [static]	Creates the notification channel; required on Android 8.0 (API 26) and above.
PostAlert(Context context, string message) [static]	Posts a local alert notification with the title "Securio Password Alert" and the supplied message body.
BuildForegroundNotification(Context context) [static]	Builds the persistent ongoing notification displayed while PasswordMonitorService is running.
PostCheckResult(Context context, PasswordCheckResult result) [static]	Calls PasswordCheckDecision.ShouldNotify; if true, posts an alert notification via PostAlert.

מקלט אתחול (BootReceiver)

Android BroadcastReceiver שמאזין ל-BOOT_COMPLETED ו- MY_PACKAGE_REPLACED. מפעיל מחדש את PasswordMonitorService לאחר הפעלת המכשיר או עדכון האפליקציה.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
OnReceive(Context context, Intent intent)	Handles BOOT_COMPLETED and MY_PACKAGE_REPLACED broadcasts by creating an Intent for PasswordMonitorService and calling StartForegroundService.

שירות HIBP בלקוח (HibpClientService)

מחלקה סטטית בצד הלקוח לבדיקת דליפות ב-HIBP. מיישמת k-Anonymity: שולחת רק 5 תווים ראשונים של SHA-1, ומשווה מקומית. הסיסמה המלאה לעולם אינה עוזבת את המכשיר.

פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
IsPasswordPwnedAsync(string sha1Hash) [static]	Sends the 5-character SHA-1 prefix to <code>api.pwnedpasswords.com/range/{prefix}</code> , checks whether the full suffix appears in the result list; returns false on network failure (Fail Open).

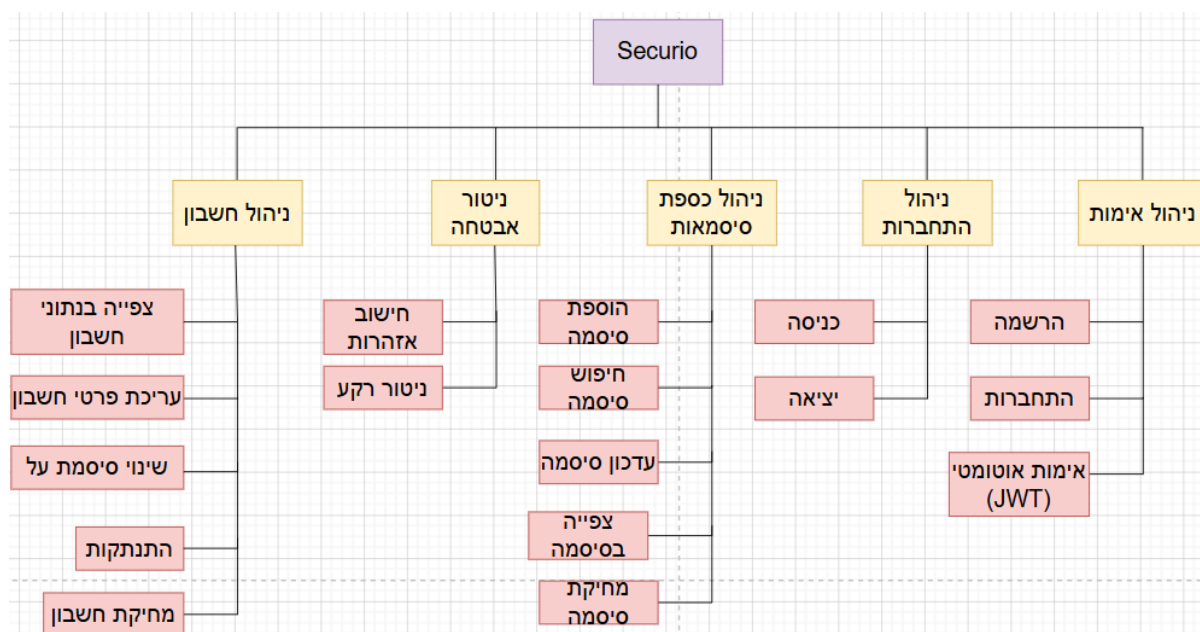
עוזר פעולות על פריטים (PasswordEntryActionsHelper)

מחלקה סטטית עם לוגיקה משותפת לפעולות על פריטי כספת (צפייה, עריכה, מחיקה).

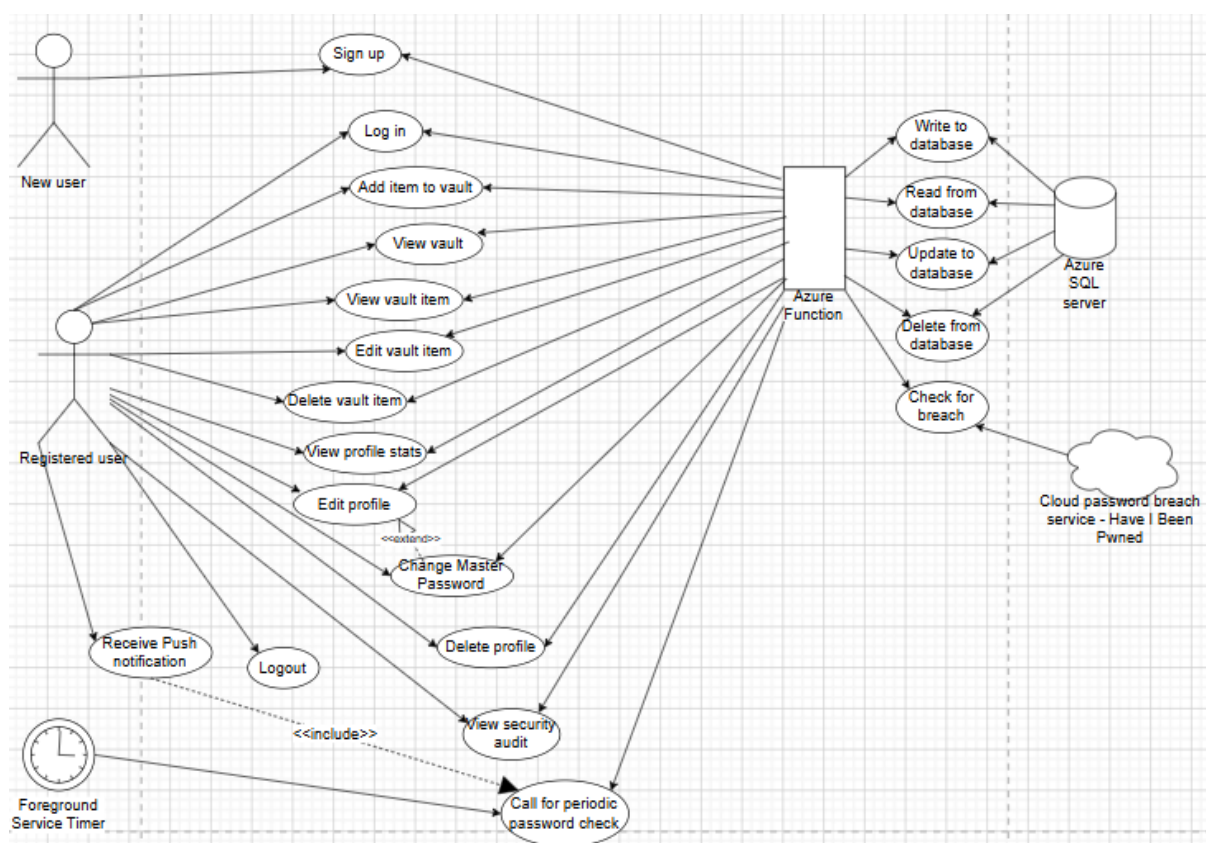
פעולות:

<u>Method Name</u>	<u>Use and Explanation</u>
ShowOptionsSheet(Activity host, VaultItem entry, int reqCodeEdit, Action<VaultItem> onDelete) [static]	Displays PasswordOptionsBottomSheet; wires View, Edit, and Delete event handlers; shows a confirmation dialog before deletion.

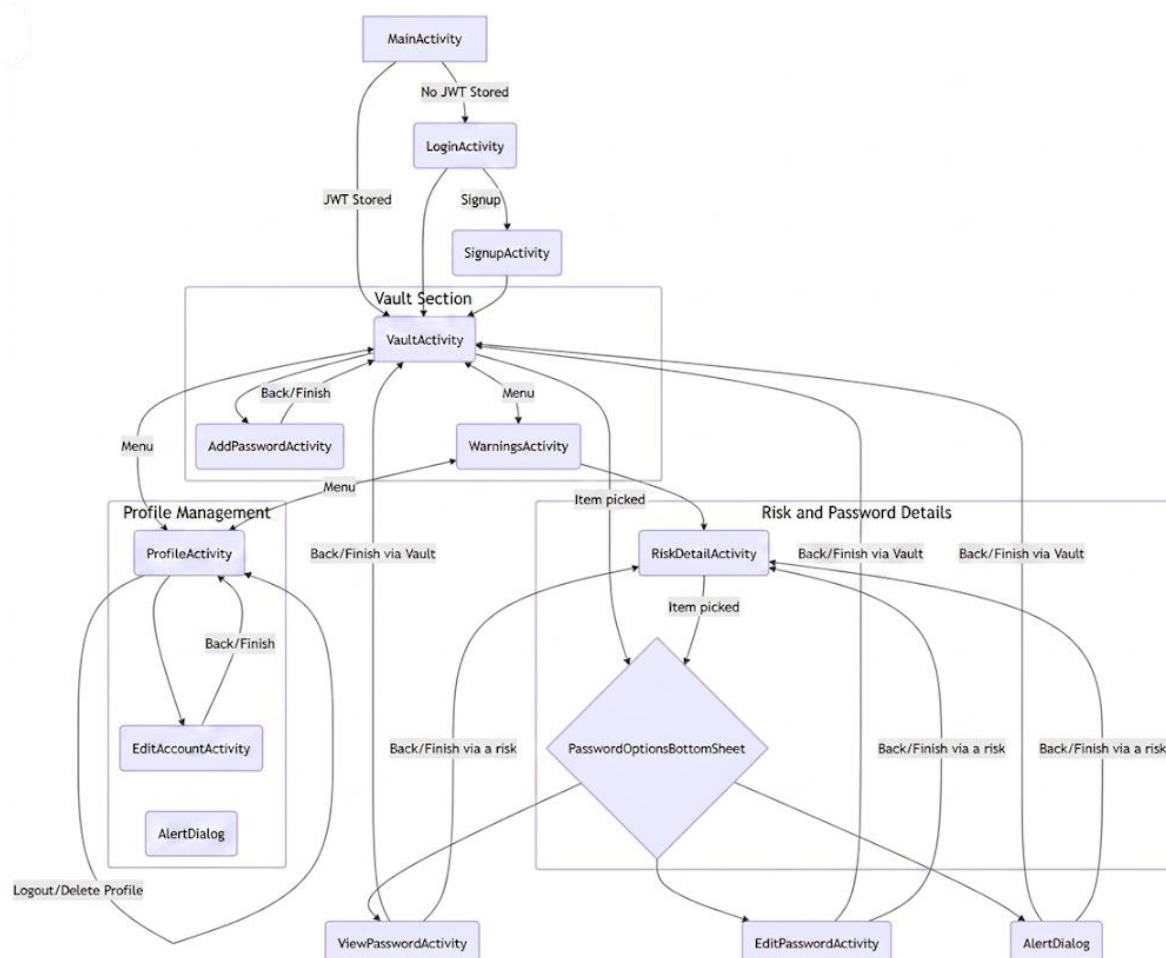
עץ תהליכים



דיאגרמת Use Case



תרשים מעבר מסכים



בסיס נתונים טבלאי

טבלת משתמשים (Users)

שומרת את פרטי המשתמש ומשמשת להתחברות.

Primary Key: ID (Integer, Auto-increment)

<u>FieldName (fieldType)</u>	<u>Description</u>
ID (int)	Primary key for the user.
Username (string)	The display name of the user.
Email (string)	Unique user's email, also used for login.
MasterPasswordKey (string)	Stretched key (PBKDF2) used for server-side login verification.
AuthSalt (string)	Salt used to derive the MasterPasswordKey on the client.
EncryptionSalt (string)	Salt used to derive the local AES encryption key.
LastLogin (DateTime)	Timestamp of the last successful authentication.
LastPasswordUpdate (DateTime)	Timestamp of when the master password was last changed.
CreatedAt (DateTime)	Timestamp of account creation.

טבלת פריטי הכספת (VaultItems)

שומרת את כל פריטי הכספת המוצפנים של המשתמשים.

Primary Key: Id (Integer, Auto-increment) | Foreign Key: UserId (References Users)

<u>FieldName (fieldType)</u>	<u>Description</u>
ID (int)	Unique identifier for the specific credential entry.
UserID (int)	Foreign key linking the entry to the owner.
AccountName (string)	Name of the service or website (e.g., Google).
AccountUsername (string)	Username or email used for that service.
IV (string)	Base64 12-byte initialization vector for AES-GCM.
Tag (string)	Base64 16-byte authentication tag for integrity check.
Ciphertext (string)	The AES-256 encrypted password.
Notes (string)	Encrypted additional information.
SHA1Hash (string)	Hash of plaintext used for HIBP monitoring.
isLeaked (bool)	Flag set if the password appears in a data breach.
LastUpdate (DateTime)	Timestamp for password rotation tracking.

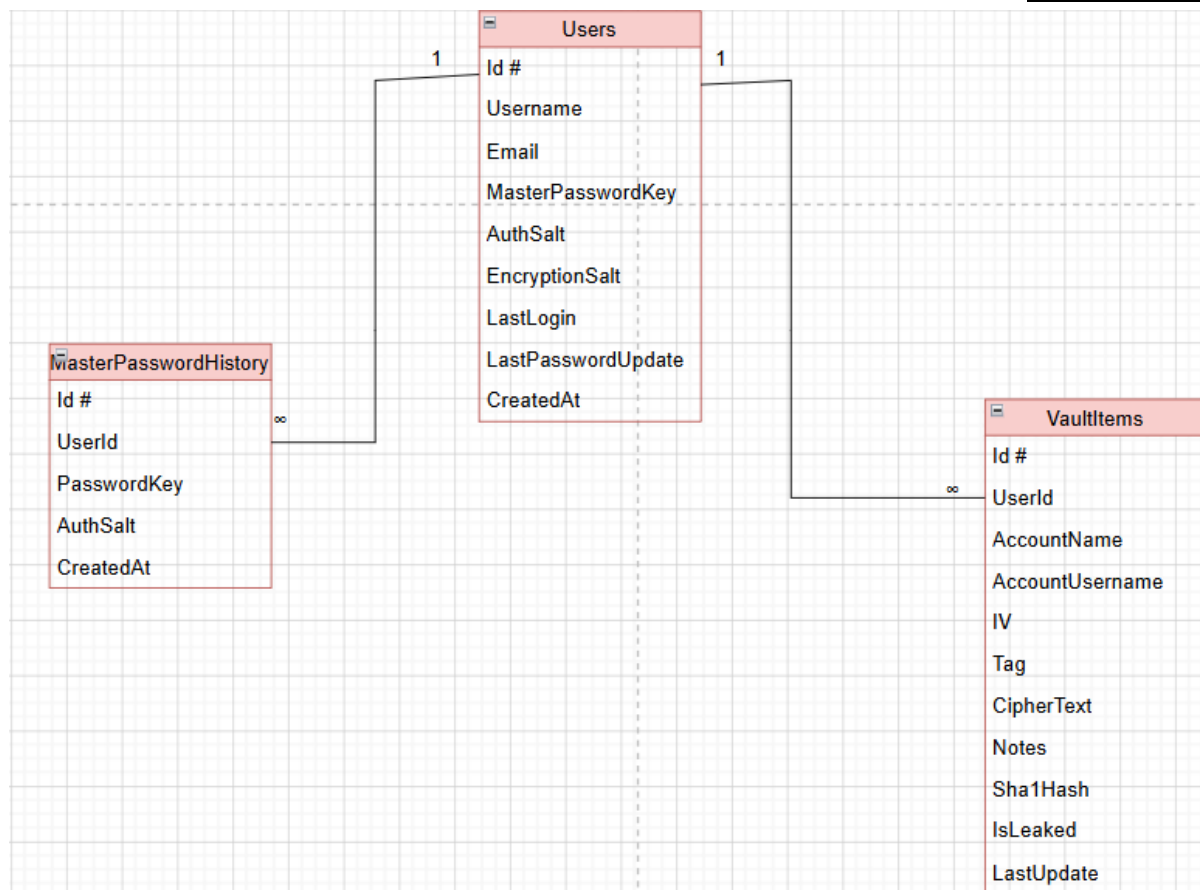
טבלת היסטוריית סיסמות על (MasterPasswordHistory)

שומרת עד 4 ערכים ישנים של סיסמת העל של כל משתמש. משמשת לאכיפת מדיניות אי-שימוש חוזר בסיסמה: בעת שינוי סיסמת על, הלקוח מביא את ההיסטוריה ומשווה מקומית לפני שליחת הבקשה לשרת.

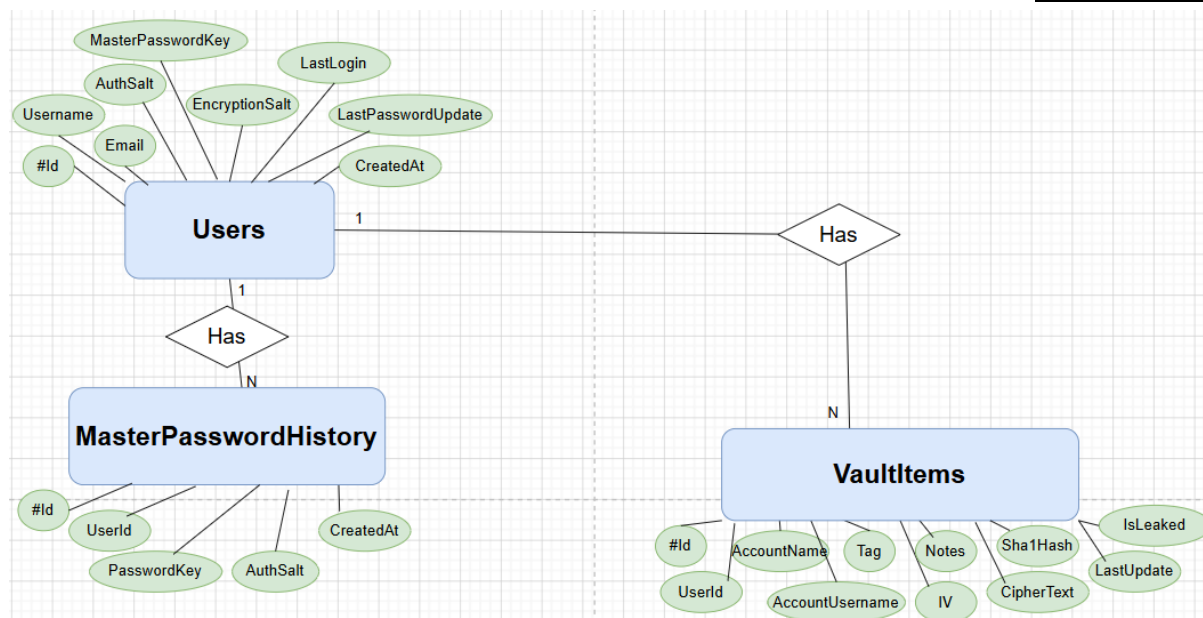
Primary Key: Id (Integer, Auto-increment) | Foreign Key: UserId (References Users)

<u>FieldName (fieldType)</u>	<u>Description</u>
ID (int)	Primary key for the history entry.
UserID (int)	Foreign key linking to the owner (User table).
PasswordKey (string)	PBKDF2-derived key of the old master password.
AuthSalt (string)	The AuthSalt paired with this PasswordKey.
CreatedAt (DateTime)	Timestamp of when this password was archived (retired).

תרשים DSD



תרשים ERD



שימוש ב-SecureStorage – SharedPreference מאובטח

SecureStorage של Xamarin.Essentials משמש כשכבת האחסון המקומית המאובטחת באפליקציה זו. בניגוד ל-SharedPreferences סטנדרטי, SecureStorage מצפין אוטומטית את הנתונים באמצעות Android Keystore System (AES-256 GCM). ממשק ה-API הפשוט — SetAsync / GetAsync / Remove — מסתיר את פרטי ההצפנה מהמפתח. נתונים רגישים כגון ה-JWT, מפתח הכספת (VaultKey), ומזהה המשתמש לשירות הניטור ברקע מוגנים מפני גישה בלתי מורשית של אפליקציות אחרות או גורמים חיצוניים.

Field (type)	Use and Description
UserId (int)	Unique identifier for the logged-in user; retained across sessions for the background service.
Username (string)	Display name of the user; retained across sessions.
JWT (string)	JSON Web Token for authorised API communication; cleared on logout.
VaultKey (string)	AES-256 vault encryption key; cleared on logout, restored on re-login.
Email (string)	User's email address; cached locally for the profile screen.
CreatedAt (string)	ISO-8601 timestamp of account creation.
PasswordCount (string)	Cached vault item count for the profile screen.
LastLogin (string)	ISO-8601 timestamp of the most recent login.
LastPasswordChange (string)	ISO-8601 timestamp of the most recent master-password change.

מדריך למשתמש

דרישות והוראות התקנה

האפליקציה פותחה ב-Visual Studio עבור מכשירי אנדרואיד גרסה 13.0 ומעלה. היא עושה שימוש ב-API 33.

המערכת נבדקה ואומתה על גבי אמולטור (מכשיר וירטואלי).

אופן הפעולה של האפליקציה

מסך התחברות והרשמה: בשימוש ראשון, תתבקשו ליצור חשבון עם שם משתמש, אימייל ו"סיסמת על". לאחר מכן, בכל כניסה תשתמשו באימייל ובסיסמה שבחרתם כדי "לפתוח את הכספת".

הכספת הדיגיטלית (מסך הבית): זהו מרכז הבקרה שלכם. כאן תראו את כל הסיסמאות ששמרתם.

- לחיצה על **כפתור ה- "+"** תעביר אתכם למסך הוספת סיסמה חדשה.
- לחיצה על חשבון קיים תפתח תפריט המאפשר לכם לצפות בפרטים, לערוך אותם או למחוק את החשבון.

מסך אזהרות אבטחה: דרך תפריט הניווט התחתון ניתן להגיע למסך המציג כמות סיסמאות שנמצאות בסיכון (למשל: סיסמאות ישנות או כאלו שדלפו לרשת). כל סיכון מיוצג ע"י כפתור, שמאפשר לכם לצפות בכל הסיסמאות תחת סיכון זה.

מסך פרופיל: דרך תפריט הניווט התחתון ניתן להגיע למסך. כאן תוכלו לראות את פרטיכם האישיים, כמה סיסמאות שמרתם, ואפשרות להתנתק בצורה בטוחה.

הודעות למשתמש ובדיקות תקינות

האפליקציה שומרת עליכם באמצעות הודעות ומגבלות קלט:

בדיקת חוזק סיסמה: בעת ההרשמה, יופיע מד המציין את חוזק סיסמת העל שלכם. מומלץ להשתמש בסיסמה חזקה הכוללת תווים מגוונים.

בדיקות קלט (Validation): אם תשכחו למלא שדה חובה (כמו אימייל או שם משתמש) או שהפורמט לא יהיה תקין, תופיע הודעת שגיאה אדומה מתחת לשדה הרלוונטי.

התראות אבטחה: המערכת סורקת אחת ליום את הסיסמאות שלכם. במידה וסיסמה מסוימת נמצאה כישנה או דלופה במאגרי מידע חיצוניים, תקבלו על כך התראה כדי שתוכלו להחליפה במהירות.

הודעות אישור: בפעולות רגישות, כמו מחיקת חשבון או התנתקות, האפליקציה תוודא איתכם את הפעולה כדי למנוע טעויות.

מגבלות ואילוצים

שדות חובה: לא ניתן לשמור פריט בכספת ללא שם אתר/אפליקציה וסיסמה.

אבטחת המכשיר: מאחר וההצפנה מתבצעת על המכשיר שלכם, חשוב לשמור על "סיסמת העל" היטב – אם תאבדו אותה, לא ניתן יהיה לשחזר את המידע המוצפן בשרת.

הרחבות

סעיף 6

הורדת נתוני מידע מהאינטרנט (API של אתר) ושימוש בנתונים שהורדו: כדי לבדוק האם סיסמה התגלתה בדליפה, האפליקציה קוראת ל - API של Have I Been Pwned, שנקרא Pwned Passwords. ה - API מקבל 5 תווים ראשונים של SHA1 Hash של סיסמה, ומחזיר רשימה של סיסמאות עם התחילית הזו. השרת משווה בין ה-Hash ששמור אצלו לרשימה, ואם הוא מופיע - הסיסמה התגלתה בדליפה.

שימוש ב - Service: שולח בקשה לשרת כל 24 שעות לדגום את הסיסמאות של הלקוח ולבדוק אם הן ישנות או שהתגלו בדליפה.

שימוש ב - RecyclerView: הסיסמאות בדף הכספת, וכן בדף הפילטר לסיכון ספציפי, מוצגות ע"י RecyclerView. בכל שורה מופיעים שם האפליקציה/אתר, שם המשתמש באותו חשבון, וכפתור לביצוע פעולות צפייה/עריכה/מחיקה על הסיסמה.

סעיף 7

נעשה שימוש בשירות Azure SQL Database, שהוא בסיס נתונים מרוחק.

סעיף 9

שימוש ב - Fragment: תפריט הניווט התחתון משולב בתור Fragment. בתפריט שלושה כפתורים: אחד המוביל לדף הכספת, אחד המוביל לדף האזהרות, ואחד המוביל לדף הפרופיל.

פיתוח ושימוש ב API בצד שרת: התקשורת מול בסיס הנתונים קורה דרך Azure Functions - שרת שנמצא בענן, ופונים אליו באמצעות API של המפתח.

סעיף 10

שימוש ב-Broadcast Receiver: האפליקציה משתמשת ב Broadcast Receiver כדי לאתחל את ה-Service של בדיקת הסיסמאות אחרי אתחול של הטלפון.

שימוש ב - Async Task: קריאות ה - API לשרת נעשות באמצעות פונקציות אסינכרוניות.

רפלקציה

היעד ההתחלתי והשגתו: היעד המקורי של הפרויקט היה פיתוח אפליקציית אבטחה מקיפה הכוללת מנהל סיסמאות בעל ארכיטקטורה ייחודית ומנגנון לזיהוי אתרי פשינג. בסופו של תהליך, היעד הושג באופן חלקי: פותחה אפליקציית ניהול סיסמאות מאובטחת ומתפקדת, אך נאלצתי לבצע שינויים טכנולוגיים משמעותיים בדרך כדי להגיע לתוצר סופי איכותי ועובד.

קשיים, חוויות והתגברות עליהם: במהלך העבודה חוויתי קושי משמעותי שנבע מתכנון לוקה בחסר בשלבים המוקדמים. מצאתי את עצמי "מפוזר", ללא תשתית צד-שרת (Backend) מוגדרת, מה שהוביל לכך שרכיבי המערכת לא התחברו. החוויה הייתה מתסכלת ומלחיצה, במיוחד כשהבנתי באיחור שעלי לשנות כיוון. התגברתי על הקושי באמצעות החלטה אמיצה לבצע "Pivot" (שינוי מסלול) – זנחתי את הקוד הישן שלא שירת את המטרה, ובניתי תוכנית עבודה חדשה וממוקדת. למרות הלחץ של כתיבת תיק הניתוח בחמישה ימים וסיום הקוד בחודש, הצלחתי לעמוד ביעדים בזכות תכנון מדוקדק ולו"ז צפוף.

מה למדתי על עצמי: למדתי שאני זקוק למסגרת תכנונית קשיחה לפני תחילת העבודה. גיליתי שדווקא תכנון מקדים משחרר אותי לעסוק בקוד בצורה נקייה ומהנה יותר. בנוסף, למדתי על היכולת שלי לא להתפשר על איכות; גם תחת לחץ זמן, בחרתי לשמור על רמת קושי גבוהה ולא "לעגל פינות". למדתי גם שאין לי פחד להתחיל מחדש כשצריך – זו תכונה שמאפשרת צמיחה מתוך טעויות.

אנשים שעזרו לי: נעזרתי באבי, מנהל צוות פיתוח, שסייע לי בזיקוק הרעיונות הטכנולוגיים (כמו עבודה עם שרת Azure) ובדיוק הארכיטקטורה של המערכת. כמו כן, המורה ליוותה אותי בהיבטים הפדגוגיים, עזרה לי לבחון את הפרויקט מנקודת המבט של הבוחן ולוודא שהרעיון עומד בכל הדרישות המקצועיות של הברגרות.

כלים להמשך, תובנות ומסקנות: אני לוקח איתי הבנה עמוקה של ניהול פרויקטים בתוכנה. התובנה המרכזית שלי היא שפרויקט מורכב הוא לא דבר "מפחיד" אם מפרקים אותו נכון לשלבים. המסקנה שלי היא שצלילה לעומק (Deep Dive) לתוך טכנולוגיה חדשה היא הדרך הטובה ביותר ללמוד, ושאסור לפחד משינויים באמצע הדרך.

פיתוח עתידי ושימוש במשאבים נוספים: בעתיד, ניתן לשדרג את המערכת על ידי הפיכת ה-JWT למנוהל בשרת עם Refresh Tokens לשיפור האבטחה, והוספת מנגנוני אימות דו-שלבי (FA2) ושחזור סיסמה. ברמת ה-UX, ניתן להוסיף מיון לקטגוריות. במידה והיו לי משאבים נוספים (כוח מחשוב ובסיסי נתונים רחבים), הייתי מטמיע מנגנון בדיקה מול רשימות ענק של סיסמאות מודלפות בזמן אמת, תוך שימוש באלגוריתמים מתקדמים לגזירת מפתחות (Key Derivation).

ביבליוגרפיה

Gemini AI Language Model

- Google DeepMind
- <https://deepmind.google/technologies/gemini/>

Claude AI Language Model

- Anthropic
- <https://www.anthropic.com/claude>

Have I Been Pwned API - Pwned Passwords Documentation

- Troy Hunt
- <https://haveibeenpwned.com/API/v3#PwnedPasswords>

Azure Functions Documentation

- Microsoft Azure
- <https://learn.microsoft.com/en-us/azure/azure-functions/>

Azure SQL Database Documentation

- Microsoft Azure
- <https://learn.microsoft.com/en-us/azure/azure-sql/database/>

Xamarin.Essentials: SecureStorage

- Google Developers (Android)
- <https://learn.microsoft.com/en-us/xamarin/essentials/secure-storage>

Verizon Data Breach Investigations Report (DBIR)

- Verizon Business
- <https://www.verizon.com/business/resources/reports/dbir/>

נספחים

```

--- FILE: FinalProject333057891\SecurioBackendFunction\Helpers\HibpService.cs ---
namespace SecurioBackendFunction.Helpers
{
    /// <summary>Checks passwords against the Have I Been Pwned Pwned Passwords API
    using the k-anonymity model.</summary>
    public class HibpService : IHibpService
    {
        private readonly HttpClient _http;
        private const string ApiBase = "https://api.pwnedpasswords.com/range/";

        /// <summary>Initializes a new instance of HibpService.</summary>
        public HibpService(HttpClient http) => _http = http;

        /// <summary>Returns true if the given SHA-1 hex hash appears in the HIBP
        dataset.</summary>
        public async Task<bool> IsPasswordPwnedAsync(string sha1Hash)
        {
            if (string.IsNullOrEmpty(sha1Hash) || sha1Hash.Length != 40)
                return false;

            string prefix = sha1Hash[..5].ToUpperInvariant();
            string suffix = sha1Hash[5..].ToUpperInvariant();

            try
            {
                string body = await _http.GetStringAsync(ApiBase + prefix);

                // Each line in the response is "SUFFIX: COUNT" (CRLF-terminated).
                foreach (string line in body.Split(new[] { "\r\n", "\n" },
                    StringSplitOptions.RemoveEmptyEntries))
                {
                    int colon = line.IndexOf(':');
                    if (colon < 0) continue;
                    if (string.Equals(line[..colon].Trim(), suffix, StringComparison.OrdinalIgnoreCase))
                        return true;
                }
            }
        }
    }
}

```

```

        return false;
    }
    catch
    {
        // Fail open: if the HIBP service is unavailable, allow the operation to proceed.
        return false;
    }
}
}
}

```

```

--- FILE: FinalProject333057891\SecurioBackendFunction\Helpers\IHibpService.cs ---
namespace SecurioBackendFunction.Helpers
{
    /// <summary>Defines the contract for checking passwords against the Have I Been Pwned
    dataset.</summary>
    public interface IHibpService
    {
        /// <summary>Returns true if the provided SHA-1 hash appears in the HIBP Pwned
        Passwords dataset.</summary>
        Task<bool> IsPasswordPwnedAsync(string sha1Hash);
    }
}

```

```

--- FILE: FinalProject333057891\SecurioBackendFunction\Helpers\JwtHelper.cs ---
using System;
using System.Collections.Generic;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
using Microsoft.IdentityModel.Tokens;

namespace SecurioBackendFunction.Helpers
{
    /// <summary>A utility for creating and validating cryptographically signed JSON Web
    Tokens.</summary>
    public static class JwtHelper
    {

```

```

{
    private static readonly string Secret = GetSecret();

    /// <summary>Reads the JwtSecret environment variable and throws if it is not
configured.</summary>
    private static string GetSecret()
    {
        var secret = Environment.GetEnvironmentVariable("JwtSecret");
        if (string.IsNullOrEmpty(secret))
            throw new InvalidOperationException("JwtSecret environment variable is not
configured.");
        return secret;
    }

    /// <summary>Creates a signed JWT string for a specific user.</summary>
    public static string GenerateJwtToken(int userId, string username)
    {
        var securityKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(Secret));
        var credentials = new SigningCredentials(securityKey,
SecurityAlgorithms.HmacSha256);

        var claims = new[]
        {
            new Claim("user_id", userId.ToString()),
            new Claim("username", username),
            new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
        };

        var token = new JwtSecurityToken(
            claims: claims,
            expires: DateTime.UtcNow.AddHours(2),
            signingCredentials: credentials);

        return new JwtSecurityTokenHandler().WriteToken(token);
    }

    /// <summary>Validates a JWT token and returns the ClaimsPrincipal if valid, or null if
invalid or expired.</summary>
    public static ClaimsPrincipal ValidateToken(string token)

```

```

{
    if (string.IsNullOrEmpty(token))
        return null;

    try
    {
        var tokenHandler = new JwtSecurityTokenHandler();
        var key = Encoding.UTF8.GetBytes(Secret);

        var validationParameters = new TokenValidationParameters
        {
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(key),
            ValidateIssuer = false,
            ValidateAudience = false,
            ValidateLifetime = true,
            ClockSkew = TimeSpan.Zero
        };

        return tokenHandler.ValidateToken(token, validationParameters, out _);
    }
    catch
    {
        return null;
    }
}

/// <summary>Extracts the user_id claim from a validated ClaimsPrincipal; returns 0 if
extraction fails.</summary>
public static int GetUserIdFromPrincipal(ClaimsPrincipal principal)
{
    var claim = principal?.FindFirst("user_id");
    return int.TryParse(claim?.Value, out int userId) ? userId : 0;
}
}

```

--- FILE: FinalProject333057891\SecurioBackendFunction\Logic\AuthManager.cs ---

```

using SecurioBackendFunction.Helpers;
using SecurioBackendFunction.Repositories;
using SecurioModels;
using SecurioModels.DataTransferObjects;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SecurioBackendFunction.Logic
{
    /// <summary>Manages authentication logic for user registration and login.</summary>
    public class AuthManager
    {
        private readonly IUserRepository _repo;
        private readonly IHibpService _hibp;

        /// <summary>Initializes a new instance of AuthManager.</summary>
        public AuthManager(IUserRepository repo, IHibpService hibp)
        {
            _repo = repo;
            _hibp = hibp;
        }

        /// <summary>Registers user and generates a token immediately for a seamless UI
        transition.</summary>
        public async Task<ServerResponse<AuthData>> RegisterAsync(User user)
        {
            if (!string.IsNullOrEmpty(user.PasswordSha1Hash))
            {
                bool isPwned = await _hibp.IsPasswordPwnedAsync(user.PasswordSha1Hash);
                if (isPwned)
                {
                    return new ServerResponse<AuthData>
                    {
                        Success = false,
                        Message = "Password has been found in a data breach. Please choose a different
password."
                    };
                }
            }
        }
    }
}

```



```

    }

    if (await _repo.EmailExistsAsync(user.Email))
        return new ServerResponse<AuthData> { Success = false, Message = "Email already
registered." };

    int newId = await _repo.CreateUserAsync(user);
    if (newId <= 0) return new ServerResponse<AuthData> { Success = false, Message =
"Database error." };

    string token = Helpers.JwtHelper.GenerateJwtToken(newId, user.Username);
    return new ServerResponse<AuthData>
    {
        Success = true,
        Message = "User registered successfully",
        Data = new AuthData
        {
            UserId = newId,
            Username = user.Username,
            Token = token
        }
    };
}

/// <summary>Validates login credentials and returns a session token.</summary>
public async Task<ServerResponse<AuthData>> VerifyLoginAsync(string email, string
key)
{
    var user = await _repo.GetUserByEmailAsync(email);
    if (user == null || user.MasterPasswordKey != key)
        return new ServerResponse<AuthData> { Success = false, Message = "Invalid email
or password." };

    string token = Helpers.JwtHelper.GenerateJwtToken(user.Id, user.Username);
    await _repo.UpdateLastLoginAsync(user.Id);
    return new ServerResponse<AuthData>
    {
        Success = true,
        Message = "Login successful",
    }
}

```

```

        Data = new AuthData
        {
            Token = token,
            UserId = user.Id,
            Username = user.Username
        }
    };
}

/// <summary>Retrieves the user's salts for the login process.</summary>
public async Task<ServerResponse<SaltData>> GetUserSaltsAsync(string email)
{
    var user = await _repo.GetUserByEmailAsync(email);
    if (user == null)
        return new ServerResponse<SaltData> { Success = false, Message = "User not found." };

    return new ServerResponse<SaltData>
    {
        Success = true,
        Message = "Salts retrieved successfully",
        Data = new SaltData
        {
            AuthSalt = user.AuthSalt,
            EncryptionSalt = user.EncryptionSalt
        }
    };
}
}
}

```

--- FILE: FinalProject333057891\SecurioBackendFunction\Logic\PasswordCheckManager.cs

```

using SecurioBackendFunction.Helpers;
using SecurioBackendFunction.Repositories;
using SecurioModels;
using SecurioModels.DataTransferObjects;

```

```

using System;
using System.Linq;
using System.Threading.Tasks;

namespace SecurioBackendFunction.Logic
{
    /// <summary>Computes a password-health summary for a specific user without requiring
    an active session.</summary>
    public class PasswordCheckManager
    {
        // Passwords (and the master password) not changed within this many days are flagged as
        old.
        private const int OldPasswordDays = 90;

        private readonly IUserRepository _userRepo;
        private readonly IVaultItemRepository _vaultRepo;

        /// <summary>Initializes the manager with the required repository
        dependencies.</summary>
        public PasswordCheckManager(IUserRepository userRepo, IVaultItemRepository
        vaultRepo)
        {
            _userRepo = userRepo;
            _vaultRepo = vaultRepo;
        }

        /// <summary>Returns a PasswordCheckResult for the given user, or null when the user is
        not found.</summary>
        public async Task<PasswordCheckResult?> GetPasswordCheckAsync(int userId)
        {
            var user = await _userRepo.GetUserProfileAsync(userId);
            if (user == null)
                return null;

            // 1. Load all vault items for the user.
            var items = await _vaultRepo.GetVaultItemsByUserIdAsync(userId);

            int breachedCount = items.Count(i => i.IsLeaked);

```

```

        int oldCount    = items.Count(i => (DateTime.UtcNow - i.LastUpdate).TotalDays >
OldPasswordDays);
        bool masterOld  = (DateTime.UtcNow - user.LastPasswordUpdate).TotalDays >
OldPasswordDays;

        return new PasswordCheckResult
        {
            BreachedCount    = breachedCount,
            OldCount         = oldCount,
            MasterPasswordOld = masterOld
        };
    }
}

```

--- FILE: FinalProject333057891\SecurioBackendFunction\Logic\UserManager.cs ---

```

using SecurioBackendFunction.Helpers;
using SecurioBackendFunction.Repositories;
using SecurioModels;
using SecurioModels.DataTransferObjects;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace SecurioBackendFunction.Logic
{
    /// <summary>Coordinates the retrieval of user-specific data and account
statistics.</summary>
    public class UserManager
    {
        private readonly IUserRepository _repo;
        private readonly IVaultItemRepository _vaultRepo;
        private readonly IHibpService _hibp;

        /// <summary>Initializes a new instance of UserManager.</summary>
        public UserManager(IUserRepository repo, IVaultItemRepository vaultRepo,
IHibpService hibp)

```

```

{
    _repo = repo;
    _vaultRepo = vaultRepo;
    _hibp = hibp;
}

/// <summary>Fetches the profile and wraps it in a standard response for the
API.</summary>
public async Task<ServerResponse<User>> GetProfileAsync(int userId)
{
    var user = await _repo.GetUserProfileAsync(userId);

    if (user == null)
    {
        return new ServerResponse<User> { Success = false, Message = "Profile not found."
};
    }

    return new ServerResponse<User>
    {
        Success = true,
        Data = user
    };
}

/// <summary>Updates the user's account details including username, email, and
optionally master password.</summary>
public async Task<ServerResponse<object>> UpdateUserAsync(int userId, User
updated, bool passwordChanged, List<VaultItem> reEncryptedItems = null)
{
    if (string.IsNullOrEmpty(updated.Username))
        return new ServerResponse<object> { Success = false, Message = "Username is
required." };

    if (string.IsNullOrEmpty(updated.Email))
        return new ServerResponse<object> { Success = false, Message = "Email is
required." };

    // Check for email uniqueness (excluding the current user)

```

```

    bool emailTaken = await _repo.EmailExistsForOtherUserAsync(updated.Email,
userId);
    if (emailTaken)
        return new ServerResponse<object> { Success = false, Message = "Email is already
in use by another account." };

    User oldUser = null;
    if (passwordChanged)
    {
        if (string.IsNullOrEmpty(updated.MasterPasswordKey))
            return new ServerResponse<object> { Success = false, Message =
"MasterPasswordKey is required when changing password." };

        if (string.IsNullOrEmpty(updated.AuthSalt))
            return new ServerResponse<object> { Success = false, Message = "AuthSalt is
required when changing password." };

        if (string.IsNullOrEmpty(updated.EncryptionSalt))
            return new ServerResponse<object> { Success = false, Message = "EncryptionSalt
is required when changing password." };

        // HIBP breach check — same pattern as registration.
        if (!string.IsNullOrEmpty(updated.PasswordSha1Hash))
        {
            bool isPwned = await
_hibp.IsPasswordPwnedAsync(updated.PasswordSha1Hash);
            if (isPwned)
                return new ServerResponse<object>
                {
                    Success = false,
                    Message = "Password has been found in a data breach. Please choose a
different password."
                };
        }

        // Fetch the current user record so we can archive the old password key before
overwriting it.
        oldUser = await _repo.GetUserByIdAsync(userId);
        if (oldUser == null)

```

```

        return new ServerResponse<object> { Success = false, Message = "Account not
found." };
    }

    updated.Id = userId;
    bool success = await _repo.UpdateUserAsync(updated, passwordChanged);

    if (!success)
        return new ServerResponse<object> { Success = false, Message = "Account not
found." };

    // When the password changed, save the old key to MasterPasswordHistory so the
    // no-reuse check can compare against it later.
    if (passwordChanged && oldUser != null)
    {
        // Use LastPasswordUpdate as the archived timestamp so history entries can be
        // sorted chronologically. If the field is missing (e.g. a legacy account that
        // pre-dates the LastPasswordUpdate column), fall back to the current time so
        // the INSERT always has a valid date.
        DateTime archivedAt = oldUser.LastPasswordUpdate != DateTime.MinValue
            ? oldUser.LastPasswordUpdate
            : DateTime.UtcNow;
        await _repo.AddPasswordHistoryAsync(userId, oldUser.MasterPasswordKey,
oldUser.AuthSalt, archivedAt);
    }

    // Bulk-update re-encrypted vault items when the master password changes
    if (passwordChanged && reEncryptedItems != null && reEncryptedItems.Count > 0)
    {
        bool vaultUpdated = await
_vaultRepo.BulkUpdateVaultItemsAsync(reEncryptedItems, userId);
        if (!vaultUpdated)
            return new ServerResponse<object> { Success = false, Message = "Account
updated but vault re-encryption failed." };
    }

    return new ServerResponse<object> { Success = true, Message = "Account updated
successfully." };
}

```

```

    /// <summary>Deletes the user account and all associated vault items via
    CASCADE.</summary>
    public async Task<ServerResponse<object>> DeleteUserAsync(int userId)
    {
        bool deleted = await _repo.DeleteUserAsync(userId);

        if (!deleted)
        {
            return new ServerResponse<object> { Success = false, Message = "Account not
found." };
        }

        return new ServerResponse<object> { Success = true, Message = "Account deleted
successfully." };
    }

    /// <summary>Returns the last 4 password-history entries for the user.</summary>
    public async Task<ServerResponse<List<MasterPasswordHistory>>>
GetPasswordHistoryAsync(int userId)
    {
        var history = await _repo.GetLastPasswordHistoryAsync(userId, 4);
        return new ServerResponse<List<MasterPasswordHistory>> { Success = true, Data =
history };
    }
}

```

--- FILE: FinalProject333057891\SecurioBackendFunction\Logic\VaultItemManager.cs ---

```

using SecurioBackendFunction.Helpers;
using SecurioBackendFunction.Repositories;
using SecurioModels;
using SecurioModels.DataTransferObjects;
using System;
using System.Collections.Generic;
using System.Threading.Tasks;

```

```

namespace SecurioBackendFunction.Logic

```



```
{
    /// <summary>Coordinates the storage of encrypted vault items for a given
user.</summary>
    public class VaultItemManager
    {
        private readonly IVaultItemRepository _repo;
        private readonly IHibpService _hibp;

        /// <summary>Initializes a new instance of VaultItemManager.</summary>
        public VaultItemManager(IVaultItemRepository repo, IHibpService hibp = null)
        {
            _repo = repo;
            _hibp = hibp;
        }

        /// <summary>Validates the incoming vault item and persists it to the
database.</summary>
        public async Task<ServerResponse<VaultItem>> AddVaultItemAsync(VaultItem item)
        {
            if (string.IsNullOrEmpty(item.AccountName))
                return new ServerResponse<VaultItem> { Success = false, Message = "Account
name is required." };

            if (string.IsNullOrEmpty(item.CipherText))
                return new ServerResponse<VaultItem> { Success = false, Message = "CipherText is
required." };

            if (string.IsNullOrEmpty(item.IV))
                return new ServerResponse<VaultItem> { Success = false, Message = "IV is
required." };

            if (string.IsNullOrEmpty(item.Tag))
                return new ServerResponse<VaultItem> { Success = false, Message = "Tag is
required." };

            if (string.IsNullOrEmpty(item.Sha1Hash))
                return new ServerResponse<VaultItem> { Success = false, Message = "Sha1Hash is
required." };
        }
    }
}
```

```
// Check HIBP and set the IsLeaked flag before persisting.
if (_hibp != null)
    item.IsLeaked = await _hibp.IsPasswordPwnedAsync(item.Sha1Hash);

int newId = await _repo.AddVaultItemAsync(item);
if (newId <= 0)
    return new ServerResponse<VaultItem> { Success = false, Message = "Database
error." };

item.Id = newId;
item.LastUpdate = DateTime.UtcNow;
return new ServerResponse<VaultItem>
{
    Success = true,
    Message = "Vault item added successfully.",
    Data = item
};
}

/// <summary>Validates the updated vault item and persists the changes to the
database.</summary>
public async Task<ServerResponse<VaultItem>> UpdateVaultItemAsync(VaultItem
item)
{
    if (item.Id <= 0)
        return new ServerResponse<VaultItem> { Success = false, Message = "Item ID is
required." };

    if (string.IsNullOrEmpty(item.AccountName))
        return new ServerResponse<VaultItem> { Success = false, Message = "Account
name is required." };

    if (string.IsNullOrEmpty(item.CipherText))
        return new ServerResponse<VaultItem> { Success = false, Message = "CipherText is
required." };

    if (string.IsNullOrEmpty(item.IV))
        return new ServerResponse<VaultItem> { Success = false, Message = "IV is
required." };
}
```

```

        if (string.IsNullOrEmpty(item.Tag))
            return new ServerResponse<VaultItem> { Success = false, Message = "Tag is
required." };

        if (string.IsNullOrEmpty(item.Sha1Hash))
            return new ServerResponse<VaultItem> { Success = false, Message = "Sha1Hash is
required." };

        // When the password was changed, re-check HIBP to update the leaked status.
        if (item.PasswordChanged && _hibp != null)
            item.IsLeaked = await _hibp.IsPasswordPwnedAsync(item.Sha1Hash);

        bool updated = await _repo.UpdateVaultItemAsync(item);
        if (!updated)
            return new ServerResponse<VaultItem> { Success = false, Message = "Item not
found or access denied." };

        if (item.PasswordChanged)
            item.LastUpdate = DateTime.UtcNow;
        return new ServerResponse<VaultItem>
        {
            Success = true,
            Message = "Vault item updated successfully.",
            Data = item
        };
    }

    /// <summary>Retrieves all vault items for the specified user.</summary>
    public async Task<ServerResponse<List<VaultItem>>> GetVaultItemsAsync(int userId)
    {
        if (userId <= 0)
            return new ServerResponse<List<VaultItem>> { Success = false, Message =
"Invalid user ID." };

        var items = await _repo.GetVaultItemsByUserIdAsync(userId);
        return new ServerResponse<List<VaultItem>>
        {
            Success = true,

```

```

        Message = "Vault items retrieved successfully.",
        Data = items
    };
}

/// <summary>Validates the request and permanently deletes the specified vault item
from the database.</summary>
public async Task<ServerResponse<object>> DeleteVaultItemAsync(int itemId, int
userId)
{
    if (itemId <= 0)
        return new ServerResponse<object> { Success = false, Message = "Item ID is
required." };

    if (userId <= 0)
        return new ServerResponse<object> { Success = false, Message = "Invalid user ID."
};

    bool deleted = await _repo.DeleteVaultItemAsync(itemId, userId);
    if (!deleted)
        return new ServerResponse<object> { Success = false, Message = "Item not found or
access denied." };

    return new ServerResponse<object> { Success = true, Message = "Vault item deleted
successfully." };
}
}
}

```

--- FILE: FinalProject333057891\SecurioBackendFunction\Repositories\IUserRepository.cs --
-

```

using SecurioModels.DataTransferObjects;
using System.Collections.Generic;
using System.Threading.Tasks;

```

```

namespace SecurioBackendFunction.Repositories
{

```

```

    /// <summary>Defines the contract for all user-related database operations.</summary>

```

```

public interface IUserRepository
{
    /// <summary>Checks whether the specified email address is already in use.</summary>
    Task<bool> EmailExistsAsync(string email);
    /// <summary>Checks whether the email is used by a different user than the excluded
one.</summary>
    Task<bool> EmailExistsForOtherUserAsync(string email, int excludeUserId);
    /// <summary>Inserts a new user record and returns the newly generated ID.</summary>
    Task<int> CreateUserAsync(User user);
    /// <summary>Retrieves a user record by email.</summary>
    Task<User> GetUserByEmailAsync(string email);
    /// <summary>Retrieves a full user record by user ID.</summary>
    Task<User> GetUserByIdAsync(int userId);
    /// <summary>Retrieves the profile for the given user.</summary>
    Task<User> GetUserProfileAsync(int userId);
    /// <summary>Updates the LastLogin timestamp for the given user.</summary>
    Task UpdateLastLoginAsync(int userId);
    /// <summary>Updates the user's profile fields.</summary>
    Task<bool> UpdateUserAsync(User user, bool passwordChanged);
    /// <summary>Deletes the user account.</summary>
    Task<bool> DeleteUserAsync(int userId);
    /// <summary>Returns the most-recent password history entries for the given
user.</summary>
    Task<List<MasterPasswordHistory>> GetLastPasswordHistoryAsync(int userId, int
count);
    /// <summary>Inserts an entry into MasterPasswordHistory.</summary>
    Task AddPasswordHistoryAsync(int userId, string passwordKey, string authSalt,
System.DateTime createdAt);
}
}

```

--- FILE:

FinalProject333057891\SecurioBackendFunction\Repositories\IVaultItemRepository.cs ---

```

using SecurioModels.DataTransferObjects;
using System.Collections.Generic;
using System.Threading.Tasks;

```

```

namespace SecurioBackendFunction.Repositories

```

```
{
    /// <summary>Defines the contract for all vault-item-related database
operations.</summary>
    public interface IVaultItemRepository
    {
        /// <summary>Inserts a new vault item and returns the newly generated ID.</summary>
        Task<int> AddVaultItemAsync(VaultItem item);
        /// <summary>Updates an existing vault item.</summary>
        Task<bool> UpdateVaultItemAsync(VaultItem item);
        /// <summary>Retrieves all vault items for the specified user.</summary>
        Task<List<VaultItem>> GetVaultItemsByUserIdAsync(int userId);
        /// <summary>Deletes a vault item by ID.</summary>
        Task<bool> DeleteVaultItemAsync(int itemId, int userId);
        /// <summary>Bulk-updates the encryption fields for all vault items belonging to the
given user.</summary>
        Task<bool> BulkUpdateVaultItemsAsync(List<VaultItem> items, int userId);
    }
}
```

--- FILE: FinalProject333057891\SecurioBackendFunction\Repositories\UserRepository.cs ---

```
using Microsoft.Data.SqlClient;
using SecurioModels.DataTransferObjects;
using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
```

```
/*
 * SQL migration — run once against the target database to create the history table:
 *
 * CREATE TABLE MasterPasswordHistory (
 *     Id      INT      IDENTITY(1,1) PRIMARY KEY,
 *     UserId  INT      NOT NULL,
 *     PasswordKey NVARCHAR(MAX) NOT NULL,
 *     AuthSalt NVARCHAR(MAX) NOT NULL,
 *     CreatedAt DATETIME NOT NULL,
```

```
* CONSTRAINT FK_MasterPasswordHistory_Users
* FOREIGN KEY (UserId) REFERENCES Users(Id) ON DELETE CASCADE
* );
*/
```

```
namespace SecurioBackendFunction.Repositories
{
    /// <summary>Manages all direct SQL database interactions for user-related
data.</summary>
    public class UserRepository : IUserRepository
    {
        private readonly string _connectionString;
        /// <summary>Initializes a new instance of UserRepository.</summary>
        public UserRepository(string connectionString) => _connectionString =
connectionString;

        /// <summary>Checks the database to see if a specific email address is already in
use.</summary>
        public async Task<bool> EmailExistsAsync(string email)
        {
            using var conn = new SqlConnection(_connectionString);
            await conn.OpenAsync();
            var sql = "SELECT COUNT(1) FROM Users WHERE Email = @email";
            using var cmd = new SqlCommand(sql, conn);
            cmd.Parameters.Add("@email", SqlDbType.NVarChar).Value = email;
            return (int)await cmd.ExecuteScalarAsync() > 0;
        }

        /// <summary>Inserts a new user record and returns the newly generated ID.</summary>
        public async Task<int> CreateUserAsync(User user)
        {
            using var conn = new SqlConnection(_connectionString);
            await conn.OpenAsync();
            var sql = @"INSERT INTO Users (Username, Email, MasterPasswordKey, AuthSalt,
EncryptionSalt, LastLogin, LastPasswordUpdate, CreatedAt)
                OUTPUT INSERTED.Id
                VALUES (@name, @email, @key, @asalt, @esalt, GETDATE(), GETDATE(),
GETDATE())";
            using var cmd = new SqlCommand(sql, conn);
```

```

cmd.Parameters.Add("@name", SqlDbType.NVarChar).Value = user.Username;
cmd.Parameters.Add("@email", SqlDbType.NVarChar).Value = user.Email;
cmd.Parameters.Add("@key", SqlDbType.NVarChar).Value =
user.MasterPasswordKey;
cmd.Parameters.Add("@asalt", SqlDbType.NVarChar).Value = user.AuthSalt;
cmd.Parameters.Add("@esalt", SqlDbType.NVarChar).Value = user.EncryptionSalt;
return (int)await cmd.ExecuteScalarAsync();
}

/// <summary>Retrieves a user record by email for authentication purposes.</summary>
public async Task<User> GetUserByEmailAsync(string email)
{
    using var conn = new SqlConnection(_connectionString);
    await conn.OpenAsync();
    var sql = @"SELECT Id, Username, Email, MasterPasswordKey, AuthSalt,
EncryptionSalt,
                LastLogin, LastPasswordUpdate, CreatedAt
                FROM Users WHERE Email = @email";
    using var cmd = new SqlCommand(sql, conn);
    cmd.Parameters.Add("@email", SqlDbType.NVarChar).Value = email;
    using var reader = await cmd.ExecuteReaderAsync();
    if (await reader.ReadAsync())
    {
        return new User
        {
            Id = (int)reader["Id"],
            Username = reader["Username"].ToString(),
            Email = reader["Email"].ToString(),
            MasterPasswordKey = reader["MasterPasswordKey"].ToString(),
            AuthSalt = reader["AuthSalt"].ToString(),
            EncryptionSalt = reader["EncryptionSalt"].ToString(),
            LastLogin = reader["LastLogin"] != DBNull.Value ?
(DateTime)reader["LastLogin"] : DateTime.MinValue,
            LastPasswordUpdate = reader["LastPasswordUpdate"] != DBNull.Value ?
(DateTime)reader["LastPasswordUpdate"] : DateTime.MinValue,
            CreatedAt = reader["CreatedAt"] != DBNull.Value ?
(DateTime)reader["CreatedAt"] : DateTime.MinValue
        };
    }
}

```



```

        return null;
    }

    /// <summary>Retrieves the fully populated profile for the given user.</summary>
    public async Task<User> GetUserProfileAsync(int userId)
    {
        using var conn = new SqlConnection(_connectionString);
        await conn.OpenAsync();

        var sql = @"SELECT Id, Username, Email, CreatedAt, LastLogin,
LastPasswordUpdate,
        (SELECT COUNT(*) FROM VaultItems WHERE UserId = @uid) AS
PasswordCount
        FROM Users WHERE Id = @uid";

        using var cmd = new SqlCommand(sql, conn);
        cmd.Parameters.Add("@uid", SqlDbType.Int).Value = userId;

        using var reader = await cmd.ExecuteReaderAsync();
        if (await reader.ReadAsync())
        {
            return new User
            {
                Id = (int)reader["Id"],
                Username = reader["Username"].ToString(),
                Email = reader["Email"].ToString(),
                CreatedAt = reader["CreatedAt"] != DBNull.Value ?
(DateTime)reader["CreatedAt"] : DateTime.MinValue,
                LastLogin = reader["LastLogin"] != DBNull.Value ?
(DateTime)reader["LastLogin"] : DateTime.MinValue,
                LastPasswordUpdate = reader["LastPasswordUpdate"] != DBNull.Value ?
(DateTime)reader["LastPasswordUpdate"] : DateTime.MinValue,
                PasswordCount = (int)reader["PasswordCount"]
            };
        }
        return null;
    }

```

/// <summary>Updates the LastLogin timestamp for the given user to the current UTC time.</summary>

```
public async Task UpdateLastLoginAsync(int userId)
{
    using var conn = new SqlConnection(_connectionString);
    await conn.OpenAsync();
    var sql = "UPDATE Users SET LastLogin = GETDATE() WHERE Id = @uid";
    using var cmd = new SqlCommand(sql, conn);
    cmd.Parameters.Add("@uid", SqlDbType.Int).Value = userId;
    await cmd.ExecuteNonQuery();
}
```

/// <summary>Checks whether the email is already used by a different user.</summary>
public async Task<bool> EmailExistsForOtherUserAsync(string email, int excludeUserId)

```
{
    using var conn = new SqlConnection(_connectionString);
    await conn.OpenAsync();
    var sql = "SELECT COUNT(1) FROM Users WHERE Email = @email AND Id != @uid";
    using var cmd = new SqlCommand(sql, conn);
    cmd.Parameters.Add("@email", SqlDbType.NVarChar).Value = email;
    cmd.Parameters.Add("@uid", SqlDbType.Int).Value = excludeUserId;
    return (int)await cmd.ExecuteScalarAsync() > 0;
}
```

/// <summary>Updates the user's profile fields, including password fields when passwordChanged is true.</summary>

```
public async Task<bool> UpdateUserAsync(User user, bool passwordChanged)
{
    using var conn = new SqlConnection(_connectionString);
    await conn.OpenAsync();

    string sql;
    if (passwordChanged)
    {
        sql = @"UPDATE Users
                SET Username = @name, Email = @email,
                    MasterPasswordKey = @key, AuthSalt = @asalt, EncryptionSalt = @esalt,
```

```

        LastPasswordUpdate = GETDATE()
        WHERE Id = @uid";
    }
    else
    {
        sql = @"UPDATE Users
            SET Username = @name, Email = @email
            WHERE Id = @uid";
    }

    using var cmd = new SqlCommand(sql, conn);
    cmd.Parameters.Add("@uid", SqlDbType.Int).Value = user.Id;
    cmd.Parameters.Add("@name", SqlDbType.NVarChar).Value = user.Username;
    cmd.Parameters.Add("@email", SqlDbType.NVarChar).Value = user.Email;

    if (passwordChanged)
    {
        cmd.Parameters.Add("@key", SqlDbType.NVarChar).Value =
user.MasterPasswordKey;
        cmd.Parameters.Add("@asalt", SqlDbType.NVarChar).Value = user.AuthSalt;
        cmd.Parameters.Add("@esalt", SqlDbType.NVarChar).Value = user.EncryptionSalt;
    }

    int rows = await cmd.ExecuteNonQueryAsync();
    return rows > 0;
}

/// <summary>Deletes the user account; CASCADE removes all associated vault items
automatically.</summary>
public async Task<bool> DeleteUserAsync(int userId)
{
    using var conn = new SqlConnection(_connectionString);
    await conn.OpenAsync();
    var sql = "DELETE FROM Users WHERE Id = @uid";
    using var cmd = new SqlCommand(sql, conn);
    cmd.Parameters.Add("@uid", SqlDbType.Int).Value = userId;
    int rows = await cmd.ExecuteNonQueryAsync();
    return rows > 0;
}

```

/// <summary>Retrieves a full user record including password key and salts by user ID.</summary>

```
public async Task<User> GetUserByIdAsync(int userId)
{
    using var conn = new SqlConnection(_connectionString);
    await conn.OpenAsync();
    var sql = @"SELECT Id, Username, Email, MasterPasswordKey, AuthSalt,
EncryptionSalt,
                LastLogin, LastPasswordUpdate, CreatedAt
                FROM Users WHERE Id = @uid";
    using var cmd = new SqlCommand(sql, conn);
    cmd.Parameters.Add("@uid", SqlDbType.Int).Value = userId;
    using var reader = await cmd.ExecuteReaderAsync();
    if (await reader.ReadAsync())
    {
        return new User
        {
            Id = (int)reader["Id"],
            Username = reader["Username"].ToString(),
            Email = reader["Email"].ToString(),
            MasterPasswordKey = reader["MasterPasswordKey"].ToString(),
            AuthSalt = reader["AuthSalt"].ToString(),
            EncryptionSalt = reader["EncryptionSalt"].ToString(),
            LastLogin = reader["LastLogin"] != DBNull.Value ?
(DateTime)reader["LastLogin"] : DateTime.MinValue,
            LastPasswordUpdate = reader["LastPasswordUpdate"] != DBNull.Value ?
(DateTime)reader["LastPasswordUpdate"] : DateTime.MinValue,
            CreatedAt = reader["CreatedAt"] != DBNull.Value ?
(DateTime)reader["CreatedAt"] : DateTime.MinValue
        };
    }
    return null;
}
```

/// <summary>Returns the most-recent password history entries for the given user, newest first.</summary>

```
public async Task<List<MasterPasswordHistory>> GetLastPasswordHistoryAsync(int
userId, int count)
```

```

{
    using var conn = new SqlConnection(_connectionString);
    await conn.OpenAsync();
    var sql = @"SELECT TOP (@count) Id, UserId, PasswordKey, AuthSalt, CreatedAt
                FROM MasterPasswordHistory
                WHERE UserId = @uid
                ORDER BY CreatedAt DESC";
    using var cmd = new SqlCommand(sql, conn);
    cmd.Parameters.Add("@count", SqlDbType.Int).Value = count;
    cmd.Parameters.Add("@uid", SqlDbType.Int).Value = userId;
    using var reader = await cmd.ExecuteReaderAsync();
    var result = new List<MasterPasswordHistory>();
    while (await reader.ReadAsync())
    {
        result.Add(new MasterPasswordHistory
        {
            Id = (int)reader["Id"],
            UserId = (int)reader["UserId"],
            PasswordKey = reader["PasswordKey"].ToString(),
            AuthSalt = reader["AuthSalt"].ToString(),
            CreatedAt = reader["CreatedAt"] != DBNull.Value ?
(DateTime)reader["CreatedAt"] : DateTime.MinValue
        });
    }
    return result;
}

/// <summary>Inserts an entry into MasterPasswordHistory to record a password that is
no longer active.</summary>
public async Task AddPasswordHistoryAsync(int userId, string passwordKey, string
authSalt, DateTime createdAt)
{
    using var conn = new SqlConnection(_connectionString);
    await conn.OpenAsync();
    var sql = @"INSERT INTO MasterPasswordHistory (UserId, PasswordKey, AuthSalt,
CreatedAt)
                VALUES (@uid, @key, @salt, @date)";
    using var cmd = new SqlCommand(sql, conn);
    cmd.Parameters.Add("@uid", SqlDbType.Int).Value = userId;

```

```

        cmd.Parameters.Add("@key", SqlDbType.NVarChar).Value = passwordKey;
        cmd.Parameters.Add("@salt", SqlDbType.NVarChar).Value = authSalt;
        cmd.Parameters.Add("@date", SqlDbType.DateTime).Value = createdAt;
        await cmd.ExecuteNonQuery();
    }
}
}

--- FILE:
FinalProject333057891\SecurioBackendFunction\Repositories\VaultItemRepository.cs ---
using Microsoft.Data.SqlClient;
using SecurioModels.DataTransferObjects;
using System;
using System.Collections.Generic;
using System.Data;
using System.Threading.Tasks;

namespace SecurioBackendFunction.Repositories
{
    /// <summary>Manages all direct SQL database interactions for vault-item
data.</summary>
    public class VaultItemRepository : IVaultItemRepository
    {
        private readonly string _connectionString;
        /// <summary>Initializes a new instance of VaultItemRepository.</summary>
        public VaultItemRepository(string connectionString) => _connectionString =
connectionString;

        /// <summary>Inserts a new vault item record and returns the newly generated
ID.</summary>
        public async Task<int> AddVaultItemAsync(VaultItem item)
        {
            using var conn = new SqlConnection(_connectionString);
            await conn.OpenAsync();
            var sql = @"INSERT INTO VaultItems (UserId, AccountName, AccountUsername,
IV, Tag, CipherText, Notes, Sha1Hash, IsLeaked, LastUpdate)
                OUTPUT INSERTED.Id

```

```
VALUES (@uid, @name, @uname, @iv, @tag, @cipher, @notes, @hash,
@leaked, GETDATE());
using var cmd = new SqlCommand(sql, conn);
cmd.Parameters.Add("@uid", SqlDbType.Int).Value = item.UserId;
cmd.Parameters.Add("@name", SqlDbType.NVarChar).Value = item.AccountName;
cmd.Parameters.Add("@uname", SqlDbType.NVarChar).Value =
(object)item.AccountUsername ?? DBNull.Value;
cmd.Parameters.Add("@iv", SqlDbType.NVarChar).Value = item.IV;
cmd.Parameters.Add("@tag", SqlDbType.NVarChar).Value = item.Tag;
cmd.Parameters.Add("@cipher", SqlDbType.NVarChar).Value = item.CipherText;
cmd.Parameters.Add("@notes", SqlDbType.NVarChar).Value = (object)item.Notes ??
DBNull.Value;
cmd.Parameters.Add("@hash", SqlDbType.NVarChar).Value = item.Sha1Hash;
cmd.Parameters.Add("@leaked", SqlDbType.Bit).Value = item.IsLeaked;
return (int)await cmd.ExecuteScalarAsync();
}
```

/// <summary>Updates an existing vault item, enforcing ownership via
UserId.</summary>

```
public async Task<bool> UpdateVaultItemAsync(VaultItem item)
{
    using var conn = new SqlConnection(_connectionString);
    await conn.OpenAsync();
    var sql = @"UPDATE VaultItems
        SET AccountName = @name,
            AccountUsername = @uname,
            IV = @iv,
            Tag = @tag,
            CipherText = @cipher,
            Notes = @notes,
            Sha1Hash = @hash,
            IsLeaked = @leaked,
            LastUpdate = CASE WHEN @passwordChanged = 1 THEN GETDATE()
ELSE LastUpdate END
        WHERE Id = @id AND UserId = @uid";
    using var cmd = new SqlCommand(sql, conn);
    cmd.Parameters.Add("@id", SqlDbType.Int).Value = item.Id;
    cmd.Parameters.Add("@uid", SqlDbType.Int).Value = item.UserId;
    cmd.Parameters.Add("@name", SqlDbType.NVarChar).Value = item.AccountName;
```

```

        cmd.Parameters.Add("@uname", SqlDbType.NVarChar).Value =
(object)item.AccountUsername ?? DBNull.Value;
        cmd.Parameters.Add("@iv", SqlDbType.NVarChar).Value = item.IV;
        cmd.Parameters.Add("@tag", SqlDbType.NVarChar).Value = item.Tag;
        cmd.Parameters.Add("@cipher", SqlDbType.NVarChar).Value = item.CipherText;
        cmd.Parameters.Add("@notes", SqlDbType.NVarChar).Value = (object)item.Notes ??
DBNull.Value;
        cmd.Parameters.Add("@hash", SqlDbType.NVarChar).Value = item.Sha1Hash;
        cmd.Parameters.Add("@leaked", SqlDbType.Bit).Value = item.IsLeaked;
        cmd.Parameters.Add("@passwordChanged", SqlDbType.Bit).Value =
item.PasswordChanged;
        int rows = await cmd.ExecuteNonQueryAsync();
        return rows > 0;
    }

```

```

/// <summary>Retrieves all vault items belonging to a specific user.</summary>
public async Task<List<VaultItem>> GetVaultItemsByUserIdAsync(int userId)
{
    using var conn = new SqlConnection(_connectionString);
    await conn.OpenAsync();
    var sql = @"SELECT Id, UserId, AccountName, AccountUsername, IV, Tag,
CipherText, Notes, Sha1Hash, IsLeaked, LastUpdate
FROM VaultItems
WHERE UserId = @uid
ORDER BY LastUpdate DESC";
    using var cmd = new SqlCommand(sql, conn);
    cmd.Parameters.Add("@uid", SqlDbType.Int).Value = userId;

    var items = new List<VaultItem>();
    using var reader = await cmd.ExecuteReaderAsync();
    while (await reader.ReadAsync())
    {
        items.Add(new VaultItem
        {
            Id = reader.GetInt32(reader.GetOrdinal("Id")),
            UserId = reader.GetInt32(reader.GetOrdinal("UserId")),
            AccountName = reader.IsDBNull(reader.GetOrdinal("AccountName")) ? null :
reader.GetString(reader.GetOrdinal("AccountName")),

```



```

        AccountUsername = reader.IsDBNull(reader.GetOrdinal("AccountUsername")) ?
null : reader.GetString(reader.GetOrdinal("AccountUsername")),
        IV = reader.IsDBNull(reader.GetOrdinal("IV")) ? null :
reader.GetString(reader.GetOrdinal("IV")),
        Tag = reader.IsDBNull(reader.GetOrdinal("Tag")) ? null :
reader.GetString(reader.GetOrdinal("Tag")),
        CipherText = reader.IsDBNull(reader.GetOrdinal("CipherText")) ? null :
reader.GetString(reader.GetOrdinal("CipherText")),
        Notes = reader.IsDBNull(reader.GetOrdinal("Notes")) ? null :
reader.GetString(reader.GetOrdinal("Notes")),
        Sha1Hash = reader.IsDBNull(reader.GetOrdinal("Sha1Hash")) ? null :
reader.GetString(reader.GetOrdinal("Sha1Hash")),
        IsLeaked = reader.GetBoolean(reader.GetOrdinal("IsLeaked")),
        LastUpdate = reader.GetDateTime(reader.GetOrdinal("LastUpdate"))
    });
}
return items;
}

```

/// <summary>Deletes a vault item by ID, enforcing ownership via UserId.</summary>

```

public async Task<bool> DeleteVaultItemAsync(int itemId, int userId)
{
    using var conn = new SqlConnection(_connectionString);
    await conn.OpenAsync();
    var sql = "DELETE FROM VaultItems WHERE Id = @id AND UserId = @uid";
    using var cmd = new SqlCommand(sql, conn);
    cmd.Parameters.Add("@id", SqlDbType.Int).Value = itemId;
    cmd.Parameters.Add("@uid", SqlDbType.Int).Value = userId;
    int rows = await cmd.ExecuteNonQueryAsync();
    return rows > 0;
}

```

/// <summary>Bulk-updates the encryption fields for all vault items belonging to the given user.</summary>

```

public async Task<bool> BulkUpdateVaultItemsAsync(List<VaultItem> items, int
userId)
{
    if (items == null || items.Count == 0) return true;

```

```

using var conn = new SqlConnection(_connectionString);
await conn.OpenAsync();
using var transaction = conn.BeginTransaction();

try
{
    foreach (var item in items)
    {
        var sql = @"UPDATE VaultItems
                    SET IV = @iv, Tag = @tag, CipherText = @cipher
                    WHERE Id = @id AND UserId = @uid";
        using var cmd = new SqlCommand(sql, conn, transaction);
        cmd.Parameters.Add("@id", SqlDbType.Int).Value = item.Id;
        cmd.Parameters.Add("@uid", SqlDbType.Int).Value = userId;
        cmd.Parameters.Add("@iv", SqlDbType.NVarChar).Value = item.IV;
        cmd.Parameters.Add("@tag", SqlDbType.NVarChar).Value = item.Tag;
        cmd.Parameters.Add("@cipher", SqlDbType.NVarChar).Value =
item.CipherText;
        await cmd.ExecuteNonQueryAsync();
    }

    transaction.Commit();
    return true;
}
catch
{
    transaction.Rollback();
    return false;
}
}
}

```

--- FILE: FinalProject333057891\SecurioBackendFunction\ServerFunctions\AuthFunctions.cs

```

using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Azure.Functions.Worker;

```

```

using Microsoft.Data.SqlClient;
using Microsoft.Extensions.Logging;
using Newtonsoft.Json;
using SecurioBackendFunction.Helpers;
using SecurioBackendFunction.Logic;
using SecurioBackendFunction.Repositories;
using SecurioModels;
using SecurioModels.DataTransferObjects;
using System.Collections.Generic;
using System.Linq;
using static System.Runtime.InteropServices.JavaScript.JSType;

namespace SecurioBackendFunction.ServerFunctions;

/// <summary>Handles HTTP endpoints for user authentication.</summary>
public class AuthFunctions
{
    private readonly AuthManager _authManager;

    /// <summary>Initializes a new instance of AuthFunctions.</summary>
    public AuthFunctions(AuthManager authManager)
    {
        _authManager = authManager;
    }

    /// <summary>Handles the Register HTTP request and catches any unexpected
    errors.</summary>
    [Function("RegisterUser")]
    public async Task<IActionResult> Register([HttpTrigger(AuthorizationLevel.Function,
    "post")] HttpRequest req)
    {
        try
        {
            var signup = JsonConvert.DeserializeObject<User>(await new
StreamReader(req.Body).ReadToEndAsync());
            var result = await _authManager.RegisterAsync(signup);
            return result.Success ? new OkObjectResult(result) : new ConflictObjectResult(result);
        }
        catch (Exception ex)
    }

```

```

    {
        // This catch ensures the client ALWAYS gets a JSON BaseResponse, never a raw
        crash string.
        return new BadRequestObjectResult(new ServerResponse<object> { Success = false,
        Message = $"An internal error occurred. Error - {ex.Message} " });
    }
}

```

```

/// <summary>Handles the Login HTTP request and ensures a secure JSON
response.</summary>
[Function("VerifyLogin")]
public async Task<IActionResult> Login([HttpTrigger(AuthorizationLevel.Function,
"post")] HttpRequest req)
{
    try
    {
        var attempt = JsonConvert.DeserializeObject<User>(await new
StreamReader(req.Body).ReadToEndAsync());
        var result = await _authManager.VerifyLoginAsync(attempt.Email,
attempt.MasterPasswordKey);
        return result.Success ? new OkObjectResult(result) : new
UnauthorizedObjectResult(result);
    }
    catch (Exception ex)
    {
        return new BadRequestObjectResult(new ServerResponse<object> { Success = false,
        Message = "An internal error occurred." });
    }
}

```

```

/// <summary>Gets the user's salts for the login process.</summary>
[Function("GetSalts")]
public async Task<IActionResult> GetSalts([HttpTrigger(AuthorizationLevel.Function,
"post")] HttpRequest req)
{
    try
    {
        var request = JsonConvert.DeserializeObject<dynamic>(await new
StreamReader(req.Body).ReadToEndAsync());
    }
}

```

```

string email = request.Email;

var user = await _authManager.GetUserSaltsAsync(email);
return user.Success ? new OkObjectResult(user) : new NotFoundObjectResult(user);
}
catch (Exception ex)
{
    return new BadRequestObjectResult(new ServerResponse<object> { Success = false,
Message = "An internal error occurred." });
}
}

/// <summary>Validates an existing JWT token; returns 200 if valid and unexpired, 401
otherwise.</summary>
[Function("ValidateToken")]
public IActionResult ValidateToken([HttpTrigger(AuthorizationLevel.Function, "get")]
HttpRequest req)
{
    try
    {
        var authHeader = req.Headers["Authorization"].FirstOrDefault();
        if (string.IsNullOrEmpty(authHeader) || !authHeader.StartsWith("Bearer "))
            return new UnauthorizedObjectResult(new ServerResponse<object> { Success =
false, Message = "Unauthorized." });

        var token = authHeader.Substring("Bearer ".Length);
        var principal = JwtHelper.ValidateToken(token);

        if (principal == null)
            return new UnauthorizedObjectResult(new ServerResponse<object> { Success =
false, Message = "Token is invalid or expired." });

        int userId = JwtHelper.GetUserIdFromPrincipal(principal);
        return new OkObjectResult(new ServerResponse<object> { Success = true, Message =
"Token is valid.", Data = new { UserId = userId } });
    }
    catch (Exception)
    {

```

```

        return new BadRequestObjectResult(new ServerResponse<object> { Success = false,
        Message = "An internal error occurred." });
    }
}
}

```

--- FILE:

FinalProject333057891\SecurioBackendFunction\ServerFunctions\PasswordCheckFunctions.cs ---

```

using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Azure.Functions.Worker;
using Newtonsoft.Json;
using SecurioBackendFunction.Logic;
using SecurioModels;
using SecurioModels.DataTransferObjects;
using System;
using System.IO;
using System.Threading.Tasks;

namespace SecurioBackendFunction.ServerFunctions
{
    /// <summary>Exposes the password-health check as an unauthenticated HTTP
    endpoint.</summary>
    public class PasswordCheckFunctions
    {
        private readonly PasswordCheckManager _manager;

        /// <summary>Initializes a new instance of PasswordCheckFunctions.</summary>
        public PasswordCheckFunctions(PasswordCheckManager manager)
        {
            _manager = manager;
        }

        /// <summary>Runs the password-health check for the specified user.</summary>
        [Function("PasswordCheck")]
        public async Task<ActionResult> PasswordCheck(

```

```
[HttpTrigger(AuthorizationLevel.Function, "post")] HttpRequest req)
{
    try
    {
        var body = await new StreamReader(req.Body).ReadToEndAsync();
        var request = JsonConvert.DeserializeObject<PasswordCheckRequest>(body);

        if (request == null || request.UserId <= 0)
        {
            return new BadRequestObjectResult(new ServerResponse<PasswordCheckResult>
            {
                Success = false,
                Message = "UserId is required."
            });
        }

        var result = await _manager.GetPasswordCheckAsync(request.UserId);

        if (result == null)
        {
            return new NotFoundObjectResult(new ServerResponse<PasswordCheckResult>
            {
                Success = false,
                Message = "User not found."
            });
        }

        return new OkObjectResult(new ServerResponse<PasswordCheckResult>
        {
            Success = true,
            Data = result
        });
    }
    catch
    {
        return new BadRequestObjectResult(new ServerResponse<PasswordCheckResult>
        {
            Success = false,
            Message = "Error running password check."
        });
    }
}
```

```

        });
    }
}
}
}

```

--- FILE: FinalProject333057891\SecurioBackendFunction\ServerFunctions\UserFunctions.cs

```

using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Azure.Functions.Worker;
using Newtonsoft.Json;
using SecurioBackendFunction.Helpers;
using SecurioBackendFunction.Logic;
using SecurioModels;
using SecurioModels.DataTransferObjects;
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```

```

namespace SecurioBackendFunction.ServerFunctions

```

```

{
    /// <summary>Manages HTTP endpoints for non-authentication user data.</summary>
    public class UserFunctions
    {
        private readonly UserManager _userManager;

        /// <summary>Initializes a new instance of UserFunctions.</summary>
        public UserFunctions(UserManager userManager)
        {
            _userManager = userManager;
        }
    }
}

```

```

    /// <summary>Retrieves the user's statistics and profile details via a GET
    request.</summary>

```



```
[Function("GetProfile")]
public async Task<IActionResult> GetProfile(
    [HttpTrigger(AuthorizationLevel.Function, "get")] HttpRequest req)
{
    try
    {
        // Extract the Bearer token from the Authorization header
        var authHeader = req.Headers["Authorization"].FirstOrDefault();
        if (string.IsNullOrEmpty(authHeader) || !authHeader.StartsWith("Bearer "))
        {
            return new UnauthorizedObjectResult(new ServerResponse<User> { Success =
false, Message = "Unauthorized." });
        }

        var token = authHeader.Substring("Bearer ".Length);
        var principal = JwtHelper.ValidateToken(token);

        if (principal == null)
        {
            return new UnauthorizedObjectResult(new ServerResponse<User> { Success =
false, Message = "Unauthorized." });
        }

        int userId = JwtHelper.GetUserIdFromPrincipal(principal);
        if (userId <= 0)
        {
            return new UnauthorizedObjectResult(new ServerResponse<User> { Success =
false, Message = "Unauthorized." });
        }

        var result = await _userManager.GetProfileAsync(userId);
        return new OkObjectResult(result);
    }
    catch
    {
        return new BadRequestObjectResult(new ServerResponse<User> { Success = false,
Message = "Error loading profile data." });
    }
}
```

```

/// <summary>Updates the user's account details including username, email, and
optionally master password.</summary>
[Function("UpdateUser")]
public async Task<IActionResult> UpdateUser(
    [HttpTrigger(AuthorizationLevel.Function, "post")] HttpRequest req)
{
    try
    {
        var authHeader = req.Headers["Authorization"].FirstOrDefault();
        if (string.IsNullOrEmpty(authHeader) || !authHeader.StartsWith("Bearer "))
        {
            return new UnauthorizedObjectResult(new ServerResponse<object> { Success =
false, Message = "Unauthorized." });
        }

        var token = authHeader.Substring("Bearer ".Length);
        var principal = JwtHelper.ValidateToken(token);

        if (principal == null)
        {
            return new UnauthorizedObjectResult(new ServerResponse<object> { Success =
false, Message = "Unauthorized." });
        }

        int userId = JwtHelper.GetUserIdFromPrincipal(principal);
        if (userId <= 0)
        {
            return new UnauthorizedObjectResult(new ServerResponse<object> { Success =
false, Message = "Unauthorized." });
        }

        var body = await new StreamReader(req.Body).ReadToEndAsync();
        var request = JsonConvert.DeserializeObject<UpdateAccountRequest>(body);

        if (request == null)
        {
            return new BadRequestObjectResult(new ServerResponse<object> { Success =
false, Message = "Invalid request body." });
        }
    }
}

```

```

    }

    var updatedUser = new User
    {
        Username = request.Username,
        Email = request.Email,
        MasterPasswordKey = request.MasterPasswordKey,
        AuthSalt = request.AuthSalt,
        EncryptionSalt = request.EncryptionSalt,
        PasswordSha1Hash = request.PasswordSha1Hash
    };

    var result = await _userManager.UpdateUserAsync(userId, updatedUser,
request.PasswordChanged, request.VaultItems);
    return result.Success
        ? new OkObjectResult(result)
        : new BadRequestObjectResult(result);
    }
    catch
    {
        return new BadRequestObjectResult(new ServerResponse<object> { Success = false,
Message = "Error updating account." });
    }
}

/// <summary>Permanently deletes the user account and all associated vault
items.</summary>
[Function("DeleteUser")]
public async Task<IActionResult> DeleteUser(
    [HttpTrigger(AuthorizationLevel.Function, "post")] HttpRequest req)
{
    try
    {
        var authHeader = req.Headers["Authorization"].FirstOrDefault();
        if (string.IsNullOrEmpty(authHeader) || !authHeader.StartsWith("Bearer "))
        {
            return new UnauthorizedObjectResult(new ServerResponse<object> { Success =
false, Message = "Unauthorized." });
        }
    }
}

```

```

var token = authHeader.Substring("Bearer ".Length);
var principal = JwtHelper.ValidateToken(token);

if (principal == null)
{
    return new UnauthorizedObjectResult(new ServerResponse<object> { Success =
false, Message = "Unauthorized." });
}

int userId = JwtHelper.GetUserIdFromPrincipal(principal);
if (userId <= 0)
{
    return new UnauthorizedObjectResult(new ServerResponse<object> { Success =
false, Message = "Unauthorized." });
}

var result = await _userManager.DeleteUserAsync(userId);
return new OkObjectResult(result);
}
catch
{
    return new BadRequestObjectResult(new ServerResponse<object> { Success = false,
Message = "Error deleting account." });
}
}

/// <summary>Returns the last 4 master-password history entries for the authenticated
user.</summary>
[Function("GetPasswordHistory")]
public async Task<IActionResult> GetPasswordHistory(
    [HttpTrigger(AuthorizationLevel.Function, "get")] HttpRequest req)
{
    try
    {
        var authHeader = req.Headers["Authorization"].FirstOrDefault();
        if (string.IsNullOrEmpty(authHeader) || !authHeader.StartsWith("Bearer "))
        {

```

```

        return new UnauthorizedObjectResult(new ServerResponse<object> { Success =
false, Message = "Unauthorized." });
    }

    var token = authHeader.Substring("Bearer ".Length);
    var principal = JwtHelper.ValidateToken(token);

    if (principal == null)
    {
        return new UnauthorizedObjectResult(new ServerResponse<object> { Success =
false, Message = "Unauthorized." });
    }

    int userId = JwtHelper.GetUserIdFromPrincipal(principal);
    if (userId <= 0)
    {
        return new UnauthorizedObjectResult(new ServerResponse<object> { Success =
false, Message = "Unauthorized." });
    }

    var result = await _userManager.GetPasswordHistoryAsync(userId);
    return new OkObjectResult(result);
}
catch
{
    return new BadRequestObjectResult(new ServerResponse<object> { Success = false,
Message = "Error loading password history." });
}
}
}
}

```

--- FILE:

```

FinalProject333057891\SecurioBackendFunction\ServerFunctions\VaultItemFunctions.cs ---
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Azure.Functions.Worker;
using Newtonsoft.Json;

```

```

using SecurioBackendFunction.Helpers;
using SecurioBackendFunction.Logic;
using SecurioModels;
using SecurioModels.DataTransferObjects;
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Threading.Tasks;

namespace SecurioBackendFunction.ServerFunctions
{
    /// <summary>Manages HTTP endpoints for vault-item operations.</summary>
    public class VaultItemFunctions
    {
        private readonly VaultItemManager _vaultItemManager;

        /// <summary>Initializes a new instance of VaultItemFunctions.</summary>
        public VaultItemFunctions(VaultItemManager vaultItemManager)
        {
            _vaultItemManager = vaultItemManager;
        }

        /// <summary>Receives an encrypted vault item from the client, validates the session, and
        stores it.</summary>
        [Function("AddVaultItem")]
        public async Task<IActionResult> AddVaultItem(
            [HttpTrigger(AuthorizationLevel.Function, "post")] HttpRequest req)
        {
            try
            {
                // Extract the Bearer token from the Authorization header
                var authHeader = req.Headers["Authorization"].FirstOrDefault();
                if (string.IsNullOrEmpty(authHeader) || !authHeader.StartsWith("Bearer "))
                {
                    return new UnauthorizedObjectResult(
                        new ServerResponse<VaultItem> { Success = false, Message = "Unauthorized."
                });
            }
        }
    }

```

```

var token = authHeader.Substring("Bearer ".Length);
var principal = JwtHelper.ValidateToken(token);

if (principal == null)
{
    return new UnauthorizedObjectResult(
        new ServerResponse<VaultItem> { Success = false, Message = "Unauthorized."
});
}

int userId = JwtHelper.GetUserIdFromPrincipal(principal);
if (userId <= 0)
{
    return new UnauthorizedObjectResult(
        new ServerResponse<VaultItem> { Success = false, Message = "Unauthorized."
});
}

var body = await new StreamReader(req.Body).ReadToEndAsync();
var item = JsonConvert.DeserializeObject<VaultItem>(body);

if (item == null)
{
    return new BadRequestObjectResult(
        new ServerResponse<VaultItem> { Success = false, Message = "Invalid request
body." });
}

// Bind the authenticated user's ID so the client cannot spoof ownership.
item.UserId = userId;

var result = await _vaultItemManager.AddVaultItemAsync(item);
return result.Success
    ? new OkObjectResult(result)
    : new BadRequestObjectResult(result);
}
catch (Exception)
{

```

```

        return new BadRequestObjectResult(
            new ServerResponse<VaultItem> { Success = false, Message = "An internal error
occurred." });
    }
}

/// <summary>Receives an updated vault item from the client, validates the session, and
persists the changes.</summary>
[Function("UpdateVaultItem")]
public async Task<IActionResult> UpdateVaultItem(
    [HttpTrigger(AuthorizationLevel.Function, "post")] HttpRequest req)
{
    try
    {
        var authHeader = req.Headers["Authorization"].FirstOrDefault();
        if (string.IsNullOrEmpty(authHeader) || !authHeader.StartsWith("Bearer "))
        {
            return new UnauthorizedObjectResult(
                new ServerResponse<VaultItem> { Success = false, Message = "Unauthorized."
});
        }

        var token = authHeader.Substring("Bearer ".Length);
        var principal = JwtHelper.ValidateToken(token);

        if (principal == null)
        {
            return new UnauthorizedObjectResult(
                new ServerResponse<VaultItem> { Success = false, Message = "Unauthorized."
});
        }

        int userId = JwtHelper.GetUserIdFromPrincipal(principal);
        if (userId <= 0)
        {
            return new UnauthorizedObjectResult(
                new ServerResponse<VaultItem> { Success = false, Message = "Unauthorized."
});
        }
    }
}

```



```

var body = await new StreamReader(req.Body).ReadToEndAsync();
var item = JsonConvert.DeserializeObject<VaultItem>(body);

if (item == null)
{
    return new BadRequestObjectResult(
        new ServerResponse<VaultItem> { Success = false, Message = "Invalid request
body." });
}

// Bind the authenticated user's ID so the client cannot spoof ownership.
item.UserId = userId;

var result = await _vaultItemManager.UpdateVaultItemAsync(item);
return result.Success
    ? new OkObjectResult(result)
    : new BadRequestObjectResult(result);
}
catch (Exception)
{
    return new BadRequestObjectResult(
        new ServerResponse<VaultItem> { Success = false, Message = "An internal error
occurred." });
}
}

/// <summary>Returns all vault items for the authenticated user.</summary>
[Function("GetVaultItems")]
public async Task<IActionResult> GetVaultItems(
    [HttpTrigger(AuthorizationLevel.Function, "get")] HttpRequest req)
{
    try
    {
        var authHeader = req.Headers["Authorization"].FirstOrDefault();
        if (string.IsNullOrEmpty(authHeader) || !authHeader.StartsWith("Bearer "))
        {
            return new UnauthorizedObjectResult(

```

```

        new ServerResponse<List<VaultItem>> { Success = false, Message =
"Unauthorized." });
    }

    var token = authHeader.Substring("Bearer ".Length);
    var principal = JwtHelper.ValidateToken(token);

    if (principal == null)
    {
        return new UnauthorizedObjectResult(
            new ServerResponse<List<VaultItem>> { Success = false, Message =
"Unauthorized." });
    }

    int userId = JwtHelper.GetUserIdFromPrincipal(principal);
    if (userId <= 0)
    {
        return new UnauthorizedObjectResult(
            new ServerResponse<List<VaultItem>> { Success = false, Message =
"Unauthorized." });
    }

    var result = await _vaultItemManager.GetVaultItemsAsync(userId);
    return result.Success
        ? new OkObjectResult(result)
        : new BadRequestObjectResult(result);
    }
    catch (Exception)
    {
        return new BadRequestObjectResult(
            new ServerResponse<List<VaultItem>> { Success = false, Message = "An internal
error occurred." });
    }
}

/// <summary>Permanently deletes a vault item owned by the authenticated
user.</summary>
[Function("DeleteVaultItem")]
public async Task<IActionResult> DeleteVaultItem(

```

```
[HttpTrigger(AuthorizationLevel.Function, "post")] HttpRequest req)
{
    try
    {
        var authHeader = req.Headers["Authorization"].FirstOrDefault();
        if (string.IsNullOrEmpty(authHeader) || !authHeader.StartsWith("Bearer "))
        {
            return new UnauthorizedObjectResult(
                new ServerResponse<object> { Success = false, Message = "Unauthorized." });
        }

        var token = authHeader.Substring("Bearer ".Length);
        var principal = JwtHelper.ValidateToken(token);

        if (principal == null)
        {
            return new UnauthorizedObjectResult(
                new ServerResponse<object> { Success = false, Message = "Unauthorized." });
        }

        int userId = JwtHelper.GetUserIdFromPrincipal(principal);
        if (userId <= 0)
        {
            return new UnauthorizedObjectResult(
                new ServerResponse<object> { Success = false, Message = "Unauthorized." });
        }

        var body = await new StreamReader(req.Body).ReadToEndAsync();
        var request = JsonConvert.DeserializeObject<DeleteVaultItemRequest>(body);

        if (request == null || request.Id <= 0)
        {
            return new BadRequestObjectResult(
                new ServerResponse<object> { Success = false, Message = "Item ID is
required." });
        }

        var result = await _vaultItemManager.DeleteVaultItemAsync(request.Id, userId);
        return result.Success
    }
}
```

```

        ? new OkObjectResult(result)
        : new BadRequestObjectResult(result);
    }
    catch (Exception)
    {
        return new BadRequestObjectResult(
            new ServerResponse<object> { Success = false, Message = "An internal error
occurred." });
    }
}
}

```

```

/// <summary>Request body for the DeleteVaultItem endpoint.</summary>
internal sealed class DeleteVaultItemRequest
{
    public int Id { get; set; }
}
}

```

```

--- FILE: FinalProject333057891\SecurioBackendFunction\Program.cs ---
using Microsoft.Azure.Functions.Worker;
using Microsoft.Azure.Functions.Worker.Builder;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using SecurioBackendFunction.Helpers;
using SecurioBackendFunction.Logic; // Adjust namespace if needed
using SecurioBackendFunction.Repositories; // Adjust namespace if needed
using System;

```

```

var builder = FunctionsApplication.CreateBuilder(args);

```

```

builder.ConfigureFunctionsWebApplication();

```

```

// --- DEPENDENCY INJECTION REGISTRATION ---

```

```

builder.Services.AddApplicationInsightsTelemetryWorkerService()
    .ConfigureFunctionsApplicationInsights();

```

```

// 1. Get the connection string from local.settings.json or Azure Environment

```

```
string sqlConn = Environment.GetEnvironmentVariable("SqlConnectionString");
if (string.IsNullOrEmpty(sqlConn))
    throw new InvalidOperationException("SqlConnectionString environment variable is not
configured.");

// 2. Register the Repositories as Singletons (one instance shared by everyone)
builder.Services.AddSingleton<IUserRepository>(new UserRepository(sqlConn));
builder.Services.AddSingleton<IVaultItemRepository>(new VaultItemRepository(sqlConn));

// 3. Register the HIBP service with a dedicated HttpClient
builder.Services.AddHttpClient<HibpService>();
builder.Services.AddScoped<IHibpService, HibpService>();

// 4. Register your Managers/Logic as Scoped (new instance created per request)
builder.Services.AddScoped<UserManager>();
builder.Services.AddScoped<AuthManager>();
builder.Services.AddScoped<VaultItemManager>();
builder.Services.AddScoped<PasswordCheckManager>();

builder.Build().Run();
```

--- FILE: FinalProject333057891\SecurioClient\Activities\AddPasswordActivity.cs ---

```
using Android.App;
using Android.Content.Res;
using Android.OS;
using Android.Views;
using Android.Widget;
using AndroidX.AppCompat.App;
using Google.Android.Material.Button;
using Google.Android.Material.TextField;
using SecurioClient.Helpers;
using SecurioClient.Helpers.ServerHelpers;
using SecurioModels.DataTransferObjects;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
```

```

namespace SecurioClient.Activities
{
    /// <summary>Activity for adding a new password entry to the vault using the shared
activity_entry.xml layout.</summary>
    [Activity(Label = "@string/app_name", Theme = "@style/AppTheme.NoActionBar")]
    public class AddPasswordActivity : AppCompatActivity
    {
        // Result extras returned to the caller.
        public const string ResultEntryId = "RESULT_ENTRY_ID";
        public const string ResultSiteName = "RESULT_SITE_NAME";
        public const string ResultUsername = "RESULT_USERNAME";
        public const string ResultNotes = "RESULT_NOTES";
        public const string ResultIV = "RESULT_IV";
        public const string ResultTag = "RESULT_TAG";
        public const string ResultCipherText = "RESULT_CIPHER_TEXT";
        public const string ResultSha1Hash = "RESULT_SHA1_HASH";
        public const string ResultIsLeaked = "RESULT_IS_LEAKED";
        public const string ResultLastUpdate = "RESULT_LAST_UPDATE";

        public const int RequestCodeAdd = 1001;

        // -- Views -----
        private ImageView imageViewBack;
        private TextView textViewTitle;
        private TextView textViewSubtitle;

        private TextInputEditText editTextSiteName;
        private TextInputEditText editTextUsername;
        private TextInputEditText editTextPassword;
        private TextInputEditText editTextConfirmPassword;
        private TextInputEditText editTextNotes;

        private TextView textViewSiteNameError;
        private TextView textViewUsernameError;
        private TextView textViewPasswordError;
        private TextView textViewConfirmPasswordError;
        private TextView textViewGeneralError;

        private ProgressBar progressBarStrength;
    }
}

```

```

private TextView textViewStrengthHint;

private MaterialButton buttonGeneratePassword;
private MaterialButton buttonSave;
private ProgressBar progressBar;

// Holds existing entries for duplicate checking.
private List<VaultItem> existingEntries = new List<VaultItem>();

// -- Lifecycle -----

/// <summary>Initializes the activity, inflates the layout, and sets up views and event
handlers.</summary>
protected override void OnCreate(Bundle savedInstanceState)
{
    base.OnCreate(savedInstanceState);
    Xamarin.Essentials.Platform.Init(this, savedInstanceState);
    SetContentView(Resource.Layout.activity_entry);

    InitializeViews();
    ConfigureForAddMode();
    PopulateExistingEntries();
    SetupEventHandlers();
}

// -- Setup helpers -----

/// <summary>Finds and assigns all view references from the layout.</summary>
private void InitializeViews()
{
    imageViewBack = FindViewById<ImageView>(Resource.Id.imageViewEntryBack);
    textViewTitle = FindViewById<TextView>(Resource.Id.textViewEntryTitle);
    textViewSubtitle = FindViewById<TextView>(Resource.Id.textViewEntrySubtitle);

    editTextSiteName =
FindViewById<TextInputEditText>(Resource.Id.editTextEntrySiteName);
    editTextUsername =
FindViewById<TextInputEditText>(Resource.Id.editTextEntryUsername);

```

```

        editTextPassword =
FindViewById<TextInputEditText>(Resource.Id.editTextEntryPassword);
        editTextConfirmPassword =
FindViewById<TextInputEditText>(Resource.Id.editTextEntryConfirmPassword);
        editTextNotes =
FindViewById<TextInputEditText>(Resource.Id.editTextEntryNotes);

        textViewSiteNameError =
FindViewById<TextView>(Resource.Id.textViewEntrySiteNameError);
        textViewUsernameError =
FindViewById<TextView>(Resource.Id.textViewEntryUsernameError);
        textViewPasswordError =
FindViewById<TextView>(Resource.Id.textViewEntryPasswordError);
        textViewConfirmPasswordError =
FindViewById<TextView>(Resource.Id.textViewEntryConfirmPasswordError);
        textViewGeneralError =
FindViewById<TextView>(Resource.Id.textViewEntryGeneralError);

        progressBarStrength =
FindViewById<ProgressBar>(Resource.Id.progressBarEntryStrength);
        textViewStrengthHint =
FindViewById<TextView>(Resource.Id.textViewEntryStrengthHint);

        buttonGeneratePassword =
FindViewById<MaterialButton>(Resource.Id.buttonEntryGeneratePassword);
        buttonSave = FindViewById<MaterialButton>(Resource.Id.buttonEntrySave);
        progressBar = FindViewById<ProgressBar>(Resource.Id.progressBarEntry);
    }

    /// <summary>Sets the title, subtitle, and button label for add mode.</summary>
    private void ConfigureForAddMode()
    {
        textViewTitle.Text = GetString(Resource.String.entry_title_add);
        textViewSubtitle.Text = GetString(Resource.String.entry_subtitle_add);
        buttonSave.Text = GetString(Resource.String.entry_button_save);
    }

    /// <summary>Loads existing vault entries from the cache for duplicate
    checking.</summary>

```



```

private void PopulateExistingEntries()
{
    existingEntries = VaultEntryCache.Entries ?? new List<VaultItem>();
}

/// <summary>Wires up click and text-change event handlers for the entry form
controls.</summary>
private void SetupEventHandlers()
{
    imageViewBack.Click += (s, e) =>
    {
        SetResult(Result.Canceled);
        Finish();
    };

    buttonGeneratePassword.Click += (s, e) =>
    {
        editTextPassword.Text = ValidationHelper.GenerateStrongPassword();
    };

    buttonSave.Click += async (s, e) => await OnSaveClicked();

    // Real-time validation: site name
    editTextSiteName.AddTextChangedListener(new SimpleTextWatcher(_ =>
    {
        string site = editTextSiteName.Text?.Trim();
        if (string.IsNullOrEmpty(site))
            ShowError(textViewSiteNameError,
                GetString(Resource.String.entry_error_site_required));
        else
            HideError(textViewSiteNameError);

        // Clear any previous duplicate error when the user changes the field.
        HideError(textViewGeneralError);
    }));

    // Real-time validation: username
    editTextUsername.AddTextChangedListener(new SimpleTextWatcher(_ =>
    {

```

```

        string username = editTextUsername.Text?.Trim();
        if (string.IsNullOrEmpty(username))
            ShowError(textViewUsernameError,
GetString(Resource.String.entry_error_username_required));
        else
            HideError(textViewUsernameError);

        HideError(textViewGeneralError);
    });

    // Real-time validation: password (strength bar â€” informational only, never blocks)
    editTextPassword.AddTextChangedListener(new SimpleTextWatcher(text =>
    {
        UpdatePasswordStrengthIndicator(text);

        // Re-validate confirm password if the user has already typed in it.
        if (!string.IsNullOrEmpty(editTextConfirmPassword.Text))
            ValidatePasswordsMatch();
    }));

    // Real-time validation: confirm password
    editTextConfirmPassword.AddTextChangedListener(new SimpleTextWatcher(_ =>
    {
        ValidatePasswordsMatch();
    }));
}

// -- Save logic -----

/// <summary>Validates inputs, encrypts the password, checks for breaches, and saves the
new vault entry.</summary>
private async Task OnSaveClicked()
{
    ClearErrors();

    string siteName = editTextSiteName.Text?.Trim();
    string username = editTextUsername.Text?.Trim();
    string password = editTextPassword.Text;
    string confirmPassword = editTextConfirmPassword.Text;

```

```

string notes = editTextNotes.Text?.Trim();

if (!ValidateInputs(siteName, username, password, confirmPassword))
    return;

if (IsDuplicate(siteName, username))
{
    ShowError(textViewGeneralError,
GetString(Resource.String.entry_error_duplicate));
    return;
}

SetLoadingState(true);

try
{
    // Encrypt the password with AES-GCM using the session vault key.
    string vaultKey = SessionHelper.SessionVaultKey;
    var (iv, tag, cipherText) = EncryptionHelper.EncryptAesGcm(password, vaultKey);
    string sha1Hash = EncryptionHelper.ComputeSha1Hash(password);
    bool isLeaked = await HibpClientService.IsPasswordPwnedAsync(sha1Hash);

    var vaultItem = new VaultItem
    {
        AccountName = siteName,
        AccountUsername = username,
        IV = iv,
        Tag = tag,
        CipherText = cipherText,
        Notes = notes ?? string.Empty,
        Sha1Hash = sha1Hash,
        IsLeaked = isLeaked
    };

    var vaultService = new VaultService();
    var result = await vaultService.AddVaultItemAsync(vaultItem);

    if (result.Success)
    {

```

```

        // Return the data to VaultActivity so it can update its local list.
        var resultIntent = new Android.Content.Intent();
        resultIntent.PutExtra(ResultEntryId, result.Data?.Id ?? 0);
        resultIntent.PutExtra(ResultSiteName, siteName);
        resultIntent.PutExtra(ResultUsername, username);
        resultIntent.PutExtra(ResultNotes, notes ?? string.Empty);
        resultIntent.PutExtra(ResultIV, iv);
        resultIntent.PutExtra(ResultTag, tag);
        resultIntent.PutExtra(ResultCipherText, cipherText);
        resultIntent.PutExtra(ResultSha1Hash, sha1Hash);
        resultIntent.PutExtra(ResultIsLeaked, result.Data?.IsLeaked ?? isLeaked);
        resultIntent.PutExtra(ResultLastUpdate, result.Data?.LastUpdate.Ticks ??
DateTime.UtcNow.Ticks);

        SetResult(Result.Ok, resultIntent);

        Toast.MakeText(this,
            GetString(Resource.String.entry_saved_success),
            ToastLength.Short).Show();

        Finish();
    }
    else
    {
        ShowError(textViewGeneralError, result.Message);
    }
}
catch (Exception ex)
{
    ShowError(textViewGeneralError,
GetString(Resource.String.entry_error_save_failed));
    System.Diagnostics.Debug.WriteLine($"[ADD PASSWORD ERROR]
{ex.Message}");
}
finally
{
    SetLoadingState(false);
}
}

```

```
// -- Validation -----

/// <summary>Validates all entry form fields and shows errors for any invalid
values.</summary>
private bool ValidateInputs(string siteName, string username, string password, string
confirmPassword)
{
    bool valid = true;

    if (string.IsNullOrEmpty(siteName))
    {
        ShowError(textViewSiteNameError,
GetString(Resource.String.entry_error_site_required));
        valid = false;
    }

    if (string.IsNullOrEmpty(username))
    {
        ShowError(textViewUsernameError,
GetString(Resource.String.entry_error_username_required));
        valid = false;
    }

    if (string.IsNullOrEmpty(password))
    {
        ShowError(textViewPasswordError,
GetString(Resource.String.entry_error_password_required));
        valid = false;
    }

    if (string.IsNullOrEmpty(confirmPassword))
    {
        ShowError(textViewConfirmPasswordError,
GetString(Resource.String.entry_error_confirm_password_required));
        valid = false;
    }
    else if (!string.IsNullOrEmpty(password) && password != confirmPassword)
    {

```

```

        ShowError(textViewConfirmPasswordError,
GetString(Resource.String.entry_error_passwords_mismatch));
        valid = false;
    }

    // NOTE: Weak passwords are intentionally allowed because the user cannot
    // control what password policy external sites enforce (e.g. a 4-digit PIN).

    return valid;
}

/// <summary>Returns true when an entry with the same site name and username already
exists in the vault.</summary>
private bool IsDuplicate(string siteName, string username)
{
    return existingEntries.Any(e =>
        string.Equals(e.AccountName, siteName, StringComparison.OrdinalIgnoreCase)
        && string.Equals(e.AccountUsername, username,
StringComparison.OrdinalIgnoreCase));
}

/// <summary>Validates that the confirm password field matches the password field and
shows or hides the error accordingly.</summary>
private void ValidatePasswordsMatch()
{
    string password = editTextPassword.Text;
    string confirm = editTextConfirmPassword.Text;
    if (string.IsNullOrEmpty(confirm))
    {
        HideError(textViewConfirmPasswordError);
        return;
    }
    if (password != confirm)
        ShowError(textViewConfirmPasswordError,
GetString(Resource.String.entry_error_passwords_mismatch));
    else
        HideError(textViewConfirmPasswordError);
}

```

```
// -- Password strength indicator (informational) -----

/// <summary>Updates the password strength progress bar and hint text based on the
given password.</summary>
private void UpdatePasswordStrengthIndicator(string password)
{
    if (string.IsNullOrEmpty(password))
    {
        progressBarStrength.Visibility = ViewStates.Gone;
        textViewStrengthHint.Visibility = ViewStates.Gone;
        HideError(textViewPasswordError);
        return;
    }

    int score = ValidationHelper.GetPasswordScore(password);
    progressBarStrength.Visibility = ViewStates.Visible;
    progressBarStrength.Progress = score;
    progressBarStrength.ProgressTintList =
        ColorStateList.ValueOf(new
        Android.Graphics.Color(Resources.GetColor(ScoreToColorRes(score))));

    string hint = ValidationHelper.GetMissingCriteriaHint(password);
    textViewStrengthHint.Text = hint;
    textViewStrengthHint.Visibility = ViewStates.Visible;
    textViewStrengthHint.SetTextColor(new Android.Graphics.Color(
        Resources.GetColor(score == 5
            ? Resource.Color.passwordStrengthVeryStrong
            : Resource.Color.signupHintText)));
}

/// <summary>Maps a password score (1â€“5) to the corresponding color resource
ID.</summary>
private static int ScoreToColorRes(int score)
{
    switch (score)
    {
        case 1: return Resource.Color.passwordStrengthWeak;
        case 2: return Resource.Color.passwordStrengthPoor;
        case 3: return Resource.Color.passwordStrengthFair;
```

```

        case 4: return Resource.Color.passwordStrengthStrong;
        default: return Resource.Color.passwordStrengthVeryStrong;
    }
}

// -- UI helpers -----

/// <summary>Sets the error text and makes the given error view visible.</summary>
private void ShowError(TextView errorView, string message)
{
    errorView.Text = message;
    errorView.Visibility = ViewStates.Visible;
}

/// <summary>Clears the error text and hides the given error view.</summary>
private void HideError(TextView errorView)
{
    errorView.Text = null;
    errorView.Visibility = ViewStates.Gone;
}

/// <summary>Hides all field-level and general error messages.</summary>
private void ClearErrors()
{
    HideError(textViewSiteNameError);
    HideError(textViewUsernameError);
    HideError(textViewPasswordError);
    HideError(textViewConfirmPasswordError);
    HideError(textViewGeneralError);
}

/// <summary>Toggles the loading state, showing the progress bar and disabling form
controls.</summary>
private void SetLoadingState(bool isLoading)
{
    progressBar.Visibility = isLoading ? ViewStates.Visible : ViewStates.Gone;
    buttonSave.Enabled = !isLoading;
    buttonGeneratePassword.Enabled = !isLoading;
    editTextSiteName.Enabled = !isLoading;
}

```



```

        editTextUsername.Enabled = !isLoading;
        editTextPassword.Enabled = !isLoading;
        editTextConfirmPassword.Enabled = !isLoading;
        editTextNotes.Enabled = !isLoading;
    }
}

```

/// <summary>Static cache used to share the current entry list between VaultActivity and entry activities for duplicate checking.</summary>

```

public static class VaultEntryCache
{
    public static List<VaultItem> Entries { get; set; } = new List<VaultItem>();
}

```

--- FILE: FinalProject333057891\SecurioClient\Activities\EditAccountActivity.cs ---

```

using Android.App;
using Android.Content;
using Android.OS;
using Android.Views;
using Android.Widget;
using AndroidX.AppCompat.App;
using Google.Android.Material.Button;
using Google.Android.Material.TextField;
using SecurioClient.Helpers;
using SecurioClient.Helpers.ServerHelpers;
using SecurioModels;
using SecurioModels.DataTransferObjects;
using System;
using System.Collections.Generic;
using System.Threading.Tasks;

```

```

namespace SecurioClient.Activities

```

```

{
    [Activity(Label = "@string/app_name", Theme = "@style/AppTheme.NoActionBar")]
    /// <summary>Activity that allows the user to update their username, email, and optionally their master password.</summary>
    public class EditAccountActivity : AppCompatActivity

```

```
{
    // Request code used when launching this activity from ProfileActivity.
    public const int RequestCodeEditAccount = 2001;
    // Result key indicating the profile was updated.
    public const string ResultUpdated = "AccountUpdated";

    private TextInputEditText editTextUsername;
    private TextInputEditText editTextEmail;
    private TextInputEditText editTextCurrentPassword;
    private TextInputEditText editTextNewPassword;
    private TextInputEditText editTextConfirmNewPassword;
    private MaterialButton buttonSave;
    private MaterialButton buttonGeneratePassword;
    private ImageView imageViewBack;
    private TextView textViewUsernameError;
    private TextView textViewEmailError;
    private TextView textViewCurrentPasswordError;
    private TextView textViewNewPasswordError;
    private ProgressBar progressBarPasswordStrength;
    private TextView textViewPasswordHint;
    private TextView textViewConfirmNewPasswordError;
    private TextView textViewGeneralError;
    private ProgressBar progressBarEditAccount;

    // The current values loaded from the profile.
    private string originalUsername;
    private string originalEmail;

    /// <summary>Initializes the activity, inflates the layout, and pre-fills fields with the
current profile.</summary>
    protected override async void OnCreate(Bundle savedInstanceState)
    {
        base.OnCreate(savedInstanceState);
        Xamarin.Essentials.Platform.Init(this, savedInstanceState);
        SetContentView(Resource.Layout.activity_edit_account);

        InitializeViews();
        SetupEventHandlers();
        await LoadCurrentProfileAsync();
    }
}
```

```

    }

    /// <summary>Finds and assigns all view references from the layout.</summary>
    private void InitializeViews()
    {
        editTextUsername =
        FindViewById<TextInputEditText>(Resource.Id.editTextEditUsername);
        editTextEmail =
        FindViewById<TextInputEditText>(Resource.Id.editTextEditEmail);
        editTextCurrentPassword =
        FindViewById<TextInputEditText>(Resource.Id.editTextCurrentPassword);
        editTextNewPassword =
        FindViewById<TextInputEditText>(Resource.Id.editTextNewPassword);
        editTextConfirmNewPassword =
        FindViewById<TextInputEditText>(Resource.Id.editTextConfirmNewPassword);
        buttonSave = FindViewById<MaterialButton>(Resource.Id.buttonEditSave);
        buttonGeneratePassword =
        FindViewById<MaterialButton>(Resource.Id.buttonEditGeneratePassword);
        imageViewBack =
        FindViewById<ImageView>(Resource.Id.imageViewEditAccountBack);
        textViewUsernameError =
        FindViewById<TextView>(Resource.Id.textViewEditUsernameError);
        textViewEmailError =
        FindViewById<TextView>(Resource.Id.textViewEditEmailError);
        textViewCurrentPasswordError =
        FindViewById<TextView>(Resource.Id.textViewCurrentPasswordError);
        textViewNewPasswordError =
        FindViewById<TextView>(Resource.Id.textViewNewPasswordError);
        progressBarPasswordStrength =
        FindViewById<ProgressBar>(Resource.Id.progressBarEditPasswordStrength);
        textViewPasswordHint =
        FindViewById<TextView>(Resource.Id.textViewEditPasswordHint);
        textViewConfirmNewPasswordError =
        FindViewById<TextView>(Resource.Id.textViewConfirmNewPasswordError);
        textViewGeneralError =
        FindViewById<TextView>(Resource.Id.textViewEditGeneralError);
        progressBarEditAccount =
        FindViewById<ProgressBar>(Resource.Id.progressBarEditAccount);
    }

```

```

/// <summary>Wires up click and text-change event handlers for the edit account form
controls.</summary>
private void SetupEventHandlers()
{
    imageViewBack.Click += (sender, e) => Finish();

    buttonSave.Click += async (sender, e) => await OnSaveClicked();

    buttonGeneratePassword.Click += (sender, e) =>
    {
        string generated = ValidationHelper.GenerateStrongPassword();
        editTextNewPassword.Text = generated;
    };

    // Real-time validation â€” reuses the same ValidationHelper as signup (DRY).
    editTextUsername.AddTextChangedListener(new SimpleTextWatcher(_ =>
    {
        var result = ValidationHelper.ValidateUsername(editTextUsername.Text?.Trim());
        if (!result.IsValid) FormUiHelper.ShowError(textViewUsernameError,
result.ErrorMessage);
        else FormUiHelper.HideError(textViewUsernameError);
    }));

    editTextEmail.AddTextChangedListener(new SimpleTextWatcher(_ =>
    {
        var result = ValidationHelper.ValidateEmail(editTextEmail.Text?.Trim());
        if (!result.IsValid) FormUiHelper.ShowError(textViewEmailError,
result.ErrorMessage);
        else FormUiHelper.HideError(textViewEmailError);
    }));

    editTextNewPassword.AddTextChangedListener(new SimpleTextWatcher(text =>
    {
        FormUiHelper.UpdatePasswordStrengthIndicator(
            text, progressBarPasswordStrength, textViewPasswordHint,
textViewNewPasswordError, Resources);

        if (!string.IsNullOrEmpty(editTextConfirmNewPassword.Text))

```

```

        {
            var matchResult = ValidationHelper.ValidatePasswordsMatch(text,
editTextConfirmNewPassword.Text);
            if (!matchResult.IsValid)
FormUiHelper.ShowError(textViewConfirmNewPasswordError,
matchResult.ErrorMessage);
            else
                FormUiHelper.HideError(textViewConfirmNewPasswordError);
        }
    });

```

```

editTextConfirmNewPassword.AddTextChangedListener(new SimpleTextWatcher(_
=>
{
    var result = ValidationHelper.ValidatePasswordsMatch(
        editTextNewPassword.Text, editTextConfirmNewPassword.Text);
    if (!result.IsValid) FormUiHelper.ShowError(textViewConfirmNewPasswordError,
result.ErrorMessage);
    else
        FormUiHelper.HideError(textViewConfirmNewPasswordError);
}));
}

```

/// <summary>Loads the current username and email from the server or cache and pre-fills the form fields.</summary>

```

private async Task LoadCurrentProfileAsync()
{
    try
    {
        var profileService = new ProfileService();
        var result = await profileService.GetProfileAsync();

        if (result.Success && result.Data != null)
        {
            originalUsername = result.Data.Username;
            originalEmail = result.Data.Email;
        }
        else
        {
            var cached = await StorageHelper.GetCachedProfileAsync();
            if (cached != null)

```

```

        {
            originalUsername = cached.Username;
            originalEmail = cached.Email;
        }
    }
}
catch
{
    var cached = await StorageHelper.GetCachedProfileAsync();
    if (cached != null)
    {
        originalUsername = cached.Username;
        originalEmail = cached.Email;
    }
}

editTextUsername.Text = originalUsername ?? "";
editTextEmail.Text = originalEmail ?? "";
}

/// <summary>Validates inputs, re-encrypts vault items if the password changes, and
submits the update.</summary>
private async Task OnSaveClicked()
{
    ClearErrors();

    string username    = editTextUsername.Text?.Trim();
    string email       = editTextEmail.Text?.Trim();
    string currentPassword = editTextCurrentPassword.Text;
    string newPassword = editTextNewPassword.Text;
    string confirmPassword = editTextConfirmNewPassword.Text;

    // Determine if the user is changing their master password.
    bool isChangingPassword = !string.IsNullOrEmpty(currentPassword) ||
        !string.IsNullOrEmpty(newPassword) ||
        !string.IsNullOrEmpty(confirmPassword);

    if (!ValidateInputs(username, email, currentPassword, newPassword, confirmPassword,
isChangingPassword))

```

```

return;

SetLoadingState(true);

try
{
    var request = new UpdateAccountRequest
    {
        Username = username,
        Email = email,
        PasswordChanged = isChangingPassword
    };

    if (isChangingPassword)
    {
        // Verify the current password by deriving the auth key and comparing locally.
        // We fetch the user's salts from the server to derive the current key.
        var saltResult = await new BaseServiceProxy().GetSaltsAsync(email);

        // If salts aren't available for the current email (e.g., email changed), use original
email.
        if (!saltResult.Success)
            saltResult = await new BaseServiceProxy().GetSaltsAsync(originalEmail);

        if (!saltResult.Success)
        {

ShowGeneralError(GetString(Resource.String.edit_account_verify_password_failed));
            return;
        }

        string currentAuthKey = EncryptionHelper.DeriveKey(currentPassword,
saltResult.Data.AuthSalt);
        string currentVaultKey = EncryptionHelper.DeriveKey(currentPassword,
saltResult.Data.EncryptionSalt);

        // Verify the current vault key matches the session key.
        if (currentVaultKey != SessionHelper.SessionVaultKey)
        {

```

```

        FormUiHelper.ShowError(textViewCurrentPasswordError,
GetString(Resource.String.edit_account_current_password_wrong));
        return;
    }

    // No-reuse check: compare new password against the current one and last 4 history
entries.
    // For each entry (and the current), derive: DeriveKey(newPassword,
entry.AuthSalt).
    // A match means the password was already used â†’ reject.
    string newKeyForReuseCheck = EncryptionHelper.DeriveKey(newPassword,
saltResult.Data.AuthSalt);
    if (newKeyForReuseCheck == currentAuthKey)
    {
        FormUiHelper.ShowError(textViewNewPasswordError,
GetString(Resource.String.edit_account_password_reused));
        return;
    }

    var historyService = new ProfileService();
    var historyResult = await historyService.GetPasswordHistoryAsync();
    if (historyResult.Success && historyResult.Data != null)
    {
        foreach (var entry in historyResult.Data)
        {
            string historicCheck = EncryptionHelper.DeriveKey(newPassword,
entry.AuthSalt);
            if (historicCheck == entry.PasswordKey)
            {
                FormUiHelper.ShowError(textViewNewPasswordError,
GetString(Resource.String.edit_account_password_reused));
                return;
            }
        }
    }

    // Generate new salts and derive new keys.
    string newAuthSalt    = EncryptionHelper.GenerateSalt();
    string newEncryptionSalt = EncryptionHelper.GenerateSalt();

```



```

string newMasterPasswordKey = EncryptionHelper.DeriveKey(newPassword,
newAuthSalt);
string newVaultKey = EncryptionHelper.DeriveKey(newPassword,
newEncryptionSalt);

request.MasterPasswordKey = newMasterPasswordKey;
request.AuthSalt = newAuthSalt;
request.EncryptionSalt = newEncryptionSalt;
// Compute SHA-1 of the plaintext new password for the HIBP breach check.
// Same pattern as signup "never" never stored, discarded after server validation.
request.PasswordSha1Hash =
EncryptionHelper.ComputeSha1Hash(newPassword);

// Re-encrypt all vault items with the new key.
string oldVaultKey = SessionHelper.SessionVaultKey;
var reEncryptedItems = new List<VaultItem>();

foreach (var item in SessionHelper.CachedVault)
{
    // Decrypt with old key
    string plaintext = EncryptionHelper.DecryptAesGcm(
        item.IV, item.Tag, item.CipherText, oldVaultKey);

    // Re-encrypt with new key
    var (newIV, newTag, newCipherText) =
EncryptionHelper.EncryptAesGcm(plaintext, newVaultKey);

    reEncryptedItems.Add(new VaultItem
    {
        Id = item.Id,
        IV = newIV,
        Tag = newTag,
        CipherText = newCipherText
    });
}

request.VaultItems = reEncryptedItems;

// Send the update to the server.

```

```

var profileService = new ProfileService();
var result = await profileService.UpdateAccountAsync(request);

if (result.Success)
{
    // Update the session with the new vault key.
    SessionHelper.StartSession(new VaultKey);
    await StorageHelper.SaveVaultKey(new VaultKey);
    await StorageHelper.SaveUsername(username);

    // Update the cached vault items with the new encryption.
    for (int i = 0; i < SessionHelper.CachedVault.Count; i++)
    {
        var cached = SessionHelper.CachedVault[i];
        var reenc = reEncryptedItems.Find(r => r.Id == cached.Id);
        if (reenc != null)
        {
            cached.IV = reenc.IV;
            cached.Tag = reenc.Tag;
            cached.CipherText = reenc.CipherText;
        }
    }

    SessionHelper.InvalidateWarnings();

    Toast.MakeText(this, Resource.String.edit_account_success,
ToastLength.Short).Show();
    SetResult(Result.Ok, new Intent().PutExtra(ResultUpdated, true));
    Finish();
}
else
{
    ShowGeneralError(result.Message);
}
}
else
{
    // Username/email only change "no re-encryption needed.
    var profileService = new ProfileService();

```

```

var result = await profileService.UpdateAccountAsync(request);

if (result.Success)
{
    await StorageHelper.SaveUsername(username);

    Toast.MakeText(this, Resource.String.edit_account_success,
ToastLength.Short).Show();
    SetResult(Result.Ok, new Intent().PutExtra(ResultUpdated, true));
    Finish();
}
else
{
    ShowGeneralError(result.Message);
}
}
}
catch (Exception ex)
{
    ShowGeneralError(GetString(Resource.String.edit_account_error));
    System.Diagnostics.Debug.WriteLine($"[EDIT ACCOUNT ERROR]
{ex.Message}");
}
finally
{
    SetLoadingState(false);
}
}

/// <summary>Validates all edit account form fields and shows errors for any invalid
values.</summary>
private bool ValidateInputs(string username, string email,
    string currentPassword, string newPassword, string confirmPassword,
    bool isChangingPassword)
{
    bool isValid = true;

    var usernameResult = ValidationHelper.ValidateUsername(username);

```

```

        if (!usernameResult.IsValid) { FormUiHelper.ShowError(textViewUsernameError,
usernameResult.ErrorMessage); isValid = false; }

        var emailResult = ValidationHelper.ValidateEmail(email);
        if (!emailResult.IsValid) { FormUiHelper.ShowError(textViewEmailError,
emailResult.ErrorMessage); isValid = false; }

        if (isChangingPassword)
        {
            if (string.IsNullOrEmpty(currentPassword))
            {
                FormUiHelper.ShowError(textViewCurrentPasswordError,
GetString(Resource.String.edit_account_current_password_required));
                isValid = false;
            }

            var passwordResult = ValidationHelper.ValidatePassword(newPassword);
            if (!passwordResult.IsValid) {
FormUiHelper.ShowError(textViewNewPasswordError, passwordResult.ErrorMessage);
isValid = false; }

            var confirmResult = ValidationHelper.ValidatePasswordsMatch(newPassword,
confirmPassword);
            if (!confirmResult.IsValid) {
FormUiHelper.ShowError(textViewConfirmNewPasswordError,
confirmResult.ErrorMessage); isValid = false; }
        }

        return isValid;
    }

    // -- UI helpers delegating to the shared FormUiHelper (DRY) --

    /// <summary>Displays the general error message banner.</summary>
    private void ShowGeneralError(string message)
    {
        textViewGeneralError.Text = message;
        textViewGeneralError.Visibility = ViewStates.Visible;
    }

```

```

/// <summary>Hides all field-level and general error messages.</summary>
private void ClearErrors()
{
    FormUiHelper.HideError(textViewUsernameError);
    FormUiHelper.HideError(textViewEmailError);
    FormUiHelper.HideError(textViewCurrentPasswordError);
    FormUiHelper.HideError(textViewNewPasswordError);
    FormUiHelper.HideError(textViewConfirmNewPasswordError);
    FormUiHelper.HideError(textViewGeneralError);
}

/// <summary>Toggles the loading state, showing the progress bar and disabling form
controls.</summary>
private void SetLoadingState(bool isLoading)
{
    progressBarEditAccount.Visibility = isLoading ? ViewStates.Visible :
ViewStates.Gone;
    buttonSave.Enabled = !isLoading;
    buttonGeneratePassword.Enabled = !isLoading;
    editTextUsername.Enabled = !isLoading;
    editTextEmail.Enabled = !isLoading;
    editTextCurrentPassword.Enabled = !isLoading;
    editTextNewPassword.Enabled = !isLoading;
    editTextConfirmNewPassword.Enabled = !isLoading;
}
}

/// <summary>Lightweight proxy that exposes salt retrieval from BaseService for use in
EditAccountActivity.</summary>
internal class BaseServiceProxy : BaseService
{
    /// <summary>Fetches the auth and encryption salts for the given email
address.</summary>
    public async Task<ServerResponse<SaltData>> GetSaltsAsync(string email)
    {
        return await PostAsync<SaltData>("GetSalts", new { Email = email });
    }
}

```

}

--- FILE: FinalProject333057891\SecurioClient\Activities\EditPasswordActivity.cs ---

```
using Android.App;
using Android.Content;
using Android.Content.Res;
using Android.OS;
using Android.Views;
using Android.Widget;
using AndroidX.AppCompat.App;
using Google.Android.Material.Button;
using Google.Android.Material.TextField;
using SecurioClient.Helpers;
using SecurioClient.Helpers.ServerHelpers;
using SecurioModels.DataTransferObjects;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
```

namespace SecurioClient.Activities

{

/// <summary>Activity for editing an existing password entry in the vault using the shared activity_entry.xml layout.</summary>

[Activity(Label = "@string/app_name", Theme = "@style/AppTheme.NoActionBar")]

public class EditPasswordActivity : AppCompatActivity

{

// Intent extras used to pass the item data into this activity.

public const string ExtraEntryId = "EXTRA_ENTRY_ID";

public const string ExtraSiteName = "EXTRA_SITE_NAME";

public const string ExtraUsername = "EXTRA_USERNAME";

public const string ExtraNotes = "EXTRA_NOTES";

// Existing encrypted fields " passed in so the user does not have to re-enter the password.

public const string ExtraIV = "EXTRA_IV";

public const string ExtraTag = "EXTRA_TAG";

public const string ExtraCipherText = "EXTRA_CIPHER_TEXT";

public const string ExtraSha1Hash = "EXTRA_SHA1_HASH";

```

public const string ExtraIsLeaked = "EXTRA_IS_LEAKED";

// Result extras returned to the caller.
public const string ResultEntryId = "RESULT_ENTRY_ID";
public const string ResultSiteName = "RESULT_SITE_NAME";
public const string ResultUsername = "RESULT_USERNAME";
public const string ResultNotes = "RESULT_NOTES";
public const string ResultIV = "RESULT_IV";
public const string ResultTag = "RESULT_TAG";
public const string ResultCipherText = "RESULT_CIPHER_TEXT";
public const string ResultSha1Hash = "RESULT_SHA1_HASH";
public const string ResultIsLeaked = "RESULT_IS_LEAKED";
public const string ResultLastUpdate = "RESULT_LAST_UPDATE";

public const int RequestCodeEdit = 1002;

// -- Views -----
private ImageView imageViewBack;
private TextView textViewTitle;
private TextView textViewSubtitle;

private TextInputLayout textInputLayoutPassword;
private TextInputLayout textInputLayoutConfirmPassword;

private TextInputEditText editTextSiteName;
private TextInputEditText editTextUsername;
private TextInputEditText editTextPassword;
private TextInputEditText editTextConfirmPassword;
private TextInputEditText editTextNotes;

private TextView textViewSiteNameError;
private TextView textViewUsernameError;
private TextView textViewPasswordError;
private TextView textViewConfirmPasswordError;
private TextView textViewGeneralError;

private ProgressBar progressBarStrength;
private TextView textViewStrengthHint;

```

```
private MaterialButton buttonGeneratePassword;
private MaterialButton buttonSave;
private ProgressBar progressBar;

// The ID of the vault item being edited.
private int entryId;

// Holds existing entries for duplicate checking (excludes the item being edited).
private List<VaultItem> existingEntries = new List<VaultItem>();

// Placeholder shown in the password field to indicate an existing password is stored.
// Cleared when the user focuses the field; restored if they leave without typing.
private const string PasswordPlaceholder = "â€¢â€¢â€¢â€¢â€¢â€¢â€¢â€¢â€¢â€¢â€¢";
private bool _passwordIsPlaceholder;

// -- Lifecycle -----

/// <summary>Initializes the activity, inflates the layout, and sets up views and event
handlers.</summary>
protected override void OnCreate(Bundle savedInstanceState)
{
    base.OnCreate(savedInstanceState);
    Xamarin.Essentials.Platform.Init(this, savedInstanceState);
    SetContentView(Resource.Layout.activity_entry);

    InitializeViews();
    ConfigureEditMode();
    PopulateExistingEntries();
    PopulateFieldsFromIntent();
    SetupEventHandlers();
    ConfigurePasswordPlaceholder();
}

// -- Setup helpers -----

/// <summary>Finds and assigns all view references from the layout.</summary>
private void InitializeViews()
{
    imageViewBack = FindViewById<ImageView>(Resource.Id.imageViewEntryBack);
```



```

        textInputLayoutPassword =
FindViewById<TextInputLayout>(Resource.Id.textInputLayoutEntryPassword);
        textInputLayoutConfirmPassword =
FindViewById<TextInputLayout>(Resource.Id.textInputLayoutEntryConfirmPassword);

        editTextSiteName =
FindViewById<TextInputEditText>(Resource.Id.editTextEntrySiteName);
        editTextUsername =
FindViewById<TextInputEditText>(Resource.Id.editTextEntryUsername);
        editTextPassword =
FindViewById<TextInputEditText>(Resource.Id.editTextEntryPassword);
        editTextConfirmPassword =
FindViewById<TextInputEditText>(Resource.Id.editTextEntryConfirmPassword);
        editTextNotes =
FindViewById<TextInputEditText>(Resource.Id.editTextEntryNotes);

        textViewTitle = FindViewById<TextView>(Resource.Id.textViewEntryTitle);
        textViewSubtitle = FindViewById<TextView>(Resource.Id.textViewEntrySubtitle);
        textViewSiteNameError =
FindViewById<TextView>(Resource.Id.textViewEntrySiteNameError);
        textViewUsernameError =
FindViewById<TextView>(Resource.Id.textViewEntryUsernameError);
        textViewPasswordError =
FindViewById<TextView>(Resource.Id.textViewEntryPasswordError);
        textViewConfirmPasswordError =
FindViewById<TextView>(Resource.Id.textViewEntryConfirmPasswordError);
        textViewGeneralError =
FindViewById<TextView>(Resource.Id.textViewEntryGeneralError);

        progressBarStrength =
FindViewById<ProgressBar>(Resource.Id.progressBarEntryStrength);
        textViewStrengthHint =
FindViewById<TextView>(Resource.Id.textViewEntryStrengthHint);

        buttonGeneratePassword =
FindViewById<MaterialButton>(Resource.Id.buttonEntryGeneratePassword);
        buttonSave = FindViewById<MaterialButton>(Resource.Id.buttonEntrySave);
        progressBar = FindViewById<ProgressBar>(Resource.Id.progressBarEntry);
    }

```

```

    /// <summary>Sets the title, subtitle, button label, and hint text for edit
mode.</summary>
    private void ConfigureForEditMode()
    {
        textViewTitle.Text = GetString(Resource.String.entry_title_edit);
        textViewSubtitle.Text = GetString(Resource.String.entry_subtitle_edit);
        buttonSave.Text = GetString(Resource.String.entry_button_update);

        // Update hints to make clear that the password field is optional in edit mode.
        textInputLayoutPassword.Hint =
GetString(Resource.String.entry_password_edit_hint);
        textInputLayoutConfirmPassword.Hint =
GetString(Resource.String.entry_confirm_password_edit_hint);
    }

    /// <summary>Loads existing vault entries excluding the current item for duplicate
checking.</summary>
    private void PopulateExistingEntries()
    {
        entryId = Intent.GetIntExtra(ExtraEntryId, 0);
        existingEntries = (VaultEntryCache.Entries ?? new List<VaultItem>())
            .Where(e => e.Id != entryId)
            .ToList();
    }

    /// <summary>Pre-fills the form fields with the data passed via the launching
Intent.</summary>
    private void PopulateFieldsFromIntent()
    {
        editTextSiteName.Text = Intent.GetStringExtra(ExtraSiteName) ?? string.Empty;
        editTextUsername.Text = Intent.GetStringExtra(ExtraUsername) ?? string.Empty;
        editTextNotes.Text = Intent.GetStringExtra(ExtraNotes) ?? string.Empty;
        // Password and confirm password are intentionally left empty.
        // Existing encrypted data is carried via
ExtraIV/ExtraTag/ExtraCipherText/ExtraSha1Hash.
    }

```

```

/// <summary>Wires up click and text-change event handlers for the entry form
controls.</summary>
private void SetupEventHandlers()
{
    imageViewBack.Click += (s, e) =>
    {
        SetResult(Result.Canceled);
        Finish();
    };

    buttonGeneratePassword.Click += (s, e) =>
    {
        _passwordIsPlaceholder = false;
        string generated = ValidationHelper.GenerateStrongPassword();
        editTextPassword.Text = generated;
        editTextConfirmPassword.Text = generated;
    };

    buttonSave.Click += async (s, e) => await OnUpdateClicked();

    // Real-time validation: site name
    editTextSiteName.AddTextChangedListener(new SimpleTextWatcher(_ =>
    {
        string site = editTextSiteName.Text?.Trim();
        if (string.IsNullOrEmpty(site))
            ShowError(textViewSiteNameError,
                GetString(Resource.String.entry_error_site_required));
        else
            HideError(textViewSiteNameError);

        HideError(textViewGeneralError);
    }));

    // Real-time validation: username
    editTextUsername.AddTextChangedListener(new SimpleTextWatcher(_ =>
    {
        string username = editTextUsername.Text?.Trim();
        if (string.IsNullOrEmpty(username))
    
```

```

        ShowError(textViewUsernameError,
GetString(Resource.String.entry_error_username_required));
    else
        HideError(textViewUsernameError);

    HideError(textViewGeneralError);
});

// Real-time validation: password strength (informational only, never blocks)
editTextPassword.AddTextChangedListener(new SimpleTextWatcher(text =>
{
    if (!_passwordIsPlaceholder) return;
    UpdatePasswordStrengthIndicator(text);

    // Re-validate confirm password match whenever the password field changes.
    if (!string.IsNullOrEmpty(editTextConfirmPassword.Text))
        ValidatePasswordsMatch();
}));

// Real-time validation: confirm password
editTextConfirmPassword.AddTextChangedListener(new SimpleTextWatcher(_ =>
{
    ValidatePasswordsMatch();
}));
}

/// <summary>Seeds the password field with a masked placeholder and clears it on focus
so the user can type a new value.</summary>
private void ConfigurePasswordPlaceholder()
{
    _passwordIsPlaceholder = true;
    editTextPassword.Text = PasswordPlaceholder;

    editTextPassword.FocusChange += (s, e) =>
    {
        if (e.HasFocus && _passwordIsPlaceholder)
        {
            // User tapped the field "â€”" clear the placeholder so they can type.
            _passwordIsPlaceholder = false;

```

```

        editTextPassword.Text = string.Empty;
    }
    else if (!e.HasFocus && string.IsNullOrEmpty(editTextPassword.Text))
    {
        // User left the field without entering anything â€” restore the placeholder.
        _passwordIsPlaceholder = true;
        editTextPassword.Text = PasswordPlaceholder;
    }
    };
}

// -- Update logic -----

/// <summary>Validates inputs, optionally re-encrypts the password, and submits the
vault entry update.</summary>
private async Task OnUpdateClicked()
{
    ClearErrors();

    string siteName = editTextSiteName.Text?.Trim();
    string username = editTextUsername.Text?.Trim();
    // Treat the placeholder as an empty field (user kept the existing password).
    string password = _passwordIsPlaceholder ? string.Empty : editTextPassword.Text;
    string confirmPassword = _passwordIsPlaceholder ? string.Empty :
editTextConfirmPassword.Text;
    string notes = editTextNotes.Text?.Trim();

    if (!ValidateInputs(siteName, username, password, confirmPassword))
        return;

    if (IsDuplicate(siteName, username))
    {
        ShowError(textViewGeneralError,
GetString(Resource.String.entry_error_duplicate));
        return;
    }

    SetLoadingState(true);

```

```

try
{
    string iv, tag, cipherText, sha1Hash;
    bool isLeaked;

    if (!string.IsNullOrEmpty(password))
    {
        // User entered a new password â€” encrypt it and check HIBP.
        string vaultKey = SessionHelper.SessionVaultKey;
        (iv, tag, cipherText) = EncryptionHelper.EncryptAesGcm(password, vaultKey);
        sha1Hash = EncryptionHelper.ComputeSha1Hash(password);
        isLeaked = await HibpClientService.IsPasswordPwnedAsync(sha1Hash);
    }
    else
    {
        // Password left blank â€” keep the existing encrypted data.
        iv      = Intent.GetStringExtra(ExtraIV) ?? string.Empty;
        tag     = Intent.GetStringExtra(ExtraTag) ?? string.Empty;
        cipherText = Intent.GetStringExtra(ExtraCipherText) ?? string.Empty;
        sha1Hash  = Intent.GetStringExtra(ExtraSha1Hash) ?? string.Empty;
        isLeaked  = Intent.GetBooleanExtra(ExtraIsLeaked, false);
    }

    var vaultItem = new VaultItem
    {
        Id          = entryId,
        AccountName  = siteName,
        AccountUsername = username,
        IV          = iv,
        Tag         = tag,
        CipherText   = cipherText,
        Notes       = notes ?? string.Empty,
        Sha1Hash     = sha1Hash,
        IsLeaked     = isLeaked,
        PasswordChanged = !string.IsNullOrEmpty(password)
    };

    var vaultService = new VaultService();
    var result = await vaultService.UpdateVaultItemAsync(vaultItem);

```

```

if (result.Success)
{
    // Return the updated data to VaultActivity so it can refresh its local list.
    var resultIntent = new Intent();
    resultIntent.PutExtra(ResultEntryId, entryId);
    resultIntent.PutExtra(ResultSiteName, siteName);
    resultIntent.PutExtra(ResultUsername, username);
    resultIntent.PutExtra(ResultNotes, notes ?? string.Empty);
    resultIntent.PutExtra(ResultIV, iv);
    resultIntent.PutExtra(ResultTag, tag);
    resultIntent.PutExtra(ResultCipherText, cipherText);
    resultIntent.PutExtra(ResultSha1Hash, sha1Hash);
    resultIntent.PutExtra(ResultIsLeaked, result.Data?.IsLeaked ?? isLeaked);
    resultIntent.PutExtra(ResultLastUpdate, result.Data?.LastUpdate.Ticks ?? 0L);

    SetResult(Result.Ok, resultIntent);

    Toast.MakeText(this,
        GetString(Resource.String.entry_updated_success),
        ToastLength.Short).Show();

    Finish();
}
else
{
    ShowError(textViewGeneralError, result.Message);
}
}
catch (Exception ex)
{
    ShowError(textViewGeneralError,
        GetString(Resource.String.entry_error_update_failed));
    System.Diagnostics.Debug.WriteLine($"[EDIT PASSWORD ERROR]
    {ex.Message}");
}
finally
{
    SetLoadingState(false);
}

```

```

    }
}

// -- Validation -----

/// <summary>Validates all entry form fields and shows errors for any invalid
values.</summary>
private bool ValidateInputs(string siteName, string username, string password, string
confirmPassword)
{
    bool valid = true;

    if (string.IsNullOrEmpty(siteName))
    {
        ShowError(textViewSiteNameError,
GetString(Resource.String.entry_error_site_required));
        valid = false;
    }

    if (string.IsNullOrEmpty(username))
    {
        ShowError(textViewUsernameError,
GetString(Resource.String.entry_error_username_required));
        valid = false;
    }

    // In edit mode the password is optional; only validate when provided.
    if (!string.IsNullOrEmpty(password) || !string.IsNullOrEmpty(confirmPassword))
    {
        // If one is set, both must be set and equal.
        if (string.IsNullOrEmpty(password))
        {
            ShowError(textViewPasswordError,
GetString(Resource.String.entry_error_password_required));
            valid = false;
        }
        else if (string.IsNullOrEmpty(confirmPassword))
        {

```



```

        ShowError(textViewConfirmPasswordError,
GetString(Resource.String.entry_error_confirm_password_required));
        valid = false;
    }
    else if (password != confirmPassword)
    {
        ShowError(textViewConfirmPasswordError,
GetString(Resource.String.entry_error_passwords_mismatch));
        valid = false;
    }
}

return valid;
}

```

/// <summary>Returns true when an entry with the same site name and username already exists in the vault (excluding the item being edited).</summary>

```

private bool IsDuplicate(string siteName, string username)
{
    return existingEntries.Any(e =>
        string.Equals(e.AccountName, siteName, StringComparison.OrdinalIgnoreCase)
        && string.Equals(e.AccountUsername, username,
StringComparison.OrdinalIgnoreCase));
}

```

/// <summary>Validates that the confirm password field matches the password field and shows or hides the error accordingly.</summary>

```

private void ValidatePasswordsMatch()
{
    string password = editTextPassword.Text;
    string confirm = editTextConfirmPassword.Text;
    if (string.IsNullOrEmpty(confirm))
    {
        HideError(textViewConfirmPasswordError);
        return;
    }
    if (password != confirm)
        ShowError(textViewConfirmPasswordError,
GetString(Resource.String.entry_error_passwords_mismatch));
}

```

```

else
    HideError(textViewConfirmPasswordError);
}

// -- Password strength indicator (informational) -----

/// <summary>Updates the password strength progress bar and hint text based on the
given password.</summary>
private void UpdatePasswordStrengthIndicator(string password)
{
    if (string.IsNullOrEmpty(password))
    {
        progressBarStrength.Visibility = ViewStates.Gone;
        textViewStrengthHint.Visibility = ViewStates.Gone;
        HideError(textViewPasswordError);
        return;
    }

    int score = ValidationHelper.GetPasswordScore(password);
    progressBarStrength.Visibility = ViewStates.Visible;
    progressBarStrength.Progress = score;
    progressBarStrength.ProgressTintList =
        ColorStateList.ValueOf(new
Android.Graphics.Color(Resources.GetColor(ScoreToColorRes(score))));

    string hint = ValidationHelper.GetMissingCriteriaHint(password);
    textViewStrengthHint.Text = hint;
    textViewStrengthHint.Visibility = ViewStates.Visible;
    textViewStrengthHint.SetTextColor(new Android.Graphics.Color(
        Resources.GetColor(score == 5
            ? Resource.Color.passwordStrengthVeryStrong
            : Resource.Color.signupHintText)));
}

/// <summary>Maps a password score (1â€“5) to the corresponding color resource
ID.</summary>
private static int ScoreToColorRes(int score)
{
    switch (score)

```

```

    {
        case 1: return Resource.Color.passwordStrengthWeak;
        case 2: return Resource.Color.passwordStrengthPoor;
        case 3: return Resource.Color.passwordStrengthFair;
        case 4: return Resource.Color.passwordStrengthStrong;
        default: return Resource.Color.passwordStrengthVeryStrong;
    }
}

// -- UI helpers -----

/// <summary>Sets the error text and makes the given error view visible.</summary>
private void ShowError(TextView errorView, string message)
{
    errorView.Text = message;
    errorView.Visibility = ViewStates.Visible;
}

/// <summary>Clears the error text and hides the given error view.</summary>
private void HideError(TextView errorView)
{
    errorView.Text = null;
    errorView.Visibility = ViewStates.Gone;
}

/// <summary>Hides all field-level and general error messages.</summary>
private void ClearErrors()
{
    HideError(textViewSiteNameError);
    HideError(textViewUsernameError);
    HideError(textViewPasswordError);
    HideError(textViewConfirmPasswordError);
    HideError(textViewGeneralError);
}

/// <summary>Toggles the loading state, showing the progress bar and disabling form
controls.</summary>
private void SetLoadingState(bool isLoading)
{

```

```

        progressBar.Visibility = isLoading ? ViewStates.Visible : ViewStates.Gone;
        buttonSave.Enabled = !isLoading;
        buttonGeneratePassword.Enabled = !isLoading;
        editTextSiteName.Enabled = !isLoading;
        editTextUsername.Enabled = !isLoading;
        editTextPassword.Enabled = !isLoading;
        editTextConfirmPassword.Enabled = !isLoading;
        editTextNotes.Enabled = !isLoading;
    }
}
}

```

--- FILE: FinalProject333057891\SecurioClient\Activities\LoginActivity.cs ---

```

using Android.App;
using Android.OS;
using Android.Text;
using Android.Views;
using Android.Widget;
using AndroidX.AppCompat.App;
using Google.Android.Material.Button;
using Google.Android.Material.TextField;
using SecurioClient.Helpers;
using SecurioClient.Helpers.ServerHelpers;
using System;
using System.Threading.Tasks;

namespace SecurioClient.Activities
{
    [Activity(Label = "@string/app_name", Theme = "@style/AppTheme.NoActionBar")]
    /// <summary>Activity that provides the login form and authenticates the user.</summary>
    public class LoginActivity : AppCompatActivity
    {
        private TextInputEditText editTextEmail;
        private TextInputEditText editTextPassword;
        private MaterialButton buttonLogin;
        private TextView textViewEmailError;
        private TextView textViewPasswordError;
        private TextView textViewGeneralError;
    }
}

```

```

private TextView textViewSignupLink;
private ProgressBar progressBarLogin;

/// <summary>Initializes the activity, inflates the layout, and wires up views and
handlers.</summary>
protected override void OnCreate(Bundle savedInstanceState)
{
    base.OnCreate(savedInstanceState);
    Xamarin.Essentials.Platform.Init(this, savedInstanceState);
    SetContentView(Resource.Layout.activity_login);

    InitializeViews();
    SetupEventHandlers();
}

/// <summary>Finds and assigns all view references from the layout.</summary>
private void InitializeViews()
{
    editTextEmail =
FindViewById<TextInputEditText>(Resource.Id.editTextLoginEmail);
    editTextPassword =
FindViewById<TextInputEditText>(Resource.Id.editTextLoginPassword);
    buttonLogin = FindViewById<MaterialButton>(Resource.Id.buttonLogin);
    textViewEmailError =
FindViewById<TextView>(Resource.Id.textViewLoginEmailError);
    textViewPasswordError =
FindViewById<TextView>(Resource.Id.textViewLoginPasswordError);
    textViewGeneralError =
FindViewById<TextView>(Resource.Id.textViewLoginGeneralError);
    textViewSignupLink = FindViewById<TextView>(Resource.Id.textViewSignupLink);
    progressBarLogin = FindViewById<ProgressBar>(Resource.Id.progressBarLogin);
}

/// <summary>Wires up click and text-change event handlers for login form
controls.</summary>
private void SetupEventHandlers()
{
    buttonLogin.Click += async (sender, e) => await OnLoginClicked();
}

```

```

textViewSignupLink.Click += (sender, e) =>
{
    var intent = new Android.Content.Intent(this, typeof(SignupActivity));
    StartActivity(intent);
    Finish();
};

// Real-time validation feedback
editTextEmail.AddTextChangedListener(new SimpleTextWatcher(_ =>
{
    var result = ValidationHelper.ValidateEmail(editTextEmail.Text?.Trim());
    if (!result.IsValid)
        ShowError(textViewEmailError, result.ErrorMessage);
    else
        HideError(textViewEmailError);
}));

editTextPassword.AddTextChangedListener(new SimpleTextWatcher(_ =>
{
    if (string.IsNullOrEmpty(editTextPassword.Text))
        ShowError(textViewPasswordError,
GetString(Resource.String.login_error_password_required));
    else
        HideError(textViewPasswordError);
}));
}

/// <summary>Validates inputs and attempts to log in the user via
AuthService.</summary>
private async Task OnLoginClicked()
{
    ClearErrors();

    string email = editTextEmail.Text?.Trim();
    string password = editTextPassword.Text;

    if (!ValidateInputs(email, password))
        return;

```

```

SetLoadingState(true);

try
{
    var authService = new AuthService();
    var result = await authService.LoginAsync(email, password);

    if (result.Success)
    {
        MainActivity.StartPasswordMonitor(this);
        var intent = new Android.Content.Intent(this, typeof(VaultActivity));
        intent.SetFlags(Android.Content.ActivityFlags.NewTask |
Android.Content.ActivityFlags.ClearTask);
        StartActivity(intent);
        Finish();
    }
    else
    {
        ShowGeneralError(result.Message);
    }
}
catch (Exception ex)
{
    ShowGeneralError(GetString(Resource.String.login_error_sign_in_failed));
    System.Diagnostics.Debug.WriteLine($"[LOGIN ERROR] {ex.Message}");
}
finally
{
    SetLoadingState(false);
}
}

/// <summary>Validates the email and password fields and shows errors for any invalid
values.</summary>
private bool ValidateInputs(string email, string password)
{
    bool isValid = true;

    var emailResult = ValidationHelper.ValidateEmail(email);

```

```

        if (!emailResult.IsValid)
        {
            ShowError(textViewEmailError, emailResult.ErrorMessage);
            isValid = false;
        }

        if (string.IsNullOrEmpty(password))
        {
            ShowError(textViewPasswordError,
GetString(Resource.String.login_error_password_required));
            isValid = false;
        }

        return isValid;
    }

    /// <summary>Sets the error text and makes the given error view visible.</summary>
    private void ShowError(TextView errorView, string message)
    {
        errorView.Text = message;
        errorView.Visibility = ViewStates.Visible;
    }

    /// <summary>Clears the error text and hides the given error view.</summary>
    private void HideError(TextView errorView)
    {
        errorView.Text = null;
        errorView.Visibility = ViewStates.Gone;
    }

    /// <summary>Displays the general error message banner.</summary>
    private void ShowGeneralError(string message)
    {
        textViewGeneralError.Text = message;
        textViewGeneralError.Visibility = ViewStates.Visible;
    }

    /// <summary>Hides all field-level and general error messages.</summary>
    private void ClearErrors()

```



```

    {
        HideError(textViewEmailError);
        HideError(textViewPasswordError);
        HideError(textViewGeneralError);
    }

    /// <summary>Toggles the loading state, showing the progress bar and disabling form
    controls.</summary>
    private void SetLoadingState(bool isLoading)
    {
        progressBarLogin.Visibility = isLoading ? ViewStates.Visible : ViewStates.Gone;
        buttonLogin.Enabled = !isLoading;
        editTextEmail.Enabled = !isLoading;
        editTextPassword.Enabled = !isLoading;
    }

}
}

--- FILE: FinalProject333057891\SecurioClient\Activities\MainActivity.cs ---
using Android.App;
using Android.OS;
using Android.Runtime;
using AndroidX.AppCompat.App;
using SecurioClient.Helpers;
using SecurioClient.Helpers.ServerHelpers;

namespace SecurioClient.Activities
{
    [Activity(Label = "@string/app_name", Theme = "@style/AppTheme.NoActionBar",
    MainLauncher = true)]
    /// <summary>Entry-point activity that validates the stored JWT and routes to VaultActivity
    or LoginActivity.</summary>
    public class MainActivity : AppCompatActivity
    {
        private const int RequestCodeNotificationPermission = 1001;

        private bool _pendingVaultNavigation = false;

```

```

/// <summary>Initializes the activity, validates the stored JWT, and navigates to the
appropriate screen.</summary>
protected override async void OnCreate(Bundle savedInstanceState)
{
    base.OnCreate(savedInstanceState);
    Xamarin.Essentials.Platform.Init(this, savedInstanceState);
    SetContentView(Resource.Layout.activity_main);

    // Check whether the user already has a stored JWT and validate it with the server.
    // A valid token means the user is already authenticated and can go straight to the Vault.
    string jwt = await StorageHelper.GetJwt();

    if (!string.IsNullOrEmpty(jwt))
    {
        var authService = new AuthService();
        bool tokenValid = await authService.ValidateTokenAsync();

        if (tokenValid)
        {
            // Restore the vault key from secure storage so in-memory encryption is ready.
            string vaultKey = await StorageHelper.GetVaultKey();
            if (!string.IsNullOrEmpty(vaultKey))
                SessionHelper.StartSession(vaultKey);

            // Fetch the user's vault items into the in-memory cache.
            await authService.FetchAndCacheVaultAsync();

            // Compute password-health warnings for the restored session.
            if (!string.IsNullOrEmpty(vaultKey))
            {
                SessionHelper.CachedWarnings = await
WarningsHelper.ComputeWarningsAsync(
                SessionHelper.CachedVault, vaultKey);
            }

            StartPasswordMonitor(this);
            _pendingVaultNavigation = true;
        }
    }
}

```

```

else
{
    // Token is invalid or expired — clear stale credentials before going to login.
    StorageHelper.ClearAll();
}
}

if (Build.VERSION.SdkInt >= BuildVersionCodes.Tiramisu &&
    CheckSelfPermission(Android.Manifest.Permission.PostNotifications)
    != Android.Content.PM.Permission.Granted)
{
    if
(ShouldShowRequestPermissionRationale(Android.Manifest.Permission.PostNotifications))
    {
        System.Diagnostics.Debug.WriteLine("[NOTIFICATIONS] Showing
POST_NOTIFICATIONS permission prompt after prior denial.");
    }

    RequestPermissions(
        new[] { Android.Manifest.Permission.PostNotifications },
        RequestCodeNotificationPermission);
}
else
{
    NavigateToDest();
}
}

/// <summary>Handles the notification permission result and navigates to the destination
screen.</summary>
public override void OnRequestPermissionsResult(int requestCode, string[] permissions,
[GeneratedEnum] Android.Content.PM.Permission[] grantResults)
{
    Xamarin.Essentials.Platform.OnRequestPermissionsResult(requestCode, permissions,
grantResults);
    base.OnRequestPermissionsResult(requestCode, permissions, grantResults);

    if (requestCode == RequestCodeNotificationPermission)
    {

```

```

        bool granted = grantResults?.Length > 0 &&
            grantResults[0] == Android.Content.PM.Permission.Granted;
        System.Diagnostics.Debug.WriteLine(
            $"[NOTIFICATIONS] POST_NOTIFICATIONS permission {(granted ? "granted"
: "denied")}.");

        NavigateToDest();
    }
}

/// <summary>Navigates to VaultActivity if a valid session exists, otherwise to
LoginActivity.</summary>
private void NavigateToDest()
{
    Android.Content.Intent intent;
    if (_pendingVaultNavigation)
    {
        intent = new Android.Content.Intent(this, typeof(VaultActivity));
        intent.SetFlags(Android.Content.ActivityFlags.NewTask |
Android.Content.ActivityFlags.ClearTask);
    }
    else
    {
        intent = new Android.Content.Intent(this, typeof(LoginActivity));
    }

    StartActivity(intent);
    Finish();
}

/// <summary>Starts the PasswordMonitorService as a foreground service if it is not
already running.</summary>
public static void StartPasswordMonitor(Android.Content.Context context)
{
    var serviceIntent = new Android.Content.Intent(context,
typeof(PasswordMonitorService));
    context.StartForegroundService(serviceIntent);
}
}

```

```
}
```

--- FILE: FinalProject333057891\SecurioClient\Activities\ProfileActivity.cs ---

```
using Android.App;
using Android.Content;
using Android.OS;
using Android.Views;
using Android.Widget;
using AndroidX.AppCompat.App;
using Google.Android.Material.Button;
using SecurioClient.Helpers;
using SecurioClient.Helpers.ServerHelpers;
using SecurioModels.DataTransferObjects;
using System;
using System.Threading.Tasks;

namespace SecurioClient.Activities
{
    [Activity(Label = "@string/app_name", Theme = "@style/AppTheme.NoActionBar")]
    /// <summary>Activity that displays the user's profile information with options to edit, log
    out, or delete the account.</summary>
    public class ProfileActivity : AppCompatActivity
    {
        private TextView textViewProfileUsername;
        private TextView textViewProfileEmail;
        private TextView textViewProfileLastLogin;
        private TextView textViewProfileLastPasswordChange;
        private TextView textViewProfileCreatedAt;
        private TextView textViewProfilePasswordCount;
        private MaterialButton buttonProfileEdit;
        private MaterialButton buttonProfileLogout;
        private MaterialButton buttonProfileDelete;
        private ProgressBar progressBarProfile;

        /// <summary>Initializes the activity, inflates the layout, and loads the user
        profile.</summary>
        protected override async void OnCreate(Bundle savedInstanceState)
        {
```

```

base.OnCreate(savedInstanceState);
Xamarin.Essentials.Platform.Init(this, savedInstanceState);
SetContentView(Resource.Layout.activity_profile);

InitializeViews();
SetupBottomNavFragment(savedInstanceState);
SetupEventHandlers();

await LoadProfileAsync();
}

/// <summary>Finds and assigns all view references from the layout.</summary>
private void InitializeViews()
{
    textViewProfileUsername =
FindViewById<TextView>(Resource.Id.textViewProfileUsername);
    textViewProfileEmail =
FindViewById<TextView>(Resource.Id.textViewProfileEmail);
    textViewProfileLastLogin =
FindViewById<TextView>(Resource.Id.textViewProfileLastLogin);
    textViewProfileLastPasswordChange =
FindViewById<TextView>(Resource.Id.textViewProfileLastPasswordChange);
    textViewProfileCreatedAt =
FindViewById<TextView>(Resource.Id.textViewProfileCreatedAt);
    textViewProfilePasswordCount =
FindViewById<TextView>(Resource.Id.textViewProfilePasswordCount);
    buttonProfileEdit = FindViewById<MaterialButton>(Resource.Id.buttonProfileEdit);
    buttonProfileLogout =
FindViewById<MaterialButton>(Resource.Id.buttonProfileLogout);
    buttonProfileDelete =
FindViewById<MaterialButton>(Resource.Id.buttonProfileDelete);
    progressBarProfile = FindViewById<ProgressBar>(Resource.Id.progressBarProfile);
}

/// <summary>Adds the BottomNavFragment on first creation to avoid duplicate
fragments on configuration change.</summary>
private void SetupBottomNavFragment(Bundle savedInstanceState)
{
    if (savedInstanceState == null)

```

```

    {
        var fragment = BottomNavFragment.NewInstance("profile");
        fragment.TabSelected += OnBottomNavTabSelected;

        SupportFragmentManager
            .BeginTransaction()
            .Replace(Resource.Id.frameBottomNav, fragment)
            .Commit();
    }
}

/// <summary>Wires up click handlers for the edit, logout, and delete account
buttons.</summary>
private void SetupEventHandlers()
{
    buttonProfileEdit.Click += (sender, e) =>
    {
        var intent = new Intent(this, typeof(EditAccountActivity));
        StartActivityForResult(intent, EditAccountActivity.RequestCodeEditAccount);
    };

    buttonProfileLogout.Click += (sender, e) => ConfirmLogout();
    buttonProfileDelete.Click += (sender, e) => ConfirmDeleteAccount();
}

/// <summary>Delegates bottom navigation tab selection to
BottomNavHelper.</summary>
private void OnBottomNavTabSelected(object sender, string tab)
    => BottomNavHelper.Navigate(this, tab, "profile");

/// <summary>Reloads the profile after a successful account edit.</summary>
protected override async void OnActivityResult(int requestCode, Result resultCode,
Intent data)
{
    base.OnActivityResult(requestCode, resultCode, data);

    if (requestCode == EditAccountActivity.RequestCodeEditAccount
        && resultCode == Result.Ok
        && data?.GetBooleanExtra(EditAccountActivity.ResultUpdated, false) == true)

```

```

    {
        // Refresh the displayed profile after a successful account edit.
        await LoadProfileAsync();
    }
}

/// <summary>Fetches the user profile from the server and displays it, falling back to
cached data on failure.</summary>
private async Task LoadProfileAsync()
{
    progressBarProfile.Visibility = ViewStates.Visible;

    try
    {
        var profileService = new ProfileService();
        var result = await profileService.GetProfileAsync();

        if (result.Success && result.Data != null)
        {
            DisplayProfile(result.Data);
        }
        else
        {
            // Fall back to cached profile data
            var cached = await StorageHelper.GetCachedProfileAsync();
            if (cached != null)
            {
                DisplayProfile(cached);
                Toast.MakeText(this, Resource.String.profile_load_error,
ToastLength.Short).Show();
            }
            else
            {
                Toast.MakeText(this, result.Message ?? "Unable to load profile.",
ToastLength.Long).Show();
            }
        }
    }
    catch (Exception ex)

```



```

    {
        System.Diagnostics.Debug.WriteLine($"[PROFILE LOAD ERROR]
{ex.Message}");

        // Fall back to cached profile data
        var cached = await StorageHelper.GetCachedProfileAsync();
        if (cached != null)
        {
            DisplayProfile(cached);
            Toast.MakeText(this, Resource.String.profile_load_error,
ToastLength.Short).Show();
        }
    }
    finally
    {
        progressBarProfile.Visibility = ViewStates.Gone;
    }
}

/// <summary>Populates the UI text views with the given user profile data.</summary>
private void DisplayProfile(User profile)
{
    textViewProfileUsername.Text = profile.Username ?? "â€";
    textViewProfileEmail.Text = profile.Email ?? "â€";
    textViewProfileLastLogin.Text = FormatDate(profile.LastLogin);
    textViewProfileLastPasswordChange.Text =
FormatDate(profile.LastPasswordUpdate);
    textViewProfileCreatedAt.Text = FormatDate(profile.CreatedAt);
    textViewProfilePasswordCount.Text = profile.PasswordCount.ToString();
}

/// <summary>Formats a DateTime as a locale-friendly string, returning "â€" for the
minimum value.</summary>
private string FormatDate(DateTime date)
{
    if (date == DateTime.MinValue)
        return "â€";

    return date.ToLocalTime().ToString("MMM dd, yyyy h:mm tt");
}

```

```

}

/// <summary>Shows a confirmation dialog before logging out.</summary>
private void ConfirmLogout()
{
    new AndroidX.AppCompat.App.AlertDialog.Builder(this)
        .SetTitle(Resource.String.profile_logout_confirm_title)
        .SetMessage(Resource.String.profile_logout_confirm_message)
        .SetPositiveButton(Resource.String.profile_logout_confirm_yes, (s, e) =>
        {
            PerformLogout();
        })
        .SetNegativeButton(Resource.String.profile_logout_confirm_no, (s, e) => { })
        .Show();
}

/// <summary>Clears the session and navigates to LoginActivity.</summary>
private async void PerformLogout()
{
    SessionHelper.EndSession();
    await StorageHelper.ClearSessionAsync();

    var intent = new Intent(this, typeof(LoginActivity));
    intent.SetFlags(ActivityFlags.NewTask | ActivityFlags.ClearTask);
    StartActivity(intent);
    Finish();
}

/// <summary>Shows a confirmation dialog before deleting the account.</summary>
private void ConfirmDeleteAccount()
{
    new AndroidX.AppCompat.App.AlertDialog.Builder(this)
        .SetTitle(Resource.String.profile_delete_confirm_title)
        .SetMessage(Resource.String.profile_delete_confirm_message)
        .SetPositiveButton(Resource.String.profile_delete_confirm_yes, async (s, e) =>
        {
            await DeleteAccountAsync();
        })
        .SetNegativeButton(Resource.String.profile_delete_confirm_no, (s, e) => { })

```

```

        .Show();
    }

    /// <summary>Calls ProfileService to delete the account and navigates to LoginActivity
on success.</summary>
    private async Task DeleteAccountAsync()
    {
        progressBarProfile.Visibility = ViewStates.Visible;
        buttonProfileDelete.Enabled = false;

        try
        {
            var profileService = new ProfileService();
            var result = await profileService.DeleteAccountAsync();

            if (result.Success)
            {
                Toast.MakeText(this, Resource.String.profile_deleted_toast,
ToastLength.Short).Show();

                SessionHelper.EndSession();
                StorageHelper.ClearAll();

                var intent = new Intent(this, typeof(LoginActivity));
                intent.SetFlags(ActivityFlags.NewTask | ActivityFlags.ClearTask);
                StartActivity(intent);
                Finish();
            }
            else
            {
                Toast.MakeText(this, Resource.String.profile_delete_error,
ToastLength.Long).Show();
            }
        }
        catch (Exception ex)
        {
            System.Diagnostics.Debug.WriteLine($"[PROFILE DELETE ERROR]
{ex.Message}");

```

```

        Toast.MakeText(this, Resource.String.profile_delete_error,
ToastLength.Long).Show();
    }
    finally
    {
        progressBarProfile.Visibility = ViewStates.Gone;
        buttonProfileDelete.Enabled = true;
    }
}
}
}

```

--- FILE: FinalProject333057891\SecurioClient\Activities\RiskDetailActivity.cs ---

```

using Android.App;
using Android.Content;
using Android.OS;
using Android.Views;
using Android.Widget;
using AndroidX.AppCompat.App;
using AndroidX.RecyclerView.Widget;
using SecurioClient.Helpers;
using SecurioModels.DataTransferObjects;
using System;
using System.Collections.Generic;
using System.Linq;

namespace SecurioClient.Activities
{
    /// <summary>
    /// Displays the list of vault passwords that fall under a specific risk category
    /// (leaked, weak, reused, or old). Receives the category via an Intent extra and
    /// filters <see cref="SessionHelper.CachedVault"/> accordingly.
    /// Reuses the same <see cref="PasswordBannerAdapter"/> and search-bar pattern as
    /// <see cref="VaultActivity"/>. The kebab icon on each banner opens the same
    /// <see cref="PasswordOptionsBottomSheet"/> as the vault via
    /// <see cref="PasswordEntryActionsHelper"/>.
    /// </summary>
    [Activity(Label = "@string/app_name", Theme = "@style/AppTheme.NoActionBar")]

```

```

public class RiskDetailActivity : AppCompatActivity
{
    /// <summary>Intent extra key for the risk category string ("leaked", "weak", "reused",
    "old").</summary>
    public const string ExtraRiskCategory = "EXTRA_RISK_CATEGORY";

    // Risk category constants matching the values passed via intent.
    public const string CategoryLeaked = "leaked";
    public const string CategoryWeak  = "weak";
    public const string CategoryReused = "reused";
    public const string CategoryOld   = "old";

    // -- Views -----
    private ImageView imageViewBack;
    private TextView textViewEmoji;
    private TextView textViewTitle;
    private TextView textViewSubtitle;
    private EditText editTextSearch;
    private RecyclerView recyclerView;
    private LinearLayout layoutEmpty;

    private PasswordBannerAdapter adapter;
    private List<VaultItem> riskEntries = new List<VaultItem>();
    private string category;

    // -- Lifecycle -----

    protected override void OnCreate(Bundle savedInstanceState)
    {
        base.OnCreate(savedInstanceState);
        Xamarin.Essentials.Platform.Init(this, savedInstanceState);
        SetContentView(Resource.Layout.activity_risk_detail);

        category = Intent.GetStringExtra(ExtraRiskCategory) ?? CategoryLeaked;

        InitializeViews();
        ConfigureHeader();
        SetupRecyclerView();
        SetupEventHandlers();
    }
}

```

```

    _ = LoadRiskEntriesAsync();
}

// -- Setup helpers -----

private void InitializeViews()
{
    imageViewBack = FindViewById<ImageView>(Resource.Id.imageViewRiskBack);
    textViewEmoji = FindViewById<TextView>(Resource.Id.textViewRiskEmoji);
    textViewTitle = FindViewById<TextView>(Resource.Id.textViewRiskTitle);
    textViewSubtitle = FindViewById<TextView>(Resource.Id.textViewRiskSubtitle);
    editTextSearch = FindViewById<EditText>(Resource.Id.editTextRiskSearch);
    recyclerView =
FindViewById<RecyclerView>(Resource.Id.recyclerViewRiskPasswords);
    layoutEmpty = FindViewById<LinearLayout>(Resource.Id.layoutRiskEmpty);
}

/// <summary>
/// Configures the header text and emoji based on the risk category.
/// </summary>
private void ConfigureHeader()
{
    switch (category)
    {
        case CategoryLeaked:
            textViewEmoji.Text = "ðŸ””";
            textViewTitle.Text = GetString(Resource.String.warnings_leaked_title);
            textViewSubtitle.Text = GetString(Resource.String.risk_detail_leaked_subtitle);
            break;
        case CategoryWeak:
            textViewEmoji.Text = "ðŸ””";
            textViewTitle.Text = GetString(Resource.String.warnings_weak_title);
            textViewSubtitle.Text = GetString(Resource.String.risk_detail_weak_subtitle);
            break;
        case CategoryReused:
            textViewEmoji.Text = "â„”•";
            textViewTitle.Text = GetString(Resource.String.warnings_reused_title);
            textViewSubtitle.Text = GetString(Resource.String.risk_detail_reused_subtitle);
    }
}

```

```

        break;
    case CategoryOld:
        textViewEmoji.Text = "⚠️";
        textViewTitle.Text = GetString(Resource.String.warnings_old_title);
        textViewSubtitle.Text = GetString(Resource.String.risk_detail_old_subtitle);
        break;
    }
}

private void SetupRecyclerView()
{
    adapter = new PasswordBannerAdapter(riskEntries);
    recyclerView.SetLayoutManager(new LinearLayoutManager(this));
    recyclerView.SetAdapter(adapter);

    // Both the full-banner tap and the kebab icon open the options bottom sheet.
    adapter.ItemClick += (sender, position) => OnBannerActionAt(position);
    adapter.EditClick += (sender, position) => OnBannerActionAt(position);
}

/// <summary>
/// Resolves the entry at <paramref name="position"/> and opens the options
/// bottom sheet via the shared <see cref="PasswordEntryActionsHelper"/>.
/// </summary>
private void OnBannerActionAt(int position)
{
    var displayed = GetDisplayedEntries();
    if (position >= 0 && position < displayed.Count)
        PasswordEntryActionsHelper.ShowOptionsSheet(
            this,
            displayed[position],
            SyncEntryCache,
            OnEntryDeleted);
}

private void OnEntryDeleted(VaultItem entry)
{
    riskEntries.RemoveAll(x => x.Id == entry.Id);
    RefreshList();
}

```

```

    }

    private void SetupEventHandlers()
    {
        imageViewBack.Click += (s, e) => Finish();

        editTextSearch.AddTextChangedListener(new SimpleTextWatcher(query =>
        {
            FilterPasswords(query);
        }));
    }

    // -- Activity result -----

    protected override void OnActivityResult(int requestCode, Result resultCode, Intent data)
    {
        base.OnActivityResult(requestCode, resultCode, data);

        if (resultCode != Result.Ok || data == null)
            return;

        if (requestCode == EditPasswordActivity.RequestCodeEdit)
        {
            int editedId = data.GetIntExtra(EditPasswordActivity.ResultEntryId, 0);
            var existing = riskEntries.FirstOrDefault(e => e.Id == editedId);
            if (existing != null)
            {
                existing.AccountName =
data.GetStringExtra(EditPasswordActivity.ResultSiteName);
                existing.AccountUsername =
data.GetStringExtra(EditPasswordActivity.ResultUsername);
                existing.Notes = data.GetStringExtra(EditPasswordActivity.ResultNotes);
                existing.IV = data.GetStringExtra(EditPasswordActivity.ResultIV);
                existing.Tag = data.GetStringExtra(EditPasswordActivity.ResultTag);
                existing.CipherText =
data.GetStringExtra(EditPasswordActivity.ResultCipherText);
                existing.Sha1Hash =
data.GetStringExtra(EditPasswordActivity.ResultSha1Hash);
            }
        }
    }

```



```

        existing.IsLeaked    =
data.GetBooleanExtra(EditPasswordActivity.ResultIsLeaked, false);
        existing.LastUpdate  = new
DateTime(data.GetLongExtra(EditPasswordActivity.ResultLastUpdate,
existing.LastUpdate.Ticks));

        SessionHelper.UpdateVaultItem(existing);
        SessionHelper.InvalidateWarnings();
    }

    // Reload the risk list from the updated vault cache so entries that
    // are no longer at risk (e.g. a breached password replaced with a
    // strong one) are removed immediately.
    _ = LoadRiskEntriesAsync();
}

// -- Data loading -----

/// <summary>
/// Filters the cached vault to only items that match the current risk category.
/// </summary>
private async System.Threading.Tasks.Task LoadRiskEntriesAsync()
{
    riskEntries = await WarningsHelper.GetItemsAtRiskAsync(
        SessionHelper.CachedVault,
        SessionHelper.SessionVaultKey,
        category);

    RefreshList();
}

// -- Search / filter helpers -----

private void FilterPasswords(string query)
{
    if (string.IsNullOrEmpty(query))
        adapter.UpdateData(riskEntries);
    else

```

```

        adapter.UpdateData(GetFilteredEntries(query));

        UpdateEmptyState();
    }

    private List<VaultItem> GetFilteredEntries(string query)
    {
        return riskEntries
            .Where(e => (e.AccountName != null && e.AccountName.IndexOf(query,
StringComparison.OrdinalIgnoreCase) >= 0)
                || (e.AccountUsername != null && e.AccountUsername.IndexOf(query,
StringComparison.OrdinalIgnoreCase) >= 0))
            .ToList();
    }

    private List<VaultItem> GetDisplayedEntries()
    {
        string query = editTextSearch.Text?.Trim();
        return string.IsNullOrEmpty(query) ? riskEntries : GetFilteredEntries(query);
    }

    private void RefreshList()
    {
        string query = editTextSearch.Text?.Trim();
        FilterPasswords(query);
    }

    private void UpdateEmptyState()
    {
        bool isEmpty = adapter.ItemCount == 0;
        layoutEmpty.Visibility = isEmpty ? ViewStates.Visible : ViewStates.Gone;
        recyclerView.Visibility = isEmpty ? ViewStates.Gone : ViewStates.Visible;
    }

    /// <summary>
    /// Pushes the full vault cache into <see cref="VaultEntryCache"/> so that
    /// <see cref="EditPasswordActivity"/> can perform duplicate checking.
    /// </summary>
    private void SyncEntryCache()

```

```

    {
        VaultEntryCache.Entries = new List<VaultItem>(SessionHelper.CachedVault ?? new
List<VaultItem>());
    }
}
}

```

--- FILE: FinalProject333057891\SecurioClient\Activities\SignupActivity.cs ---

```

using Android.App;
using Android.OS;
using Android.Text;
using Android.Views;
using Android.Widget;
using AndroidX.AppCompat.App;
using Google.Android.Material.Button;
using Google.Android.Material.TextField;
using SecurioClient.Helpers;
using SecurioClient.Helpers.ServerHelpers;
using SecurioModels.DataTransferObjects;
using System;
using System.Threading.Tasks;

namespace SecurioClient.Activities
{
    [Activity(Label = "@string/app_name", Theme = "@style/AppTheme.NoActionBar")]
    /// <summary>Activity that provides the registration form and creates a new user
account.</summary>
    public class SignupActivity : AppCompatActivity
    {
        private TextInputEditText editTextUsername;
        private TextInputEditText editTextEmail;
        private TextInputEditText editTextPassword;
        private TextInputEditText editTextConfirmPassword;
        private MaterialButton buttonSignUp;
        private MaterialButton buttonGeneratePassword;
        private TextView textViewUsernameError;
        private TextView textViewEmailError;
    }
}

```

```

private TextView textViewPasswordError;
private ProgressBar progressBarPasswordStrength;
private TextView textViewPasswordHint;
private TextView textViewConfirmPasswordError;
private TextView textViewGeneralError;
private TextView textViewLoginLink;
private ProgressBar progressBarSignup;

/// <summary>Initializes the activity, inflates the layout, and wires up views and
handlers.</summary>
protected override void OnCreate(Bundle savedInstanceState)
{
    base.OnCreate(savedInstanceState);
    Xamarin.Essentials.Platform.Init(this, savedInstanceState);
    SetContentView(Resource.Layout.activity_signup);

    InitializeViews();
    SetupEventHandlers();
}

/// <summary>Finds and assigns all view references from the layout.</summary>
private void InitializeViews()
{
    editTextUsername =
    FindViewById<TextInputEditText>(Resource.Id.editTextUsername);
    editTextEmail =
    FindViewById<TextInputEditText>(Resource.Id.editTextEmail);
    editTextPassword =
    FindViewById<TextInputEditText>(Resource.Id.editTextPassword);
    editTextConfirmPassword =
    FindViewById<TextInputEditText>(Resource.Id.editTextConfirmPassword);
    buttonSignUp = FindViewById<MaterialButton>(Resource.Id.buttonSignUp);
    buttonGeneratePassword =
    FindViewById<MaterialButton>(Resource.Id.buttonGeneratePassword);
    textViewUsernameError =
    FindViewById<TextView>(Resource.Id.textViewUsernameError);
    textViewEmailError =
    FindViewById<TextView>(Resource.Id.textViewEmailError);

```

```

        textViewPasswordError =
FindViewById<TextView>(Resource.Id.textViewPasswordError);
        progressBarPasswordStrength =
FindViewById<ProgressBar>(Resource.Id.progressBarPasswordStrength);
        textViewPasswordHint =
FindViewById<TextView>(Resource.Id.textViewPasswordHint);
        textViewConfirmPasswordError =
FindViewById<TextView>(Resource.Id.textViewConfirmPasswordError);
        textViewGeneralError =
FindViewById<TextView>(Resource.Id.textViewGeneralError);
        textViewLoginLink =
FindViewById<TextView>(Resource.Id.textViewLoginLink);
        progressBarSignup =
FindViewById<ProgressBar>(Resource.Id.progressBarSignup);
    }

```

/// <summary>Wires up click and text-change event handlers for signup form controls.</summary>

```
private void SetupEventHandlers()
```

```
{
```

```
    buttonSignUp.Click += async (sender, e) => await OnSignUpClicked();
```

```
    textViewLoginLink.Click += (sender, e) =>
```

```
    {
```

```
        var intent = new Android.Content.Intent(this, typeof(LoginActivity));
```

```
        StartActivity(intent);
```

```
        Finish();
```

```
    };
```

```
// Generate a strong password and fill ONLY the password field.
```

```
buttonGeneratePassword.Click += (sender, e) =>
```

```
{
```

```
    string generated = ValidationHelper.GenerateStrongPassword();
```

```
    editTextPassword.Text = generated;
```

```
    // Confirm password intentionally left blank â€” user must type it themselves.
```

```
};
```

```
// Real-time validation feedback as the user types
```

```
editTextUsername.AddTextChangedListener(new SimpleTextWatcher(_ =>
```

```

{
    var result = ValidationHelper.ValidateUsername(editTextUsername.Text?.Trim());
    if (!result.IsValid) FormUiHelper.ShowError(textViewUsernameError,
result.ErrorMessage);
    else FormUiHelper.HideError(textViewUsernameError);
});

editTextEmail.AddTextChangedListener(new SimpleTextWatcher(_ =>
{
    var result = ValidationHelper.ValidateEmail(editTextEmail.Text?.Trim());
    if (!result.IsValid) FormUiHelper.ShowError(textViewEmailError,
result.ErrorMessage);
    else FormUiHelper.HideError(textViewEmailError);
}));

editTextPassword.AddTextChangedListener(new SimpleTextWatcher(text =>
{
    FormUiHelper.UpdatePasswordStrengthIndicator(
        text, progressBarPasswordStrength, textViewPasswordHint,
textViewPasswordError, Resources);

    // Re-validate confirm password if the user has already typed in it.
    if (!string.IsNullOrEmpty(editTextConfirmPassword.Text))
    {
        var matchResult = ValidationHelper.ValidatePasswordsMatch(text,
editTextConfirmPassword.Text);
        if (!matchResult.IsValid)
FormUiHelper.ShowError(textViewConfirmPasswordError, matchResult.ErrorMessage);
        else FormUiHelper.HideError(textViewConfirmPasswordError);
    }
}));

editTextConfirmPassword.AddTextChangedListener(new SimpleTextWatcher(_ =>
{
    var result = ValidationHelper.ValidatePasswordsMatch(
        editTextPassword.Text, editTextConfirmPassword.Text);
    if (!result.IsValid) FormUiHelper.ShowError(textViewConfirmPasswordError,
result.ErrorMessage);
    else FormUiHelper.HideError(textViewConfirmPasswordError);
}));

```

```

    });
}

/// <summary>Validates inputs and registers a new user via AuthService.</summary>
private async Task OnSignUpClicked()
{
    ClearErrors();

    string username    = editTextUsername.Text?.Trim();
    string email       = editTextEmail.Text?.Trim();
    string password    = editTextPassword.Text;
    string confirmPassword = editTextConfirmPassword.Text;

    if (!ValidateInputs(username, email, password, confirmPassword))
        return;

    SetLoadingState(true);

    try
    {
        // Compute SHA-1 of the plaintext password BEFORE key derivation.
        // This hash is only used for the HIBP breach check on the server
        // and is never persisted to the database.
        string passwordSha1Hash = EncryptionHelper.ComputeSha1Hash(password);

        string authSalt    = EncryptionHelper.GenerateSalt();
        string encryptionSalt = EncryptionHelper.GenerateSalt();
        string masterPasswordKey = EncryptionHelper.DeriveKey(password, authSalt);

        var newUser = new User
        {
            Username    = username,
            Email       = email,
            MasterPasswordKey = masterPasswordKey,
            AuthSalt    = authSalt,
            EncryptionSalt = encryptionSalt,
            PasswordSha1Hash = passwordSha1Hash
        };
    }
}

```

```

var authService = new AuthService();
var result = await authService.RegisterAsync(newUser, password);

if (result.Success)
{
    var intent = new Android.Content.Intent(this, typeof(VaultActivity));
    intent.SetFlags(Android.Content.ActivityFlags.NewTask |
Android.Content.ActivityFlags.ClearTask);
    StartActivity(intent);
    Finish();
}
else
{
    ShowGeneralError(result.Message);
}
}
catch (Exception ex)
{
    ShowGeneralError(GetString(Resource.String.signup_error_create_failed));
    System.Diagnostics.Debug.WriteLine($"[SIGNUP ERROR] {ex.Message}");
}
finally
{
    SetLoadingState(false);
}
}

/// <summary>Validates all signup form fields and shows errors for any invalid
values.</summary>
private bool ValidateInputs(string username, string email, string password, string
confirmPassword)
{
    bool isValid = true;

    var usernameResult = ValidationHelper.ValidateUsername(username);
    if (!usernameResult.IsValid) { FormUiHelper.ShowError(textViewUsernameError,
usernameResult.ErrorMessage); isValid = false; }

    var emailResult = ValidationHelper.ValidateEmail(email);

```



```

        if (!emailResult.IsValid) { FormUiHelper.ShowError(textViewEmailError,
emailResult.ErrorMessage); isValid = false; }

        var passwordResult = ValidationHelper.ValidatePassword(password);
        if (!passwordResult.IsValid) { FormUiHelper.ShowError(textViewPasswordError,
passwordResult.ErrorMessage); isValid = false; }

        var confirmResult = ValidationHelper.ValidatePasswordsMatch(password,
confirmPassword);
        if (!confirmResult.IsValid) {
FormUiHelper.ShowError(textViewConfirmPasswordError, confirmResult.ErrorMessage);
isValid = false; }

        return isValid;
    }

    /// <summary>Displays the general error message banner.</summary>
    private void ShowGeneralError(string message)
    {
        textViewGeneralError.Text = message;
        textViewGeneralError.Visibility = ViewStates.Visible;
    }

    /// <summary>Hides all field-level and general error messages.</summary>
    private void ClearErrors()
    {
        FormUiHelper.HideError(textViewUsernameError);
        FormUiHelper.HideError(textViewEmailError);
        FormUiHelper.HideError(textViewPasswordError);
        FormUiHelper.HideError(textViewConfirmPasswordError);
        FormUiHelper.HideError(textViewGeneralError);
    }

    /// <summary>Toggles the loading state, showing the progress bar and disabling form
controls.</summary>
    private void SetLoadingState(bool isLoading)
    {
        progressBarSignup.Visibility = isLoading ? ViewStates.Visible : ViewStates.Gone;
        buttonSignUp.Enabled = !isLoading;
    }

```

```

        buttonGeneratePassword.Enabled = !isLoading;
        editTextUsername.Enabled      = !isLoading;
        editTextEmail.Enabled         = !isLoading;
        editTextPassword.Enabled      = !isLoading;
        editTextConfirmPassword.Enabled = !isLoading;
    }
}
}

```

--- FILE: FinalProject333057891\SecurioClient\Activities\VaultActivity.cs ---

```

using Android.App;
using Android.Content;
using Android.Content.PM;
using Android.OS;
using Android.Views;
using Android.Widget;
using AndroidX.AppCompat.App;
using AndroidX.RecyclerView.Widget;
using SecurioClient.Helpers;
using SecurioModels.DataTransferObjects;
using System;
using System.Collections.Generic;
using System.Linq;

```

```

namespace SecurioClient.Activities

```

```

{
    [Activity(Label = "@string/app_name", Theme = "@style/AppTheme.NoActionBar")]
    /// <summary>Activity that displays the user's password vault entries with search and
    management capabilities.</summary>
    public class VaultActivity : AppCompatActivity
    {
        private const int RequestCodeNotificationPermission = 9001;
        private const string PostNotificationsPermission =
"android.permission.POST_NOTIFICATIONS";

```

```

        private TextView textViewVaultTitle;
        private TextView textViewVaultSubtitle;
        private EditText editTextVaultSearch;

```

```

private RecyclerView recyclerViewPasswords;
private LinearLayout layoutVaultEmpty;

private PasswordBannerAdapter adapter;
private List<VaultItem> allEntries = new List<VaultItem>();

/// <summary>Initializes the activity, inflates the layout, and loads vault entries from the
session cache.</summary>
protected override void OnCreate(Bundle savedInstanceState)
{
    base.OnCreate(savedInstanceState);
    Xamarin.Essentials.Platform.Init(this, savedInstanceState);
    SetContentView(Resource.Layout.activity_vault);

    InitializeViews();
    SetupRecyclerView();
    SetupBottomNavFragment(savedInstanceState);
    SetupEventHandlers();

    // Load entries from the in-memory session cache.
    LoadVaultFromSession();

    // Request notification permission (required at runtime on Android 13+).
    RequestNotificationPermissionIfNeeded();
}

/// <summary>Finds and assigns all view references from the layout.</summary>
private void InitializeViews()
{
    textViewVaultTitle = FindViewById<TextView>(Resource.Id.textViewVaultTitle);
    textViewVaultSubtitle =
FindViewById<TextView>(Resource.Id.textViewVaultSubtitle);
    editTextVaultSearch = FindViewById<EditText>(Resource.Id.editTextVaultSearch);
    recyclerViewPasswords =
FindViewById<RecyclerView>(Resource.Id.recyclerViewPasswords);
    layoutVaultEmpty = FindViewById<LinearLayout>(Resource.Id.layoutVaultEmpty);
}

```

```

/// <summary>Configures the RecyclerView with its adapter and item-click
handlers.</summary>
private void SetupRecyclerView()
{
    adapter = new PasswordBannerAdapter(allEntries);
    recyclerViewPasswords.SetLayoutManager(new LinearLayoutManager(this));
    recyclerViewPasswords.SetAdapter(adapter);

    // Both the full-banner tap and the more-options icon tap open the options sheet.
    adapter.ItemClick += (sender, position) => OnBannerActionAt(position);
    adapter.EditClick += (sender, position) => OnBannerActionAt(position);
}

/// <summary>Resolves the entry at the given position in the displayed list and opens the
options bottom sheet.</summary>
private void OnBannerActionAt(int position)
{
    var displayed = GetDisplayedEntries();
    if (position >= 0 && position < displayed.Count)
        PasswordEntryActionsHelper.ShowOptionsSheet(
            this,
            displayed[position],
            SyncEntryCache,
            OnEntryDeleted);
}

/// <summary>Removes the deleted entry from the local list and refreshes the
RecyclerView.</summary>
private void OnEntryDeleted(VaultItem entry)
{
    allEntries.RemoveAll(x => x.Id == entry.Id);
    RefreshList();
}

/// <summary>Adds the BottomNavFragment on first creation to avoid duplicate
fragments on configuration change.</summary>
private void SetupBottomNavFragment(Bundle savedInstanceState)
{
    // Only add the fragment on fresh creation to avoid duplicates on configuration change.

```

```

if (savedInstanceState == null)
{
    var fragment = BottomNavFragment.NewInstance("vault");
    fragment.TabSelected += OnBottomNavTabSelected;

    SupportFragmentManager
        .BeginTransaction()
        .Replace(Resource.Id.frameBottomNav, fragment)
        .Commit();
}
}

/// <summary>Wires up the search field and FAB click handlers.</summary>
private void SetupEventHandlers()
{
    // Live search filtering
    editTextVaultSearch.AddTextChangedListener(new SimpleTextWatcher(query =>
    {
        FilterPasswords(query);
    }));

    // FAB "open AddPasswordActivity.
    FindViewById(Resource.Id.fabAddPassword).Click += (sender, e) =>
    {
        SyncEntryCache();
        var intent = new Intent(this, typeof(AddPasswordActivity));
        StartActivityForResult(intent, AddPasswordActivity.RequestCodeAdd);
    };
}

/// <summary>Delegates bottom navigation tab selection to
BottomNavHelper.</summary>
private void OnBottomNavTabSelected(object sender, string tab)
    => BottomNavHelper.Navigate(this, tab, "vault");

// -----
// Activity result handling
// -----

```

```

/// <summary>Handles results from AddPasswordActivity and EditPasswordActivity,
updating the local vault list.</summary>
protected override void OnActivityResult(int requestCode, Result resultCode, Intent data)
{
    base.OnActivityResult(requestCode, resultCode, data);

    if (resultCode != Result.Ok || data == null)
        return;

    if (requestCode == AddPasswordActivity.RequestCodeAdd)
    {
        var newItem = new VaultItem
        {
            Id = data.GetIntExtra(AddPasswordActivity.ResultEntryId, 0),
            AccountName = data.GetStringExtra(AddPasswordActivity.ResultSiteName),
            AccountUsername = data.GetStringExtra(AddPasswordActivity.ResultUsername),
            Notes = data.GetStringExtra(AddPasswordActivity.ResultNotes),
            IV = data.GetStringExtra(AddPasswordActivity.ResultIV),
            Tag = data.GetStringExtra(AddPasswordActivity.ResultTag),
            CipherText = data.GetStringExtra(AddPasswordActivity.ResultCipherText),
            Sha1Hash = data.GetStringExtra(AddPasswordActivity.ResultSha1Hash),
            IsLeaked = data.GetBooleanExtra(AddPasswordActivity.ResultIsLeaked,
false),
            LastUpdate = new
DateTime(data.GetLongExtra(AddPasswordActivity.ResultLastUpdate,
DateTime.UtcNow.Ticks))
        };

        allEntries.Add(newItem);
        SessionHelper.AddVaultItem(newItem);
        SessionHelper.InvalidateWarnings();
        RefreshList();
    }
    else if (requestCode == EditPasswordActivity.RequestCodeEdit)
    {
        int editedId = data.GetIntExtra(EditPasswordActivity.ResultEntryId, 0);
        var existing = allEntries.FirstOrDefault(e => e.Id == editedId);
        if (existing != null)
        {

```

```

        existing.AccountName =
data.GetStringExtra(EditPasswordActivity.ResultSiteName);
        existing.AccountUsername =
data.GetStringExtra(EditPasswordActivity.ResultUsername);
        existing.Notes = data.GetStringExtra(EditPasswordActivity.ResultNotes);
        existing.IV = data.GetStringExtra(EditPasswordActivity.ResultIV);
        existing.Tag = data.GetStringExtra(EditPasswordActivity.ResultTag);
        existing.CipherText =
data.GetStringExtra(EditPasswordActivity.ResultCipherText);
        existing.Sha1Hash =
data.GetStringExtra(EditPasswordActivity.ResultSha1Hash);
        existing.IsLeaked =
data.GetBooleanExtra(EditPasswordActivity.ResultIsLeaked, false);
        existing.LastUpdate = new
DateTime(data.GetLongExtra(EditPasswordActivity.ResultLastUpdate,
existing.LastUpdate.Ticks));

        long lastUpdateTicks =
data.GetLongExtra(EditPasswordActivity.ResultLastUpdate, 0L);
        if (lastUpdateTicks > 0)
            existing.LastUpdate = new DateTime(lastUpdateTicks, DateTimeKind.Utc);

        SessionHelper.UpdateVaultItem(existing);
        SessionHelper.InvalidateWarnings();
    }

    RefreshList();
}

// -----
// Data helpers
// -----

/// <summary>Loads vault entries from the in-memory session cache into the local list
and refreshes the RecyclerView.</summary>
private void LoadVaultFromSession()
{

```

```

        allEntries = new List<VaultItem>(SessionHelper.CachedVault ?? new
List<VaultItem>());
        RefreshList();
    }

    /// <summary>Filters the displayed entries by the given search query and updates the
empty state.</summary>
    private void FilterPasswords(string query)
    {
        if (string.IsNullOrEmpty(query))
        {
            adapter.UpdateData(allEntries);
        }
        else
        {
            adapter.UpdateData(GetFilteredEntries(query));
        }

        UpdateEmptyState();
    }

    /// <summary>Returns the vault entries whose site name or username contains the given
query string.</summary>
    private List<VaultItem> GetFilteredEntries(string query)
    {
        return allEntries
            .Where(e => (e.AccountName != null && e.AccountName.IndexOf(query,
StringComparison.OrdinalIgnoreCase) >= 0)
                || (e.AccountUsername != null && e.AccountUsername.IndexOf(query,
StringComparison.OrdinalIgnoreCase) >= 0))
            .ToList();
    }

    /// <summary>Returns the list currently shown in the adapter (filtered or
full).</summary>
    private List<VaultItem> GetDisplayedEntries()
    {
        string query = editTextVaultSearch.Text?.Trim();
        return string.IsNullOrEmpty(query) ? allEntries : GetFilteredEntries(query);
    }

```



```

    }

    /// <summary>Re-applies the current search filter to refresh the
RecyclerView.</summary>
    private void RefreshList()
    {
        string query = editTextVaultSearch.Text?.Trim();
        FilterPasswords(query);
    }

    /// <summary>Toggles the empty-state placeholder and RecyclerView visibility based on
adapter item count.</summary>
    private void UpdateEmptyState()
    {
        bool isEmpty = adapter.ItemCount == 0;
        layoutVaultEmpty.Visibility = isEmpty ? ViewStates.Visible : ViewStates.Gone;
        recyclerViewPasswords.Visibility = isEmpty ? ViewStates.Gone : ViewStates.Visible;
    }

    /// <summary>Pushes the current entry list into the static cache so that entry activities can
perform duplicate checking.</summary>
    private void SyncEntryCache()
    {
        VaultEntryCache.Entries = new List<VaultItem>(allEntries);
    }

    /// <summary>Requests the POST_NOTIFICATIONS runtime permission on Android
13+ if not already granted.</summary>
    private void RequestNotificationPermissionIfNeeded()
    {
        if (Build.VERSION.SdkInt >= BuildVersionCodes.Tiramisu)
        {
            if (CheckSelfPermission(PostNotificationsPermission) != Permission.Granted)
            {
                RequestPermissions(new[] { PostNotificationsPermission },
RequestCodeNotificationPermission);
            }
        }
    }
}

```

```
}
}
```

--- FILE: FinalProject333057891\SecurioClient\Activities\ViewPasswordActivity.cs ---

```
using Android.App;
using Android.Content;
using Android.OS;
using Android.Widget;
using AndroidX.AppCompat.App;
using SecurioClient.Helpers;
using System;

namespace SecurioClient.Activities
{
    /// <summary>Read-only activity that displays a single vault entry's details with the
    password decrypted client-side.</summary>
    [Activity(Label = "@string/app_name", Theme = "@style/AppTheme.NoActionBar")]
    public class ViewPasswordActivity : AppCompatActivity
    {
        // Intent extras used to pass the item data into this activity.
        public const string ExtraSiteName = "EXTRA_SITE_NAME";
        public const string ExtraUsername = "EXTRA_USERNAME";
        public const string ExtraNotes = "EXTRA_NOTES";
        public const string ExtraIV = "EXTRA_IV";
        public const string ExtraTag = "EXTRA_TAG";
        public const string ExtraCipherText = "EXTRA_CIPHER_TEXT";
        public const string ExtraLastUpdate = "EXTRA_LAST_UPDATE";

        // -- Views -----
        private ImageView imageViewBack;
        private TextView textViewSiteName;
        private TextView textViewUsername;
        private TextView textViewPassword;
        private TextView textViewNotes;
        private TextView textViewLastChanged;
        private ImageView buttonCopyUsername;
        private ImageView buttonTogglePassword;
```

```

private ImageView buttonCopyPassword;

// The decrypted password; kept in memory only for the lifetime of this activity.
private string decryptedPassword;
private bool passwordVisible;

// -- Lifecycle -----

/// <summary>Initializes the activity, inflates the layout, populates fields, and wires up
event handlers.</summary>
protected override void OnCreate(Bundle savedInstanceState)
{
    base.OnCreate(savedInstanceState);
    Xamarin.Essentials.Platform.Init(this, savedInstanceState);
    SetContentView(Resource.Layout.activity_view_password);

    InitializeViews();
    PopulateFields();
    SetupEventHandlers();
}

// -- Setup helpers -----

/// <summary>Finds and assigns all view references from the layout.</summary>
private void InitializeViews()
{
    imageViewBack = FindViewById<ImageView>(Resource.Id.imageViewViewBack);
    textViewSiteName =
FindViewById<TextView>(Resource.Id.textViewViewSiteName);
    textViewUsername =
FindViewById<TextView>(Resource.Id.textViewViewUsername);
    textViewPassword =
FindViewById<TextView>(Resource.Id.textViewViewPassword);
    textViewNotes = FindViewById<TextView>(Resource.Id.textViewViewNotes);
    textViewLastChanged =
FindViewById<TextView>(Resource.Id.textViewViewLastChanged);
    buttonCopyUsername =
FindViewById<ImageView>(Resource.Id.buttonCopyUsername);

```

```

        buttonTogglePassword =
FindViewById<ImageView>(Resource.Id.buttonTogglePassword);
        buttonCopyPassword =
FindViewById<ImageView>(Resource.Id.buttonCopyPassword);
    }

    private void PopulateFields()
    {
        string siteName = Intent.GetStringExtra(ExtraSiteName) ?? string.Empty;
        string username = Intent.GetStringExtra(ExtraUsername) ?? string.Empty;
        string notes = Intent.GetStringExtra(ExtraNotes) ?? string.Empty;

        textViewSiteName.Text = siteName;
        textViewUsername.Text = username;
        textViewNotes.Text = string.IsNullOrEmpty(notes)
            ? GetString(Resource.String.view_no_notes)
            : notes;

        // Display "Last changed" date.
        long lastUpdateTicks = Intent.GetLongExtra(ExtraLastUpdate, 0L);
        if (lastUpdateTicks > 0)
        {
            var lastUpdate = new DateTime(lastUpdateTicks, DateTimeKind.Utc);
            textViewLastChanged.Text = lastUpdate.ToString("MMM d, yyyy 'at' h: mm tt
'UTC'");
        }
        else
        {
            textViewLastChanged.Text = "â€";
        }

        // Decrypt the password.
        string iv = Intent.GetStringExtra(ExtraIV) ?? string.Empty;
        string tag = Intent.GetStringExtra(ExtraTag) ?? string.Empty;
        string cipherText = Intent.GetStringExtra(ExtraCipherText) ?? string.Empty;

        if (!string.IsNullOrEmpty(iv) && !string.IsNullOrEmpty(tag) &&
!string.IsNullOrEmpty(cipherText))
        {

```

```

        try
        {
            decryptedPassword = EncryptionHelper.DecryptAesGcm(iv, tag, cipherText,
SessionHelper.SessionVaultKey);
        }
        catch
        {
            System.Diagnostics.Debug.WriteLine("[VIEW PASSWORD ERROR] Failed to
decrypt password");
            decryptedPassword = null;
            Toast.MakeText(this, GetString(Resource.String.view_decrypt_error),
ToastLength.Short).Show();
        }
    }
else
    {
        decryptedPassword = null;
    }

    // Show masked password by default.
    passwordVisible = false;
    UpdatePasswordDisplay();
}

private void SetupEventHandlers()
{
    imageViewBack.Click += (s, e) => Finish();

    buttonCopyUsername.Click += (s, e) =>
    {
        CopyToClipboard("username", textViewUsername.Text);
        Toast.MakeText(this, GetString(Resource.String.view_copied_username),
ToastLength.Short).Show();
    };

    buttonTogglePassword.Click += (s, e) =>
    {
        passwordVisible = !passwordVisible;
        UpdatePasswordDisplay();
    }
}

```

```

    };

    buttonCopyPassword.Click += (s, e) =>
    {
        if (!string.IsNullOrEmpty(decryptedPassword))
        {
            CopyToClipboard("password", decryptedPassword);
            Toast.MakeText(this, GetString(Resource.String.view_copied_password),
                ToastLength.Short).Show();
        }
    };
}

// -- Helpers -----

private void UpdatePasswordDisplay()
{
    if (string.IsNullOrEmpty(decryptedPassword))
    {
        textViewPassword.Text = "â€¢â€¢â€¢â€¢â€¢â€¢â€¢â€¢â€¢";
        return;
    }

    textViewPassword.Text = passwordVisible
        ? decryptedPassword
        : new string('â€¢', decryptedPassword.Length);
}

private void CopyToClipboard(string label, string text)
{
    var clipboard = (ClipboardManager)GetSystemService(ClipboardService);
    var clip = ClipData.NewPlainText(label, text);
    clipboard.PrimaryClip = clip;
}
}
}

```

--- FILE: FinalProject333057891\SecurioClient\Activities\WarningsActivity.cs ---

```

using Android.App;
using Android.Content;
using Android.OS;
using Android.Widget;
using AndroidX.AppCompat.App;
using SecurioClient.Helpers;

namespace SecurioClient.Activities
{
    /// <summary>
    /// Displays four password-risk warning cards (Leaked, Weak, Reused, Old)
    /// with the number of affected passwords in each category.
    /// Counters are read from <see cref="SessionHelper.CachedWarnings"/>;
    /// if the cache has been invalidated the counters are recomputed on-the-fly.
    /// </summary>
    [Activity(Label = "@string/app_name", Theme = "@style/AppTheme.NoActionBar")]
    public class WarningsActivity : AppCompatActivity
    {
        private TextView textViewLeakedCount;
        private TextView textViewWeakCount;
        private TextView textViewReusedCount;
        private TextView textViewOldCount;

        private TextView buttonViewAllLeaked;
        private TextView buttonViewAllWeak;
        private TextView buttonViewAllReused;
        private TextView buttonViewAllOld;

        protected override void OnCreate(Bundle savedInstanceState)
        {
            base.OnCreate(savedInstanceState);
            Xamarin.Essentials.Platform.Init(this, savedInstanceState);
            SetContentView(Resource.Layout.activity_warnings);

            InitializeViews();
            SetupBottomNavFragment(savedInstanceState);
            SetupViewAllButtons();
            DisplayWarnings();
        }
    }
}

```

```

protected override void OnResume()
{
    base.OnResume();
    DisplayWarnings();
}

private void InitializeViews()
{
    textViewLeakedCount =
FindViewById<TextView>(Resource.Id.textViewLeakedCount);
    textViewWeakCount =
FindViewById<TextView>(Resource.Id.textViewWeakCount);
    textViewReusedCount =
FindViewById<TextView>(Resource.Id.textViewReusedCount);
    textViewOldCount = FindViewById<TextView>(Resource.Id.textViewOldCount);

    buttonViewAllLeaked =
FindViewById<TextView>(Resource.Id.buttonViewAllLeaked);
    buttonViewAllWeak =
FindViewById<TextView>(Resource.Id.buttonViewAllWeak);
    buttonViewAllReused =
FindViewById<TextView>(Resource.Id.buttonViewAllReused);
    buttonViewAllOld = FindViewById<TextView>(Resource.Id.buttonViewAllOld);
}

private void SetupBottomNavFragment(Bundle savedInstanceState)
{
    if (savedInstanceState == null)
    {
        var fragment = BottomNavFragment.NewInstance("warnings");
        fragment.TabSelected += OnBottomNavTabSelected;

        SupportFragmentManager
            .BeginTransaction()
            .Replace(Resource.Id.frameBottomNav, fragment)
            .Commit();
    }
}

```



```

private void SetupViewAllButtons()
{
    buttonViewAllLeaked.Click += (s, e) =>
OpenRiskDetail(RiskDetailActivity.CategoryLeaked);
    buttonViewAllWeak.Click += (s, e) =>
OpenRiskDetail(RiskDetailActivity.CategoryWeak);
    buttonViewAllReused.Click += (s, e) =>
OpenRiskDetail(RiskDetailActivity.CategoryReused);
    buttonViewAllOld.Click += (s, e) =>
OpenRiskDetail(RiskDetailActivity.CategoryOld);
}

private void OpenRiskDetail(string category)
{
    var intent = new Intent(this, typeof(RiskDetailActivity));
    intent.PutExtra(RiskDetailActivity.ExtraRiskCategory, category);
    StartActivity(intent);
}

private void OnBottomNavBarTabSelected(object sender, string tab)
    => BottomNavHelper.Navigate(this, tab, "warnings");

/// <summary>
/// Reads the cached warnings or recomputes them synchronously (using stored
/// IsLeaked flags), then populates the four counter TextViews.
/// Because this runs synchronously in OnCreate the correct numbers are always
/// visible before the first UI draw â€” no "0" real number flash.
/// </summary>
private void DisplayWarnings()
{
    var warnings = SessionHelper.CachedWarnings;

    if (warnings == null)
    {
        // Cache was invalidated (vault changed) â€” recompute synchronously using
        // stored flags (no network calls), so the correct numbers are always
        // visible before the first UI draw.
        warnings = WarningsHelper.ComputeWarningsSync(

```

```

        SessionHelper.CachedVault,
        SessionHelper.SessionVaultKey);

    SessionHelper.CachedWarnings = warnings;
}

    textViewLeakedCount.Text = warnings.LeakedCount.ToString();
    textViewWeakCount.Text = warnings.WeakCount.ToString();
    textViewReusedCount.Text = warnings.ReusedCount.ToString();
    textViewOldCount.Text = warnings.OldCount.ToString();
}
}
}

--- FILE: FinalProject333057891\SecurioClient\Helpers\ServerHelpers\AuthService.cs ---
using SecurioClient.Helpers;
using SecurioClient;
using System.Threading.Tasks;
using Newtonsoft.Json;
using System.Collections.Generic;
using SecurioModels.DataTransferObjects;

namespace SecurioClient.Helpers.ServerHelpers
{
    /// <summary>A specialized service that manages identity-related tasks like registering new
    accounts and verifying credentials.</summary>
    public class AuthService : BaseService
    {
        /// <summary>Registers a new user account and sets up an authenticated session on
        success.</summary>
        public async Task<(bool Success, string Message)> RegisterAsync(User newUser, string
        plainTextPassword)
        {
            // 'result' IS the BaseResponse<AuthData>.
            var result = await PostAsync<AuthData>("RegisterUser", newUser);

            // One Success, One Message. Clean.
            if (result.Success)

```

```

    {
        await SetupAuthenticatedSession(result.Data, plainTextPassword,
newUser.EncryptionSalt);
    }

    return (result.Success, result.Message);
}

/// <summary>Validates credentials by first fetching user salts to derive keys
locally.</summary>
public async Task<(bool Success, string Message)> LoginAsync(string email, string
password)
{
    // 1. Get Salts (Server sends AuthSalt and EncryptionSalt)
    var saltResult = await PostAsync<SaltData>("GetSalts", new { Email = email });

    if (!saltResult.Success)
        return (false, saltResult.Message);

    // 2. Hash the password locally using the AuthSalt
    string hashedMPK = EncryptionHelper.DeriveKey(password,
saltResult.Data.AuthSalt);

    // 3. Authenticate with the hashed key
    var authResult = await PostAsync<AuthData>("VerifyLogin", new
    {
        Email = email,
        MasterPasswordKey = hashedMPK
    });

    if (authResult.Success)
    {
        // 4. Start the session using the EncryptionSalt from step 1
        await SetupAuthenticatedSession(authResult.Data, password,
saltResult.Data.EncryptionSalt);
    }

    return (authResult.Success, authResult.Message);
}

```

```
/// <summary>Persists the session credentials, derives the vault key, and warms up the in-
memory vault cache.</summary>
```

```
private async Task SetupAuthenticatedSession(AuthData data, string password, string
salt)
```

```
{
    // 1. Save to SecureStorage
    await StorageHelper.SaveUserId(data.UserId);
    await StorageHelper.SaveJwt(data.Token);
    await StorageHelper.SaveUsername(data.Username);

    // 2. Derive the vault key and start the in-memory session
    string vaultKey = EncryptionHelper.DeriveKey(password, salt);
    SessionHelper.StartSession(vaultKey);

    // 3. Persist the vault key so it can be restored after an app restart
    await StorageHelper.SaveVaultKey(vaultKey);

    // 4. Fetch the user's vault items and cache them in memory
    await FetchAndCacheVaultAsync();

    // 5. Compute password-health warnings and cache them for the session
    SessionHelper.CachedWarnings = await WarningsHelper.ComputeWarningsAsync(
        SessionHelper.CachedVault, vaultKey);
}
```

```
/// <summary>Fetches the user's vault items from the server and stores them in
SessionHelper.CachedVault.</summary>
```

```
public static async Task FetchAndCacheVaultAsync()
{
    var vaultService = new VaultService();
    var vaultResult = await vaultService.GetVaultItemsAsync();
    if (vaultResult.Success && vaultResult.Data != null)
        SessionHelper.CachedVault = vaultResult.Data;
    else
        SessionHelper.CachedVault = new List<VaultItem>();
}
```

```

    /// <summary>Asks the server to verify that the stored JWT is still valid and
unexpired.</summary>
    public async Task<bool> ValidateTokenAsync()
    {
        var result = await GetAsync<object>("ValidateToken");
        return result.Success;
    }
}

```

--- FILE: FinalProject333057891\SecurioClient\Helpers\ServerHelpers\BaseService.cs ---

```

using System;
using System.Net.Http;
using System.Text;
using System.Threading.Tasks;
using Newtonsoft.Json;
using System.Collections.Generic;
using SecurioModels;
using Android.Security.Identity;
using SecurioModels.DataTransferObjects;

namespace SecurioClient.Helpers
{
    /// <summary>The central engine for HTTP communication that handles JSON serialization
and validates the success of every API call.</summary>
    public abstract class BaseService
    {
        protected static readonly HttpClient Client = new HttpClient();

        /// Update this to your local or Azure URL
        protected const string BaseUrl = "http://10.0.2.2:7071/api/";

        /// <summary>Initializes a new instance of BaseService.</summary>
        public BaseService()
        {
            if (Client.BaseAddress == null)
                Client.BaseAddress = new Uri(BaseUrl);
        }
    }
}

```

```

/// <summary>Sends a POST request to the given endpoint and returns a typed server
response.</summary>
protected async Task<ServerResponse<T>> PostAsync<T>(string endpoint, object data)
{
    try
    {
        var request = new HttpRequestMessage(HttpMethod.Post, endpoint);
        request.Content = new StringContent(JsonConvert.SerializeObject(data),
Encoding.UTF8, "application/json");

        var jwt = await StorageHelper.GetJwt();
        if (!string.IsNullOrEmpty(jwt))
            request.Headers.TryAddWithoutValidation("Authorization", $"Bearer {jwt}");

        var response = await Client.SendAsync(request);
        var json = await response.Content.ReadAsStringAsync();

        if (response.IsSuccessStatusCode)
        {
            // Return the actual BaseResponse<T> from the server
            return JsonConvert.DeserializeObject<ServerResponse<T>>(json);
        }

        // If server returns 401/500, try to parse the error message
        var error = JsonConvert.DeserializeObject<ServerResponse<T>>(json);
        return error ?? new ServerResponse<T> { Success = false, Message = "Server error
occurred." };
    }
    catch
    {
        // If the internet is down, we return a new instance with the error
        return new ServerResponse<T> { Success = false, Message = $"Connection failed."
};
    }
}

/// <summary>Sends a GET request to the given endpoint and returns a typed server
response.</summary>

```

```
protected async Task<ServerResponse<T>> GetAsync<T>(string endpoint)
{
    try
    {
        var request = new HttpRequestMessage(HttpMethod.Get, endpoint);

        var jwt = await StorageHelper.GetJwt();
        if (!string.IsNullOrEmpty(jwt))
            request.Headers.TryAddWithoutValidation("Authorization", $"Bearer {jwt}");

        var response = await Client.SendAsync(request);
        var json = await response.Content.ReadAsStringAsync();

        if (response.IsSuccessStatusCode)
            return JsonConvert.DeserializeObject<ServerResponse<T>>(json);

        // Return the server's error message mapped to the object
        var error = JsonConvert.DeserializeObject<ServerResponse<T>>(json);
        return error ?? new ServerResponse<T> { Success = false, Message = "Could not
retrieve data." };
    }
    catch (Exception)
    {
        return new ServerResponse<T> { Success = false, Message = "Network error." };
    }
}
}
```

--- FILE:

FinalProject333057891\SecurioClient\Helpers\ServerHelpers\PasswordCheckServerService.cs

```
using Newtonsoft.Json;
using SecurioModels;
using SecurioModels.DataTransferObjects;
using System;
using System.Net.Http;
```

```

using System.Text;
using System.Threading.Tasks;

namespace SecurioClient.Helpers.ServerHelpers
{
    /// <summary>Contacts the backend PasswordCheck endpoint on behalf of the background
    monitor service using only the stored UserId.</summary>
    public static class PasswordCheckServerService
    {
        private static readonly HttpClient _http = new HttpClient();
        private const string Endpoint = "http://10.0.2.2:7071/api/PasswordCheck";

        /// <summary>Calls POST /api/PasswordCheck with the given userId; returns null on
        failure.</summary>
        public static async Task<PasswordCheckResult?> FetchAsync(int userId)
        {
            try
            {
                var payload = JsonConvert.SerializeObject(new PasswordCheckRequest { UserId =
userId });
                using var content = new StringContent(payload, Encoding.UTF8,
"application/json");
                using var response = await _http.PostAsync(Endpoint, content);

                if (!response.IsSuccessStatusCode) return null;

                var json = await response.Content.ReadAsStringAsync();
                var result =
JsonConvert.DeserializeObject<ServerResponse<PasswordCheckResult>>(json);
                return result?.Success == true ? result.Data : null;
            }
            catch
            {
                // Fail open: a connectivity issue must not produce false positives.
                return null;
            }
        }
    }
}

```



```

--- FILE:
FinalProject333057891\SecurioClient\Helpers\ServerHelpers\PasswordCheckService.cs ---
using System.Threading.Tasks;
using SecurioModels.DataTransferObjects;

namespace SecurioClient.Helpers.ServerHelpers
{
    /// <summary>Calls the PasswordCheck endpoint used by the background worker to poll
    for password-health issues.</summary>
    public class PasswordCheckService : BaseService
    {
        /// <summary>Sends the user's ID to the server and receives breach/old/master-old
        counts.</summary>
        public async Task<(bool Success, string Message, PasswordCheckResult Data)>
        CheckAsync(int userId)
        {
            var result = await PostAsync<PasswordCheckResult>("PasswordCheck", new { UserId
            = userId });
            return (result.Success, result.Message, result.Data);
        }
    }
}

```

```

--- FILE: FinalProject333057891\SecurioClient\Helpers\ServerHelpers\ProfileService.cs ---
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Android.App;
using Android.Content;
using Android.OS;
using Android.Runtime;
using Android.Views;
using Android.Widget;
using SecurioModels.DataTransferObjects;

```

```

using SecurioModels;

namespace SecurioClient.Helpers.ServerHelpers
{
    /// <summary>Manages the retrieval and caching of user profile information.</summary>
    public class ProfileService : BaseService
    {
        /// <summary>Fetches the full profile stats from the server.</summary>
        public async Task<ServerResponse<User>> GetProfileAsync()
        {
            var response = await GetAsync<User>("GetProfile");

            if (response.Success && response.Data != null)
            {
                // Cache it for the next time they open the page
                await StorageHelper.SaveProfileAsync(response.Data);
            }

            return response;
        }

        /// <summary>Sends the updated account details and optionally re-encrypted vault items
        to the server.</summary>
        public async Task<ServerResponse<object>>
        UpdateAccountAsync(UpdateAccountRequest request)
        {
            return await PostAsync<object>("UpdateUser", request);
        }

        /// <summary>Permanently deletes the user account and all associated vault items on the
        server.</summary>
        public async Task<ServerResponse<object>> DeleteAccountAsync()
        {
            return await PostAsync<object>("DeleteUser", new { });
        }

        /// <summary>Fetches the last 4 master-password history entries for the authenticated
        user.</summary>

```

```

        public async Task<ServerResponse<List<MasterPasswordHistory>>>
GetPasswordHistoryAsync()
    {
        return await GetAsync<List<MasterPasswordHistory>>("GetPasswordHistory");
    }
}

```

--- FILE: FinalProject333057891\SecurioClient\Helpers\ServerHelpers\VaultService.cs ---

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Android.App;
using Android.Content;
using Android.OS;
using Android.Runtime;
using Android.Views;
using Android.Widget;
using SecurioModels.DataTransferObjects;

```

```

namespace SecurioClient.Helpers.ServerHelpers

```

```

{
    /// <summary>Manages vault-item operations against the server.</summary>
    public class VaultService : BaseService
    {
        /// <summary>Sends an encrypted vault item to the server for storage.</summary>
        public async Task<(bool Success, string Message, VaultItem Data)>
AddVaultItemAsync(VaultItem item)
        {
            var result = await PostAsync<VaultItem>("AddVaultItem", item);
            return (result.Success, result.Message, result.Data);
        }

        /// <summary>Sends an updated vault item to the server for persistence.</summary>
        public async Task<(bool Success, string Message, VaultItem Data)>
UpdateVaultItemAsync(VaultItem item)

```

```

    {
        var result = await PostAsync<VaultItem>("UpdateVaultItem", item);
        return (result.Success, result.Message, result.Data);
    }

    /// <summary>Retrieves all vault items for the authenticated user.</summary>
    public async Task<(bool Success, string Message, List<VaultItem> Data)>
GetVaultItemsAsync()
    {
        var result = await GetAsync<List<VaultItem>>("GetVaultItems");
        return (result.Success, result.Message, result.Data);
    }

    /// <summary>Permanently deletes a vault item from the server.</summary>
    public async Task<(bool Success, string Message)> DeleteVaultItemAsync(int itemId)
    {
        var result = await PostAsync<object>("DeleteVaultItem", new { Id = itemId });
        return (result.Success, result.Message);
    }
}

```

--- FILE: FinalProject333057891\SecurioClient\Helpers\BottomNavHelper.cs ---

```

using Android.Content;
using AndroidX.AppCompat.App;
using SecurioClient.Activities;

namespace SecurioClient.Helpers
{
    /// <summary>
    /// Shared helper for bottom-navigation tab switching.
    /// Centralises the mapping of tab name â†’ activity so that each host activity
    /// only needs a single call instead of duplicating a switch/if chain.
    /// </summary>
    public static class BottomNavHelper
    {
        /// <summary>
        /// Navigates from <paramref name="activity"/> to the activity that corresponds

```

```

/// to <paramref name="tab"/>. If <paramref name="tab"/> equals
/// <paramref name="currentTab"/> the call is a no-op (user re-tapped the
/// active tab). The current activity is finished so the back-stack does not
/// accumulate stacked instances of the same main screens.
/// </summary>
public static void Navigate(AppCompatActivity activity, string tab, string currentTab)
{
    if (tab == currentTab)
        return;

    Intent intent = tab switch
    {
        "vault" => new Intent(activity, typeof(VaultActivity)),
        "warnings" => new Intent(activity, typeof(WarningsActivity)),
        "profile" => new Intent(activity, typeof(ProfileActivity)),
        _ => null
    };

    if (intent != null)
    {
        activity.StartActivity(intent);
        activity.Finish();
    }
}
}

```

--- FILE: FinalProject333057891\SecurioClient\Helpers\EncryptionHelper.cs ---

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Security.Cryptography;
using System.Text;
using Android.App;
using Android.Content;
using Android.OS;
using Android.Runtime;
using Android.Views;

```

```

using Android.Widget;
using Java.Util;
using static Android.Provider.Settings;

namespace SecurioClient
{
    /// <summary>A utility class for generating salts, deriving secure keys, and encrypting vault
    data using AES-GCM.</summary>
    public static class EncryptionHelper
    {
        // High iterations make brute-force attacks much harder
        private const int Iterations = 600000;

        /// <summary>Creates a cryptographically strong 32-byte random salt encoded as a
        Base64 string.</summary>
        public static string GenerateSalt()
        {
            byte[] saltBytes = new byte[32];
            RandomNumberGenerator.Fill(saltBytes);
            return Convert.ToBase64String(saltBytes);
        }

        /// <summary>Transforms a plaintext password and salt into a secure 256-bit key through
        multiple hashing iterations.</summary>
        public static string DeriveKey(string password, string saltBase64)
        {
            byte[] saltBytes = Convert.FromBase64String(saltBase64);

            using (var pbkdf2 = new Rfc2898DeriveBytes(password, saltBytes, Iterations,
                HashAlgorithmName.SHA256))
            {
                byte[] key = pbkdf2.GetBytes(32); // 256-bit key
                return Convert.ToBase64String(key);
            }
        }

        /// <summary>Encrypts plaintext using AES-GCM with the given Base64-encoded 256-
        bit key.</summary>
    }
}

```

```

public static (string IV, string Tag, string CipherText) EncryptAesGcm(string plaintext,
string base64Key)
{
    byte[] keyBytes = Convert.FromBase64String(base64Key);

    // Generate a random 96-bit IV (recommended size for GCM).
    byte[] ivBytes = new byte[12];
    using (var rng = RandomNumberGenerator.Create())
        rng.GetBytes(ivBytes);

    // Use Java's Cipher API which is fully supported on Android.
    var cipher = Javax.Crypto.Cipher.GetInstance("AES/GCM/NoPadding");
    var keySpec = new Javax.Crypto.Spec.SecretKeySpec(keyBytes, "AES");
    var gcmSpec = new Javax.Crypto.Spec.GCMParameterSpec(128, ivBytes); // 128-bit
auth tag
    cipher.Init(Javax.Crypto.CipherMode.EncryptMode, keySpec, gcmSpec);

    byte[] plaintextBytes = Encoding.UTF8.GetBytes(plaintext);
    byte[] encrypted = cipher.DoFinal(plaintextBytes);

    // Java AES-GCM appends the 16-byte authentication tag to the ciphertext.
    int tagLengthBytes = 16;
    byte[] cipherBytes = new byte[encrypted.Length - tagLengthBytes];
    byte[] tagBytes = new byte[tagLengthBytes];
    Array.Copy(encrypted, 0, cipherBytes, 0, cipherBytes.Length);
    Array.Copy(encrypted, cipherBytes.Length, tagBytes, 0, tagLengthBytes);

    return (
        Convert.ToBase64String(ivBytes),
        Convert.ToBase64String(tagBytes),
        Convert.ToBase64String(cipherBytes)
    );
}

```

/// <summary>Decrypts AES-GCM encrypted data using the given Base64-encoded 256-bit key.</summary>

```

public static string DecryptAesGcm(string base64IV, string base64Tag, string
base64CipherText, string base64Key)
{

```

```

byte[] keyBytes = Convert.FromBase64String(base64Key);
byte[] ivBytes = Convert.FromBase64String(base64IV);
byte[] tagBytes = Convert.FromBase64String(base64Tag);
byte[] cipherBytes = Convert.FromBase64String(base64CipherText);

// Java AES-GCM expects the authentication tag appended to the ciphertext.
byte[] encrypted = new byte[cipherBytes.Length + tagBytes.Length];
Array.Copy(cipherBytes, 0, encrypted, 0, cipherBytes.Length);
Array.Copy(tagBytes, 0, encrypted, cipherBytes.Length, tagBytes.Length);

var cipher = Javax.Crypto.Cipher.GetInstance("AES/GCM/NoPadding");
var keySpec = new Javax.Crypto.Spec.SecretKeySpec(keyBytes, "AES");
var gcmSpec = new Javax.Crypto.Spec.GCMParameterSpec(128, ivBytes);
cipher.Init(Javax.Crypto.CipherMode.DecryptMode, keySpec, gcmSpec);

byte[] plainBytes = cipher.DoFinal(encrypted);
return Encoding.UTF8.GetString(plainBytes);
}

/// <summary>Computes an unsalted SHA-1 hash of the input in uppercase hex, used for
HIBP breach checking only.</summary>
public static string ComputeSha1Hash(string input)
{
    using (var sha1 = SHA1.Create())
    {
        byte[] hashBytes = sha1.ComputeHash(Encoding.UTF8.GetBytes(input));
        return BitConverter.ToString(hashBytes).Replace("-", "").ToUpperInvariant();
    }
}
}
}

```

--- FILE: FinalProject333057891\SecurioClient\Helpers\FormUiHelper.cs ---

```

using Android.Content.Res;
using Android.Views;
using Android.Widget;

namespace SecurioClient.Helpers

```



```
{
    /// <summary>Shared UI helpers for form activities that display inline field errors and
    password-strength feedback.</summary>
    public static class FormUiHelper
    {
        /// <summary>Shows an inline error message below a form field.</summary>
        public static void ShowError(TextView errorView, string message)
        {
            errorView.Text = message;
            errorView.Visibility = ViewStates.Visible;
        }

        /// <summary>Hides an inline error message.</summary>
        public static void HideError(TextView errorView)
        {
            errorView.Text = null;
            errorView.Visibility = ViewStates.Gone;
        }

        /// <summary>Updates a 5-segment password-strength ProgressBar and constructive hint
        label.</summary>
        public static void UpdatePasswordStrengthIndicator(
            string password,
            ProgressBar strengthBar,
            TextView hintLabel,
            TextView errorLabel,
            Android.Content.Res.Resources resources)
        {
            if (string.IsNullOrEmpty(password))
            {
                strengthBar.Visibility = ViewStates.Gone;
                hintLabel.Visibility = ViewStates.Gone;
                HideError(errorLabel);
                return;
            }

            // Inline validation error (first unmet rule)
            var passResult = ValidationHelper.ValidatePassword(password);
            if (!passResult.IsValid) ShowError(errorLabel, passResult.ErrorMessage);
        }
    }
}
```

```

else
    HideError(errorLabel);

// Strength bar  progress = number of criteria met (0-5)
int score = ValidationHelper.GetPasswordScore(password);
strengthBar.Visibility = ViewStates.Visible;
strengthBar.Progress = score;
strengthBar.ProgressTintList =
    ColorStateList.ValueOf(new
Android.Graphics.Color(resources.GetColor(ScoreToColorRes(score))));

// Constructive hint below the bar
string hint = ValidationHelper.GetMissingCriteriaHint(password);
hintLabel.Text = hint;
hintLabel.Visibility = ViewStates.Visible;
hintLabel.SetTextColor(new Android.Graphics.Color(
    resources.GetColor(score == 5
        ? Resource.Color.passwordStrengthVeryStrong
        : Resource.Color.signupHintText)));
}

/// <summary>Maps a cumulative score (1-5) to the matching color resource for the
progress bar tint.</summary>
public static int ScoreToColorRes(int score)
{
    switch (score)
    {
        case 1: return Resource.Color.passwordStrengthWeak;
        case 2: return Resource.Color.passwordStrengthPoor;
        case 3: return Resource.Color.passwordStrengthFair;
        case 4: return Resource.Color.passwordStrengthStrong;
        default: return Resource.Color.passwordStrengthVeryStrong;
    }
}
}
}
}

```

--- FILE: FinalProject333057891\SecurioClient\Helpers\HibpClientService.cs ---
using System;

```

using System.Net.Http;
using System.Threading.Tasks;

namespace SecurioClient.Helpers
{
    /// <summary>
    /// Client-side HIBP Pwned Passwords check using the k-anonymity model.
    /// Only the first 5 characters of the SHA-1 hash are ever transmitted
    /// to the remote API; the full hash never leaves the device.
    /// </summary>
    public static class HibpClientService
    {
        private static readonly HttpClient _http = new HttpClient();
        private const string ApiBase = "https://api.pwnedpasswords.com/range/";

        /// <summary>
        /// Returns <c>true</c> if the given 40-character uppercase SHA-1 hex hash
        /// appears in the HIBP Pwned Passwords dataset.
        /// Returns <c>false</c> if the API is unreachable (fail-open), so a
        /// temporary outage never produces false positives.
        /// </summary>
        public static async Task<bool> IsPasswordPwnedAsync(string sha1Hash)
        {
            if (string.IsNullOrEmpty(sha1Hash) || sha1Hash.Length != 40)
                return false;

            string prefix = sha1Hash.Substring(0, 5).ToUpperInvariant();
            string suffix = sha1Hash.Substring(5).ToUpperInvariant();

            try
            {
                string body = await _http.GetStringAsync(ApiBase + prefix);

                // Each line in the response is "SUFFIX: COUNT" (CRLF-terminated).
                foreach (string line in body.Split(new[] { "\r\n", "\n" },
                    StringSplitOptions.RemoveEmptyEntries))
                {
                    int colon = line.IndexOf(':');
                    if (colon < 0) continue;
                }
            }
        }
    }
}

```

```

        if (string.Equals(line.Substring(0, colon).Trim(), suffix,
StringComparison.OrdinalIgnoreCase))
            return true;
    }

    return false;
}
catch
{
    // Fail open: a network or API error must not be treated as a breach.
    return false;
}
}
}
}

```

--- FILE: FinalProject333057891\SecurioClient\Helpers\NotificationHelper.cs ---

```

using Android.App;
using Android.Content;
using Android.OS;
using SecurioModels.DataTransferObjects;

namespace SecurioClient.Helpers
{
    /// <summary>Handles all Android-specific notification operations for the password-
monitor service.</summary>
    public static class NotificationHelper
    {
        // Notification channel for background password-health alerts.
        public const string ChannelId = "securio_monitor_channel";
        public const string ChannelName = "Password Monitor";

        // Notification IDs
        public const int AlertNotificationId = 1001;
        public const int ForegroundNotificationId = 1002;

        /// <summary>Creates the notification channel required on Android 8+.</summary>
        public static void CreateChannel(Context context)
    }
}

```

```

{
    if (Build.VERSION.SdkInt < BuildVersionCodes.O) return;

    var channel = new NotificationChannel(
        ChannelId,
        ChannelName,
        NotificationImportance.Default)
    {
        Description = "Securio password-health background alerts"
    };

    var nm =
(NotificationManager)context.GetService(Context.NotificationService);
    nm?.CreateNotificationChannel(channel);
}

/// <summary>Posts a local alert notification with the given message text.</summary>
public static void PostAlert(Context context, string message)
{
    var builder = new Notification.Builder(context, ChannelId)
        .SetContentTitle("Securio Password Alert")
        .SetContentText(message)
        .SetSmallIcon(Android.Resource.Drawable.IcDialogAlert)
        .SetAutoCancel(true);

    var nm =
(NotificationManager)context.GetService(Context.NotificationService);
    nm?.Notify(AlertNotificationId, builder.Build());
}

/// <summary>Builds the persistent foreground-service notification shown while the
PasswordMonitorService is running.</summary>
public static Notification BuildForegroundNotification(Context context)
{
    return new Notification.Builder(context, ChannelId)
        .SetContentTitle("Securio")
        .SetContentText("Monitoring your passwords in the background")
        .SetSmallIcon(Android.Resource.Drawable.IcDialogInfo)
        .SetOngoing(true)

```

```

        .Build();
    }

    /// <summary>Evaluates the PasswordCheckResult and posts an alert only when there is
    at least one issue to report.</summary>
    public static void PostCheckResult(Context context, PasswordCheckResult result)
    {
        if (!PasswordCheckDecision.ShouldNotify(result)) return;
        PostAlert(context, PasswordCheckDecision.BuildMessage(result));
    }
}

```

--- FILE: FinalProject333057891\SecurioClient\Helpers\PasswordCheckDecision.cs ---

```

using SecurioModels.DataTransferObjects;
using System.Text;

```

```

namespace SecurioClient.Helpers

```

```

{
    /// <summary>Pure, platform-independent helper that decides whether the user should be
    notified after a PasswordCheck server response.</summary>

```

```

    public static class PasswordCheckDecision
    {

```

```

        /// <summary>Returns true when at least one password-health issue was
        detected.</summary>

```

```

        public static bool ShouldNotify(PasswordCheckResult result)
        {
            if (result == null) return false;
            return result.BreachedCount > 0 || result.OldCount > 0 || result.MasterPasswordOld;
        }

```

```

        /// <summary>Builds a human-readable notification body summarising the issues
        found.</summary>

```

```

        public static string BuildMessage(PasswordCheckResult result)
        {
            if (result == null || !ShouldNotify(result))
                return string.Empty;

```

```

var parts = new System.Collections.Generic.List<string>();

if (result.BreachedCount > 0)
    parts.Add($"{result.BreachedCount} leaked password{(result.BreachedCount == 1 ?
"" : "s")});

if (result.OldCount > 0)
    parts.Add($"{result.OldCount} old password{(result.OldCount == 1 ? "" : "s")});

if (result.MasterPasswordOld)
    parts.Add("master password not changed in 90+ days");

return "Securio alert: " + string.Join(", ", parts) + ".";
}
}
}

```

--- FILE: FinalProject333057891\SecurioClient\Helpers\PasswordEntryActionsHelper.cs ---

```

using Android.Content;
using Android.Widget;
using AndroidX.AppCompat.App;
using SecurioClient.Helpers.ServerHelpers;
using SecurioModels.DataTransferObjects;
using System;
using System.Threading.Tasks;
using SecurioClient.Activities;

namespace SecurioClient.Helpers
{
    /// <summary>
    /// Shared helper that encapsulates the View / Edit / Delete bottom-sheet logic
    /// used by both <see cref="VaultActivity"/> and <see cref="RiskDetailActivity"/>.
    /// </summary>
    public static class PasswordEntryActionsHelper
    {
        /// <summary>
        /// Shows the <see cref="PasswordOptionsBottomSheet"/> for <paramref
        name="entry"/>

```

```

/// and wires up the View, Edit, and Delete callbacks.
/// </summary>
/// <param name="activity">The hosting activity (used for navigation and
dialogs).</param>
/// <param name="entry">The vault item the user acted on.</param>
/// <param name="beforeEdit">
/// Optional action invoked just before starting <see cref="EditPasswordActivity"/>,
/// e.g. to sync the duplicate-check entry cache.
/// </param>
/// <param name="onDeleted">
/// Callback invoked (on the UI thread) after the entry has been successfully
/// deleted so the caller can update its local list.
/// </param>
public static void ShowOptionsSheet(
    AppCompatActivity activity,
    VaultItem entry,
    Action beforeEdit,
    Action<VaultItem> onDeleted)
{
    var sheet = PasswordOptionsBottomSheet.NewInstance(entry);

    sheet.ViewClicked += (s, e) =>
    {
        var intent = new Intent(activity, typeof(ViewPasswordActivity));
        intent.PutExtra(ViewPasswordActivity.ExtraSiteName, entry.AccountName);
        intent.PutExtra(ViewPasswordActivity.ExtraUsername, entry.AccountUsername);
        intent.PutExtra(ViewPasswordActivity.ExtraNotes, entry.Notes);
        intent.PutExtra(ViewPasswordActivity.ExtraIV, entry.IV);
        intent.PutExtra(ViewPasswordActivity.ExtraTag, entry.Tag);
        intent.PutExtra(ViewPasswordActivity.ExtraCipherText, entry.CipherText);
        intent.PutExtra(ViewPasswordActivity.ExtraLastUpdate, entry.LastUpdate.Ticks);
        activity.StartActivity(intent);
    };

    sheet.EditClicked += (s, e) =>
    {
        beforeEdit?.Invoke();
        var intent = new Intent(activity, typeof(EditPasswordActivity));
        intent.PutExtra(EditPasswordActivity.ExtraEntryId, entry.Id);
    };
}

```



```

        intent.PutExtra(EditPasswordActivity.ExtraSiteName, entry.AccountName);
        intent.PutExtra(EditPasswordActivity.ExtraUsername, entry.AccountUsername);
        intent.PutExtra(EditPasswordActivity.ExtraNotes, entry.Notes);
        intent.PutExtra(EditPasswordActivity.ExtraIV, entry.IV);
        intent.PutExtra(EditPasswordActivity.ExtraTag, entry.Tag);
        intent.PutExtra(EditPasswordActivity.ExtraCipherText, entry.CipherText);
        intent.PutExtra(EditPasswordActivity.ExtraSha1Hash, entry.Sha1Hash);
        intent.PutExtra(EditPasswordActivity.ExtraIsLeaked, entry.IsLeaked);
        activity.StartActivityForResult(intent, EditPasswordActivity.RequestCodeEdit);
    };

    sheet.DeleteClicked += (s, e) => ConfirmDelete(activity, entry, onDeleted);

    sheet.Show(activity.SupportFragmentManager,
        PasswordOptionsBottomSheet.TagName);
}

private static void ConfirmDelete(AppCompatActivity activity, VaultItem entry,
    Action<VaultItem> onDeleted)
{
    string message = string.Format(
        activity.GetString(Resource.String.sheet_delete_confirm_message),
        entry.AccountName);

    new AlertDialog.Builder(activity)
        .SetTitle(Resource.String.sheet_delete_confirm_title)
        .SetMessage(message)
        .SetPositiveButton(Resource.String.sheet_delete_confirm_yes, async (s, e) =>
        {
            await DeleteEntryAsync(activity, entry, onDeleted);
        })
        .SetNegativeButton(Resource.String.sheet_delete_confirm_no, (s, e) => { })
        .Show();
}

private static async Task DeleteEntryAsync(AppCompatActivity activity, VaultItem
    entry, Action<VaultItem> onDeleted)
{
    try

```

```

{
    var vaultService = new VaultService();
    var result = await vaultService.DeleteVaultItemAsync(entry.Id);

    if (result.Success)
    {
        SessionHelper.RemoveVaultItem(entry.Id);
        SessionHelper.InvalidateWarnings();
        onDelete?.Invoke(entry);
        Toast.MakeText(activity, Resource.String.sheet_deleted_toast,
ToastLength.Short).Show();
    }
    else
    {
        Toast.MakeText(activity, Resource.String.sheet_delete_error,
ToastLength.Long).Show();
    }
}
catch (Exception)
{
    Toast.MakeText(activity, Resource.String.sheet_delete_error,
ToastLength.Long).Show();
}
}
}

```

--- FILE: FinalProject333057891\SecurioClient\Helpers\SessionHelper.cs ---

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

```

```

using Android.App;
using Android.Content;
using Android.OS;
using Android.Runtime;
using Android.Views;

```

```

using Android.Widget;

using SecurioModels;
using SecurioModels.DataTransferObjects;

namespace SecurioClient.Helpers
{
    /// <summary>A volatile, memory-only manager that holds the active vault encryption key
    and cached vault data during a user session.</summary>
    public static class SessionHelper
    {
        // The AES key derived from the Master Password; kept in RAM and never saved to disk.
        public static string SessionVaultKey { get; private set; }

        // Cached list of vault items for the authenticated user. Kept in RAM only.
        // This is the single source of truth for the vault list during a session.
        public static List<VaultItem> CachedVault { get; set; } = new List<VaultItem>();

        // Cached password-risk warning counters. Computed at login and
        // invalidated (set to null) whenever the vault contents change.
        public static WarningsData CachedWarnings { get; set; }

        // Indicates if the session is currently active with a valid key.
        public static bool IsAuthenticated => !string.IsNullOrEmpty(SessionVaultKey);

        /// <summary>Starts a session by storing the derived AES key in memory.</summary>
        public static void StartSession(string aesKey)
        {
            SessionVaultKey = aesKey;
        }

        /// <summary>Ends the session by wiping the AES key and cached data from
        memory.</summary>
        public static void EndSession()
        {
            SessionVaultKey = null;
            CachedVault.Clear();
            CachedWarnings = null;
        }
    }
}

```

```

        GC.Collect();
    }

    /// <summary>Clears the cached warnings so they will be recomputed on the next
access.</summary>
    public static void InvalidateWarnings()
    {
        CachedWarnings = null;
    }

    /// <summary>Adds a vault item to the in-memory cache.</summary>
    public static void AddVaultItem(VaultItem item)
    {
        if (item != null)
            CachedVault.Add(item);
    }

    /// <summary>Updates an existing vault item in the in-memory cache by matching its
Id.</summary>
    public static void UpdateVaultItem(VaultItem updatedItem)
    {
        if (updatedItem == null) return;

        var existing = CachedVault.FirstOrDefault(v => v.Id == updatedItem.Id);
        if (existing != null)
        {
            existing.AccountName = updatedItem.AccountName;
            existing.AccountUsername = updatedItem.AccountUsername;
            existing.IV = updatedItem.IV;
            existing.Tag = updatedItem.Tag;
            existing.CipherText = updatedItem.CipherText;
            existing.Notes = updatedItem.Notes;
            existing.Sha1Hash = updatedItem.Sha1Hash;
            existing.IsLeaked = updatedItem.IsLeaked;
            existing.LastUpdate = updatedItem.LastUpdate;
        }
    }

    /// <summary>Removes a vault item from the in-memory cache by Id.</summary>

```

```

    public static void RemoveVaultItem(int itemId)
    {
        CachedVault.RemoveAll(v => v.Id == itemId);
    }
}

```

--- FILE: FinalProject333057891\SecurioClient\Helpers\SimpleTextWatcher.cs ---

```

using Android.Text;
using System;

```

```

namespace SecurioClient.Helpers

```

```

{
    /// <summary>Minimal ITextWatcher adapter that allows a lambda to be used as a text-
change listener on Android EditText fields.</summary>
    public sealed class SimpleTextWatcher : Java.Lang.Object, ITextWatcher
    {
        private readonly Action<string> _onChanged;

        /// <summary>Initializes a new instance of SimpleTextWatcher.</summary>
        public SimpleTextWatcher(Action<string> onChanged) => _onChanged = onChanged;

        /// <summary>Called after the text has changed; invokes the provided
callback.</summary>
        public void AfterTextChanged(IEditable s) => _onChanged(s?.ToString());

        /// <summary>Called before the text changes; not used.</summary>
        public void BeforeTextChanged(Java.Lang.ICharSequence s, int start, int count, int after)
        { }

        /// <summary>Called when the text is changing; not used.</summary>
        public void OnTextChanged(Java.Lang.ICharSequence s, int start, int before, int count) {
        }
    }
}

```

--- FILE: FinalProject333057891\SecurioClient\Helpers\StorageHelper.cs ---

```

using System;

```

```

using System.Collections.Generic;
using System.Linq;
using System.Text;

using Android.App;
using Android.Content;
using Android.OS;
using Android.Runtime;
using Android.Views;
using Android.Widget;
using Xamarin.Essentials;
using System.Threading.Tasks;
using Java.Awt.Font;
using SecurioModels.DataTransferObjects;

namespace SecurioClient.Helpers
{
    /// <summary>Manages the persistent, hardware-encrypted storage of the user's ID,
    username, and authentication token on the device.</summary>
    public static class StorageHelper
    {
        // These keys identify the data in the encrypted file
        private const string KeyUserId = "user_id";
        private const string KeyUsername = "user_name";
        private const string KeyJwt = "jwt_token";
        private const string KeyVaultKey = "vault_key";
        private const string KeyEmail = "email";
        private const string KeyCreatedAt = "created_at";
        private const string KeyPasswordCount = "password_count";
        private const string KeyLastLogin = "last_login";
        private const string KeyLastPasswordChange = "last_password_change";

        /// <summary>Saves the user's unique ID to secure storage.</summary>
        public static async Task SaveUserId(int id)
        {
            await SecureStorage.SetAsync(KeyUserId, id.ToString());
        }
    }
}

```

/// <summary>Retrieves the user's unique ID from secure storage, returning 0 if not found or invalid.</summary>

```
public static async Task<int> GetUserId()
{
    var id = await SecureStorage.GetAsync(KeyUserId);
    return int.TryParse(id, out int result) ? result : 0;
}
```

/// <summary>Persists the user's display name to the device's secure storage.</summary>

```
public static async Task SaveUsername(string name)
{
    await SecureStorage.SetAsync(KeyUsername, name);
}
```

/// <summary>Retrieves the stored display name for UI personalization.</summary>

```
public static async Task<string> GetUsername()
{
    return await SecureStorage.GetAsync(KeyUsername);
}
```

/// <summary>Saves the JSON Web Token to secure storage to maintain the user's authenticated session.</summary>

```
public static async Task SaveJwt(string token)
{
    await SecureStorage.SetAsync(KeyJwt, token);
}
```

/// <summary>Retrieves the JSON Web Token from secure storage, returning null if not found.</summary>

```
public static async Task<string> GetJwt()
{
    return await SecureStorage.GetAsync(KeyJwt);
}
```

/// <summary>Saves the derived AES vault key to secure storage so the session can be restored after an app restart.</summary>

```
public static async Task SaveVaultKey(string vaultKey)
{
    await SecureStorage.SetAsync(KeyVaultKey, vaultKey);
}
```

```

    }

    /// <summary>Retrieves the stored AES vault key, returning null if not
present.</summary>
    public static async Task<string> GetVaultKey()
    {
        return await SecureStorage.GetAsync(KeyVaultKey);
    }

    /// <summary>Persists the full user profile including history timestamps to secure local
storage.</summary>
    public static async Task SaveProfileAsync(User profile)
    {
        await SecureStorage.SetAsync(KeyEmail, profile.Email);
        await SecureStorage.SetAsync(KeyPasswordCount,
profile.PasswordCount.ToString());
        await SecureStorage.SetAsync(KeyCreatedAt, profile.CreatedAt.ToString("o"));
        await SecureStorage.SetAsync(KeyLastLogin, profile.LastLogin.ToString("o"));
        await SecureStorage.SetAsync(KeyLastPasswordChange,
profile.LastPasswordUpdate.ToString("o"));
    }

    /// <summary>Retrieves the fully populated user profile from the local
cache.</summary>
    public static async Task<User> GetCachedProfileAsync()
    {
        var email = await SecureStorage.GetAsync(KeyEmail);
        if (string.IsNullOrEmpty(email)) return null;

        return new User
        {
            Id = await GetUserId(),
            Username = await GetUsername(),
            Email = email,
            PasswordCount = int.TryParse(await SecureStorage.GetAsync(KeyPasswordCount),
out var cnt) ? cnt : 0,
            CreatedAt = DateTime.TryParse(await SecureStorage.GetAsync(KeyCreatedAt), out
var createdAt) ? createdAt : DateTime.MinValue,

```



```

        LastLogin = DateTime.TryParse(await SecureStorage.GetAsync(KeyLastLogin), out
var lastLogin) ? lastLogin : DateTime.MinValue,
        LastPasswordUpdate = DateTime.TryParse(await
SecureStorage.GetAsync(KeyLastPasswordChange), out var lastPwChange) ? lastPwChange :
DateTime.MinValue
    };
}

```

/// <summary>Clears session-sensitive data from secure storage while preserving user ID and username.</summary>

```

public static Task ClearSessionAsync()
{
    SecureStorage.Remove(KeyJwt);
    SecureStorage.Remove(KeyVaultKey);
    SecureStorage.Remove(KeyEmail);
    SecureStorage.Remove(KeyCreatedAt);
    SecureStorage.Remove(KeyPasswordCount);
    SecureStorage.Remove(KeyLastLogin);
    SecureStorage.Remove(KeyLastPasswordChange);
    return Task.CompletedTask;
}

```

/// <summary>Clears ALL data from secure storage including user ID and username; use only when deleting the account.</summary>

```

public static void ClearAll()
{
    SecureStorage.RemoveAll();
}
}

```

--- FILE: FinalProject333057891\SecurioClient\Helpers\ValidationHelper.cs ---

```

using System;
using System.Collections.Generic;
using System.Security.Cryptography;
using System.Text.RegularExpressions;

```

```

namespace SecurioClient.Helpers

```

```
{
  /// <summary>Classifies how strong a password is across five levels.</summary>
  public enum PasswordStrength
  {
    Weak,    // score 1
    Poor,    // score 2
    Fair,    // score 3
    Strong,  // score 4
    VeryStrong // score 5
  }

  /// <summary>Carries the result of a single field validation.</summary>
  public sealed class ValidationResult
  {
    public bool IsValid { get; }
    public string ErrorMessage { get; }

    /// <summary>Initializes a new instance of ValidationResult.</summary>
    private ValidationResult(bool isValid, string errorMessage)
    {
      IsValid = isValid;
      ErrorMessage = errorMessage;
    }

    /// <summary>Returns a successful validation result.</summary>
    public static ValidationResult Ok() => new ValidationResult(true, null);
    /// <summary>Returns a failed validation result with the specified error
    message.</summary>
    public static ValidationResult Fail(string message) => new ValidationResult(false,
    message);
  }

  /// <summary>Industry-standard field validation for the Securio signup and login forms
  following OWASP and NIST SP 800-63B guidance.</summary>
  public static class ValidationHelper
  {
    // -----
    // Regex constants
    // -----
  }
}
```

```
// Username: starts with a letter; only letters, digits, underscores,
// or hyphens; 3-30 characters total.
private static readonly Regex UsernameRegex =
    new Regex(@"^[a-zA-Z][a-zA-Z0-9_\-]{2,29}$", RegexOptions.Compiled);

// Email: RFC 5321-compatible loose check that covers real-world addresses.
private static readonly Regex EmailRegex =
    new Regex(@"^[a-zA-Z0-9._%+\-]+\@[a-zA-Z0-9\-\.]?\.[a-zA-Z]{2,}$",
        RegexOptions.Compiled | RegexOptions.IgnoreCase);

// Password component detectors
private static readonly Regex HasUppercase = new Regex(@"[A-Z]",
    RegexOptions.Compiled);
private static readonly Regex HasLowercase = new Regex(@"[a-z]",
    RegexOptions.Compiled);
private static readonly Regex HasDigit = new Regex(@"\d", RegexOptions.Compiled);

// Accepted special characters: ! " # $ % & ' ( ) * + , - . / : ; < = > ? @ [ \ ] ^ _ ` { | } ~
// These cover the full set of ASCII printable non-alphanumeric characters (OWASP
// recommended).
private static readonly Regex HasSpecial =
    new Regex(@"[!@#$%^&*()_+\-=\[\]{}|;':",./<>?\\`~]", RegexOptions.Compiled);

// Character pools used by GenerateStrongPassword
private const string UpperPool = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
private const string LowerPool = "abcdefghijklmnopqrstuvwxyz";
private const string DigitPool = "0123456789";
private const string SpecialPool = "!@#$%^&*()-_+=[]{}|;:.,<>?";

// -----
// Public validators
// -----

/// <summary>Validates a username against Securio naming rules.</summary>
public static ValidationResult ValidateUsername(string username)
{
    if (string.IsNullOrEmpty(username))
        return ValidationResult.Fail("Username is required.");
}
```

```

    if (username.Length < 3)
        return ValidationResult.Fail("Username must be at least 3 characters.");

    if (username.Length > 30)
        return ValidationResult.Fail("Username must be 30 characters or fewer.");

    if (!char.IsLetter(username[0]))
        return ValidationResult.Fail("Username must start with a letter.");

    if (!UsernameRegex.IsMatch(username))
        return ValidationResult.Fail("Username may only contain letters, digits, underscores,
or hyphens.");

    return ValidationResult.Ok();
}

/// <summary>Validates an email address format.</summary>
public static ValidationResult ValidateEmail(string email)
{
    if (string.IsNullOrEmpty(email))
        return ValidationResult.Fail("Email address is required.");

    if (!EmailRegex.IsMatch(email))
        return ValidationResult.Fail("Please enter a valid email address.");

    return ValidationResult.Ok();
}

/// <summary>
/// Validates password strength. Returns an error for the first unmet
/// requirement so the message is actionable rather than overwhelming.
/// </summary>
public static ValidationResult ValidatePassword(string password)
{
    if (string.IsNullOrEmpty(password))
        return ValidationResult.Fail("Password is required.");

    if (password.Length < 8)

```

```

        return ValidationResult.Fail("Password must be at least 8 characters.");

        if (!HasUppercase.IsMatch(password))
            return ValidationResult.Fail("Password must include at least one uppercase letter.");

        if (!HasLowercase.IsMatch(password))
            return ValidationResult.Fail("Password must include at least one lowercase letter.");

        if (!HasDigit.IsMatch(password))
            return ValidationResult.Fail("Password must include at least one digit.");

        if (!HasSpecial.IsMatch(password))
            return ValidationResult.Fail("Password must include at least one special character
(e.g. !@#$.)");

        return ValidationResult.Ok();
    }

    /// <summary>Checks that the two password fields are identical.</summary>
    public static ValidationResult ValidatePasswordsMatch(string password, string
confirmPassword)
    {
        if (string.IsNullOrEmpty(confirmPassword))
            return ValidationResult.Fail("Please confirm your password.");

        if (password != confirmPassword)
            return ValidationResult.Fail("Passwords do not match.");

        return ValidationResult.Ok();
    }

    // -----
    // Password strength meter (5-criterion cumulative score)
    // -----

    /// <summary>
    /// Returns a score from 0 to 5 reflecting how many of the five
    /// complexity criteria the password satisfies:
    /// 1. Length >= 8

```

```

/// 2. Contains a lowercase letter
/// 3. Contains an uppercase letter
/// 4. Contains a digit
/// 5. Contains a special character
/// </summary>
public static int GetPasswordScore(string password)
{
    if (string.IsNullOrEmpty(password)) return 0;

    int score = 0;
    if (password.Length >= 8) score++;
    if (HasLowercase.IsMatch(password)) score++;
    if (HasUppercase.IsMatch(password)) score++;
    if (HasDigit.IsMatch(password)) score++;
    if (HasSpecial.IsMatch(password)) score++;

    return score;
}

/// <summary>Maps a cumulative score (0-5) to a <see cref="PasswordStrength"/>
level.</summary>
public static PasswordStrength GetPasswordStrength(string password)
{
    int score = GetPasswordScore(password);
    if (score <= 1) return PasswordStrength.Weak;
    if (score == 2) return PasswordStrength.Poor;
    if (score == 3) return PasswordStrength.Fair;
    if (score == 4) return PasswordStrength.Strong;
    return PasswordStrength.VeryStrong;
}

/// <summary>
/// Returns a short, constructive hint listing which criteria are still unmet,
/// or a success message when all five criteria are satisfied.
/// </summary>
public static string GetMissingCriteriaHint(string password)
{
    if (string.IsNullOrEmpty(password)) return null;

```

```

var missing = new List<string>();
if (password.Length < 8)      missing.Add("8+ chars");
if (!HasLowercase.IsMatch(password)) missing.Add("lowercase");
if (!HasUppercase.IsMatch(password)) missing.Add("uppercase");
if (!HasDigit.IsMatch(password))  missing.Add("digit");
if (!HasSpecial.IsMatch(password)) missing.Add("special char");

return missing.Count == 0
    ? "All requirements met âœ“"
    : "Missing: " + string.Join(", ", missing);
}

// -----
// Password generator
// -----

/// <summary>
/// Generates a cryptographically random password that satisfies all five
/// complexity criteria. The result is <paramref name="length"/> characters
/// long (minimum 12, default 16) and is NOT placed in the confirm-password
/// field automatically.
/// </summary>
public static string GenerateStrongPassword(int length = 16)
{
    if (length < 12) length = 12;

    string allChars = UpperPool + LowerPool + DigitPool + SpecialPool;

    // Start with one guaranteed character from each required category.
    var chars = new List<char>
    {
        UpperPool[SecureRandomIndex(UpperPool.Length)],
        LowerPool[SecureRandomIndex(LowerPool.Length)],
        DigitPool[SecureRandomIndex(DigitPool.Length)],
        SpecialPool[SecureRandomIndex(SpecialPool.Length)]
    };

    // Fill the remainder with random characters from the combined pool.
    for (int i = chars.Count; i < length; i++)

```

```

        chars.Add(allChars[SecureRandomIndex(allChars.Length)]);

        // Fisher-Yates shuffle so the guaranteed characters aren't always at the start.
        for (int i = chars.Count - 1; i > 0; i--)
        {
            int j = SecureRandomIndex(i + 1);
            (chars[i], chars[j]) = (chars[j], chars[i]);
        }

        return new string(chars.ToArray());
    }

    /// <summary>Returns a cryptographically secure random index in [0, max) using
    rejection sampling to eliminate modulo bias.</summary>
    private static int SecureRandomIndex(int max)
    {
        // Largest multiple of max that fits in a uint, used as the rejection threshold.
        uint limit = uint.MaxValue - (uint.MaxValue % (uint)max);
        byte[] buf = new byte[4];
        uint raw;
        do
        {
            RandomNumberGenerator.Fill(buf);
            raw = BitConverter.ToUInt32(buf, 0);
        }
        while (raw >= limit);

        return (int)(raw % (uint)max);
    }
}

```

--- FILE: FinalProject333057891\SecurioClient\Helpers\WarningsHelper.cs ---

```

using SecurioModels.DataTransferObjects;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

```



```

namespace SecurioClient.Helpers
{
    /// <summary>
    /// Holds the four password-risk counters computed from the vault.
    /// </summary>
    public sealed class WarningsData
    {
        public int LeakedCount { get; set; }
        public int WeakCount { get; set; }
        public int ReusedCount { get; set; }
        public int OldCount { get; set; }
    }

    /// <summary>
    /// Computes password-health warning counters from the in-memory vault.
    /// Counters are designed to be calculated once at login and cached in
    /// <see cref="SessionHelper.CachedWarnings"/>; the cache is flushed
    /// whenever the vault contents change.
    /// </summary>
    public static class WarningsHelper
    {
        /// <summary>
        /// Best-practice password age threshold. NIST SP 800-63B does not mandate
        /// forced rotation, but 90 days is the widely adopted industry standard
        /// for flagging stale credentials in consumer password managers.
        /// </summary>
        private const int OldPasswordDays = 90;

        /// <summary>
        /// Synchronously computes warning counters from the vault using stored flags
        /// (no live HIBP network calls). The leaked count uses each item's stored
        /// <see cref="VaultItem.IsLeaked"/> flag, which is set by the server at
        /// add/edit time. All other checks are computed locally.
        /// </summary>
        public static WarningsData ComputeWarningsSync(ICollection<VaultItem> vault, string
        vaultKey)
        {
            if (vault == null || vault.Count == 0)

```

```

        return new WarningsData();

        int leaked = vault.Count(item => item.IsLeaked);
        int weak = GetWeakItems(vault, vaultKey).Count;
        int reused = GetReusedItems(vault).Count;
        int old = GetOldItems(vault).Count;

        return new WarningsData
        {
            LeakedCount = leaked,
            WeakCount = weak,
            ReusedCount = reused,
            OldCount = old
        };
    }

    /// <summary>
    /// Analyses every item in <paramref name="vault"/> and returns aggregated
    /// warning counters. Password decryption (needed for the "weak" check)
    /// uses the provided <paramref name="vaultKey"/>.
    /// The "leaked" check performs a live HIBP k-anonymity query for each
    /// password's SHA-1 hash.
    /// </summary>
    public static async Task<WarningsData> ComputeWarningsAsync(ICollection<VaultItem>
vault, string vaultKey)
    {
        if (vault == null || vault.Count == 0)
            return new WarningsData();

        int leaked = 0;
        int weak = 0;
        int reused = 0;
        int old = 0;

        DateTime oldThreshold = DateTime.UtcNow.AddDays(-OldPasswordDays);

        // -- Leaked -----
        // Query HIBP for each password's SHA-1 hash using the k-anonymity
        // model so only the first 5 characters are ever transmitted.

```

```
// The IsLeaked flag on each item is updated so that subsequent
// synchronous recomputations (ComputeWarningsSync) stay accurate.
foreach (var item in vault)
{
    if (string.IsNullOrEmpty(item.Sha1Hash))
        continue;

    bool pwned = await HibpClientService.IsPasswordPwnedAsync(item.Sha1Hash);
    item.IsLeaked = pwned;
    if (pwned) leaked++;
}

// -- Weak -----
// Decrypt each password and run it through the same
// validation rules enforced on the signup page.
foreach (var item in vault)
{
    try
    {
        if (string.IsNullOrEmpty(item.IV) ||
            string.IsNullOrEmpty(item.Tag) ||
            string.IsNullOrEmpty(item.CipherText))
            continue;

        string plaintext = EncryptionHelper.DecryptAesGcm(
            item.IV, item.Tag, item.CipherText, vaultKey);

        var result = ValidationHelper.ValidatePassword(plaintext);
        if (!result.IsValid)
            weak++;
    }
    catch
    {
        // If decryption fails for any reason, skip the item.
    }
}

// -- Reused -----
// Two or more items that share the same SHA-1 hash are
```

```
// reusing the same password. Every member of such a
// group is counted (not just the duplicates).
var hashGroups = vault
    .Where(v => !string.IsNullOrEmpty(v.Sha1Hash))
    .GroupBy(v => v.Sha1Hash, StringComparer.OrdinalIgnoreCase)
    .Where(g => g.Count() > 1);

foreach (var group in hashGroups)
    reused += group.Count();

// -- Old -----
// Only passwords with a known LastUpdate that is older than
// the threshold are flagged. Items whose LastUpdate has never
// been set (DateTime.MinValue / default) are skipped â€” they
// are typically newly created entries whose server-assigned
// timestamp has not yet been propagated to the local cache.
foreach (var item in vault)
{
    if (item.LastUpdate != default && item.LastUpdate < oldThreshold)
        old++;
}

return new WarningsData
{
    LeakedCount = leaked,
    WeakCount = weak,
    ReusedCount = reused,
    OldCount = old
};
}

/// <summary>
/// Returns the subset of <paramref name="vault"/> items that fall under
/// the specified <paramref name="category"/> risk.
/// Categories: "leaked", "weak", "reused", "old".
/// </summary>
public static async Task<List<VaultItem>> GetItemsAtRiskAsync(
    IList<VaultItem> vault, string vaultKey, string category)
{
```

```

if (vault == null || vault.Count == 0)
    return new List<VaultItem>();

switch (category)
{
    case "leaked":
        return await GetLeakedItemsAsync(vault);

    case "weak":
        return GetWeakItems(vault, vaultKey);

    case "reused":
        return GetReusedItems(vault);

    case "old":
        return GetOldItems(vault);

    default:
        return new List<VaultItem>();
}
}

/// <summary>Returns the subset of vault items whose password appears in a known data
breach.</summary>
private static async Task<List<VaultItem>> GetLeakedItemsAsync(IList<VaultItem>
vault)
{
    var result = new List<VaultItem>();
    foreach (var item in vault)
    {
        if (!string.IsNullOrEmpty(item.Sha1Hash) &&
            await HibpClientService.IsPasswordPwnedAsync(item.Sha1Hash))
            result.Add(item);
    }
    return result;
}

/// <summary>Returns the subset of vault items that fail the password strength
validation.</summary>

```

```
private static List<VaultItem> GetWeakItems(IList<VaultItem> vault, string vaultKey)
{
    var result = new List<VaultItem>();
    foreach (var item in vault)
    {
        try
        {
            if (string.IsNullOrEmpty(item.IV) ||
                string.IsNullOrEmpty(item.Tag) ||
                string.IsNullOrEmpty(item.CipherText))
                continue;

            string plaintext = EncryptionHelper.DecryptAesGcm(
                item.IV, item.Tag, item.CipherText, vaultKey);

            var validationResult = ValidationHelper.ValidatePassword(plaintext);
            if (!validationResult.IsValid)
                result.Add(item);
        }
        catch
        {
            // Skip items that cannot be decrypted.
        }
    }
    return result;
}
```

/// <summary>Returns the subset of vault items that share a SHA-1 hash with at least one other item.</summary>

```
private static List<VaultItem> GetReusedItems(IList<VaultItem> vault)
{
    var hashGroups = vault
        .Where(v => !string.IsNullOrEmpty(v.Sha1Hash))
        .GroupBy(v => v.Sha1Hash, StringComparer.OrdinalIgnoreCase)
        .Where(g => g.Count() > 1);

    var result = new List<VaultItem>();
    foreach (var group in hashGroups)
        result.AddRange(group);
}
```

```

        return result;
    }

    /// <summary>Returns the subset of vault items whose password has not been changed in
    over 90 days.</summary>
    private static List<VaultItem> GetOldItems(IList<VaultItem> vault)
    {
        DateTime oldThreshold = DateTime.UtcNow.AddDays(-OldPasswordDays);
        return vault
            .Where(item => item.LastUpdate != default && item.LastUpdate < oldThreshold)
            .ToList();
    }
}

```

--- FILE: FinalProject333057891\SecurioClient\Properties\AndroidManifest.xml ---

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    android:versionCode="1"
    android:versionName="1.0"
    package="com.companyname.securioclient">
    <uses-sdk android:minSdkVersion="33" android:targetSdkVersion="33" />
    <application android:usesCleartextTraffic="true" android:label="Securio"
        android:allowBackup="true" android:icon="@mipmap/ic_launcher"
        android:roundIcon="@mipmap/ic_launcher_round" android:supportsRtl="true"
        android:theme="@style/AppTheme">
        <!-- Foreground service for 24-hour background password-health monitoring. -->
        <service
            android:name="com.companyname.securioclient.PasswordMonitorService"
            android:exported="false"
            android:foregroundServiceType="dataSync" />
        <!-- Restarts the monitor service after reboot or app update. -->
        <receiver
            android:name="com.companyname.securioclient.BootReceiver"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.BOOT_COMPLETED" />

```

```

        <action android:name="android.intent.action.MY_PACKAGE_REPLACED" />
    </intent-filter>
</receiver>
</application>
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
<uses-permission
android:name="android.permission.FOREGROUND_SERVICE_DATA_SYNC" />
<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"
/>
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
</manifest>

```

--- FILE: FinalProject333057891\SecurioClient\Resources\drawable\bg_bottom_nav.xml ---

```

<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <solid android:color="@color/vaultBottomNavBackground" />
    <stroke
        android:width="1dp"
        android:color="@color/vaultDivider" />
</shape>

```

--- FILE: FinalProject333057891\SecurioClient\Resources\drawable\bg_bottom_sheet.xml ---

```

<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <solid android:color="@color/vaultCardBackground" />
    <corners
        android:topLeftRadius="20dp"
        android:topRightRadius="20dp" />
</shape>

```

--- FILE: FinalProject333057891\SecurioClient\Resources\drawable\bg_icon_circle.xml ---

```

<?xml version="1.0" encoding="utf-8"?>

```



```
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="oval">
    <solid android:color="@color/vaultCardIconBackground" />
    <size
        android:width="40dp"
        android:height="40dp" />
</shape>
```

--- FILE: FinalProject333057891\SecurioClient\Resources\drawable\bg_search_bar.xml ---

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <solid android:color="@color/vaultSearchBackground" />
    <corners android:radius="12dp" />
</shape>
```

--- FILE: FinalProject333057891\SecurioClient\Resources\drawable\bg_warning_card.xml ---

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <solid android:color="@color/vaultCardBackground" />
    <corners android:radius="12dp" />
</shape>
```

--- FILE:

FinalProject333057891\SecurioClient\Resources\drawable\bg_warning_circle_leaked.xml ---

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="oval">
    <solid android:color="@color/warningLeakedAccent" />
    <size android:width="56dp" android:height="56dp" />
</shape>
```

--- FILE:

FinalProject333057891\SecurioClient\Resources\drawable\bg_warning_circle_old.xml ---

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="oval">
    <solid android:color="@color/warningOldAccent" />
    <size android:width="56dp" android:height="56dp" />
</shape>
```

--- FILE:

FinalProject333057891\SecurioClient\Resources\drawable\bg_warning_circle_reused.xml ---

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="oval">
    <solid android:color="@color/warningReusedAccent" />
    <size android:width="56dp" android:height="56dp" />
</shape>
```

--- FILE:

FinalProject333057891\SecurioClient\Resources\drawable\bg_warning_circle_weak.xml ---

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="oval">
    <solid android:color="@color/warningWeakAccent" />
    <size android:width="56dp" android:height="56dp" />
</shape>
```

--- FILE: FinalProject333057891\SecurioClient\Resources\drawable\ic_delete.xml ---

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24">
    <path
        android:fillColor="#E74C3C"
        android:pathData="M6,19c0,1.1 0.9,2 2,2h8c1.1,0 2,-0.9 2,-2V7H6v12zM19,4h-3.5l-1,-1h-
5l-1,1H5v2h14V4Z" />
```

</vector>

--- FILE: FinalProject333057891\SecurioClient\Resources\drawable\ic_edit.xml ---

<?xml version="1.0" encoding="utf-8"?>

<vector xmlns:android="http://schemas.android.com/apk/res/android"

android:width="24dp"

android:height="24dp"

android:viewportWidth="24"

android:viewportHeight="24">

<path

android:fillColor="#FFFFFF"

android:pathData="M3,17.25V21h3.75L17.81,9.94l-3.75,-3.75L3,17.25zM20.71,7.04c0.39,-0.39 0.39,-1.02 0,-1.41l-2.34,-2.34c-0.39,-0.39 -1.02,-0.39 -1.41,0l-1.83,1.83 3.75,3.75 1.83,-1.83Z"/>

</vector>

--- FILE: FinalProject333057891\SecurioClient\Resources\drawable\ic_eye.xml ---

<?xml version="1.0" encoding="utf-8"?>

<vector xmlns:android="http://schemas.android.com/apk/res/android"

android:width="24dp"

android:height="24dp"

android:viewportWidth="24"

android:viewportHeight="24">

<path

android:fillColor="#FFFFFF"

android:pathData="M12,4.5C7,4.5 2.73,7.61 1,12c1.73,4.39 6,7.5 11,7.5s9.27,-3.11 11,-7.5c-1.73,-4.39 -6,-7.5 -11,-7.5zM12,17c-2.76,0 -5,-2.24 -5,-5s2.24,-5 5,-5 5,2.24 5,-2.24,5 -5,5zM12,9c-1.66,0 -3,1.34 -3,3s1.34,3 3,3 3,-1.34 3,-3 -1.34,-3 -3,-3Z" />

</vector>

--- FILE: FinalProject333057891\SecurioClient\Resources\drawable\ic_more_vert.xml ---

<?xml version="1.0" encoding="utf-8"?>

<vector xmlns:android="http://schemas.android.com/apk/res/android"

android:width="24dp"

android:height="24dp"

android:viewportWidth="24"

```

    android:viewportHeight="24">
    <path
        android:fillColor="#FFFFFF"
        android:pathData="M12,8c1.1,0 2,-0.9 2,-2s-0.9,-2 -2,-2 -2,-2 0.9,-2,2 0.9,2 2,2zM12,10c-1.1,0 -
2,0.9 -2,2s0.9,2 2,2 2,-0.9 2,-2 -0.9,-2 -2,-2zM12,16c-1.1,0 -2,0.9 -2,2s0.9,2 2,2 2,-0.9 2,-2 -0.9,-2 -
2,-2z" />
</vector>

```

--- FILE: FinalProject333057891\SecurioClient\Resources\layout\activity_edit_account.xml --

```

<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true"
    android:background="@color/signupBackground">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:gravity="center_horizontal"
        android:paddingStart="32dp"
        android:paddingEnd="32dp"
        android:paddingTop="48dp"
        android:paddingBottom="32dp">

        <!-- Back arrow row -->
        <RelativeLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginBottom="8dp">

            <ImageView
                android:id="@+id/imageViewEditAccountBack"
                android:layout_width="36dp"
                android:layout_height="36dp"

```

```

        android:layout_alignParentStart="true"
        android:layout_centerVertical="true"
        android:src="@android:drawable/ic_menu_revert"
        android:padding="4dp"
        android:background="?:android:attr/selectableItemBackgroundBorderless"
        android:contentDescription="Back" />

```

```
</RelativeLayout>
```

```

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="•"
    android:textSize="64sp"
    android:layout_marginBottom="8dp" />

```

```

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/edit_account_title"
    style="@style/SignupTitleStyle"
    android:layout_marginBottom="8dp" />

```

```

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/edit_account_subtitle"
    style="@style/SignupSubtitleStyle"
    android:layout_marginBottom="32dp" />

```

```
<!-- Account details card -->
```

```

<androidx.cardview.widget.CardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:cardCornerRadius="16dp"
    app:cardElevation="8dp"
    app:cardBackgroundColor="@color/signupCardBackground"
    android:layout_marginBottom="24dp">

```

```

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="24dp">

    <!-- Username -->
    <com.google.android.material.textfield.TextInputLayout
        style="@style/SignupInputLayoutStyle">
        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/editTextEditUsername"
            style="@style/SignupEditTextStyle"
            android:hint="@string/edit_account_username_hint"
            android:inputType="textPersonName"
            android:drawableStart="@android:drawable/ic_menu_info_details"
            android:drawablePadding="8dp" />
    </com.google.android.material.textfield.TextInputLayout>

    <TextView
        android:id="@+id/textViewEditUsernameError"
        style="@style/SignupErrorTextStyle"
        android:layout_marginBottom="8dp" />

    <!-- Email -->
    <com.google.android.material.textfield.TextInputLayout
        style="@style/SignupInputLayoutStyle">
        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/editTextEditEmail"
            style="@style/SignupEditTextStyle"
            android:hint="@string/edit_account_email_hint"
            android:inputType="textEmailAddress"
            android:drawableStart="@android:drawable/ic_dialog_email"
            android:drawablePadding="8dp" />
    </com.google.android.material.textfield.TextInputLayout>

    <TextView
        android:id="@+id/textViewEditEmailError"
        style="@style/SignupErrorTextStyle"
        android:layout_marginBottom="12dp" />

```

```

<!-- Divider -->
<View
    android:layout_width="match_parent"
    android:layout_height="1dp"
    android:background="@color/signupDivider"
    android:layout_marginBottom="16dp" />

<!-- Password change section label -->
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/edit_account_password_section_title"
    android:textSize="14sp"
    android:textColor="@color/signupSubtitleText"
    android:fontFamily="sans-serif-medium"
    android:layout_marginBottom="12dp" />

<!-- Current Master Password -->
<com.google.android.material.textfield.TextInputLayout
    style="@style/SignupInputLayoutStyle"
    app:passwordToggleEnabled="true">
    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/editTextCurrentPassword"
        style="@style/SignupEditTextStyle"
        android:hint="@string/edit_account_current_password_hint"
        android:inputType="textPassword"
        android:drawableStart="@android:drawable/ic_lock_idle_lock"
        android:drawablePadding="8dp" />
</com.google.android.material.textfield.TextInputLayout>

<TextView
    android:id="@+id/textViewCurrentPasswordError"
    style="@style/SignupErrorTextStyle"
    android:layout_marginBottom="8dp" />

<!-- New Master Password -->
<com.google.android.material.textfield.TextInputLayout
    style="@style/SignupInputLayoutStyle"

```

```

app: passwordToggleEnabled="true">
<com.google.android.material.textfield.TextInputEditText
    android: id="@+id/editTextNewPassword"
    style="@style/SignupEditTextStyle"
    android: hint="@string/edit_account_new_password_hint"
    android: inputType="textPassword"
    android: drawableStart="@android: drawable/ic_lock_idle_lock"
    android: drawablePadding="8dp" />
</com.google.android.material.textfield.TextInputLayout>

<!-- Generate password button -->
<com.google.android.material.button.MaterialButton
    android: id="@+id/buttonEditGeneratePassword"
    style="@style/SignupGenerateButtonStyle"
    android: text="@string/edit_account_generate_password" />

<TextView
    android: id="@+id/textViewNewPasswordError"
    style="@style/SignupErrorTextStyle"
    android: layout_marginBottom="4dp" />

<!-- 5-segment password strength bar -->
<ProgressBar
    android: id="@+id/progressBarEditPasswordStrength"
    style="?android: attr/progressBarStyleHorizontal"
    android: layout_width="match_parent"
    android: layout_height="8dp"
    android: max="5"
    android: progress="0"
    android: visibility="gone"
    android: progressBackgroundTint="@color/signupDivider"
    android: progressTint="@color/passwordStrengthWeak"
    android: layout_marginBottom="4dp" />

<!-- Constructive feedback hint -->
<TextView
    android: id="@+id/textViewEditPasswordHint"
    android: layout_width="wrap_content"
    android: layout_height="wrap_content"

```



```

        android:textSize="12sp"
        android:fontFamily="sans-serif"
        android:visibility="gone"
        android:layout_marginBottom="12dp" />

<!-- Confirm New Master Password -->
<com.google.android.material.textfield.TextInputLayout
    style="@style/SignupInputLayoutStyle"
    app:passwordToggleEnabled="true">
    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/editTextConfirmNewPassword"
        style="@style/SignupEditTextStyle"
        android:hint="@string/edit_account_confirm_password_hint"
        android:inputType="textPassword"
        android:drawableStart="@android:drawable/ic_lock_idle_lock"
        android:drawablePadding="8dp" />
</com.google.android.material.textfield.TextInputLayout>

<TextView
    android:id="@+id/textViewConfirmNewPasswordError"
    style="@style/SignupErrorTextStyle"
    android:layout_marginBottom="16dp" />

<com.google.android.material.button.MaterialButton
    android:id="@+id/buttonEditSave"
    style="@style/SignupButtonStyle"
    android:text="@string/edit_account_button_save"
    android:backgroundTint="@color/signupButtonBackground"
    app:cornerRadius="12dp" />

<TextView
    android:id="@+id/textViewEditGeneralError"
    style="@style/SignupErrorTextStyle"
    android:layout_gravity="center"
    android:layout_marginTop="8dp" />

</LinearLayout>
</androidx.cardview.widget.CardView>

```

```

<!-- Indeterminate spinner shown while the server request is in flight -->
<ProgressBar
    android:id="@+id/progressBarEditAccount"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:visibility="gone"
    android:indeterminateTint="@color/colorAccent" />

</LinearLayout>
</ScrollView>

--- FILE: FinalProject333057891\SecurioClient\Resources\layout\activity_entry.xml ---
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true"
    android:background="@color/entryBackground">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:gravity="center_horizontal"
        android:paddingStart="32dp"
        android:paddingEnd="32dp"
        android:paddingTop="48dp"
        android:paddingBottom="32dp">

        <!-- Back arrow + shield icon row -->
        <RelativeLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginBottom="8dp">

            <ImageView
                android:id="@+id/imageViewEntryBack"

```

```

android:layout_width="36dp"
android:layout_height="36dp"
android:layout_alignParentStart="true"
android:layout_centerVertical="true"
android:src="@android:drawable/ic_menu_revert"
android:padding="4dp"
android:background="?android:attr/selectableItemBackgroundBorderless"
android:contentDescription="Back" />

```

```

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_centerInParent="true"
    android:text="דף זה ייבטל"
    android:textSize="36sp" />

```

```

</RelativeLayout>

```

```

<!-- Title -->

```

```

<TextView
    android:id="@+id/textViewEntryTitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/entry_title_add"
    style="@style/SignupTitleStyle"
    android:layout_marginBottom="4dp" />

```

```

<!-- Subtitle -->

```

```

<TextView
    android:id="@+id/textViewEntrySubtitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/entry_subtitle_add"
    style="@style/SignupSubtitleStyle"
    android:layout_marginBottom="28dp" />

```

```

<!-- Form card -->

```

```

<androidx.cardview.widget.CardView
    android:layout_width="match_parent"

```

```

android:layout_height="wrap_content"
app:cardCornerRadius="16dp"
app:cardElevation="8dp"
app:cardBackgroundColor="@color/entryCardBackground"
android:layout_marginBottom="24dp">

```

```

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="24dp">

```

```

        <!-- Site / App name -->
        <com.google.android.material.textfield.TextInputLayout
            style="@style/SignupInputLayoutStyle">

```

```

<com.google.android.material.textfield.TextInputEditText
    android:id="@+id/editTextEntrySiteName"
    style="@style/SignupEditTextStyle"
    android:hint="@string/entry_site_name_hint"
    android:inputType="text"
    android:drawableStart="@android:drawable/ic_menu_view"
    android:drawablePadding="8dp" />
</com.google.android.material.textfield.TextInputLayout>

```

```

        <TextView
            android:id="@+id/textViewEntrySiteNameError"
            style="@style/SignupErrorTextStyle"
            android:layout_marginBottom="8dp" />

```

```

        <!-- Username / Email -->
        <com.google.android.material.textfield.TextInputLayout
            style="@style/SignupInputLayoutStyle">

```

```

<com.google.android.material.textfield.TextInputEditText
    android:id="@+id/editTextEntryUsername"
    style="@style/SignupEditTextStyle"
    android:hint="@string/entry_username_hint"
    android:inputType="textEmailAddress"

```

```

        android:drawableStart="@android:drawable/ic_menu_info_details"
        android:drawablePadding="8dp" />
    </com.google.android.material.textfield.TextInputLayout>

    <TextView
        android:id="@+id/textViewEntryUsernameError"
        style="@style/SignupErrorTextStyle"
        android:layout_marginBottom="8dp" />

    <!-- Password -->
    <com.google.android.material.textfield.TextInputLayout
        android:id="@+id/textInputLayoutEntryPassword"
        style="@style/SignupInputLayoutStyle"
        app:passwordToggleEnabled="true">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/editTextEntryPassword"
        style="@style/SignupEditTextStyle"
        android:hint="@string/entry_password_hint"
        android:inputType="textPassword"
        android:drawableStart="@android:drawable/ic_lock_idle_lock"
        android:drawablePadding="8dp" />
    </com.google.android.material.textfield.TextInputLayout>

    <!-- Generate password button -->
    <com.google.android.material.button.MaterialButton
        android:id="@+id/buttonEntryGeneratePassword"
        style="@style/SignupGenerateButtonStyle"
        android:text="@string/entry_generate_password" />

    <!-- Password strength bar (informational only) -->
    <ProgressBar
        android:id="@+id/progressBarEntryStrength"
        style="?android:attr/progressBarStyleHorizontal"
        android:layout_width="match_parent"
        android:layout_height="8dp"
        android:max="5"
        android:progress="0"
        android:visibility="gone"

```

```

        android:progressBackgroundTint="@color/entryStrengthBackground"
        android:progressTint="@color/passwordStrengthWeak"
        android:layout_marginBottom="4dp" />

        <!-- Strength hint text -->
        <TextView
            android:id="@+id/textViewEntryStrengthHint"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textSize="12sp"
            android:fontFamily="sans-serif"
            android:visibility="gone"
            android:layout_marginBottom="8dp" />

        <TextView
            android:id="@+id/textViewEntryPasswordError"
            style="@style/SignupErrorTextStyle"
            android:layout_marginBottom="8dp" />

        <!-- Confirm Password -->
        <com.google.android.material.textfield.TextInputLayout
            android:id="@+id/textInputLayoutEntryConfirmPassword"
            style="@style/SignupInputLayoutStyle"
            app:passwordToggleEnabled="true">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/editTextEntryConfirmPassword"
            style="@style/SignupEditTextStyle"
            android:hint="@string/entry_confirm_password_hint"
            android:inputType="textPassword"
            android:drawableStart="@android:drawable/ic_lock_idle_lock"
            android:drawablePadding="8dp" />
        </com.google.android.material.textfield.TextInputLayout>

        <TextView
            android:id="@+id/textViewEntryConfirmPasswordError"
            style="@style/SignupErrorTextStyle"
            android:layout_marginBottom="8dp" />

```

```

        <!-- Notes (optional, multiline) -->
        <com.google.android.material.textfield.TextInputLayout
style="@style/SignupInputLayoutStyle">

<com.google.android.material.textfield.TextInputEditText
    android:id="@+id/editTextEntryNotes"
    style="@style/SignupEditTextStyle"
    android:hint="@string/entry_notes_hint"
    android:inputType="textMultiLine"
    android:singleLine="false"
    android:minLines="2"
    android:maxLines="4"
    android:layout_height="wrap_content"
    android:drawableStart="@android:drawable/ic_menu_edit"
    android:drawablePadding="8dp"
    android:gravity="center_vertical" />
        </com.google.android.material.textfield.TextInputLayout>

        <!-- General error (e.g. duplicate entry) -->
        <TextView
android:id="@+id/textViewEntryGeneralError"
style="@style/SignupErrorTextStyle"
android:layout_gravity="center"
android:layout_marginTop="8dp"
android:layout_marginBottom="12dp" />

        <!-- Save / Update button -->
        <com.google.android.material.button.MaterialButton
android:id="@+id/buttonEntrySave"
style="@style/SignupButtonStyle"
android:text="@string/entry_button_save"
android:backgroundTint="@color/entryButtonBackground"
app:cornerRadius="12dp" />

    </LinearLayout>
</androidx.cardview.widget.CardView>

<!-- Loading spinner -->
<ProgressBar

```

```

        android:id="@+id/progressBarEntry"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:visibility="gone"
        android:indeterminateTint="@color/colorAccent" />

    </LinearLayout>
</ScrollView>

--- FILE: FinalProject333057891\SecurioClient\Resources\layout\activity_login.xml ---
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true"
    android:background="@color/signupBackground">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:gravity="center_horizontal"
        android:paddingStart="32dp"
        android:paddingEnd="32dp"
        android:paddingTop="64dp"
        android:paddingBottom="32dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="ðŸ›;ï•"
            android:textSize="64sp"
            android:layout_marginBottom="8dp" />

        <TextView
            android:layout_width="wrap_content"

```



```

        android:layout_height="wrap_content"
        android:text="@string/login_title"
        style="@style/SignupTitleStyle"
        android:layout_marginBottom="8dp" />

```

```
<TextView
```

```

        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/login_subtitle"
        style="@style/SignupSubtitleStyle"
        android:layout_marginBottom="40dp" />

```

```
<androidx.cardview.widget.CardView
```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:cardCornerRadius="16dp"
        app:cardElevation="8dp"
        app:cardBackgroundColor="@color/signupCardBackground"
        android:layout_marginBottom="24dp">

```

```
<LinearLayout
```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="24dp">

```

```
<!-- Email -->
```

```
<com.google.android.material.textfield.TextInputLayout
```

```

    style="@style/SignupInputLayoutStyle">

```

```
<com.google.android.material.textfield.TextInputEditText
```

```

    android:id="@+id/editTextLoginEmail"

```

```

    style="@style/SignupEditTextStyle"

```

```

    android:hint="@string/login_email_hint"

```

```

    android:inputType="textEmailAddress"

```

```

    android:drawableStart="@android:drawable/ic_dialog_email"

```

```

    android:drawablePadding="8dp" />

```

```
</com.google.android.material.textfield.TextInputLayout>
```

```
<TextView
```

```

        android: id="@+id/textViewLoginEmailError"
        style="@style/SignupErrorTextStyle"
        android: layout_marginBottom="8dp" />

<!-- Master Password -->
<com.google.android.material.textfield.TextInputLayout
    style="@style/SignupInputLayoutStyle"
    app: passwordToggleEnabled="true">
    <com.google.android.material.textfield.TextInputEditText
        android: id="@+id/editTextLoginPassword"
        style="@style/SignupEditTextStyle"
        android: hint="@string/login_password_hint"
        android: inputType="textPassword"
        android: drawableStart="@android:drawable/ic_lock_idle_lock"
        android: drawablePadding="8dp" />
    </com.google.android.material.textfield.TextInputLayout>

    <TextView
        android: id="@+id/textViewLoginPasswordError"
        style="@style/SignupErrorTextStyle"
        android: layout_marginBottom="16dp" />

    <com.google.android.material.button.MaterialButton
        android: id="@+id/buttonLogin"
        style="@style/SignupButtonStyle"
        android: text="@string/login_button_text"
        android: backgroundTint="@color/signupButtonBackground"
        app: cornerRadius="12dp" />

    <TextView
        android: id="@+id/textViewLoginGeneralError"
        style="@style/SignupErrorTextStyle"
        android: layout_gravity="center"
        android: layout_marginTop="8dp" />

</LinearLayout>
</androidx.cardview.widget.CardView>

<!-- Indeterminate spinner shown while the server request is in flight -->

```

```

<ProgressBar
    android:id="@+id/progressBarLogin"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:visibility="gone"
    android:indeterminateTint="@color/colorAccent" />

<LinearLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center"
    android:layout_marginTop="16dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/login_signup_prompt"
        android:textSize="14sp"
        android:textColor="@color/signupSubtitleText"
        android:layout_marginEnd="4dp" />

    <TextView
        android:id="@+id/textViewSignupLink"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/login_signup_link"
        style="@style/SignupLinkTextStyle"
        android:clickable="true"
        android:focusable="true" />
</LinearLayout>

</LinearLayout>
</ScrollView>

--- FILE: FinalProject333057891\SecurioClient\Resources\layout\activity_main.xml ---
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"

```

```

android:layout_width="match_parent"
android:layout_height="match_parent"
android:background="@color/signupBackground">

```

<!-- Centred brand block -->

```

<LinearLayout
    android:id="@+id/layoutSplashBrand"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_centerInParent="true"
    android:orientation="vertical"
    android:gravity="center">

```

<!-- Shield logo -->

```

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="ðŸ›"
    android:textSize="80sp"
    android:layout_marginBottom="16dp" />

```

<!-- App name -->

```

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/splash_app_name"
    android:textSize="36sp"
    android:textColor="@color/signupTitleText"
    android:textStyle="bold"
    android:fontFamily="sans-serif-medium"
    android:layout_marginBottom="8dp" />

```

<!-- Tagline -->

```

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/splash_tagline"
    android:textSize="14sp"
    android:textColor="@color/signupSubtitleText"

```

```

        android:fontFamily="sans-serif-light" />

</LinearLayout>

<!-- Loading indicator pinned below the brand block -->
<LinearLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@id/layoutSplashBrand"
    android:layout_centerHorizontal="true"
    android:orientation="vertical"
    android:gravity="center"
    android:layout_marginTop="40dp">

    <ProgressBar
        android:layout_width="40dp"
        android:layout_height="40dp"
        android:indeterminate="true"
        android:indeterminateTint="@color/colorAccent"
        android:layout_marginBottom="12dp" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/splash_verifying"
        android:textSize="13sp"
        android:textColor="@color/signupSubtitleText"
        android:fontFamily="sans-serif-light" />

</LinearLayout>

</RelativeLayout>

--- FILE: FinalProject333057891\SecurioClient\Resources\layout\activity_profile.xml ---
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"

```

```

android:layout_height="match_parent"
android:background="@color/vaultBackground">

```

```

<!-- Bottom navigation fragment container -->

```

```

<FrameLayout
    android:id="@+id/frameBottomNav"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_alignParentBottom="true" />

```

```

<!-- Scrollable content above the bottom nav -->

```

```

<ScrollView
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_above="@id/frameBottomNav"
    android:fillViewport="true">

```

```

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:gravity="center_horizontal"
    android:paddingStart="24dp"
    android:paddingEnd="24dp"
    android:paddingTop="48dp"
    android:paddingBottom="32dp">

```

```

<!-- Header icon -->

```

```

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="ðŸ‘"
    android:textSize="36sp"
    android:layout_marginBottom="4dp" />

```

```

<!-- Title -->

```

```

<TextView
    android:id="@+id/textViewProfileTitle"
    android:layout_width="wrap_content"

```

```

        android:layout_height="wrap_content"
        android:text="@string/profile_title"
        style="@style/VaultTitleStyle"
        android:layout_marginBottom="4dp" />

<!-- Subtitle -->
<TextView
    android:id="@+id/textViewProfileSubtitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/profile_subtitle"
    style="@style/VaultSubtitleStyle"
    android:layout_marginBottom="24dp" />

<!-- Loading indicator -->
<ProgressBar
    android:id="@+id/progressBarProfile"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:visibility="gone" />

<!-- Profile stats card -->
<androidx.cardview.widget.CardView
    android:id="@+id/cardProfileStats"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:cardCornerRadius="16dp"
    app:cardElevation="8dp"
    app:cardBackgroundColor="@color/entryCardBackground"
    android:layout_marginBottom="20dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="24dp">

        <!-- Username -->

```

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/profile_label_username"
    android:textSize="12sp"
    android:textColor="@color/profileStatLabel"
    android:fontFamily="sans-serif"
    android:layout_marginBottom="4dp" />
```

```
<TextView
    android:id="@+id/textViewProfileUsername"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="16sp"
    android:textColor="@color/profileStatValue"
    android:fontFamily="sans-serif-medium"
    android:drawableStart="@android:drawable/ic_menu_myplaces"
    android:drawablePadding="8dp"
    android:layout_marginBottom="20dp" />
```

```
<!-- Email -->
```

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/profile_label_email"
    android:textSize="12sp"
    android:textColor="@color/profileStatLabel"
    android:fontFamily="sans-serif"
    android:layout_marginBottom="4dp" />
```

```
<TextView
    android:id="@+id/textViewProfileEmail"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="16sp"
    android:textColor="@color/profileStatValue"
    android:fontFamily="sans-serif"
    android:drawableStart="@android:drawable/ic_dialog_email"
    android:drawablePadding="8dp"
```



```
android:layout_marginBottom="20dp" />
```

```
<!-- Divider -->
```

```
<View
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="1dp"
```

```
    android:background="@color/signupDivider"
```

```
    android:layout_marginBottom="20dp" />
```

```
<!-- Last Login -->
```

```
<TextView
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="@string/profile_label_last_login"
```

```
    android:textSize="12sp"
```

```
    android:textColor="@color/profileStatLabel"
```

```
    android:fontFamily="sans-serif"
```

```
    android:layout_marginBottom="4dp" />
```

```
<TextView
```

```
    android:id="@+id/textViewProfileLastLogin"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
    android:textSize="14sp"
```

```
    android:textColor="@color/profileStatValue"
```

```
    android:fontFamily="sans-serif"
```

```
    android:drawableStart="@android:drawable/ic_menu_recent_history"
```

```
    android:drawablePadding="8dp"
```

```
    android:layout_marginBottom="16dp" />
```

```
<!-- Last Master Password Change -->
```

```
<TextView
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="@string/profile_label_last_password_change"
```

```
    android:textSize="12sp"
```

```
    android:textColor="@color/profileStatLabel"
```

```
    android:fontFamily="sans-serif"
```

```
    android:layout_marginBottom="4dp" />
```

```
<TextView
    android:id="@+id/textViewProfileLastPasswordChange"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="14sp"
    android:textColor="@color/profileStatValue"
    android:fontFamily="sans-serif"
    android:drawableStart="@android:drawable/ic_lock_idle_lock"
    android:drawablePadding="8dp"
    android:layout_marginBottom="16dp" />
```

```
<!-- Account Created -->
```

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/profile_label_created_at"
    android:textSize="12sp"
    android:textColor="@color/profileStatLabel"
    android:fontFamily="sans-serif"
    android:layout_marginBottom="4dp" />
```

```
<TextView
    android:id="@+id/textViewProfileCreatedAt"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="14sp"
    android:textColor="@color/profileStatValue"
    android:fontFamily="sans-serif"
    android:drawableStart="@android:drawable/ic_menu_today"
    android:drawablePadding="8dp"
    android:layout_marginBottom="16dp" />
```

```
<!-- Divider -->
```

```
<View
    android:layout_width="match_parent"
    android:layout_height="1dp"
    android:background="@color/signupDivider"
    android:layout_marginBottom="20dp" />
```

```

<!-- Password Count -->
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/profile_label_password_count"
    android:textSize="12sp"
    android:textColor="@color/profileStatLabel"
    android:fontFamily="sans-serif"
    android:layout_marginBottom="4dp" />

<TextView
    android:id="@+id/textViewProfilePasswordCount"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="20sp"
    android:textColor="@color/colorAccent"
    android:textStyle="bold"
    android:fontFamily="sans-serif-medium"
    android:drawableStart="@android:drawable/ic_lock_idle_lock"
    android:drawablePadding="8dp" />

</LinearLayout>
</androidx.cardview.widget.CardView>

<!-- Action buttons -->

<!-- Edit Credentials button -->
<com.google.android.material.button.MaterialButton
    android:id="@+id/buttonProfileEdit"
    android:layout_width="match_parent"
    android:layout_height="52dp"
    android:text="@string/profile_button_edit"
    android:textSize="16sp"
    android:textColor="@color/entryButtonText"
    android:textStyle="bold"
    android:textAllCaps="false"
    android:fontFamily="sans-serif-medium"
    android:backgroundTint="@color/entryButtonBackground"

```

```
app: cornerRadius="12dp"
android: layout_marginBottom="12dp" />
```

```
<!-- Log Out button -->
```

```
<com.google.android.material.button.MaterialButton
    android: id="@+id/buttonProfileLogout"
    android: layout_width="match_parent"
    android: layout_height="52dp"
    android: text="@string/profile_button_logout"
    android: textSize="16sp"
    android: textColor="@color/entryButtonText"
    android: textStyle="bold"
    android: textAllCaps="false"
    android: fontFamily="sans-serif-medium"
    android: backgroundTint="@color/profileLogoutButton"
    app: cornerRadius="12dp"
    android: layout_marginBottom="12dp" />
```

```
<!-- Remove Account button -->
```

```
<com.google.android.material.button.MaterialButton
    android: id="@+id/buttonProfileDelete"
    android: layout_width="match_parent"
    android: layout_height="52dp"
    android: text="@string/profile_button_delete"
    android: textSize="16sp"
    android: textColor="@color/entryButtonText"
    android: textStyle="bold"
    android: textAllCaps="false"
    android: fontFamily="sans-serif-medium"
    android: backgroundTint="@color/profileDeleteButton"
    app: cornerRadius="12dp"
    android: layout_marginBottom="16dp" />
```

```
</LinearLayout>
```

```
</ScrollView>
```

```
</RelativeLayout>
```

--- FILE: FinalProject333057891\SecurioClient\Resources\layout\activity_risk_detail.xml ---

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/vaultBackground">

    <!-- Header area -->
    <LinearLayout
        android:id="@+id/layoutRiskHeader"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:paddingStart="24dp"
        android:paddingEnd="24dp"
        android:paddingTop="48dp"
        android:paddingBottom="8dp"
        android:layout_alignParentTop="true">

        <!-- Back arrow + icon row -->
        <RelativeLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginBottom="8dp">

            <ImageView
                android:id="@+id/imageViewRiskBack"
                android:layout_width="36dp"
                android:layout_height="36dp"
                android:layout_alignParentStart="true"
                android:layout_centerVertical="true"
                android:src="@android:drawable/ic_menu_revert"
                android:padding="4dp"
                android:background="?android:attr/selectableItemBackgroundBorderless"
                android:contentDescription="Back" />

            <TextView
                android:id="@+id/textViewRiskEmoji"
                android:layout_width="wrap_content"
```

```

        android:layout_height="wrap_content"
        android:layout_centerInParent="true"
        android:text="אָי, •"
        android:textSize="36sp" />

```

```
</RelativeLayout>
```

```

<TextView
    android:id="@+id/textViewRiskTitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    style="@style/VaultTitleStyle"
    android:layout_marginBottom="4dp" />

```

```

<TextView
    android:id="@+id/textViewRiskSubtitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    style="@style/VaultSubtitleStyle"
    android:layout_marginBottom="16dp" />

```

```
<!-- Search bar -->
```

```

<EditText
    android:id="@+id/editTextRiskSearch"
    android:layout_width="match_parent"
    android:layout_height="44dp"
    android:background="@drawable/bg_search_bar"
    android:hint="@string/vault_search_hint"
    android:textColor="@color/vaultSearchText"
    android:textColorHint="@color/vaultSearchHint"
    android:textSize="14sp"
    android:fontFamily="sans-serif"
    android:singleLine="true"
    android:paddingStart="16dp"
    android:paddingEnd="16dp"
    android:drawableStart="@android:drawable/ic_menu_search"
    android:drawablePadding="8dp"
    android:inputType="text"
    android:imeOptions="actionSearch"

```

```

        android:layout_marginBottom="8dp" />

</LinearLayout>

<!-- Password list (RecyclerView) fills space between header and bottom -->
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recyclerViewRiskPasswords"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_below="@id/layoutRiskHeader"
    android:paddingStart="16dp"
    android:paddingEnd="16dp"
    android:paddingTop="8dp"
    android:paddingBottom="16dp"
    android:clipToPadding="false"
    android:scrollbars="vertical" />

<!-- Empty state (shown when there are no passwords at risk) -->
<LinearLayout
    android:id="@+id/layoutRiskEmpty"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_centerInParent="true"
    android:orientation="vertical"
    android:gravity="center"
    android:visibility="gone">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="⚠️..."
        android:textSize="48sp"
        android:layout_marginBottom="12dp" />

    <TextView
        android:id="@+id/textViewRiskEmptyTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/risk_empty_title"

```

```

        android:textSize="18sp"
        android:textColor="@color/vaultHeaderText"
        android:textStyle="bold"
        android:fontFamily="sans-serif-medium"
        android:layout_marginBottom="4dp" />

```

```

<TextView
    android:id="@+id/textViewRiskEmptySubtitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/risk_empty_subtitle"
    android:textSize="14sp"
    android:textColor="@color/vaultEmptyText"
    android:fontFamily="sans-serif" />

```

```

</LinearLayout>

```

```

</RelativeLayout>

```

--- FILE: FinalProject333057891\SecurioClient\Resources\layout\activity_signup.xml ---

```

<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true"
    android:background="@color/signupBackground">

```

```

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:gravity="center_horizontal"
    android:paddingStart="32dp"
    android:paddingEnd="32dp"
    android:paddingTop="48dp"
    android:paddingBottom="32dp">

```



```

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="ðŸ›;ï•"
    android:textSize="64sp"
    android:layout_marginBottom="8dp" />

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/signup_title"
    style="@style/SignupTitleStyle"
    android:layout_marginBottom="8dp" />

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/signup_subtitle"
    style="@style/SignupSubtitleStyle"
    android:layout_marginBottom="32dp" />

<androidx.cardview.widget.CardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:cardCornerRadius="16dp"
    app:cardElevation="8dp"
    app:cardBackgroundColor="@color/signupCardBackground"
    android:layout_marginBottom="24dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="24dp">

        <!-- Username -->
        <com.google.android.material.textfield.TextInputLayout
            style="@style/SignupInputLayoutStyle">
            <com.google.android.material.textfield.TextInputEditText

```

```

        android: id="@+id/editTextUsername"
        style="@style/SignupEditTextStyle"
        android: hint="@string/signup_username_hint"
        android: inputType="textPersonName"
        android: drawableStart="@android:drawable/ic_menu_info_details"
        android: drawablePadding="8dp" />
</com.google.android.material.textfield.TextInputLayout>

```

```

<TextView
    android: id="@+id/textViewUsernameError"
    style="@style/SignupErrorTextStyle"
    android: layout_marginBottom="8dp" />

```

<!-- Email -->

```

<com.google.android.material.textfield.TextInputLayout
    style="@style/SignupInputLayoutStyle">
    <com.google.android.material.textfield.TextInputEditText
        android: id="@+id/editTextEmail"
        style="@style/SignupEditTextStyle"
        android: hint="@string/signup_email_hint"
        android: inputType="textEmailAddress"
        android: drawableStart="@android:drawable/ic_dialog_email"
        android: drawablePadding="8dp" />
</com.google.android.material.textfield.TextInputLayout>

```

```

<TextView
    android: id="@+id/textViewEmailError"
    style="@style/SignupErrorTextStyle"
    android: layout_marginBottom="8dp" />

```

<!-- Master Password -->

```

<com.google.android.material.textfield.TextInputLayout
    style="@style/SignupInputLayoutStyle"
    app: passwordToggleEnabled="true">
    <com.google.android.material.textfield.TextInputEditText
        android: id="@+id/editTextPassword"
        style="@style/SignupEditTextStyle"
        android: hint="@string/signup_password_hint"
        android: inputType="textPassword"

```

```

        android:drawableStart="@android:drawable/ic_lock_idle_lock"
        android:drawablePadding="8dp" />
</com.google.android.material.textfield.TextInputLayout>

```

```

<!-- Generate password button -->
<com.google.android.material.button.MaterialButton
    android:id="@+id/buttonGeneratePassword"
    style="@style/SignupGenerateButtonStyle"
    android:text="@string/signup_generate_password" />

```

```

<TextView
    android:id="@+id/textViewPasswordError"
    style="@style/SignupErrorTextStyle"
    android:layout_marginBottom="4dp" />

```

```

<!-- 5-segment password strength bar -->
<ProgressBar
    android:id="@+id/progressBarPasswordStrength"
    style="?android:attr/progressBarStyleHorizontal"
    android:layout_width="match_parent"
    android:layout_height="8dp"
    android:max="5"
    android:progress="0"
    android:visibility="gone"
    android:progressBackgroundTint="@color/signupDivider"
    android:progressTint="@color/passwordStrengthWeak"
    android:layout_marginBottom="4dp" />

```

```

<!-- Constructive feedback hint -->
<TextView
    android:id="@+id/textViewPasswordHint"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:textSize="12sp"
    android:fontFamily="sans-serif"
    android:visibility="gone"
    android:layout_marginBottom="12dp" />

```

```

<!-- Confirm Master Password -->

```

```

<com.google.android.material.textfield.TextInputLayout
    style="@style/SignupInputLayoutStyle"
    app:passwordToggleEnabled="true">
    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/editTextConfirmPassword"
        style="@style/SignupEditTextStyle"
        android:hint="@string/signup_confirm_password_hint"
        android:inputType="textPassword"
        android:drawableStart="@android:drawable/ic_lock_idle_lock"
        android:drawablePadding="8dp" />
</com.google.android.material.textfield.TextInputLayout>

<TextView
    android:id="@+id/textViewConfirmPasswordError"
    style="@style/SignupErrorTextStyle"
    android:layout_marginBottom="16dp" />

<com.google.android.material.button.MaterialButton
    android:id="@+id/buttonSignUp"
    style="@style/SignupButtonStyle"
    android:text="@string/signup_button_text"
    android:backgroundTint="@color/signupButtonBackground"
    app:cornerRadius="12dp" />

<TextView
    android:id="@+id/textViewGeneralError"
    style="@style/SignupErrorTextStyle"
    android:layout_gravity="center"
    android:layout_marginTop="8dp" />

</LinearLayout>
</androidx.cardview.widget.CardView>

<!-- Indeterminate spinner shown while the server request is in flight -->
<ProgressBar
    android:id="@+id/progressBarSignup"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:visibility="gone"

```

```

        android:indeterminateTint="@color/colorAccent" />

<LinearLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center"
    android:layout_marginTop="8dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/signup_login_prompt"
        android:textSize="14sp"
        android:textColor="@color/signupSubtitleText"
        android:layout_marginEnd="4dp" />

    <TextView
        android:id="@+id/textViewLoginLink"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/signup_login_link"
        style="@style/SignupLinkTextStyle"
        android:clickable="true"
        android:focusable="true" />
</LinearLayout>

</LinearLayout>
</ScrollView>

--- FILE: FinalProject333057891\SecurioClient\Resources\layout\activity_vault.xml ---
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/vaultBackground">

```

```

<!-- Header area -->
<LinearLayout
    android:id="@+id/layoutVaultHeader"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:paddingStart="24dp"
    android:paddingEnd="24dp"
    android:paddingTop="48dp"
    android:paddingBottom="8dp"
    android:layout_alignParentTop="true">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="ðŸ›;ï•"
        android:textSize="36sp"
        android:layout_marginBottom="4dp" />

    <TextView
        android:id="@+id/textViewVaultTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/vault_title"
        style="@style/VaultTitleStyle"
        android:layout_marginBottom="4dp" />

    <TextView
        android:id="@+id/textViewVaultSubtitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/vault_subtitle"
        style="@style/VaultSubtitleStyle"
        android:layout_marginBottom="16dp" />

<!-- Search bar -->
<EditText
    android:id="@+id/editTextVaultSearch"

```

```

android:layout_width="match_parent"
android:layout_height="44dp"
android:background="@drawable/bg_search_bar"
android:hint="@string/vault_search_hint"
android:textColor="@color/vaultSearchText"
android:textColorHint="@color/vaultSearchHint"
android:textSize="14sp"
android:fontFamily="sans-serif"
android:singleLine="true"
android:paddingStart="16dp"
android:paddingEnd="16dp"
android:drawableStart="@android:drawable/ic_menu_search"
android:drawablePadding="8dp"
android:inputType="text"
android:imeOptions="actionSearch"
android:layout_marginBottom="8dp" />

```

```
</LinearLayout>
```

```
<!-- Bottom navigation fragment container -->
```

```
<FrameLayout
```

```

    android:id="@+id/frameBottomNav"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_alignParentBottom="true" />

```

```
<!-- Password list (RecyclerView) fills space between header and bottom nav -->
```

```
<androidx.recyclerview.widget.RecyclerView
```

```

    android:id="@+id/recyclerViewPasswords"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_below="@id/layoutVaultHeader"
    android:layout_above="@id/frameBottomNav"
    android:paddingStart="16dp"
    android:paddingEnd="16dp"
    android:paddingTop="8dp"
    android:paddingBottom="80dp"
    android:clipToPadding="false"
    android:scrollbars="vertical" />

```

```
<!-- Empty state (shown when there are no passwords) -->
```

```
<LinearLayout
```

```
    android:id="@+id/layoutVaultEmpty"
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:layout_centerInParent="true"
```

```
    android:orientation="vertical"
```

```
    android:gravity="center"
```

```
    android:visibility="gone">
```

```
<TextView
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="ðŸ”•"
```

```
    android:textSize="48sp"
```

```
    android:layout_marginBottom="12dp" />
```

```
<TextView
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="@string/vault_empty_title"
```

```
    android:textSize="18sp"
```

```
    android:textColor="@color/vaultHeaderText"
```

```
    android:textStyle="bold"
```

```
    android:fontFamily="sans-serif-medium"
```

```
    android:layout_marginBottom="4dp" />
```

```
<TextView
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="@string/vault_empty_subtitle"
```

```
    android:textSize="14sp"
```

```
    android:textColor="@color/vaultEmptyText"
```

```
    android:fontFamily="sans-serif" />
```

```
</LinearLayout>
```

```
<!-- Floating Action Button to add a new password -->
```



```
<com.google.android.material.button.MaterialButton
    android:id="@+id/fabAddPassword"
    android:layout_width="56dp"
    android:layout_height="56dp"
    android:layout_alignParentEnd="true"
    android:layout_above="@id/frameBottomNav"
    android:layout_marginEnd="20dp"
    android:layout_marginBottom="16dp"
    android:text="+"
    android:textSize="24sp"
    android:textColor="@color/vaultFabIcon"
    android:backgroundTint="@color/vaultFabBackground"
    android:gravity="center"
    android:padding="0dp"
    android:insetTop="0dp"
    android:insetBottom="0dp"
    app:cornerRadius="28dp"
    app:elevation="6dp" />
```

```
</RelativeLayout>
```

```
--- FILE: FinalProject333057891\SecurioClient\Resources\layout\activity_view_password.xml
---
```

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true"
    android:background="@color/entryBackground">
```

```
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:gravity="center_horizontal"
        android:paddingStart="32dp"
        android:paddingEnd="32dp"
```

```
android:paddingTop="48dp"
android:paddingBottom="32dp">
```

```
<!-- Back arrow + shield icon row -->
```

```
<RelativeLayout
```

```
android:layout_width="match_parent"
```

```
android:layout_height="wrap_content"
```

```
android:layout_marginBottom="8dp">
```

```
<ImageView
```

```
android:id="@+id/imageViewBack"
```

```
android:layout_width="36dp"
```

```
android:layout_height="36dp"
```

```
android:layout_alignParentStart="true"
```

```
android:layout_centerVertical="true"
```

```
android:src="@android:drawable/ic_menu_revert"
```

```
android:padding="4dp"
```

```
android:background="?android:attr/selectableItemBackgroundBorderless"
```

```
android:contentDescription="Back" />
```

```
<TextView
```

```
android:layout_width="wrap_content"
```

```
android:layout_height="wrap_content"
```

```
android:layout_centerInParent="true"
```

```
android:text="ðŸŒ™;ï‚•"
```

```
android:textSize="36sp" />
```

```
</RelativeLayout>
```

```
<!-- Title -->
```

```
<TextView
```

```
android:id="@+id/textViewTitle"
```

```
android:layout_width="wrap_content"
```

```
android:layout_height="wrap_content"
```

```
android:text="@string/view_title"
```

```
style="@style/SignupTitleStyle"
```

```
android:layout_marginBottom="4dp" />
```

```
<!-- Subtitle -->
```

```

<TextView
    android:id="@+id/textViewViewSubtitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/view_subtitle"
    style="@style/SignupSubtitleStyle"
    android:layout_marginBottom="28dp" />

```

```

<!-- Details card -->
<androidx.cardview.widget.CardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:cardCornerRadius="16dp"
    app:cardElevation="8dp"
    app:cardBackgroundColor="@color/entryCardBackground"
    android:layout_marginBottom="24dp">

```

```

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="24dp">

```

```

<!-- Site / App name -->
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/view_label_site_name"
    android:textSize="12sp"
    android:textColor="@color/signupHintText"
    android:fontFamily="sans-serif"
    android:layout_marginBottom="4dp" />

```

```

<TextView
    android:id="@+id/textViewViewSiteName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="16sp"
    android:textColor="@color/signupInputText"

```

```

android: fontFamily="sans-serif-medium"
android: drawableStart="@android: drawable/ic_menu_view"
android: drawablePadding="8dp"
android: layout_marginBottom="20dp" />

```

```

        <!-- Username / Email -->
        <TextView
            android: layout_width="wrap_content"
            android: layout_height="wrap_content"
            android: text="@string/view_label_username"
            android: textSize="12sp"
            android: textColor="@color/signupHintText"
            android: fontFamily="sans-serif"
            android: layout_marginBottom="4dp" />

```

```

        <RelativeLayout
            android: layout_width="match_parent"
            android: layout_height="wrap_content"
            android: layout_marginBottom="20dp">

```

```

            <TextView
                android: id="@+id/textViewUsername"
                android: layout_width="match_parent"
                android: layout_height="wrap_content"
                android: layout_toStartOf="@+id/buttonCopyUsername"
                android: layout_centerVertical="true"
                android: textSize="16sp"
                android: textColor="@color/signupInputText"
                android: fontFamily="sans-serif"
                android: drawableStart="@android: drawable/ic_menu_info_details"
                android: drawablePadding="8dp" />

```

```

            <ImageView
                android: id="@+id/buttonCopyUsername"
                android: layout_width="36dp"
                android: layout_height="36dp"
                android: layout_alignParentEnd="true"
                android: layout_centerVertical="true"
                android: src="@android: drawable/ic_menu_set_as"

```

```

        android:padding="6dp"
        android:background="?android:attr/selectableItemBackgroundBorderless"
        android:contentDescription="Copy username" />

```

```

    </RelativeLayout>

```

```

    <!-- Password -->

```

```

    <TextView

```

```

        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/view_label_password"
        android:textSize="12sp"
        android:textColor="@color/signupHintText"
        android:fontFamily="sans-serif"
        android:layout_marginBottom="4dp" />

```

```

    <RelativeLayout

```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="20dp">

```

```

        <TextView

```

```

            android:id="@+id/textViewViewPassword"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_toStartOf="@+id/layoutPasswordActions"
            android:layout_centerVertical="true"
            android:textSize="16sp"
            android:textColor="@color/signupInputText"
            android:fontFamily="monospace"
            android:drawableStart="@android:drawable/ic_lock_idle_lock"
            android:drawablePadding="8dp" />

```

```

        <LinearLayout

```

```

            android:id="@+id/layoutPasswordActions"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignParentEnd="true"
            android:layout_centerVertical="true"

```

```
android:orientation="horizontal">
```

```

<ImageView
    android:id="@+id/buttonTogglePassword"
    android:layout_width="36dp"
    android:layout_height="36dp"
    android:src="@drawable/ic_eye"
    android:tint="@color/signupHintText"
    android:padding="6dp"
    android:background="?android:attr/selectableItemBackgroundBorderless"
    android:contentDescription="Toggle password visibility" />

```

```

<ImageView
    android:id="@+id/buttonCopyPassword"
    android:layout_width="36dp"
    android:layout_height="36dp"
    android:src="@android:drawable/ic_menu_set_as"
    android:padding="6dp"
    android:background="?android:attr/selectableItemBackgroundBorderless"
    android:contentDescription="Copy password" />

```

```
</LinearLayout>
```

```
</RelativeLayout>
```

```
<!-- Notes -->
```

```

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/view_label_notes"
    android:textSize="12sp"
    android:textColor="@color/signupHintText"
    android:fontFamily="sans-serif"
    android:layout_marginBottom="4dp" />

```

```

<TextView
    android:id="@+id/textViewViewNotes"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"

```

```

        android:textSize="15sp"
        android:textColor="@color/signupInputText"
        android:fontFamily="sans-serif"
        android:drawableStart="@android:drawable/ic_menu_edit"
        android:drawablePadding="8dp"
        android:lineSpacingExtra="4dp" />

```

```

        <!-- Last Changed -->

```

```

        <TextView

```

```

        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/view_label_last_changed"
        android:textSize="12sp"
        android:textColor="@color/signupHintText"
        android:fontFamily="sans-serif"
        android:layout_marginTop="20dp"
        android:layout_marginBottom="4dp" />

```

```

        <TextView

```

```

        android:id="@+id/textViewViewLastChanged"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="14sp"
        android:textColor="@color/signupInputText"
        android:fontFamily="sans-serif"
        android:drawableStart="@android:drawable/ic_menu_recent_history"
        android:drawablePadding="8dp" />

```

```

    </LinearLayout>

```

```

</androidx.cardview.widget.CardView>

```

```

</LinearLayout>

```

```

</ScrollView>

```

--- FILE: FinalProject333057891\SecurioClient\Resources\layout\activity_warnings.xml ---

```

<?xml version="1.0" encoding="utf-8"?>

```

```

<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"

```

```

    android:layout_width="match_parent"

```

```
android:layout_height="match_parent"
android:background="@color/vaultBackground">
```

```
<!-- Header area -->
```

```
<LinearLayout
    android:id="@+id/layoutWarningsHeader"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:paddingStart="24dp"
    android:paddingEnd="24dp"
    android:paddingTop="48dp"
    android:paddingBottom="8dp"
    android:layout_alignParentTop="true">
```

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="אשׁי • "
    android:textSize="36sp"
    android:layout_marginBottom="4dp" />
```

```
<TextView
    android:id="@+id/textViewWarningsTitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/warnings_title"
    style="@style/VaultTitleStyle"
    android:layout_marginBottom="4dp" />
```

```
<TextView
    android:id="@+id/textViewWarningsSubtitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/warnings_subtitle"
    style="@style/VaultSubtitleStyle"
    android:layout_marginBottom="16dp" />
```

```
</LinearLayout>
```



```

<!-- Bottom navigation fragment container -->
<FrameLayout
    android:id="@+id/frameBottomNav"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_alignParentBottom="true" />

<!-- Scrollable risk cards -->
<ScrollView
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_below="@id/layoutWarningsHeader"
    android:layout_above="@id/frameBottomNav"
    android:paddingStart="16dp"
    android:paddingEnd="16dp"
    android:paddingTop="8dp"
    android:paddingBottom="16dp"
    android:clipToPadding="false"
    android:scrollbars="vertical">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <!-- â••â••â•• Leaked / Breached card â••â••â•• -->
        <LinearLayout
            android:id="@+id/cardLeaked"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:background="@drawable/bg_warning_card"
            android:padding="16dp"
            android:layout_marginBottom="12dp"
            android:gravity="center_vertical">

            <!-- Accent bar + count circle -->
            <LinearLayout

```

```

android:layout_width="56dp"
android:layout_height="56dp"
android:gravity="center"
android:background="@drawable/bg_warning_circle_leaked">

```

```

<TextView
    android:id="@+id/textViewLeakedCount"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="0"
    android:textSize="22sp"
    android:textStyle="bold"
    android:textColor="#FFFFFF"
    android:fontFamily="sans-serif-medium" />

```

```

</LinearLayout>

```

```

<LinearLayout
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_weight="1"
    android:orientation="vertical"
    android:layout_marginStart="16dp">

```

```

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/warnings_leaked_title"
    android:textSize="16sp"
    android:textStyle="bold"
    android:textColor="@color/warningLeakedAccent"
    android:fontFamily="sans-serif-medium" />

```

```

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/warnings_leaked_desc"
    android:textSize="12sp"
    android:textColor="@color/vaultCardSubtitleText"
    android:fontFamily="sans-serif"

```

```
android:layout_marginTop="2dp" />
```

```
<TextView
```

```
    android:id="@+id/buttonViewAllLeaked"
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="@string/warnings_view_all"
```

```
    android:textSize="13sp"
```

```
    android:textStyle="bold"
```

```
    android:textColor="@color/warningLeakedAccent"
```

```
    android:fontFamily="sans-serif-medium"
```

```
    android:layout_marginTop="6dp"
```

```
    android:padding="4dp"
```

```
    android:background="?android:attr/selectableItemBackground" />
```

```
</LinearLayout>
```

```
</LinearLayout>
```

```
<!-- â••â•• Weak card â••â•• -->
```

```
<LinearLayout
```

```
    android:id="@+id/cardWeak"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
    android:orientation="horizontal"
```

```
    android:background="@drawable/bg_warning_card"
```

```
    android:padding="16dp"
```

```
    android:layout_marginBottom="12dp"
```

```
    android:gravity="center_vertical">
```

```
<LinearLayout
```

```
    android:layout_width="56dp"
```

```
    android:layout_height="56dp"
```

```
    android:gravity="center"
```

```
    android:background="@drawable/bg_warning_circle_weak">
```

```
<TextView
```

```
    android:id="@+id/textViewWeakCount"
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```

        android:text="0"
        android:textSize="22sp"
        android:textStyle="bold"
        android:textColor="#FFFFFF"
        android:fontFamily="sans-serif-medium" />
</LinearLayout>

<LinearLayout
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_weight="1"
    android:orientation="vertical"
    android:layout_marginStart="16dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/warnings_weak_title"
        android:textSize="16sp"
        android:textStyle="bold"
        android:textColor="@color/warningWeakAccent"
        android:fontFamily="sans-serif-medium" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/warnings_weak_desc"
        android:textSize="12sp"
        android:textColor="@color/vaultCardSubtitleText"
        android:fontFamily="sans-serif"
        android:layout_marginTop="2dp" />

    <TextView
        android:id="@+id/buttonViewAllWeak"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/warnings_view_all"
        android:textSize="13sp"
        android:textStyle="bold"

```

```

        android:textColor="@color/warningWeakAccent"
        android:fontFamily="sans-serif-medium"
        android:layout_marginTop="6dp"
        android:padding="4dp"
        android:background="?android:attr/selectableItemBackground" />
    </LinearLayout>

```

```

</LinearLayout>

```

```

<!-- â••â••â•• Reused card â••â••â•• -->

```

```

<LinearLayout
    android:id="@+id/cardReused"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:background="@drawable/bg_warning_card"
    android:padding="16dp"
    android:layout_marginBottom="12dp"
    android:gravity="center_vertical">

```

```

<LinearLayout
    android:layout_width="56dp"
    android:layout_height="56dp"
    android:gravity="center"
    android:background="@drawable/bg_warning_circle_reused">

```

```

<TextView
    android:id="@+id/textViewReusedCount"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="0"
    android:textSize="22sp"
    android:textStyle="bold"
    android:textColor="#FFFFFF"
    android:fontFamily="sans-serif-medium" />
</LinearLayout>

```

```

<LinearLayout
    android:layout_width="0dp"

```

```

android:layout_height="wrap_content"
android:layout_weight="1"
android:orientation="vertical"
android:layout_marginStart="16dp">

```

```

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/warnings_reused_title"
    android:textSize="16sp"
    android:textStyle="bold"
    android:textColor="@color/warningReusedAccent"
    android:fontFamily="sans-serif-medium" />

```

```

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/warnings_reused_desc"
    android:textSize="12sp"
    android:textColor="@color/vaultCardSubtitleText"
    android:fontFamily="sans-serif"
    android:layout_marginTop="2dp" />

```

```

<TextView
    android:id="@+id/buttonViewAllReused"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/warnings_view_all"
    android:textSize="13sp"
    android:textStyle="bold"
    android:textColor="@color/warningReusedAccent"
    android:fontFamily="sans-serif-medium"
    android:layout_marginTop="6dp"
    android:padding="4dp"
    android:background="?android:attr/selectableItemBackground" />

```

```

</LinearLayout>

```

```

</LinearLayout>

```

```

<!-- â••â•• Old card â••â•• -->
<LinearLayout
    android:id="@+id/cardOld"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:background="@drawable/bg_warning_card"
    android:padding="16dp"
    android:layout_marginBottom="12dp"
    android:gravity="center_vertical">

    <LinearLayout
        android:layout_width="56dp"
        android:layout_height="56dp"
        android:gravity="center"
        android:background="@drawable/bg_warning_circle_old">

        <TextView
            android:id="@+id/textViewOldCount"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="0"
            android:textSize="22sp"
            android:textStyle="bold"
            android:textColor="#FFFFFF"
            android:fontFamily="sans-serif-medium" />
    </LinearLayout>

    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:orientation="vertical"
        android:layout_marginStart="16dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/warnings_old_title"

```

```

        android:textSize="16sp"
        android:textStyle="bold"
        android:textColor="@color/warningOldAccent"
        android:fontFamily="sans-serif-medium" />

```

```

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/warnings_old_desc"
    android:textSize="12sp"
    android:textColor="@color/vaultCardSubtitleText"
    android:fontFamily="sans-serif"
    android:layout_marginTop="2dp" />

```

```

<TextView
    android:id="@+id/buttonViewAllOld"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/warnings_view_all"
    android:textSize="13sp"
    android:textStyle="bold"
    android:textColor="@color/warningOldAccent"
    android:fontFamily="sans-serif-medium"
    android:layout_marginTop="6dp"
    android:padding="4dp"
    android:background="?android:attr/selectableItemBackground" />

```

```

</LinearLayout>

```

```

</LinearLayout>

```

```

</LinearLayout>

```

```

</ScrollView>

```

```

</RelativeLayout>

```


--- FILE:

FinalProject333057891\SecurioClient\Resources\layout\bottom_sheet_password_options.xml -

--

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:background="@drawable/bg_bottom_sheet"
    android:paddingBottom="24dp">

    <!-- Drag handle -->
    <View
        android:layout_width="40dp"
        android:layout_height="4dp"
        android:layout_gravity="center_horizontal"
        android:layout_marginTop="12dp"
        android:layout_marginBottom="16dp"
        android:background="@color/vaultDivider" />

    <!-- Site name and username header -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center_vertical"
        android:paddingStart="20dp"
        android:paddingEnd="20dp"
        android:paddingBottom="16dp">

        <!-- Circular icon -->
        <FrameLayout
            android:layout_width="44dp"
            android:layout_height="44dp"
            android:layout_marginEnd="14dp">

            <View
                android:layout_width="match_parent"
                android:layout_height="match_parent"
```

```

        android:background="@drawable/bg_icon_circle" />

<TextView
    android:id="@+id/textViewSheetIcon"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:textSize="18sp"
    android:textColor="@color/vaultCardIconText"
    android:textStyle="bold"
    android:fontFamily="sans-serif-medium" />

</FrameLayout>

<!-- Account info -->
<LinearLayout
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_weight="1"
    android:orientation="vertical">

    <TextView
        android:id="@+id/textViewSheetTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="17sp"
        android:textColor="@color/vaultCardTitleText"
        android:textStyle="bold"
        android:fontFamily="sans-serif-medium"
        android:singleLine="true"
        android:ellipsize="end" />

    <TextView
        android:id="@+id/textViewSheetUsername"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="13sp"
        android:textColor="@color/vaultCardSubtitleText"
        android:fontFamily="sans-serif"

```

```

        android:layout_marginTop="2dp"
        android:singleLine="true"
        android:ellipsize="end" />

```

```

</LinearLayout>

```

```

</LinearLayout>

```

```

<!-- Divider -->

```

```

<View

```

```

    android:layout_width="match_parent"
    android:layout_height="1dp"
    android:background="@color/vaultDivider"
    android:layout_marginBottom="8dp" />

```

```

<!-- View option -->

```

```

<LinearLayout

```

```

    android:id="@+id/layoutOptionView"
    android:layout_width="match_parent"
    android:layout_height="56dp"
    android:orientation="horizontal"
    android:gravity="center_vertical"
    android:paddingStart="20dp"
    android:paddingEnd="20dp"
    android:background="?android:attr/selectableItemBackground">

```

```

<ImageView

```

```

    android:layout_width="24dp"
    android:layout_height="24dp"
    android:src="@drawable/ic_eye"
    android:layout_marginEnd="16dp"
    android:contentDescription="@string/sheet_option_view" />

```

```

<TextView

```

```

    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/sheet_option_view"
    android:textSize="15sp"
    android:textColor="@color/vaultCardTitleText"

```

```

        android: fontFamily="sans-serif" />

</LinearLayout>

<!-- Edit option -->
<LinearLayout
    android: id="@+id/layoutOptionEdit"
    android: layout_width="match_parent"
    android: layout_height="56dp"
    android: orientation="horizontal"
    android: gravity="center_vertical"
    android: paddingStart="20dp"
    android: paddingEnd="20dp"
    android: background="?android: attr/selectableItemBackground">

    <ImageView
        android: layout_width="24dp"
        android: layout_height="24dp"
        android: src="@drawable/ic_edit"
        android: layout_marginEnd="16dp"
        android: contentDescription="@string/sheet_option_edit" />

    <TextView
        android: layout_width="wrap_content"
        android: layout_height="wrap_content"
        android: text="@string/sheet_option_edit"
        android: textSize="15sp"
        android: textColor="@color/vaultCardTitleText"
        android: fontFamily="sans-serif" />

</LinearLayout>

<!-- Delete option -->
<LinearLayout
    android: id="@+id/layoutOptionDelete"
    android: layout_width="match_parent"
    android: layout_height="56dp"
    android: orientation="horizontal"
    android: gravity="center_vertical"

```

```

android:paddingStart="20dp"
android:paddingEnd="20dp"
android:background="?android:attr/selectableItemBackground">

```

```

<ImageView
    android:layout_width="24dp"
    android:layout_height="24dp"
    android:src="@drawable/ic_delete"
    android:layout_marginEnd="16dp"
    android:contentDescription="@string/sheet_option_delete" />

```

```

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/sheet_option_delete"
    android:textSize="15sp"
    android:textColor="@color/deleteActionText"
    android:fontFamily="sans-serif" />

```

```

</LinearLayout>

```

```

</LinearLayout>

```

--- FILE: FinalProject333057891\SecurioClient\Resources\layout\fragment_bottom_nav.xml ---

-

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="56dp"
    android:orientation="horizontal"
    android:gravity="center_vertical"
    android:background="@drawable/bg_bottom_nav"
    android:paddingStart="8dp"
    android:paddingEnd="8dp"
    android:baselineAligned="false">

```

```

<!-- Vault tab -->

```

```

<LinearLayout

```

```

android: id="@+id/navTabVault"
android: layout_width="0dp"
android: layout_height="match_parent"
android: layout_weight="1"
android: orientation="vertical"
android: gravity="center"
android: clickable="true"
android: focusable="true"
android: background="?android: attr/selectableItemBackgroundBorderless">

```

```

<ImageView
    android: id="@+id/iconNavVault"
    android: layout_width="24dp"
    android: layout_height="24dp"
    android: src="@android: drawable/ic_lock_idle_lock"
    android: contentDescription="@string/vault_nav_vault" />

```

```

<TextView
    android: id="@+id/textNavVault"
    android: layout_width="wrap_content"
    android: layout_height="wrap_content"
    android: text="@string/vault_nav_vault"
    style="@style/VaultBottomNavTextStyle"
    android: textColor="@color/vaultBottomNavSelected" />

```

```

</LinearLayout>

```

```

<!-- Warnings tab -->

```

```

<LinearLayout
    android: id="@+id/navTabWarnings"
    android: layout_width="0dp"
    android: layout_height="match_parent"
    android: layout_weight="1"
    android: orientation="vertical"
    android: gravity="center"
    android: clickable="true"
    android: focusable="true"
    android: background="?android: attr/selectableItemBackgroundBorderless">

```

```
<ImageView
    android:id="@+id/iconNavWarnings"
    android:layout_width="24dp"
    android:layout_height="24dp"
    android:src="@android:drawable/ic_dialog_alert"
    android:contentDescription="@string/vault_nav_warnings" />
```

```
<TextView
    android:id="@+id/textNavWarnings"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/vault_nav_warnings"
    style="@style/VaultBottomNavTextStyle"
    android:textColor="@color/vaultBottomNavUnselected" />
```

```
</LinearLayout>
```

```
<!-- Profile tab -->
```

```
<LinearLayout
    android:id="@+id/navTabProfile"
    android:layout_width="0dp"
    android:layout_height="match_parent"
    android:layout_weight="1"
    android:orientation="vertical"
    android:gravity="center"
    android:clickable="true"
    android:focusable="true"
    android:background="?android:attr/selectableItemBackgroundBorderless">
```

```
<ImageView
    android:id="@+id/iconNavProfile"
    android:layout_width="24dp"
    android:layout_height="24dp"
    android:src="@android:drawable/ic_menu_myplaces"
    android:contentDescription="@string/vault_nav_profile" />
```

```
<TextView
    android:id="@+id/textNavProfile"
    android:layout_width="wrap_content"
```

```

        android:layout_height="wrap_content"
        android:text="@string/vault_nav_profile"
        style="@style/VaultBottomNavTextStyle"
        android:textColor="@color/vaultBottomNavUnselected" />

```

```

</LinearLayout>

```

```

</LinearLayout>

```

```

--- FILE: FinalProject333057891\SecurioClient\Resources\layout\item_password_banner.xml

```

```

---

```

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="10dp"
    app:cardCornerRadius="14dp"
    app:cardElevation="4dp"
    app:cardBackgroundColor="@color/vaultCardBackground">

```

```

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center_vertical"
        android:padding="14dp">

```

```

        <!-- Circular icon with the first letter of the site name -->

```

```

        <FrameLayout
            android:layout_width="42dp"
            android:layout_height="42dp"
            android:layout_marginEnd="14dp">

```

```

            <View
                android:layout_width="match_parent"
                android:layout_height="match_parent"

```



```

        android:background="@drawable/bg_icon_circle" />

<TextView
    android:id="@+id/textViewBannerIcon"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:textSize="18sp"
    android:textColor="@color/vaultCardIconText"
    android:textStyle="bold"
    android:fontFamily="sans-serif-medium" />

</FrameLayout>

<!-- Site name and username -->
<LinearLayout
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_weight="1"
    android:orientation="vertical">

    <TextView
        android:id="@+id/textViewBannerSiteName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        style="@style/VaultBannerTitleStyle" />

    <TextView
        android:id="@+id/textViewBannerUsername"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        style="@style/VaultBannerSubtitleStyle"
        android:layout_marginTop="2dp" />

</LinearLayout>

<!-- More options button -->
<ImageView
    android:id="@+id/imageViewBannerEdit"

```

```

        android:layout_width="36dp"
        android:layout_height="36dp"
        android:padding="6dp"
        android:src="@drawable/ic_more_vert"
        android:contentDescription="@string/vault_more_options"
        android:background="?android:attr/selectableItemBackgroundBorderless" />

</LinearLayout>

</androidx.cardview.widget.CardView>

```

--- FILE: FinalProject333057891\SecurioClient\Resources\mipmap-anydpi-v26\ic_launcher.xml ---

```

<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
    <background android:drawable="@color/ic_launcher_background"/>
    <foreground android:drawable="@mipmap/ic_launcher_foreground"/>
</adaptive-icon>

```

--- FILE: FinalProject333057891\SecurioClient\Resources\mipmap-anydpi-v26\ic_launcher_round.xml ---

```

<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
    <background android:drawable="@color/ic_launcher_background"/>
    <foreground android:drawable="@mipmap/ic_launcher_foreground"/>
</adaptive-icon>

```

--- FILE: FinalProject333057891\SecurioClient\Resources\values\colors.xml ---

```

<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="ic_launcher_background">#282A3E</color>
    <color name="colorPrimary">#2c3e50</color>
    <color name="colorPrimaryDark">#1B3147</color>
    <color name="colorAccent">#3498db</color>

```

```

<!-- Signup page colors -->

```

```
<color name="signupBackground">#1B2838</color>
<color name="signupCardBackground">#FFFFFF</color>
<color name="signupButtonBackground">#3498db</color>
<color name="signupButtonText">#FFFFFF</color>
<color name="signupHintText">#9E9E9E</color>
<color name="signupInputBackground">#F5F5F5</color>
<color name="signupInputText">#212121</color>
<color name="signupTitleText">#FFFFFF</color>
<color name="signupSubtitleText">#B0BEC5</color>
<color name="signupLinkText">#3498db</color>
<color name="signupErrorText">#E74C3C</color>
<color name="signupDivider">#E0E0E0</color>
<color name="passwordStrengthWeak">#E74C3C</color>
<color name="passwordStrengthPoor">#E67E22</color>
<color name="passwordStrengthFair">#F39C12</color>
<color name="passwordStrengthStrong">#27AE60</color>
<color name="passwordStrengthVeryStrong">#1E8449</color>
```

```
<!-- Vault page colors -->
```

```
<color name="vaultBackground">#1B2838</color>
<color name="vaultHeaderText">#FFFFFF</color>
<color name="vaultSubtitleText">#B0BEC5</color>
<color name="vaultSearchBackground">#253649</color>
<color name="vaultSearchText">#FFFFFF</color>
<color name="vaultSearchHint">#78909C</color>
<color name="vaultCardBackground">#253649</color>
<color name="vaultCardTitleText">#FFFFFF</color>
<color name="vaultCardSubtitleText">#B0BEC5</color>
<color name="vaultCardIconBackground">#3498db</color>
<color name="vaultCardIconText">#FFFFFF</color>
<color name="vaultFabBackground">#3498db</color>
<color name="vaultFabIcon">#FFFFFF</color>
<color name="vaultBottomNavBackground">#1B2838</color>
<color name="vaultBottomNavSelected">#3498db</color>
<color name="vaultBottomNavUnselected">#78909C</color>
<color name="vaultDivider">#37474F</color>
<color name="vaultEmptyText">#78909C</color>
<color name="deleteActionText">#E74C3C</color>
```

```

<!-- Warnings page colors -->
<color name="warningLeakedAccent">#E74C3C</color>
<color name="warningWeakAccent">#E67E22</color>
<color name="warningReusedAccent">#F39C12</color>
<color name="warningOldAccent">#3498db</color>

<!-- Entry (Add / Edit) page colors -->
<color name="entryBackground">#1B2838</color>
<color name="entryCardBackground">#FFFFFF</color>
<color name="entryButtonBackground">#3498db</color>
<color name="entryButtonText">#FFFFFF</color>
<color name="entryStrengthBackground">#E0E0E0</color>

<!-- Profile page colors -->
<color name="profileStatValue">#212121</color>
<color name="profileStatLabel">#9E9E9E</color>
<color name="profileLogoutButton">#E67E22</color>
<color name="profileDeleteButton">#E74C3C</color>
</resources>

--- FILE:
FinalProject333057891\SecurioClient\Resources\values\ic_launcher_background.xml ---
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="ic_launcher_background">#2C3E50</color>
</resources>

--- FILE: FinalProject333057891\SecurioClient\Resources\values\strings.xml ---
<resources>
    <string name="app_name">Securio</string>
    <string name="action_settings">Settings</string>

    <!-- Signup page strings -->
    <string name="signup_title">Create Account</string>
    <string name="signup_subtitle">Secure your passwords with Securio</string>
    <string name="signup_username_hint">Username</string>
    <string name="signup_email_hint">Email</string>

```

```

<string name="signup_password_hint">Master Password</string>
<string name="signup_confirm_password_hint">Confirm Master Password</string>
<string name="signup_button_text">Sign Up</string>
<string name="signup_login_prompt">Already have an account?</string>
<string name="signup_login_link">Log In</string>
<string name="signup_error_empty_fields">Please fill in all fields</string>
<string name="signup_error_invalid_email">Please enter a valid email address</string>
<string name="signup_error_password_mismatch">Passwords do not match</string>
<string name="signup_error_password_length">Password must be at least 8
characters</string>
<string name="signup_error_username_length">Username must be at least 3
characters</string>
<string name="signup_error_username_start">Username must start with a letter</string>
<string name="signup_error_username_chars">Username may only contain letters, digits,
underscores, or hyphens</string>
<string name="signup_error_password_uppercase">Password must include at least one
uppercase letter</string>
<string name="signup_error_password_lowercase">Password must include at least one
lowercase letter</string>
<string name="signup_error_password_digit">Password must include at least one
digit</string>
<string name="signup_error_password_special">Password must include at least one
special character (e.g. !@#$)</string>
<string name="signup_password_strength_weak">Strength: Weak</string>
<string name="signup_password_strength_fair">Strength: Fair</string>
<string name="signup_password_strength_strong">Strength: Strong</string>
<string name="signup_password_strength_very_strong">Strength: Very Strong
&acirc;</string>
<string name="signup_generate_password">ðŸŽ² Generate strong password</string>
<string name="signup_progress_message">Creating your secure account&acirc;</string>

<!-- Login page strings -->
<string name="login_title">Welcome Back</string>
<string name="login_subtitle">Sign in to your Securio vault</string>
<string name="login_email_hint">Email</string>
<string name="login_password_hint">Master Password</string>
<string name="login_button_text">Log In</string>
<string name="login_signup_prompt">Don't have an account?</string>
<string name="login_signup_link">Sign Up</string>

```

```

<string name="login_error_empty_fields">Please fill in all fields</string>
<string name="login_error_invalid_email">Please enter a valid email address</string>

<!-- Splash / loading screen strings -->
<string name="splash_app_name">Securio</string>
<string name="splash_tagline">Your passwords, locked and loaded</string>
<string name="splash_verifying">Verifying your sessionâ€¦</string>

<!-- Vault page strings -->
<string name="vault_title">My Vault</string>
<string name="vault_subtitle">Your saved passwords</string>
<string name="vault_search_hint">Search passwordsâ€¦</string>
<string name="vault_empty_title">No passwords yet</string>
<string name="vault_empty_subtitle">Tap + to add your first password</string>
<string name="vault_add_password">Add Password</string>
<string name="vault_edit_password">Edit Password</string>
<string name="vault_more_options">More options</string>

<!-- Password options bottom sheet strings -->
<string name="sheet_option_view">View</string>
<string name="sheet_option_edit">Edit</string>
<string name="sheet_option_delete">Delete</string>
<string name="sheet_delete_confirm_title">Delete password?</string>
<string name="sheet_delete_confirm_message">This will permanently remove the entry
for {0}. This action cannot be undone.</string>
<string name="sheet_delete_confirm_yes">Delete</string>
<string name="sheet_delete_confirm_no">Cancel</string>
<string name="sheet_deleted_toast">Password deleted</string>
<string name="sheet_delete_error">Failed to delete. Please check your connection and try
again.</string>
<string name="vault_nav_vault">Vault</string>
<string name="vault_nav_warnings">Warnings</string>
<string name="vault_nav_profile">Profile</string>

<!-- Entry (Add / Edit) page strings -->
<string name="entry_title_add">Add Password</string>
<string name="entry_title_edit">Edit Password</string>
<string name="entry_subtitle_add">Save a new credential to your vault</string>
<string name="entry_subtitle_edit">Update your saved credential</string>

```

```

<string name="entry_site_name_hint">App or site name</string>
<string name="entry_username_hint">Username or email</string>
<string name="entry_password_hint">Password</string>
<string name="entry_url_hint">Website URL (optional)</string>
<string name="entry_notes_hint">Notes (optional)</string>
<string name="entry_generate_password">ðŸŽ² Generate password</string>
<string name="entry_button_save">Save</string>
<string name="entry_button_update">Update</string>
<string name="entry_error_site_required">App or site name is required</string>
<string name="entry_error_username_required">Username or email is required</string>
<string name="entry_error_password_required">Password is required</string>
<string name="entry_error_duplicate">An entry with this site name and username already
exists</string>
<string name="entry_saved_success">Password saved successfully</string>
<string name="entry_updated_success">Password updated successfully</string>
<string name="entry_error_save_failed">Unable to save password. Please check your
connection and try again.</string>
<string name="entry_error_update_failed">Unable to update password. Please check your
connection and try again.</string>
<string name="entry_confirm_password_hint">Confirm Password</string>
<string name="entry_confirm_password_edit_hint">Confirm New Password</string>
<string name="entry_error_confirm_password_required">Please confirm your
password</string>
<string name="entry_error_passwords_mismatch">Passwords do not match</string>
<string name="entry_password_edit_hint">New Password</string>

<!-- View Password page strings -->
<string name="view_title">View Password</string>
<string name="view_subtitle">Credential details</string>
<string name="view_label_site_name">Site / App</string>
<string name="view_label_username">Username / Email</string>
<string name="view_label_password">Password</string>
<string name="view_label_notes">Notes</string>
<string name="view_no_notes">No notes</string>
<string name="view_copied_username">Username copied to clipboard</string>
<string name="view_copied_password">Password copied to clipboard</string>
<string name="view_decrypt_error">Unable to decrypt password</string>

<!-- Warnings page strings -->

```

```

<string name="warnings_title">Warnings</string>
<string name="warnings_subtitle">Password health overview</string>
<string name="warnings_leaked_title">Leaked / Breached</string>
<string name="warnings_leaked_desc">Passwords found in known data breaches
(HIBP)</string>
<string name="warnings_weak_title">Weak</string>
<string name="warnings_weak_desc">Passwords that don't meet complexity
requirements</string>
<string name="warnings_reused_title">Reused</string>
<string name="warnings_reused_desc">Same password used across multiple
accounts</string>
<string name="warnings_old_title">Old</string>
<string name="warnings_old_desc">Passwords not updated in over 90 days</string>

<!-- Risk detail page strings -->
<string name="risk_detail_leaked_subtitle">These passwords were found in known data
breaches</string>
<string name="risk_detail_weak_subtitle">These passwords don't meet complexity
requirements</string>
<string name="risk_detail_reused_subtitle">These passwords are used across multiple
accounts</string>
<string name="risk_detail_old_subtitle">These passwords haven't been updated in over 90
days</string>
<string name="risk_empty_title">All clear!</string>
<string name="risk_empty_subtitle">No passwords at risk in this category</string>
<string name="warnings_view_all">View All</string>
<string name="view_label_last_changed">Last changed</string>

<!-- Profile page strings -->
<string name="profile_title">My Profile</string>
<string name="profile_subtitle">Your account details</string>
<string name="profile_label_username">Username</string>
<string name="profile_label_email">Email</string>
<string name="profile_label_last_login">Last Login</string>
<string name="profile_label_last_password_change">Last Master Password
Change</string>
<string name="profile_label_created_at">Account Created</string>
<string name="profile_label_password_count">Passwords in Vault</string>
<string name="profile_button_edit">Edit Credentials</string>

```



```

<string name="profile_button_logout">Log Out</string>
<string name="profile_button_delete">Remove Account</string>
<string name="profile_logout_confirm_title">Log Out?</string>
<string name="profile_logout_confirm_message">You will need to sign in again to access
your vault.</string>
<string name="profile_logout_confirm_yes">Log Out</string>
<string name="profile_logout_confirm_no">Cancel</string>
<string name="profile_delete_confirm_title">Delete Account?</string>
<string name="profile_delete_confirm_message">This will permanently delete your
account and all saved passwords. This action cannot be undone.</string>
<string name="profile_delete_confirm_yes">Delete</string>
<string name="profile_delete_confirm_no">Cancel</string>
<string name="profile_deleted_toast">Account deleted successfully</string>
<string name="profile_delete_error">Failed to delete account. Please try again.</string>
<string name="profile_load_error">Unable to load profile. Showing cached data.</string>
<!-- Edit Account page strings -->
<string name="edit_account_title">Edit Account</string>
<string name="edit_account_subtitle">Update your credentials</string>
<string name="edit_account_username_hint">Username</string>
<string name="edit_account_email_hint">Email</string>
<string name="edit_account_current_password_hint">Current Master Password</string>
<string name="edit_account_new_password_hint">New Master Password</string>
<string name="edit_account_confirm_password_hint">Confirm New Master
Password</string>
<string name="edit_account_generate_password">ðŸŽ² Generate strong
password</string>
<string name="edit_account_button_save">Save Changes</string>
<string name="edit_account_password_section_title">Change Master Password
(optional)</string>
<string name="edit_account_success">Account updated successfully.</string>
<string name="edit_account_error">Unable to update account. Please check your
connection and try again.</string>
<string name="edit_account_current_password_wrong">Current password is
incorrect.</string>
<string name="edit_account_password_breached">This password was found in a data
breach. Please choose a different password.</string>
<string name="edit_account_reencrypt_progress">Re-encrypting vaultâ€¦</string>
<string name="login_error_password_required">Password is required</string>

```

```

    <string name="login_error_sign_in_failed">Unable to sign in. Please check your
connection and try again.</string>
    <string name="signup_error_create_failed">Unable to create account. Please check your
connection and try again.</string>
    <string name="edit_account_verify_password_failed">Unable to verify current password.
Please try again.</string>
    <string name="edit_account_current_password_required">Current password is required to
change your master password.</string>
    <string name="edit_account_password_reused">This password has been used recently.
Please choose a different one.</string>
</resources>

```

--- FILE: FinalProject333057891\SecurioClient\Resources\values\styles.xml ---

```

<resources>

    <!-- Base application theme. -->
    <style name="AppTheme" parent="Theme.MaterialComponents.Light.DarkActionBar">
        <!-- Customize your theme here. -->
        <item name="colorPrimary">@color/colorPrimary</item>
        <item name="colorPrimaryDark">@color/colorPrimaryDark</item>
        <item name="colorAccent">@color/colorAccent</item>
    </style>

    <!-- Signup page theme (no action bar, dark background) -->
    <style name="AppTheme.NoActionBar"
parent="Theme.MaterialComponents.Light.NoActionBar">
        <item name="colorPrimary">@color/colorPrimary</item>
        <item name="colorPrimaryDark">@color/colorPrimaryDark</item>
        <item name="colorAccent">@color/colorAccent</item>
        <item name="android:windowBackground">@color/signupBackground</item>
        <item name="android:statusBarColor">@color/colorPrimaryDark</item>
    </style>

    <!-- Reusable TextInputLayout style "outline box with rounded corners" -->
    <style name="SignupInputLayoutStyle">
        <item name="android:layout_width">match_parent</item>
        <item name="android:layout_height">wrap_content</item>
        <item name="android:layout_marginBottom">4dp</item>
    </style>

```

```

<item name="android:textColorHint">@color/colorAccent</item>
<item name="boxBackgroundMode">outline</item>
<item name="boxCornerRadiusTopStart">12dp</item>
<item name="boxCornerRadiusTopEnd">12dp</item>
<item name="boxCornerRadiusBottomStart">12dp</item>
<item name="boxCornerRadiusBottomEnd">12dp</item>
</style>

<!-- Signup title text style -->
<style name="SignupTitleStyle">
    <item name="android:textSize">28sp</item>
    <item name="android:textColor">@color/signupTitleText</item>
    <item name="android:textStyle">bold</item>
    <item name="android:fontFamily">sans-serif-medium</item>
</style>

<!-- Signup subtitle text style -->
<style name="SignupSubtitleStyle">
    <item name="android:textSize">14sp</item>
    <item name="android:textColor">@color/signupSubtitleText</item>
    <item name="android:fontFamily">sans-serif-light</item>
</style>

<!-- Signup input field style -->
<style name="SignupEditTextStyle">
    <item name="android:layout_width">match_parent</item>
    <item name="android:layout_height">52dp</item>
    <item name="android:paddingStart">16dp</item>
    <item name="android:paddingEnd">16dp</item>
    <item name="android:textSize">15sp</item>
    <item name="android:textColor">@color/signupInputText</item>
    <item name="android:textColorHint">@color/signupHintText</item>
    <item name="android:background">@color/signupInputBackground</item>
    <item name="android:fontFamily">sans-serif</item>
    <item name="android:singleLine">true</item>
</style>

<!-- Signup primary action button style -->
<style name="SignupButtonStyle">

```

```

<item name="android: layout_width">match_parent</item>
<item name="android: layout_height">52dp</item>
<item name="android: textSize">16sp</item>
<item name="android: textColor">@color/signupButtonText</item>
<item name="android: textStyle">bold</item>
<item name="android: textAllCaps">false</item>
<item name="android: fontFamily">sans-serif-medium</item>
</style>

<!-- Small secondary button for generating a password -->
<style name="SignupGenerateButtonStyle">
    <item name="android: layout_width">wrap_content</item>
    <item name="android: layout_height">36dp</item>
    <item name="android: layout_gravity">center</item>
    <item name="android: layout_marginBottom">4dp</item>
    <item name="android: textSize">12sp</item>
    <item name="android: textColor">@color/colorAccent</item>
    <item name="android: textAllCaps">false</item>
    <item name="android: backgroundTint">@android: color/transparent</item>
    <item name="android: fontFamily">sans-serif-medium</item>
</style>

<!-- Inline field error text style (hidden by default) -->
<style name="SignupErrorTextStyle">
    <item name="android: layout_width">wrap_content</item>
    <item name="android: layout_height">wrap_content</item>
    <item name="android: textSize">12sp</item>
    <item name="android: textColor">@color/signupErrorText</item>
    <item name="android: fontFamily">sans-serif</item>
    <item name="android: visibility">gone</item>
</style>

<!-- Signup link text style -->
<style name="SignupLinkTextStyle">
    <item name="android: textSize">14sp</item>
    <item name="android: textColor">@color/signupLinkText</item>
    <item name="android: textStyle">bold</item>
    <item name="android: fontFamily">sans-serif-medium</item>
</style>

```

```

<!-- Vault page styles -->
<style name="VaultTitleStyle">
    <item name="android: textSize">26sp</item>
    <item name="android: textColor">@color/vaultHeaderText</item>
    <item name="android: textStyle">bold</item>
    <item name="android: fontFamily">sans-serif-medium</item>
</style>

<style name="VaultSubtitleStyle">
    <item name="android: textSize">14sp</item>
    <item name="android: textColor">@color/vaultSubtitleText</item>
    <item name="android: fontFamily">sans-serif-light</item>
</style>

<style name="VaultBannerTitleStyle">
    <item name="android: textSize">16sp</item>
    <item name="android: textColor">@color/vaultCardTitleText</item>
    <item name="android: textStyle">bold</item>
    <item name="android: fontFamily">sans-serif-medium</item>
    <item name="android: singleLine">true</item>
    <item name="android: ellipsis">end</item>
</style>

<style name="VaultBannerSubtitleStyle">
    <item name="android: textSize">13sp</item>
    <item name="android: textColor">@color/vaultCardSubtitleText</item>
    <item name="android: fontFamily">sans-serif</item>
    <item name="android: singleLine">true</item>
    <item name="android: ellipsis">end</item>
</style>

<style name="VaultBottomNavTextStyle">
    <item name="android: textSize">11sp</item>
    <item name="android: fontFamily">sans-serif-medium</item>
</style>

</resources>

```

```

--- FILE: FinalProject333057891\SecurioClient\BootReceiver.cs ---
using Android.App;
using Android.Content;
using Android.Util;

namespace SecurioClient
{
    /// <summary>Receives BOOT_COMPLETED and MY_PACKAGE_REPLACED
    broadcasts to restart the PasswordMonitorService after reboot or update.</summary>
    [BroadcastReceiver(
        Exported = true,
        DirectBootAware = false)]
    [IntentFilter(new[]
    {
        Intent.ActionBootCompleted,
        Intent.ActionMyPackageReplaced
    })]
    public class BootReceiver : BroadcastReceiver
    {
        private const string Tag = "BootReceiver";

        /// <summary>Handles the broadcast and starts the PasswordMonitorService as a
        foreground service.</summary>
        public override void OnReceive(Context context, Intent intent)
        {
            Log.Info(Tag, $"Received broadcast: {intent?.Action}");
            var serviceIntent = new Intent(context, typeof(PasswordMonitorService));
            context.StartForegroundService(serviceIntent);
        }
    }
}

```

```

--- FILE: FinalProject333057891\SecurioClient\BottomNavFragment.cs ---
using Android.Graphics;
using Android.OS;
using Android.Views;

```

```

using Android.Widget;
using System;

namespace SecurioClient
{
    /// <summary>
    /// Fragment that renders the bottom navigation bar with three tabs: Vault, Warnings, and
    Profile.
    /// The host activity can subscribe to <see cref="TabSelected"/> to react to tab changes.
    /// </summary>
    public class BottomNavFragment : AndroidX.Fragment.App.Fragment
    {
        /// <summary>Raised when the user selects a different tab. The string is "vault",
        "warnings", or "profile".</summary>
        public event EventHandler<string> TabSelected;

        private const string ArgSelectedTab = "selectedTab";
        private string currentTab = "vault";

        /// <summary>
        /// Creates a new <see cref="BottomNavFragment"/> with the specified tab pre-selected.
        /// </summary>
        public static BottomNavFragment NewInstance(string selectedTab)
        {
            var fragment = new BottomNavFragment();
            var args = new Bundle();
            args.PutString(ArgSelectedTab, selectedTab);
            fragment.Arguments = args;
            return fragment;
        }

        public override View OnCreateView(LayoutInflater inflater, ViewGroup container,
        Bundle savedInstanceState)
        {
            return inflater.Inflate(Resource.Layout.fragment_bottom_nav, container, false);
        }

        public override void OnViewCreated(View view, Bundle savedInstanceState)
        {

```

```

base.OnViewCreated(view, savedInstanceState);

currentTab = Arguments?.GetString(ArgSelectedTab, "vault") ?? "vault";

var tabVault = view.FindViewById<LinearLayout>(Resource.Id.navTabVault);
var tabWarnings =
view.FindViewById<LinearLayout>(Resource.Id.navTabWarnings);
var tabProfile = view.FindViewById<LinearLayout>(Resource.Id.navTabProfile);

tabVault.Click += (s, e) => SelectTab(view, "vault");
tabWarnings.Click += (s, e) => SelectTab(view, "warnings");
tabProfile.Click += (s, e) => SelectTab(view, "profile");

// Default selection
SelectTab(view, currentTab);
}

private void SelectTab(View root, string tab)
{
    currentTab = tab;

    var selectedColor = new
Color(Resources.GetColor(Resource.Color.vaultBottomNavSelected));
    var unselectedColor = new
Color(Resources.GetColor(Resource.Color.vaultBottomNavUnselected));

    // Vault
    root.FindViewById<ImageView>(Resource.Id.iconNavVault)
        .SetColorFilter(tab == "vault" ? selectedColor : unselectedColor,
PorterDuff.Mode.SrcIn);
    root.FindViewById<TextView>(Resource.Id.textNavVault)
        .SetTextColor(tab == "vault" ? selectedColor : unselectedColor);

    // Warnings
    root.FindViewById<ImageView>(Resource.Id.iconNavWarnings)
        .SetColorFilter(tab == "warnings" ? selectedColor : unselectedColor,
PorterDuff.Mode.SrcIn);
    root.FindViewById<TextView>(Resource.Id.textNavWarnings)
        .SetTextColor(tab == "warnings" ? selectedColor : unselectedColor);

```



```
// Profile
root.FindViewById<ImageView>(Resource.Id.iconNavProfile)
    .SetColorFilter(tab == "profile" ? selectedColor : unselectedColor,
PorterDuff.Mode.SrcIn);
root.FindViewById<TextView>(Resource.Id.textNavProfile)
    .SetTextColor(tab == "profile" ? selectedColor : unselectedColor);

    TabSelected?.Invoke(this, tab);
}
}
}
```

--- FILE: FinalProject333057891\SecurioClient\PasswordBannerAdapter.cs ---

```
using Android.Views;
using Android.Widget;
using AndroidX.RecyclerView.Widget;
using SecurioModels.DataTransferObjects;
using System;
using System.Collections.Generic;

namespace SecurioClient
{
    /// <summary>
    /// Adapter that feeds <see cref="VaultItem"/> items into the vault RecyclerView.
    /// </summary>
    public class PasswordBannerAdapter : RecyclerView.Adapter
    {
        private List<VaultItem> items;

        /// <summary>Raised when the user taps a password banner.</summary>
        public event EventHandler<int> ItemClick;

        /// <summary>Raised when the user taps the edit icon on a banner.</summary>
        public event EventHandler<int> EditClick;

        public PasswordBannerAdapter(List<VaultItem> items)
        {
```

```

        this.items = items ?? new List<VaultItem>();
    }

    public override int ItemCount => items.Count;

    public override RecyclerView.ViewHolder OnCreateViewHolder(ViewGroup parent, int
viewType)
    {
        View view = LayoutInflater.From(parent.Context)
            .Inflate(Resource.Layout.item_password_banner, parent, false);
        return new PasswordBannerViewHolder(view);
    }

    public override void OnBindViewHolder(RecyclerView.ViewHolder holder, int position)
    {
        var vh = (PasswordBannerViewHolder)holder;
        var entry = items[position];

        vh.TextViewIcon.Text = string.IsNullOrEmpty(entry.AccountName)
            ? "?"
            : entry.AccountName.Substring(0, 1).ToUpperInvariant();

        vh.TextViewSiteName.Text = entry.AccountName ?? string.Empty;
        vh.TextViewUsername.Text = entry.AccountUsername ?? string.Empty;

        vh.ItemView.Click -= vh.OnItemClick;
        vh.ItemView.Click += vh.OnItemClick;
        vh.ItemClickAction = pos => ItemClick?.Invoke(this, pos);

        vh.ImageViewEdit.Click -= vh.OnEditClick;
        vh.ImageViewEdit.Click += vh.OnEditClick;
        vh.EditClickAction = pos => EditClick?.Invoke(this, pos);
    }

    /// <summary>
    /// Replaces the full data set and refreshes the list.
    /// </summary>
    public void UpdateData(List<VaultItem> newItems)
    {

```

```

        items = newItems ?? new List<VaultItem>();
        NotifyDataSetChanged();
    }

    // -----
    // ViewHolder
    // -----
    private class PasswordBannerViewHolder : RecyclerView.ViewHolder
    {
        public TextView TextViewIcon { get; }
        public TextView TextViewSiteName { get; }
        public TextView TextViewUsername { get; }
        public ImageView ImageViewEdit { get; }

        public Action<int> ItemClickAction { get; set; }
        public Action<int> EditClickAction { get; set; }

        public PasswordBannerViewHolder(View itemView) : base(itemView)
        {
            TextViewIcon =
itemView.FindViewById<TextView>(Resource.Id.textViewBannerIcon);
            TextViewSiteName =
itemView.FindViewById<TextView>(Resource.Id.textViewBannerSiteName);
            TextViewUsername =
itemView.FindViewById<TextView>(Resource.Id.textViewBannerUsername);
            ImageViewEdit =
itemView.FindViewById<ImageView>(Resource.Id.imageViewBannerEdit);
        }

        public void OnItemClick(object sender, EventArgs e)
        {
            ItemClickAction?.Invoke(AdapterPosition);
        }

        public void OnEditClick(object sender, EventArgs e)
        {
            EditClickAction?.Invoke(AdapterPosition);
        }
    }

```

```
}
}
```

--- FILE: FinalProject333057891\SecurioClient\PasswordCheckWorker.cs ---

```
using Android.Content;
using Android.Util;
using SecurioClient.Helpers;
using SecurioClient.Helpers.ServerHelpers;
using System.Threading.Tasks;

namespace SecurioClient
{
    /// <summary>Static helper that performs the password-health check and fires local
    notifications for any problems found.</summary>
    public static class PasswordCheckWorker
    {
        private const string Tag = "PasswordCheckWorker";
        private const string DefaultUsername = "User";

        /// <summary>Runs the password-health check and posts notifications for each category
        that has issues.</summary>
        public static async Task RunCheckAsync(Context context)
        {
            // Retrieve the persisted user ID from secure storage (survives logout).
            int userId = await StorageHelper.GetUserId();
            if (userId <= 0)
            {
                Log.Info(Tag, "No stored user ID â€" skipping password check.");
                return;
            }

            var service = new PasswordCheckService();
            var (success, message, data) = await service.CheckAsync(userId);

            if (!success || data == null)
            {
                Log.Warn(Tag, $"Server check failed: {message}");
                return;
            }
        }
    }
}
```

```

    }

    // Retrieve stored username for personalised notification text.
    string username = await StorageHelper.GetUsername() ?? DefaultUsername;

    // Fire individual notifications for each category that has issues.
    if (data.BreachedCount > 0)
    {
        NotificationHelper.Show(
            context,
            NotificationHelper.NotificationIdBreach,
            "âš• Breached Passwords Detected",
            $" {username}, {data.BreachedCount} of your passwords appeared in known data breaches. Change them now.");
    }

    if (data.OldCount > 0)
    {
        NotificationHelper.Show(
            context,
            NotificationHelper.NotificationIdOld,
            "•• Old Passwords Detected",
            $" {username}, {data.OldCount} of your passwords haven't been updated in over 90 days.");
    }

    if (data.MasterPasswordOld)
    {
        NotificationHelper.Show(
            context,
            NotificationHelper.NotificationIdMaster,
            "•• Master Password Needs Update",
            $" {username}, your master password hasn't been changed in over 90 days. Consider updating it.");
    }

    Log.Info(Tag, $"Check done â€" breached={data.BreachedCount}, old={data.OldCount}, masterOld={data.MasterPasswordOld}.");
}

```

```
}
}
```

```
--- FILE: FinalProject333057891\SecurioClient\PasswordCheckWorkerFactory.cs ---
// File removed â€” WorkManager factory no longer used.
// Periodic password monitoring is handled by PasswordMonitorService (foreground service).
```

```
--- FILE: FinalProject333057891\SecurioClient\PasswordMonitorService.cs ---
using Android.App;
using Android.Content;
using Android.OS;
using SecurioClient.Helpers;
using SecurioClient.Helpers.ServerHelpers;
using System.Threading.Tasks;

namespace SecurioClient
{
    /// <summary>Foreground service that performs a password-health check once every 24
    hours.</summary>
    [Service(Exported = false)]
    public class PasswordMonitorService : Service
    {
        private const long IntervalMs = 24 * 60 * 60 * 1000L; // 24 hours

        private Android.OS.Handler _handler;
        private Java.Lang.Runnable _runnable;

        /// <summary>Creates the notification channel, starts the foreground notification, and
        schedules the first check.</summary>
        public override void OnCreate()
        {
            base.OnCreate();

            NotificationHelper.CreateChannel(this);

            // Show the mandatory persistent notification for foreground services.
            StartForeground(
```

```

        NotificationHelper.ForegroundNotificationId,
        NotificationHelper.BuildForegroundNotification(this));

        _handler = new Android.OS.Handler(Looper.MainLooper);
        _runnable = new Java.Lang.Runnable(async () => await RunCycleAsync());

        // Run the first check immediately, then repeat every 24 h.
        _handler.Post(_runnable);
    }

    /// <summary>Returns START_STICKY so Android restarts the service if it is
    killed.</summary>
    public override StartCommandResult OnStartCommand(Intent intent,
        StartCommandFlags flags, int startId)
    {
        // START_STICKY ensures Android restarts the service if it is killed.
        return StartCommandResult.Sticky;
    }

    /// <summary>Not a bound service; returns null.</summary>
    public override IBinder OnBind(Intent intent) => null;

    /// <summary>Removes pending callbacks and cleans up when the service is
    destroyed.</summary>
    public override void OnDestroy()
    {
        _handler?.RemoveCallbacks(_runnable);
        base.OnDestroy();
    }

    /// <summary>Runs the password-check once and then re-schedules itself for the next
    interval.</summary>
    private async Task RunCycleAsync()
    {
        await PerformCheckAsync(this);
        _handler.PostDelayed(_runnable, IntervalMs);
    }

```

```

    /// <summary>Performs a single password-health check and posts a notification when
issues are found.</summary>
    internal static async Task PerformCheckAsync(Context context)
    {
        int userId = await StorageHelper.GetUserId();
        if (userId <= 0) return;

        var result = await PasswordCheckServerService.FetchAsync(userId);
        if (result == null) return;

        NotificationHelper.PostCheckResult(context, result);
    }
}

```

--- FILE: FinalProject333057891\SecurioClient\PasswordOptionsBottomSheet.cs ---

```

using Android.OS;
using Android.Views;
using Android.Widget;
using Google.Android.Material.BottomSheet;
using SecurioModels.DataTransferObjects;
using System;

namespace SecurioClient
{
    /// <summary>
    /// BottomSheetDialogFragment that presents View, Edit, and Delete options
    /// for a single <see cref="VaultItem"/> password entry.
    /// </summary>
    public class PasswordOptionsBottomSheet : BottomSheetDialogFragment
    {
        /// <summary>Tag used when showing the fragment via the support fragment
manager.</summary>
        public const string TagName = "PasswordOptions";

        private VaultItem _entry;

        /// <summary>Raised when the user taps the View option.</summary>

```



```

public event EventHandler ViewClicked;

/// <summary>Raised when the user taps the Edit option.</summary>
public event EventHandler EditClicked;

/// <summary>Raised when the user taps the Delete option.</summary>
public event EventHandler DeleteClicked;

/// <summary>
/// Creates a new instance pre-loaded with the given vault entry.
/// </summary>
public static PasswordOptionsBottomSheet NewInstance(VaultItem entry)
{
    return new PasswordOptionsBottomSheet { _entry = entry };
}

public override View OnCreateView(LayoutInflater inflater, ViewGroup container,
Bundle savedInstanceState)
{
    var view = inflater.Inflate(Resource.Layout.bottom_sheet_password_options,
container, false);

    // Populate header
    string accountName = _entry?.AccountName ?? string.Empty;
    view.FindViewById<TextView>(Resource.Id.textViewSheetTitle).Text =
accountName;
    view.FindViewById<TextView>(Resource.Id.textViewSheetUsername).Text =
_entry?.AccountUsername ?? string.Empty;
    view.FindViewById<TextView>(Resource.Id.textViewSheetIcon).Text =
    string.IsNullOrEmpty(accountName) ? "?" : accountName.Substring(0,
1).ToUpperInvariant();

    // View option
    view.FindViewById(Resource.Id.layoutOptionView).Click += (s, e) =>
    {
        Dismiss();
        ViewClicked?.Invoke(this, EventArgs.Empty);
    };
}

```

```
// Edit option
view.FindViewById(Resource.Id.layoutOptionEdit).Click += (s, e) =>
{
    Dismiss();
    EditClicked?.Invoke(this, EventArgs.Empty);
};

// Delete option
view.FindViewById(Resource.Id.layoutOptionDelete).Click += (s, e) =>
{
    Dismiss();
    DeleteClicked?.Invoke(this, EventArgs.Empty);
};

return view;
}
}
}

--- FILE: FinalProject333057891\SecurioClient\PasswordStrength.cs ---
public enum PasswordStrength { Weak, Fair, Strong, VeryStrong }

public static class ValidationHelper
{
    public static (bool IsValid, string ErrorMessage) ValidateUsername(string username)
        => string.IsNullOrEmpty(username) ? (false, "Username required") : (true, null);

    public static (bool IsValid, string ErrorMessage) ValidateEmail(string email)
        => string.IsNullOrEmpty(email) || !email.Contains("@") ? (false, "Invalid email") :
        (true, null);

    public static (bool IsValid, string ErrorMessage) ValidatePassword(string password)
        => string.IsNullOrEmpty(password) || password.Length < 6 ? (false, "Password too
short") : (true, null);

    public static (bool IsValid, string ErrorMessage) ValidatePasswordsMatch(string p1, string
p2)
        => p1 == p2 ? (true, null) : (false, "Passwords do not match");
}
```

```
public static PasswordStrength GetPasswordStrength(string password)
{
    if (string.IsNullOrEmpty(password)) return PasswordStrength.Weak;
    if (password.Length < 6) return PasswordStrength.Weak;
    if (password.Length < 10) return PasswordStrength.Fair;
    if (password.Length < 14) return PasswordStrength.Strong;
    return PasswordStrength.VeryStrong;
}
}
```

--- FILE: FinalProject333057891\SecurioClient\SecurioClientApplication.cs ---

```
using Android.App;
using Android.Runtime;
using System;

namespace SecurioClient
{
    /// <summary>Custom Application class that provides Android application-level
    initialization.</summary>
    [Application]
    public class SecurioClientApplication : Application
    {
        /// <summary>Initializes a new instance of SecurioClientApplication.</summary>
        public SecurioClientApplication(IntPtr handle, JniHandleOwnership transfer)
            : base(handle, transfer) { }
    }
}
```

--- FILE: FinalProject333057891\SecurioClient\SimpleTextWatcher.cs ---

```
using Android.Text;
using System;

public class SimpleTextWatcher : Java.Lang.Object, ITextWatcher
{
    private readonly Action<string> _onTextChanged;
```

```

    public SimpleTextWatcher(Action<string> onTextChanged) => _onTextChanged =
onTextChanged;
    public void AfterTextChanged(IEditable s) { }
    public void BeforeTextChanged(Java.Lang.ICharSequence s, int start, int count, int after) {
}
    public void OnTextChanged(Java.Lang.ICharSequence s, int start, int before, int count)
        => _onTextChanged?.Invoke(s?.ToString());
}

```

--- FILE: FinalProject333057891\SecurioModels\DataTransferObjects\AuthData.cs ---
using System;

```

namespace SecurioModels.DataTransferObjects
{
    /// <summary>Holds session data returned after successful login or signup.</summary>
    public class AuthData
    {
        public int UserId { get; set; }
        public string Username { get; set; }
        public string Token { get; set; }
    }
}

```

--- FILE:
FinalProject333057891\SecurioModels\DataTransferObjects\MasterPasswordHistory.cs ---
using System;

```

namespace SecurioModels.DataTransferObjects
{
    /// <summary>Represents a single entry in the master-password history log.</summary>
    public class MasterPasswordHistory
    {
        public int Id { get; set; }
        public int UserId { get; set; }
        // The PBKDF2-derived key that was stored in Users.MasterPasswordKey at the time.
        public string PasswordKey { get; set; }
    }
}

```

```
// The AuthSalt that was used to derive PasswordKey â€” needed by the client for
comparison.
public string AuthSalt { get; set; }
// The date when this password was originally created (equals Users.LastPasswordUpdate
at time of archival).
public DateTime CreatedAt { get; set; }
}
}
```

--- FILE:

FinalProject333057891\SecurioModels\DataTransferObjects\PasswordCheckRequest.cs ---
namespace SecurioModels.DataTransferObjects

```
{
    /// <summary>Request body for the PasswordCheck endpoint.</summary>
    public class PasswordCheckRequest
    {
        public int UserId { get; set; }
    }
}
```

--- FILE:

FinalProject333057891\SecurioModels\DataTransferObjects\PasswordCheckResult.cs ---
namespace SecurioModels.DataTransferObjects

```
{
    /// <summary>Carries the password-health summary returned by the PasswordCheck
endpoint.</summary>
    public class PasswordCheckResult
    {
        // Number of vault items whose stored password has been found in a data breach.
        public int BreachedCount { get; set; }

        // Number of vault items whose password has not been changed in more than 90 days.
        public int OldCount { get; set; }

        // True when the user's master password itself has not been changed in 90+ days.
        public bool MasterPasswordOld { get; set; }
    }
}
```

```
}
```

--- FILE: FinalProject333057891\SecurioModels\DataTransferObjects\SaltData.cs ---

```
using System;
```

```
using System.Collections.Generic;
```

```
using System.Text;
```

```
namespace SecurioModels.DataTransferObjects
```

```
{
```

```
    /// <summary>Carries the cryptographic salts required for a client to perform local  
    hashing.</summary>
```

```
    public class SaltData
```

```
    {
```

```
        public string AuthSalt { get; set; }
```

```
        public string EncryptionSalt { get; set; }
```

```
    }
```

```
}
```

--- FILE:

FinalProject333057891\SecurioModels\DataTransferObjects\UpdateAccountRequest.cs ---

```
using System.Collections.Generic;
```

```
namespace SecurioModels.DataTransferObjects
```

```
{
```

```
    /// <summary>Carries the data needed for updating a user's account details.</summary>
```

```
    public class UpdateAccountRequest
```

```
    {
```

```
        public string Username { get; set; }
```

```
        public string Email { get; set; }
```

```
        public bool PasswordChanged { get; set; }
```

```
        // Only populated when PasswordChanged == true
```

```
        public string MasterPasswordKey { get; set; }
```

```
        public string AuthSalt { get; set; }
```

```
        public string EncryptionSalt { get; set; }
```

```
        // SHA-1 hash of the plaintext new password, used for the HIBP breach check.
```

```
// Never stored, discarded by the server after the check.
public string PasswordSha1Hash { get; set; }

// Re-encrypted vault items " sent when the master password changes
// so the server can bulk-update them in one transaction.
public List<VaultItem> VaultItems { get; set; }
}
}

--- FILE: FinalProject333057891\SecurioModels\DataTransferObjects\User.cs ---
using System;
using System.Collections.Generic;
using System.Text;

namespace SecurioModels.DataTransferObjects
{
    /// <summary>Represents a user account including security salts and hashed keys stored in
    the database.</summary>
    public class User
    {
        public int Id { get; set; }
        public string Username { get; set; }
        public string Email { get; set; }
        public string MasterPasswordKey { get; set; } // The key derivation result
        public string AuthSalt { get; set; } // The salt for deriving the master password key
        public string EncryptionSalt { get; set; } // The salt for the AES vault
        public DateTime LastLogin { get; set; }
        public DateTime LastPasswordUpdate { get; set; }
        public DateTime CreatedAt { get; set; }
        public int PasswordCount { get; set; }
        // SHA-1 hash of the plaintext master password, used for the HIBP breach check.
        // Never stored in the database; consumed only during registration validation.
        public string PasswordSha1Hash { get; set; }
    }
}
```

```
--- FILE: FinalProject333057891\SecurioModels\DataTransferObjects\VaultItem.cs ---
```

```
using System;

namespace SecurioModels.DataTransferObjects
{
    /// <summary>The secure data carrier for a stored credential with AES-GCM encryption
    components.</summary>
    public class VaultItem
    {
        public int Id { get; set; }
        public int UserId { get; set; }
        public string AccountName { get; set; }
        public string AccountUsername { get; set; }
        public string IV { get; set; } // AES-GCM initialisation vector
        public string Tag { get; set; } // AES-GCM authentication tag
        public string CipherText { get; set; } // AES-GCM encrypted password
        public string Notes { get; set; }
        public string Sha1Hash { get; set; } // Unsalted SHA-1 hash for HIBP breach lookup
        public bool IsLeaked { get; set; }
        public DateTime LastUpdate { get; set; }
        // Transient flag set by the client to indicate whether the password ciphertext was
        changed.
        // Never persisted to the database; used only during the UpdateVaultItem request.
        public bool PasswordChanged { get; set; }
    }
}

--- FILE: FinalProject333057891\SecurioModels\ServerResponse.cs ---
using System;
using System.Collections.Generic;
using System.Text;

namespace SecurioModels
{
    /// <summary>The standard template for every server response, ensuring a success flag and
    message are always present.</summary>
    public class ServerResponse<T>
    {
        public bool Success { get; set; }
    }
}
```



```

    public string Message { get; set; }
    public T Data { get; set; }
}

```